



大模型必知必会

从 0 到 1 构建你的大模型

BYO-LLM-FZ20 · Build Your Own LLM from Zero to One

pithub.com/QinHsiu/BYO-LLM-FZ20

大模型必知必会：一份数据清洗&治理的保姆级指南

原创 QinHsiu 阿Q的AI笔记 2026年8月16日 12:53 北京

——结合BYO-LLM-FZ2O项目实战，讲透LLM数据清洗的流程、工具与面试考点

一、为什么数据清洗与治理如此重要？

大模型研发进入深水区的2026年，行业共识正经历从“模型中心（Model-Centric）”向“数据中心（Data-Centric）”的深刻演进。单纯堆砌Token数量已边际效应递减，数据的语义密度与工程精度成为了突破模型性能上限的关键。

根据Gartner的研究，在基础设施与运营领域，**仅有28%的AI项目能够完全成功并达到预期投资回报率，而有高达20%的项目以彻底失败告终**。57%的受访者在将AI应用于其领域时至少遭遇过一次失败。失败的核心原因之一，就是数据问题。

治理数据即治理AI。数据偏差会传导为模型偏差，而模型上线后产生的新数据又会反过来污染数据资产。更严峻的是，北京大学DCAI团队在研发DataFlow的过程中观察到：虽然模型已进入“自动驾驶时代”，但数据准备仍处于“**手工坊**”阶段——碎片化的Python脚本、不可复用的正则表达式、缺乏观测性的黑盒流程，已成为大模型落地企业级应用的最大瓶颈。

💡 **面试考点**：面试官可能会问——“你参与过的AI项目中，数据准备阶段占了多少工作量？”据IBM统计，**数据准备通常消耗AI项目总工作量的60%-80%**。如果你能答出这个数字，并解释为什么，面试官会知道你是真正在一线干过活的人。

二、BYO-LLM-FZ2O：从零构建大模型的数据基石

再聪明的模型，也不能“空腹上岗”。开源项目 **BYO-LLM-FZ2O**（Build Your Own LLM from Zero to One）的 01 Data 目录，系统梳理了LLM训练与评估所需的数据资源。

2.1 主流预训练数据集一览

BYO-LLM-FZ2O整理的核心数据集包括：

RedPajama	1.21万亿Token	开源复刻LLaMA的预训练数据集
SlimPajama	6270亿Token	Cerebras基于RedPajama清洗去重后的高质量版本
The Pile	800G	22个高质量数据集混合
FineWeb	15T Token	HuggingFace从Common Crawl清洗消重后发布
FineWeb-Edu	5.4T Token	从FineWeb中筛选的高质量教育水平数据
Zyda	1.3T Token	高质量小规模混合预训练数据集
MNBVC	目标40T	里屋社区发起的中文互联网语料集

CBook-150K	15万本	复旦开源的中文图书下载和抽取方案
书生万卷	1T文本+150G图文	多模态数据集

💡 **面试考点：**面试官常问——“大模型预训练数据一般来自哪些数据源？主流预训练语料库各自有什么特点？”如果你能说出RedPajama、SlimPajama、FineWeb的区别，以及它们各自的清洗特点，就是加分项。

2.2 从原始数据到高质量语料：清洗全流程

以 **SlimPajama** 为例，Cerebras公司对RedPajama进行了系统化的清洗和去重。其处理流程包括多个阶段：**NFC正规化、清理、去重、文档交错、文档重排、训练集和保留集拆分**，以及训练集与保留集中相似数据去重等步骤。

具体如下：

- 1. NFC正规化 (NFC Normalization)：** 移除非Unicode字符，统一文本编码
- 2. 过滤短文档 (Filter Short Documents)：** 删除短小且质量较低的文档
- 3. 全局去重 (Global Deduplication)：** 使用精确 (13-Grams) 和模糊 (MinHashLSH) 算法，通过分布式并行和内存池优化，实现了对1.2T Token的全量去重
- 4. 文档交错与重排 (Interleave & Shuffle)：** 打乱文档顺序，避免数据偏差

最终，SlimPajama **滤掉了49.6%的重复Token**——这意味着原始数据中近一半的内容是冗余的。

💡 **面试考点：**面试官常问——“什么是MinHash？请描述MinHash算法的原理及其在大规模文本去重中的应用。”你需要能解释MinHash如何通过最小哈希签名来近似估计文档相似度，以及在大规模去重中的工程实现。

FineWeb 则是另一种清洗范式的代表。它从Common Crawl的WARC HTML中提取主文本，然后对每个爬取批次独立进行MinHash去重（使用5-gram分片和14×8个哈希函数）。其数据处理逻辑从最初的8步优化到了5步，整合了Gopher质量过滤、Minhash去重、C4筛选等操作。

💡 **面试考点：**面试官可能会问——“Common Crawl数据的清洗流程一般包含哪些步骤？从原始WARC到最终可用文本的完整pipeline是什么样的？”你需要能描述：WARC解析→HTML清理→文本提取→质量过滤→去重→PII脱敏的完整链路。

三、LLM数据清洗的核心方法论

综合以上实践，一套工业级的LLM数据清洗流程通常包含以下环节：

第一阶段：数据采集与发现

在开始清洗之前，必须先完成数据盘点。你需要问自己：

- 数据在哪里？有哪些数据源？（参考BYO-LLM-FZ2O整理的数据集清单）
- 数据的格式是什么？结构化还是非结构化？

- 数据的质量和完整性如何？

第二阶段：清洗与标准化

这是数据准备中最核心的工作。但在动手清洗之前，必须先做一件事：**了解自己的训练数据。**

2.1 先分析，再清洗——可视化分析是清洗策略的起点

“不做分析就盲目清洗，就像医生不拍片就开刀。”

开源项目**Paper_Rec_Skill**的 `Exp_Sandbox` 技能提供了一个可借鉴的自动化实验框架，其核心原则是：**分析 (Analysis) → 生成方案 → 小规模验证 → 清洗/训练 → 评估 → 迭代**。该框架提供了两个核心分析命令：

命令	分析对象	核心任务
<code>/exp_analysis train</code>	训练集	分析数据分布、质量、标签平衡性、文本长度分布等
<code>/exp_analysis eval</code>	测试集/评估集	聚类Badcase，定位失败模式，生成多套改进方案

分析阶段的核心产出：

- **分布统计**：标签分布、文本长度分布、数据来源分布
- **质量画像**：噪声代理指标（Noise Proxies）、质量评分分布
- **Badcase聚类**：将失败样本按模式聚类（如“手写拼音”“低分辨率扫描”），保证**聚类内多样性**，避免重复分析
- **问题清单**：每个聚类的具体问题描述

可视化分析的核心方法：

图表类型	用途	发现什么问题
直方图	展示数值型特征的分布	数据偏态、长尾分布、异常值
条形图	展示类别型特征的频次	标签不平衡、类别缺失
箱线图	展示数据的离散程度	离群点、数据范围异常
t-SNE/UMAP降维图	将高维文本嵌入降维到2D	语义聚类、离群样本、数据覆盖盲区
文本长度分布图	展示序列长度的分布	超长文本、截断风险、批次填充浪费
困惑度分布图	评估文本的“自然度”	低质量文本、机器生成噪声
重复率分布图	展示文档级别的重复率	冗余数据、需要去重

💡面试官常问——“你如何评估一种清洗策略是否有效？什么时候决定全量应用？”
参考答案：在小规模子集上做A/B对比验证，设定明确的收益阈值（如F1提升 $\geq 2\%$ ），只有验证通过才全量推广——这叫 **Predict-then-Verify原则**。

2026年数据可视化分析的新工具：

工具	定位	特点
Lilac	LLM数据探索与质量控制	可视化、量化和改进预训练/微调数据质量，被Coher e、Databricks使用
TokenSmith	大规模训练数据编辑与检查	支持Megatron-style框架中的搜索、查看、导出、采样等交互操作
Data-Forge MCP	MCP协议数据科学助手	将任意LLM转化为数据科学助手，支持加载、清洗、验证、画像、可视化
DocWrangler	LLM数据处理IDE	实时可视化、情境感知协助、持续改进
Paper_Rec_Skill	训练&评估的可视化	原生支持训练、评估的自动化可视化分析

2.2 清洗与标准化的具体操作

完成分析并制定清洗策略后，进入具体清洗环节。对于LLM数据，主要包括：

1. 文本清洗（Text Cleaning）

- 移除HTML标签、链接、邮箱
- 规范化空白字符，修复Unicode字符

2. 文本过滤（Text Filtering）

- 按文本长度、字母数字比率、特殊字符比率、字符重复模式进行过滤
- 使用困惑度（Perplexity）做质量过滤
- 移除包含不安全术语的文档

3. 去重（Deduplication）

- 精确去重：MD5哈希去重
- 模糊去重：MinHashLSH、Bloom过滤器
- 不同粒度：句子级、文档级、数据集级

4. 敏感信息处理

- PII（个人可识别信息）识别与移除
- 邮件、URL、电话号码打码

5. 版权与合规

- 著作权信息去除
- 将安全能力前置到数据入库、知识切片、检索召回全流程

💡 **面试考点：**面试官常问——“预训练数据的质量过滤有哪些常见方法？”你需要能列举：长度过滤、语言检测、困惑度过滤、重复率过滤、安全内容过滤等。

第三阶段：数据配比与课程学习

清洗完成后，还需要考虑不同数据源的配比。不同配比对模型能力有显著影响。例如：

- Chinchilla Scaling Law对预训练数据量有明确指导意义
- 领域大模型预训练时，通用语料和领域语料的比例需要精心设计
- 数据量不够时，需要权衡是增大epoch数、增加合成数据还是进行上采样

💡 **面试考点：**面试官可能会问——“预训练数据的数据配比为什么重要？不同配比对模型能力有什么影响？”你需要能解释数据配比如何影响模型的多任务能力和领域适应性。

四、2026年的工具与实践

4.1 开源数据清洗工具生态

工具	定位	特点
NeMo Curator	NVIDIA出品的大规模数据预处理工具	支持文本清洗、语言识别、质量过滤、去重，GPU加速
Data-Juicer	“数据榨汁机”	将数据处理视为可组合基础设施，支持文本/图像/音频/视频多模态
llm-tab-cleaner	表格数据清洗	完整审计追踪+逐单元置信度评分，基于LLM可移除>70%数据质量问题的研究
ZeroDC	零训练数据清洗系统	利用LLM生成清洗规则和思维链，无需模型训练，准确率提升5.36%，速度加快8.16倍
DeepPrep	LLM驱动的自主数据准备系统	PVLDB 2026发表，支持31+算子，覆盖清洗、连接、聚合等

4.2 LLM正在重写数据治理的规则

传统的数据清洗依赖人工编写规则，但2026年，**AI正在改变游戏规则。**

- **llm-tab-cleaner** 基于arXiv论文证明LLMs可以移除超过70%的数据质量问题
- **ZeroDC** 实现了零训练的可解释数据清洗，自动为每个修正结果生成解释
- **TabClean** 将LLM推理编译为可重用的确定性Python程序，大幅降低重复运行时的API成本

北京大学DCAI团队推出的 **DataFlow** 框架更是将数据工程推向新高度：

- 将近**200个内置算子**严格分类

- DataFlow-WebUI支持**拖拉拽编排**和**实时数据探针**，实现数据治理的“白盒化”
- 技术报告登顶Hugging Face Daily Papers榜首

💡 **面试考点：**面试官可能会问——“你如何确保数据清洗的质量和可复现性？”**参考答案：**使用带审计追踪的工具（如llm-tab-cleaner）、建立数据版本管理、设计数据质量监控指标。

五、AI时代的数据治理新范式

5.1 从“事后清洗”到“事前设计”

2026年数据治理的最大变化，是从“**事后清洗**”向“**事前设计 (Data by Design)**”的转变。正如Atlan所建议的：**在数据进入系统之前就做好规范设计。**

这要求：

- **数据版本管理：**训练数据集需要像代码一样进行版本控制
- **数据血缘追踪：**AI模型的输出需要能够追溯到训练数据的来源
- **数据质量监控：****数据漂移检测**能够在生产数据分布偏离训练数据时发出预警

5.2 为LLM知识库准备数据：七步法

如果数据是用于构建LLM知识库（如RAG系统），Atlan提出了**七步准备法**：

1. **盘点并分类：**按领域和敏感度标签对源文档进行分类
2. **去重：**为每个概念强制建立一个权威版本
3. **规范化格式与结构：**确保分块一致性
4. **丰富元数据：**添加业务上下文、术语表链接和数据血缘
5. **映射访问控制：**在索引之前而非之后配置权限
6. **领域专家认证：**作为硬性准入关卡
7. **建立新鲜度策略：**防止语料库衰减

这七步的核心思想是：**数据准备不仅是预处理任务，更是一个治理行为。**

5.3 2026年面试必考：数据工程25题

根据2026年大模型面试的最新趋势，**数据工程已成为独立的高频考点模块**。以下是高频题型分类：

预训练数据处理与清洗（5题）

- Q1：大模型预训练数据一般来自哪些数据源？
- Q2：Common Crawl数据的清洗流程包含哪些步骤？
- Q3：预训练数据的质量过滤有哪些常见方法？
- Q4：如何利用困惑度做数据质量过滤？
- Q5：基于规则的启发式数据过滤包含哪些规则？

数据去重与去污染（5题）

- Q6: 预训练数据为什么需要去重?
- Q7: 什么是MinHash? 原理是什么?
- Q8: 数据去重有哪几种粒度?
- Q9: 什么是数据去污染?
- Q10: 如何检测预训练数据中是否包含benchmark的测试集数据?

数据配比与课程学习（4题）

- Q11: 预训练数据的数据配比为什么重要?
- Q12: Chinchilla Scaling Law对预训练数据量有什么指导意义?
- Q13: 领域大模型预训练时, 通用语料和领域语料的比例如何设置?
- Q14: 预训练数据量不够时, 是增大epoch数还是增加合成数据?

合成数据与指令构造（5题）

- Q15: 什么是Self-Instruct?
- Q16: 什么是Evol-Instruct?
- Q17: 合成数据的质量控制有哪些常见方法?
- Q18: LIMA论文的核心发现是什么?
- Q19: 合成数据是否存在“模型坍塌”问题?

六、结语

高质量的数据是LLM成功的基石。从数据采集、清洗、标注到治理, 每一个环节都决定了最终模型的天花板。

在2026年, 数据治理已从“后台清洁工”跃升为**决定企业智能化上限的核心战略基础设施**。无论是正在准备面试的求职者, 还是在一线写代码的工程师, 掌握系统化的数据清洗与治理能力, 都将是AI时代不可或缺的基本功。

“数据是新时代的石油”——但未经提炼的原油, 无法驱动任何引擎。而数据清洗与治理, 正是将原油提炼为高纯度燃料的那道核心工序。

大模型必知必会：从纯文本LLM到多模态大模型的演进之路

原创 QinHsiu 阿Q的AI笔记 2026年8月17日 09:00 北京

从Transformer到原生多模态，一文看懂大模型架构的“变”与“不变”

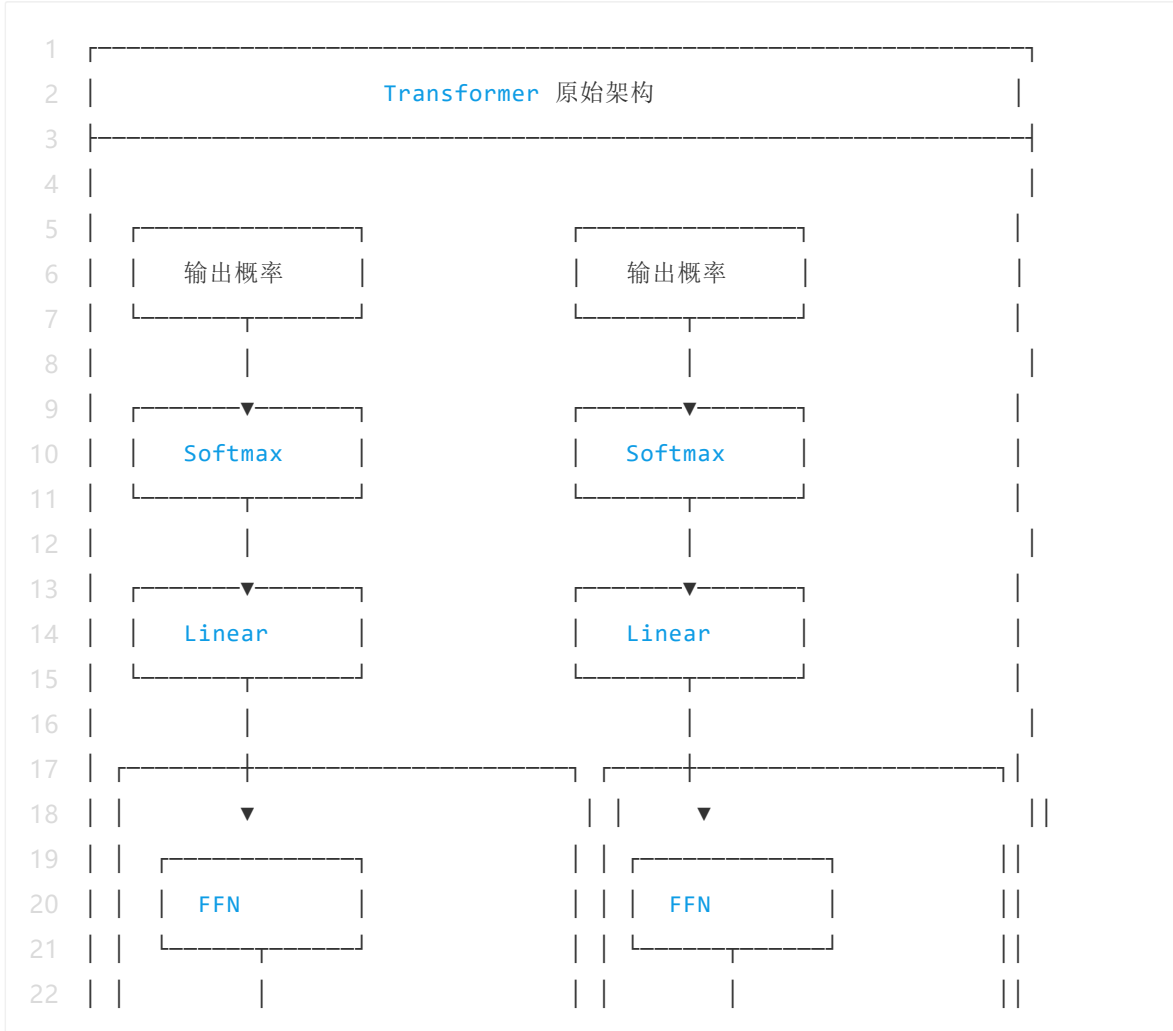
2017年，一篇名为《Attention Is All You Need》的论文横空出世，Transformer架构的提出成为人工智能发展的根本性转折点。此后，GPT系列等基于解码器的大模型通过极致的规模扩展，在参数与数据量的双重驱动下，涌现出强大的通用任务处理能力。

短短几年间，模型架构经历了从纯文本到视觉语言、再到全模态统一的深刻演变。截至2026年8月，已有超过**155种**VLM/MLLM架构被系统性整理和收录。本文将以BYO-LLM-FZ2O项目为参照框架，系统梳理三类架构的演进脉络、核心设计理念与主流开源模型。

一、纯文本大语言模型（LLM）架构

现代所有大语言模型的架构根基都源自Transformer。其核心创新在于用**自注意力机制（Self-Attention）**取代了RNN/LSTM的序列递归处理，实现了并行化训练和远距离依赖关系的优越建模能力。

原始Transformer采用**编码器-解码器（Encoder-Decoder）** 对称结构——编码器由6个相同模块堆叠，每个模块包含多头自注意力子层和前馈网络子层；解码器同样由6个模块组成，额外增加了编码器-解码器交叉注意力子层。



Encoder-only架构在最新的LLM中已较少采用。

面试考点：Encoder-Only（如BERT）双向注意力，擅长理解类任务但不能做生成；Decoder-Only（如GPT、LLaMA）因果注意力，擅长生成式任务，是当前LLM的主流；Encoder-Decoder（如T5）适合Seq2Seq任务（翻译、摘要）。追问：为何现在大模型大部分是Decoder-Only？①生成任务需求主导；②自回归训练目标简洁有效；③训练效率高；④统一了预训练和下游任务形式。

1.3 核心组件的技术演进

现代LLM在微观架构层面进行了大量优化，细节决定成败：

（1）归一化层（Normalization）

大多数现代LLM已从LayerNorm转向**RMSNorm（Root Mean Square Layer Normalization）**，在保持性能的同时计算开销更小。

面试考点：RMSNorm去掉了LayerNorm中的均值中心化步骤，只做均方根归一化，计算量更小且效果相当，已成为LLaMA等模型的标配。

（2）位置编码（Positional Embeddings）

传统的绝对位置编码逐渐被**RoPE（Rotary Positional Embeddings）**或ALiBi取代。RoPE特别受到青睐，因为它能更好地处理长文本序列并保持相对位置关系。在视觉-语言模型中，还发展出了专为多模态设计的**M-RoPE**。

面试考点：RoPE通过在Q和K上按位置角度旋转来编码相对位置信息。优势：①天然支持相对位置；②可外推到更长序列；③不影响注意力计算效率。劣势：实现稍复杂。

（3）激活函数（Activation Functions）

ReLU已成为过去，**SwiGLU**成为Llama等高性能模型的核心选择，通过更复杂的门控机制提升了非线性表达能力。Qwen的3.5系列全部采用**Gated-Attention**。

（4）注意力机制优化

为了应对超长上下文，**MQA（Multi-Query Attention）**和**GQA（Grouped-Query Attention）**逐渐取代标准多头注意力，大幅降低了推理时的KV Cache显存占用。GLM 5系列从常规GQA转向了MLA→DSA的架构。

面试考点（面试高频）：MHA（Multi-Head Attention）每个头有独立的K、V，显存占用最大；MQA（Multi-Query Attention）所有头共享一个K、V，显存最优但质量略降；GQA（Grouped-Query Attention）将头分组，组内共享K、V，是MHA和MQA的折中；MLA（Multi-head Latent Attention）DeepSeek提出，通过低秩压缩KV Cache。

面试考点：在自回归生成中，每生成一个新token都需要重新计算之前所有token的K、V。KV Cache将已计算的K、V缓存下来避免重复计算。显存占用 = $2 \times (\text{K和V}) \times \text{batch_size} \times \text{seq_len} \times \text{num_heads} \times \text{head_dim} \times \text{dtype_bytes}$ 。

面试考点：FlashAttention的原理：通过分块计算和重计算减少HBM访问次数，将注意力计算的复杂度从 $O(n^2)$ 的IO变成 $O(n^2)$ 的计算，显著加速长序列训练和推理。

(5) MoE (混合专家)

如Mixtral 8x7B，通过引入专家路由机制，模型在推理时仅激活部分参数（稀疏激活），从而在拥有超大规模参数的同时保持较低推理成本。DeepSeek-V3/V4、Qwen3-VL等均采用MoE架构。

面试考点：MoE通过路由器（Router）将每个token只分配给少数几个“专家”（Expert）处理，实现稀疏激活——总参数巨大但推理时只激活一小部分。

(6) SSM (状态空间模型)

以Mamba为代表，试图突破Transformer的 $O(n^2)$ 复杂度瓶颈，在处理无限长序列时展现出线性增长的计算优势。

1.4 2026年上半年开源LLM架构演进趋势

根据2026年上半年的开源大模型架构演进总结，主流架构方向呈现以下特点：

- **小米mimo系列：**采用SWA+GQA和MTP，最新HySparse论文已进行稀疏注意力尝试（top-k+SWA）
- **Step模型：**GQA+SWA架构
- **DeepSeek-V4：**在DSA基础上进行token压缩，采用SWA+top-k筛选，使用MHC
- **美团Longcat系列：**借鉴DeepSeek的MLA等前沿进展
- **GLM 5系列：**从GQA转向MLA→DSA架构，使用Muon优化器，参数量翻倍
- **MiniMax：**M2采用GQA架构，M3改为基于GQA的稀疏attention架构MSA
- **Kimi：**采用MLA+MuonClip
- **MiniCPM5：**采用InfLLM-V2（top-k稀疏）+ Lightning Attention

可以看出，目前MoE已发展得相对完善之后，各大厂商又回到了Attention部分进行优化，主要方向为：SWA（截断）、top-k（筛选/裁剪）、GatedDeltaNet/KDA/MLA（压缩）三大方向。

二、视觉语言模型（VLM）架构

视觉语言模型（Vision-Language Models, VLMs）是连接视觉与语言两大模态的关键技术，使机器能够同时“看懂”图像和“理解”文字。

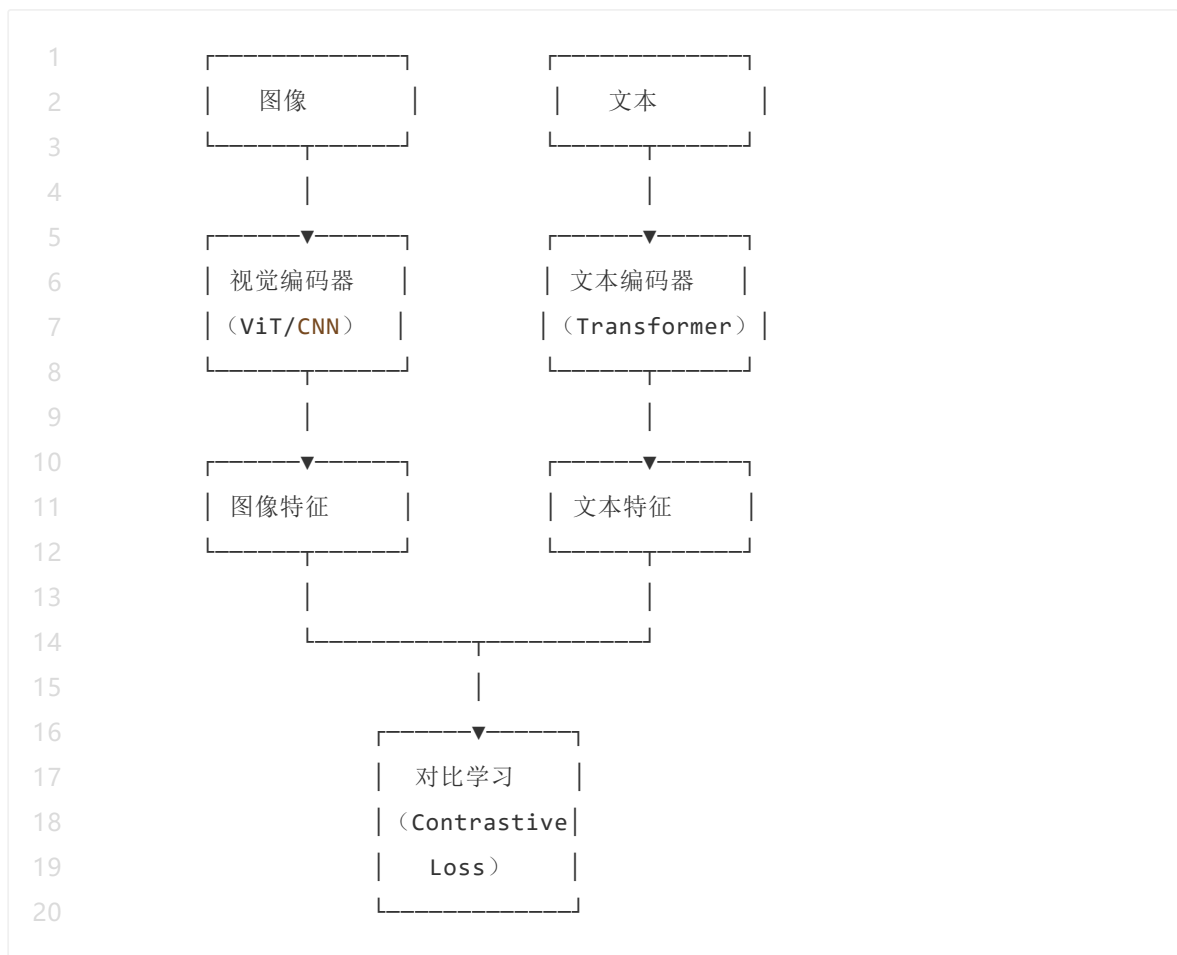
2.1 VLM的三大架构框架

根据架构设计的不同，VLM主要分为以下三类：

面试考点：如何实现不同模态信息（视觉和语言）的有效对齐和融合。视觉特征是连续的、高维的、空间结构的，文本特征是离散的、序列的、语义的，两者之间存在巨大的“语义鸿沟”。

面试考点：VLM的幻觉（Hallucination），VLM除了一般的语义幻觉外，还有关系幻觉——正确识别物体但错误描述空间位置或交互关系（如“书在杯子里面”而非“旁边”）。

(1) 双塔架构 (Two-Tower / Dual-Encoder)



- **代表模型：**CLIP、ALIGN
- **特点：**视觉与文本分别通过独立编码器处理，通过对比损失对齐特征空间
- **优势：**模态特异性表征和高效检索
- **局限：**晚期融合，细粒度跨模态交互受限

面试考点：CLIP模型的工作原理：CLIP采用双塔结构——图像编码器（ViT/ResNet）和文本编码器（Transformer），通过对比学习将图文对映射到同一向量空间。训练时一个batch有N个图文对，形成 $N \times N$ 相似度矩阵，对角线为正样本，其余为负样本，损失函数为InfoNCE。追问：batch中重复图片导致的“伪负样本”如何处理。

面试考点：CLIP是纯对比学习，只能做检索不能生成文本；BLIP联合优化对比学习、图文匹配（ITM）和图像语言生成（LM）三个目标，既能检索也能生成。

面试考点：Q-Former是视觉特征和LLM之间的“翻译官”——通过可学习的query向量，用交叉注意力从冻结的ViT输出中“按需提取”LLM需要的视觉信息。追问：为什么LLaVA反而用回MLP？因为当LLM足够强且数据量足够大时，简单MLP就能学到好映射，且训练更稳定、推理更快。

(2) 单塔架构 (One-Tower)

将视觉与文本输入拼接后送入统一的Transformer，通过随机遮盖视觉或文本片段实现自监督学习。单塔架构更加偏向于效率和轻量化。

- **代表模型：**ViLT、SAIL
- **特点：**单一多模态Transformer同时接收图像patch和文本token

- **优势：** 深层跨模态交互、参数效率高

面试考点： LLaVA采用“ViT + 线性投影器 + LLM”三段式架构。视觉特征通过可训练的MLP投影层映射到LLM的文本嵌入空间。第一阶段冻结LLM和ViT，只训练投影器；第二阶段解冻LLM进行联合微调。

(3) 混合架构（结合双塔与单塔优势）

在浅层分离编码、深层融合交互。例如，Qwen3-VL在Embedding阶段利用双塔结构将异构数据映射为定长向量完成初筛，在Reranker阶段采用计算密集的单塔结构进行精确相关性打分。

2.2 VLM的核心技术组件

VLM围绕三个核心组件构建：

(1) 视觉编码器（Vision Encoder）

将图像分割为patch（通常14×14或16×16像素），生成patch嵌入序列。主流视觉编码器包括：

编码器	特点
CLIP ViT-L/14	24层Transformer，LLaVA系列常用
SigLIP	CLIP改进版，sigmoid损失，数据效率更高
InternViT-6B	60亿参数，NVLM和InternVL使用
Pixtral-ViT	专为变分辨率输入设计

(2) 语言解码器（Language Decoder）

通常是预训练的纯文本LLM，如Llama系列、Qwen系列、Mistral等。

(3) 跨模态连接器（Connector / Fusion）

将视觉特征映射到LLM的token空间，是VLM架构中最关键的设计选择。主要有三种方案：

- **投影器（Projection）**：视觉特征通过小型FFN映射为LLM Token，代表：LLaVA
- **Q-Former**：通过Query Transformer提取视觉特征，代表：BLIP-2
- **交叉注意力（Cross-Attention）**：视觉特征通过交叉注意力层注入LLM，代表：Flamingo

2.3 主流开源VLM（截至2026年8月）

根据2026年初的评估基准，主流开源VLM涵盖8个家族，参数规模从2B到38B不等：

模型家族	规模	架构特点
Qwen2.5-VL	3B, 7B	Dense, M-RoPE

模型家族	规模	架构特点
Qwen3-VL	2B, 8B, 30B-A3B	Dense/MoE, DeepStack
Qwen3.5-VL	2B, 9B, 27B	Dense, GDN
InternVL3.5	2B, 8B, 38B	Dense, InternViT
Gemma 3	4B, 12B, 27B	Dense
GLM-4.1V	9B	Dense, thinking
LLaVA-OV	8B	Dense, SigLIP
MiniCPM-V	8B	Dense

其他值得关注的VLM包括：**Llama 3.2 Vision**（Meta）、**Phi-4 Multimodal**（微软）、**PaliGemma**、**DeepSeek-VL2**、**SmolVLM2**、**EXAONE 4.5**（LG, 33B）。

三、多模态大模型（MLLM）架构

多模态大语言模型（Multimodal Large Language Models, MLLMs）代表了AI架构的最新前沿，致力于整合文本、图像、音频、视频等多种信息模态，实现类人的推理和认知能力。

面试考点：LLM基本是纯Decoder-only Transformer；MLLM在LLM基础上新增两个模块——模态编码器（如CLIP-ViT、Whisper）和投影器（MLP/Q-Former）。输入从纯文本token变为“文本token + 模态token”的统一序列。

3.1 范式迁移：从“外挂式”到“原生统一”

业界正经历从“语言模型 + 多模态外挂”到“原生全模态统一”的深刻范式迁移：

- (1) 连接器范式（Connector-based）
- 代表模型：Flamingo、BLIP/BLIP-2
- 冻结视觉编码器和语言模型，引入轻量级连接器
 - BLIP-2提出Q-Former（Query Transformer）
 - 特点：参数高效、模块化设计
- (2) 投影器范式（Projector-based / LLaVA-style）
- 代表模型：LLaVA系列
- 冻结LLM + 冻结视觉编码器 + 小型可训练投影器
 - LLaVA是较早将视觉特征作为token输入LLM的工作
 - 优势：实现简单、训练高效
- (3) 原生多模态范式（Native Multimodal）

代表模型：Chameleon (Meta) 、GPT-4o (OpenAI) 、Kimi K3

- 在语言模型预训练阶段直接引入多模态序列
- 实现真正的端到端统一

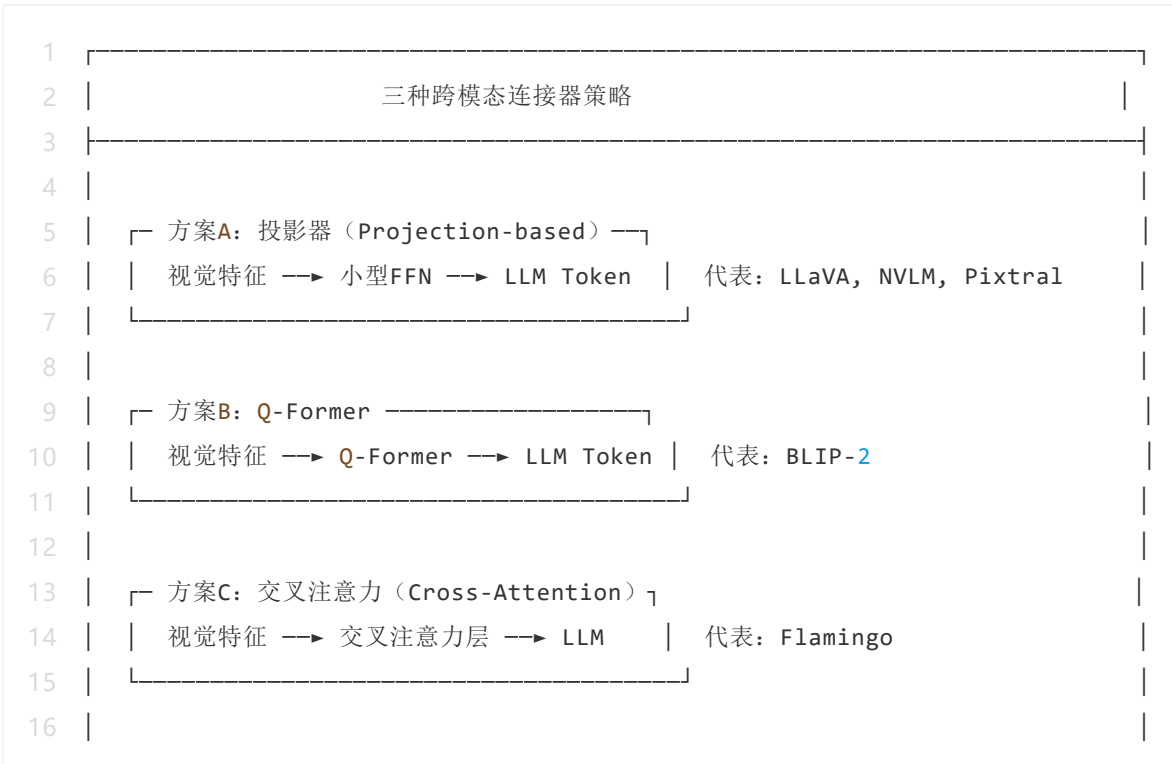
3.2 MLLM架构的完整演进图谱

从架构演进的角度，MLLM的发展可归纳为以下路径：



面试考点：传统独立编码→连接器范式 (Flamingo、BLIP-2) →投影器范式 (LLaVA) →原生多模态 (Chameleon、GPT-4o) 。核心趋势：从“外挂”到“原生”。

3.3 多模态融合的三种主流策略



3.4 主流开源MLLM（截至2026年8月）

2026年发布的代表性开源MLLM：

模型	发布方	发布时间&关键特点
Kimi K3	月之暗面	2026年7月 全球首款2.8万亿参数开源原生多模态MoE大模型
MiniCPM-V 4.6	OpenBMB	2026年5月 1.3B，口袋级多模态模型，256K上下文
Ming-flash-omni 2.0	Inclusion AI	2026年2月 100B总参数/6B激活参数，SOTA全模态MLLM
Gemma 4	Google	31B，Apache 2.0，GPQA 84.3
Qwen3.5	阿里	2026年2月 原生多模态，0.8B–397B（MoE，17B激活）
GLM-5	z.AI	2026年2月 744B总参数/40B激活
MiniMax-M2.5	MiniMax	2026年2月 229B总参数/10B激活

2025年发布的重要开源MLLM：

模型	发布方	关键特点
Qwen2.5-Omni	阿里	全模态（含音频）
InternVL3 / 3.5	OpenGVLab	最强完全许可替代方案
Llama 4 Scout / Maverick	Meta	多模态
Phi-4-multimodal	微软	多模态
MiniCPM-o-2.6	OpenBMB	多模态

超大规模多模态模型（MoE架构）：

模型	总参数	激活参数&&特点
Kimi K3	2.8T	— 全球最大开源原生多模态MoE
GLM-5	744B	40B 三层MoE架构
Qwen3.5	397B	17B 原生多模态

面试考点：Qwen2.5-VL的核心架构特点，①视觉编码器采用类SigLIP的ViT，支持动态分辨率和视频帧率训练；②通过MLP将视觉特征投影到语言模型嵌入空间；③mRoPE（多模态旋转位置编码）扩展RoPE以处理图像空间位置和视频时序信息；④图像、视频、文本统一token化；⑤原生多模态预训练——从0开始联合训练而非先训LLM再接视觉模块。

四、架构演进的底层逻辑

从纯文本LLM到VLM再到MLLM，模型架构的演进呈现出清晰的脉络：

- 1. 从单一到融合。 纯文本→视觉语言→全模态，感知能力不断拓展。
- 2. 从独立到统一。 双塔编码→单塔融合→原生多模态。从2017年到2025年，各种基于Transformer的模型不断被开发，逐步应对日益复杂的任务。
- 3. 从理解到生成+理解。 侧重理解→理解与生成统一。
- 4. 从稠密到稀疏高效。 MoE、GQA等技术使模型在规模扩大的同时保持推理效率。
- 5. 从外挂到原生。 连接器/投影器范式→原生多模态预训练。
- 6. 从模块化到端到端。 组件拼凑→端到端统一架构。

五、通用高频考点速查表&手撕代码

5.1 通用高频考点速查表

考点分类	核心知识点	考察频率
Transformer基础	Self-Attention公式、多头注意力、除以 $\sqrt{d_k}$	★★★★★
架构选型	Encoder-Only vs Decoder-Only vs Encoder-Decoder	★★★★★

位置编码	绝对位置编码、RoPE原理与优势	★★★★★
注意力变体	MHA/MQA/GQA/MLA 的区别与演进动机	★★★★★
归一化	LayerNorm vs BatchNorm vs RMSNorm	★★★★
激活函数	ReLU→GELU→SwiGLU 的演进	★★★★
推理优化	KV Cache、FlashAttention、PagedAttention	★★★★★
MoE	稀疏激活原理、Router机制	★★★★
VLM架构	CLIP 双塔、BLIP-2 Q-Former、LLaVA投影器	★★★★★
MLLM	模态编码器+投影器+LLM三段式	★★★★
Scaling Laws	幂律关系、模型规模扩展	★★★
解码策略	Greedy、Beam Search、Top-K、Top-P	★★★★

5.2 高频手撕代码题

- 1. 手撕Self-Attention (PyTorch实现)
- 2. 手撕RoPE实现
- 3. 手撕多头注意力 (MHA)
- 4. 计算Transformer某层的参数量
- 5. 实现KV Cache的推理逻辑

六、结语

回顾模型架构的演进历程，可以清晰地看到一条从简到繁、从单一到融合、从外挂到原生的技术主线。

当前，原生多模态已成为最前沿的架构方向——Kimi K3（2.8万亿参数）、Qwen3.5（原生多模态）、Gemma 4等模型正在重新定义多模态智能的边界。同时，MoE与稠密模型的融合、SSM与Transformer的混合架构也在不断涌现。

对于开发者和研究者而言，理解这些架构差异至关重要：

- 如果追求极致的生成质量和生态支持，基于**Decoder-only**的模型（如Llama、Qwen）是首选
- 如果需要处理极长文档或在边缘设备部署，应关注**Mamba**或优化过的**GQA**模型
- 如果涉及多模态任务，**Qwen2.5-VL**、**InternVL3.5**等是开源权重自部署的强力候选
- 如果追求前沿的原生多模态能力，**Kimi K3**、**Qwen3.5**值得重点关注

LLM的底层设计正变得越来越模块化和精细化。模型的强弱，本质上是顶尖技术人员能力的体现。理解这些架构演进的脉络，不仅有助于把握技术发展方向，更能为实际模型选型和架构设计提供清晰的决策依据。

作者提示: 个人观点，仅供参考

大模型必知必会：一文带你了解大模型的组成部件

原创 QinHsiu 阿Q的AI笔记 2026年8月26日 07:45 北京

2017年Google发表《Attention is All You Need》时，Transformer还只是一篇8页的论文。9年后，它支撑起了GPT-5.6、Claude Fable 5、DeepSeek-V4等千亿参数巨兽。但很多开发者对它的理解仍停留在“注意力机制”四个字上。

这篇文章，我们从工程视角把大模型“拆开”——Tokenization、Position Embedding、Attention、Normalization、Activation、MoE、Loss Function，一层层拆解，附带代码实现和前沿演进。

一、Tokenizer

1.1 什么是Tokenizer?

Token是大模型处理文本的最小单元。Tokenizer负责将原始自然语言文本转换为模型能够理解的Token ID序列。

输入：“你好世界”

输出：[12345, 67890, 23456]（假设的Token ID）

为什么需要Tokenizer？神经网络只能处理数字，无法直接理解字符。需要一种无损、高效的方式将文本映射为数字序列。这与传统NLP中的分词任务有本质区别：传统分词追求“词的边界”（词法分析），Tokenizer追求“编码效率与语义保留的平衡”。

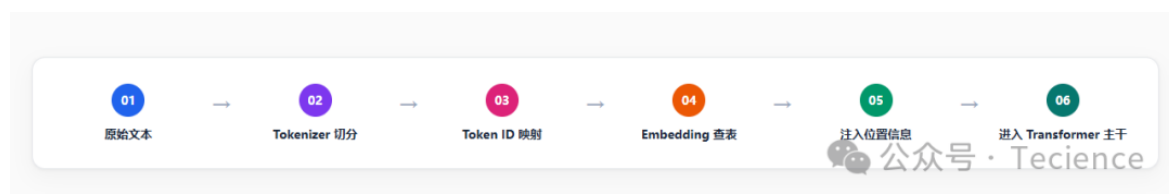
两种极端方案：

- **字符级：**把每个字符当Token。词表极小（英文26个字母），但序列极长（一个词占5-6个Token），上下文窗口很快被撑满。
- **词级：**把完整单词当Token。序列最短，但词表极大（英文常见词5万以上，无法覆盖所有形态变化），罕见词无法处理。

Tokenizer在这两者之间寻找最优解——将常见词合并为单个Token，罕见词拆分为子词。词表大小和序列长度共同决定了模型的**编码效率**（压缩率）。同样的文本，编码效率越高，在有限上下文窗口内能处理的信息就越多。

1.2 为什么Tokenizer如此重要？

从工程实现视角看，标准Decoder-only大模型的输入链路是：



Tokenization决定了三个关键指标：

- 1. **序列长度**：同一段文本，不同分词粒度影响Token总量，直接决定推理成本和训练吞吐
- 2. **多语言/代码能力**：BPE对英文友好，但对中文、代码符号的压缩效率差异巨大
- 3. **词表大小**：决定Embedding层和LM Head的参数规模（由词表大小和隐藏层维度决定， $\text{vocab_size} \times \text{hidden_dim}$ ），占模型总参数的10%-20%

一个被低估的事实：**Tokenizer的设计选择对模型能力的影响，可能比增加几层Transformer更大**。Qwen2.5拥有超过15万超大词表，编码中文的效率极高，同样文本编码出的Token数量通常比GPT-4o更少，这意味着在相同上下文内可以处理更多信息。

1.3 主流分词算法

主流子词分词算法对比如下：

算法	操作单元	构建方向	优势	劣势	代表模型
BPE	字符/符号	自底向上（合并高频 pair）	实现简单，训练速度快；序列长度可控，推理效率高	多语言支持差，易产生 OOV；词表易膨胀；无法处理生僻字和 Emoji	早期 GPT
BBPE	UTF-8 字节	自底向上（字节级合并）	无 OOV 问题，天然支持所有语言符号；词表紧凑，通用性强	序列长度增加（中文词量比 BPE 长约 30-50%）；推理加速略低	GPT 系列, Qwen, Llama
WordPiece	字符	自底向上（最大化熵减）	基于熵减的合并策略更优，比 BPE 更“聪明”；对训练数据适配性强	实现复杂度较高；需要全局排序计算，训练速度较慢	BERT 及其变体
Unigram	子词	自顶向下（递归最小熵减）	构造模型可解释性强；自顶向下策略灵活，词表更精简	初始词表选择敏感训练（通常先用 BPE 生成种子词表）；推理略慢	T5, XLNet, ALBERT

BPE (Byte Pair Encoding) ：通过统计频繁子串合并构建词表，GPT系列使用。核心逻辑：从字符级开始，迭代合并最高频的相邻符号对。

```
1 from collections import Counter, defaultdict
2
3 def get_stats(vocab):
4     """统计所有相邻token对的频次"""
5     pairs = defaultdict(int)
6     for word, freq in vocab.items():
7         symbols = word.split()
8         for i in range(len(symbols) - 1):
9             pairs[symbols[i], symbols[i+1]] += freq
10    return pairs
11
12 def merge_vocab(pair, vocab_in):
13     """合并最高频的pair"""
14     vocab_out = {}
15     bigram = ' '.join(pair)
16     replacement = ''.join(pair)
17     for word, freq in vocab_in.items():
18         new_word = word.replace(bigram, replacement)
19         vocab_out[new_word] = freq
20    return vocab_out
21
22 # 初始词表：每个词拆成字符
23 vocab = {"l o w": 5, "l o w e r": 2, "n e w": 6, "w i d e r": 3}
24 num_merges = 10
25
```

```

26 for i in range(num_merges):
27     pairs = get_stats(vocab)
28     if not pairs:
29         break
30     best = max(pairs, key=pairs.get)
31     vocab = merge_vocab(best, vocab)
32     print(f"合并 {best}: {vocab}")

```

字节级BPE (Byte-Level BPE, BBPE)

标准BPE在字符级操作，需要预定义字符集。面对多语言混合、生僻字或Emoji时，容易出现未登录词（OOV）问题。例如，标准BPE的词表中必须包含“山”“东”“大”“学”四个字符才能组成“山东大学”；若遇到训练语料中从未出现的生僻字，则无法表示。

BBPE的根本改进在于**操作单元从字符下沉到字节**。它先将文本统一转为UTF-8编码的字节流（共256种基础字节），再对字节序列执行标准BPE合并算法。由于任意字符（包括生僻汉字、Emoji、特殊符号）都可以由1-4个字节表示，BBPE**彻底消除了OOV问题**，天然支持所有语言和符号。

```

1  # BBPE 的核心逻辑：在标准BPE 前增加UTF-8 编码转换层
2  def bytes_to_unicode():
3      """将每个UTF-8字节映射为一个可打印的Unicode字符"""
4      # 256 个字节全部保留，确保无损
5      bs = list(range(ord("!"), ord("~")+1)) + list(range(ord("¡"), ord("¬")+1))
6      cs = bs[:] # 可打印字符直接映射
7      n = 0
8      for b in range(2**8):
9          if b not in bs:
10             bs.append(b)
11             cs.append(2**8 + n)
12             n += 1
13     return dict(zip(bs, cs))
14
15 # 编码流程：文本 → UTF-8 字节 → 映射为Unicode → 执行标准BPE
16 def bbpe_encode(text, bpe_merges):
17     # 1. 文本转UTF-8 字节
18     byte_array = text.encode('utf-8')
19     # 2. 字节映射为Unicode 字符（标准BPE 的词表空间）
20     mapped = [bytes_to_unicode[b] for b in byte_array]
21     # 3. 执行标准BPE 合并
22     return bpe_encode(mapped, bpe_merges)

```

WordPiece

BERT使用的分词算法。与BPE不同，WordPiece选择能**最大化训练数据似然**的子词进行合并，而非简单选频次最高的对。

```

1 import math
2 from collections import Counter, defaultdict
3
4 class WordPieceTrainer:
5     def __init__(self, vocab_size=100):
6         self.vocab_size = vocab_size
7
8     def compute_score(self, pair, pair_freq, token_freq):
9         """WordPiece核心: 用PMI (点互信息) 作为合并分数"""
10        # score = freq(pair) / (freq(first) * freq(second))[reference:9]
11        return pair_freq / (token_freq[pair[0]] * token_freq[pair[1]])
12
13    def train(self, corpus):
14        # 初始词表: 所有字符
15        vocab = set()
16        word_freq = Counter()
17        for text in corpus:
18            chars = list(text)
19            word_freq[tuple(chars)] += 1
20            vocab.update(chars)
21
22        # 迭代合并
23        while len(vocab) < self.vocab_size:
24            # 统计相邻pair频次
25            pair_freq = defaultdict(int)
26            token_freq = defaultdict(int)
27            for word, freq in word_freq.items():
28                for i in range(len(word) - 1):
29                    pair_freq[(word[i], word[i+1])] += freq
30                    token_freq[word[i]] += freq
31                if len(word) > 0:
32                    token_freq[word[-1]] += freq
33
34            if not pair_freq:
35                break
36
37            # 选PMI最高的pair[reference:10]
38            best_pair = max(pair_freq.keys(),
39                            key=lambda p: self.compute_score(p, pair_freq[p], t
40
41            # 合并
42            new_token = ''.join(best_pair)
43            vocab.add(new_token)
44            new_word_freq = {}
45            for word, freq in word_freq.items():
46                new_word = []

```

```

47         i = 0
48         while i < len(word):
49             if i < len(word) - 1 and word[i] == best_pair[0] and word[i+1] == best_pair[1]:
50                 new_word.append(new_token)
51                 i += 2
52             else:
53                 new_word.append(word[i])
54                 i += 1
55             new_word_freq[tuple(new_word)] += freq
56         word_freq = new_word_freq
57
58     return vocab

```

分词阶段使用**贪心最长匹配优先算法**——从头开始，每次尽可能匹配最长的子词。

Unigram

T5、XLNet等模型使用。与BPE/WordPiece相反：从大词汇表开始，逐步删除标记，直到达到目标词表大小。

```

1  import math
2  from collections import defaultdict, Counter
3
4  class UnigramTrainer:
5      def __init__(self, vocab_size=100):
6          self.vocab_size = vocab_size
7
8      def compute_loss(self, word, vocab, token_probs):
9          """用Viterbi找最优分词，计算负对数似然[reference:14]"""
10         n = len(word)
11         dp = [float('inf')] * (n + 1)
12         dp[0] = 0
13         split = [0] * (n + 1)
14
15         for i in range(n):
16             for j in range(i + 1, n + 1):
17                 token = word[i:j]
18                 if token in token_probs:
19                     score = dp[i] - math.log(token_probs[token])
20                     if score < dp[j]:
21                         dp[j] = score
22                         split[j] = i
23
24         # 回溯得到最优分词

```

```

25     tokens = []
26     j = n
27     while j > 0:
28         i = split[j]
29         tokens.append(word[i:j])
30         j = i
31     return dp[n], tokens[::-1]
32
33 def train(self, corpus):
34     # 初始词汇: 所有子串 (或BPE预训练结果) [reference:15]
35     vocab = set()
36     word_freq = Counter(corpus)
37     for word in word_freq:
38         for i in range(len(word)):
39             for j in range(i + 1, min(len(word), i + 10) + 1):
40                 vocab.add(word[i:j])
41     vocab = list(vocab)
42
43     # 估计token概率
44     total_freq = sum(word_freq.values())
45     token_probs = {}
46     for token in vocab:
47         token_probs[token] = sum(word_freq.get(w, 0) for w in word_freq)
48
49     # 迭代删除: 每次删除对损失影响最小的token [reference:16]
50     while len(vocab) > self.vocab_size:
51         loss_without = {}
52         for token in vocab:
53             if len(token) == 1: # 单字符不删 [reference:17]
54                 continue
55             temp_vocab = [t for t in vocab if t != token]
56             temp_probs = {t: token_probs[t] for t in temp_vocab}
57             total_loss = 0
58             for word, freq in word_freq.items():
59                 loss, _ = self.compute_loss(word, temp_vocab, temp_probs)
60                 total_loss += freq * loss
61             loss_without[token] = total_loss
62
63         if not loss_without:
64             break
65         # 删掉损失增加最少的token
66         to_remove = min(loss_without, key=loss_without.get)
67         vocab.remove(to_remove)
68         del token_probs[to_remove]
69
70

```

```
return set(vocab), token_probs
```

```
1 # SentencePiece 也支持Unigram[reference:18][reference:19]
2 import sentencepiece as spm
3
4 spm.SentencePieceTrainer.train(
5     input='data.txt',
6     model_prefix='unigram_model',
7     vocab_size=8000,
8     model_type='unigram' # 或 'bpe'
9 )
```

1.4 词表大小设计的工程权衡

词表大小与模型性能呈倒U型关系：词表太小则序列过长，计算成本高；词表太大则 Embedding/LM Head 参数爆炸，且罕见token训练不充分。

DeepSeek-V4词表约**128K**，Qwen系列约**152K**。中文场景通常需要更大的词表——常用汉字约1万，英文仅26字母。

二、Position Embedding

2.1 为什么需要位置编码？

Self-Attention是置换不变的——打乱输入顺序，输出的注意力权重不变。但自然语言中“猫追老鼠”和“老鼠追猫”意思完全不同。必须给模型注入位置信息。

早期方案（Transformer原论文）：**绝对位置编码**——将sin/cos函数生成的固定位置向量与词向量相加。

```
1 import torch
2 import math
3
4 def sinusoidal_position_encoding(seq_len, d_model):
5     """
6     生成Transformer原论文的正弦位置编码
7     seq_len: 序列长度
8     d_model: 嵌入维度
9     返回: [seq_len, d_model]
10    """
11    pe = torch.zeros(seq_len, d_model)
12    position = torch.arange(0, seq_len, dtype=torch.float).unsqueeze(1) # [s
13
14    # 计算频率项: 1 / 10000^(2i/d_model)
```

```

15     div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0 / (2 * math.pi))))
16
17     # 偶数维度用sin, 奇数维度用cos
18     pe[:, 0::2] = torch.sin(position * div_term)
19     pe[:, 1::2] = torch.cos(position * div_term)
20
21     return pe
22
23 # 示例: 生成10个位置、512维的位置编码
24 pos_encoding = sinusoidal_position_encoding(10, 512)
25 print(f"位置编码形状: {pos_encoding.shape}")

```

2.2 RoPE: 旋转位置编码

RoPE (Rotary Position Embedding) 是目前大模型的绝对主流, Qwen2.5、Llama、DeepSeek全系列采用。核心思想: **用旋转矩阵在复数域操作, 将位置信息直接编码进Q和K, 而不是与词向量相加。**

数学本质: 对二维向量, RoPE的旋转矩阵为:

$$R(\theta_m) = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}$$

对高维向量, 拆分为多个二维子空间分别旋转, 每个子空间使用不同的频率基:

$$\theta_i = 10000^{-2i/d}$$

核心优势:

1. **远程衰减性:** 内积随相对距离增大呈震荡衰减, 符合语言中邻近词关联更强的直觉
2. **外推能力:** 旋转的周期性允许处理超过训练长度的序列
3. **保留相对位置信息:** 两个Token点积的结果只取决于它们的相对位置, 而非绝对位置

RoPE的代码实现:

```

1 import torch
2 import math
3
4 def precompute_freqs_cis(dim: int, seq_len: int, theta: float = 10000.0):
5     """预计算RoPE的旋转频率"""
6     # 计算每个维度的频率基
7     freqs = 1.0 / (theta ** (torch.arange(0, dim, 2)[: (dim // 2)].float() /
8     # 为每个位置生成旋转角度

```

```

9     t = torch.arange(seq_len, device=freqs.device)
10    freqs = torch.outer(t, freqs).float() # [seq_len, dim/2]
11    freqs_cis = torch.polar(torch.ones_like(freqs), freqs) # 复数形式
12    return freqs_cis
13
14 def apply_rope(xq: torch.Tensor, xk: torch.Tensor, freqs_cis: torch.Tensor):
15     """对Q和K应用旋转位置编码"""
16     # xq, xk: [batch, seq_len, n_heads, head_dim]
17     # 将最后一维拆分为两个子空间
18     xq_ = xq.float().reshape(*xq.shape[:-1], -1, 2)
19     xk_ = xk.float().reshape(*xk.shape[:-1], -1, 2)
20
21     # 转换为复数: (x, y) -> x + yi
22     xq_complex = torch.view_as_complex(xq_)
23     xk_complex = torch.view_as_complex(xk_)
24
25     # 应用旋转: 复数乘法
26     freqs_cis = freqs_cis.unsqueeze(0).unsqueeze(2) # [1, seq_len, 1, dim/2]
27     xq_out = torch.view_as_real(xq_complex * freqs_cis).flatten(3)
28     xk_out = torch.view_as_real(xk_complex * freqs_cis).flatten(3)
29     return xq_out.type_as(xq), xk_out.type_as(xk)

```

2.3 RoPE的延伸：上下文窗口扩展

训练时固定上下文长度后，如何在推理时扩展到更长序列？核心挑战是RoPE的外推能力存在上限。

YaRN (Yet another RoPE extension)：通过调整RoPE的旋转频率基（从10000放大到更大的值），使模型在更长距离上仍能保持合理的注意力分布。Qwen2.5和Llama 3.1都使用了YaRN扩展技术。

Position Interpolation：将位置索引线性压缩到训练长度范围内，损失一定分辨率但保持分布稳定。

NTK-aware Scaling：对不同频段采用不同的缩放系数——高频分量（短距离关系）尽量保持不变，低频分量（长距离关系）进行扩展。

三、Attention

标准Self-Attention通过Q、K、V三者之间的关系建模序列依赖：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k}\right) \cdot V$$

核心逻辑分三步：计算Q和K的相关性 → softmax归一化得到权重 → 加权求和V得到输出。

vd_k 缩放因子至关重要。当 d_k 较大时，点积方差增大，softmax易进入梯度饱和区。实际工程中，忽略此缩放会导致训练初期注意力分布接近均匀分布，收敛速度下降40%。

Causal Mask: Decoder-only模型的关键设计——每个位置只能看到自己和之前的位置，不能“偷看未来”。在计算注意力分数后，将上三角区域设为 $-\text{inf}$ 实现。

```
1 import torch
2 import torch.nn.functional as F
3
4 def scaled_dot_product_attention(Q, K, V, mask=None):
5     """标准缩放点积注意力"""
6     d_k = K.shape[-1]
7     scores = torch.matmul(Q, K.transpose(-2, -1)) / torch.sqrt(torch.tensor(d_k))
8     if mask is not None:
9         scores = scores.masked_fill(mask == 0, -1e9)
10    attn_weights = F.softmax(scores, dim=-1)
11    return torch.matmul(attn_weights, V)
```

3.2 MHA → GQA → MLA: 注意力机制的演进

MHA (Multi-Head Attention) : 原始设计。每层 h 个头，每个头有独立的Q、K、V投影。参数量大，KV Cache随头数线性增长。

现代多头注意力的实现：将Q、K、V拆分为多个子空间（头），并行计算多个自注意力头，再将结果拼接融合，使模型能同时关注不同位置的多种语义信息。

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, embed_size, heads):
3         super().__init__()
4         self.heads = heads
5         self.head_dim = embed_size // heads
6
7         self.q_linear = nn.Linear(embed_size, embed_size)
8         self.k_linear = nn.Linear(embed_size, embed_size)
9         self.v_linear = nn.Linear(embed_size, embed_size)
10        self.fc_out = nn.Linear(embed_size, embed_size)
11
12    def forward(self, x):
13        batch_size = x.shape[0]
14        Q = self.q_linear(x)
15        K = self.k_linear(x)
16        V = self.v_linear(x)
17
18        # 分头
19        Q = Q.reshape(batch_size, -1, self.heads, self.head_dim)
```

```

20     K = K.reshape(batch_size, -1, self.heads, self.head_dim)
21     V = V.reshape(batch_size, -1, self.heads, self.head_dim)
22
23     scores = torch.einsum("nqhd,nkhd->nhqk", Q, K)
24     attention = torch.softmax(scores / (self.head_dim ** 0.5), dim=-1)
25     out = torch.einsum("nhq1,nlhd->nqhd", attention, V)
26     out = out.reshape(batch_size, -1, self.embed_size)
27     return self.fc_out(out)

```

GQA (Grouped-Query Attention)：将多头分为组，组内共享K、V投影，大幅减少KV Cache。Llama 3使用GQA，推理速度提升约2倍。现代Claude模型采用64个Q头、8个KV头的GQA配置，在保证表达能力的同时降低显存占用。

```

1  class GroupedQueryAttention(nn.Module):
2      def __init__(self, d_model, n_heads, n_kv_heads):
3          super().__init__()
4          self.n_heads = n_heads
5          self.n_kv_heads = n_kv_heads
6          self.head_dim = d_model // n_heads
7
8          self.q_proj = nn.Linear(d_model, n_heads * self.head_dim)
9          self.k_proj = nn.Linear(d_model, n_kv_heads * self.head_dim)
10         self.v_proj = nn.Linear(d_model, n_kv_heads * self.head_dim)
11         self.o_proj = nn.Linear(n_heads * self.head_dim, d_model)
12
13     def forward(self, x):
14         B, T, D = x.shape
15         q = self.q_proj(x).view(B, T, self.n_heads, self.head_dim)
16         k = self.k_proj(x).view(B, T, self.n_kv_heads, self.head_dim)
17         v = self.v_proj(x).view(B, T, self.n_kv_heads, self.head_dim)
18
19         # GQA 核心: 重复KV heads到与Q heads数量一致
20         k = k.repeat_interleave(self.n_heads // self.n_kv_heads, dim=2)
21         v = v.repeat_interleave(self.n_heads // self.n_kv_heads, dim=2)
22
23         out = F.scaled_dot_product_attention(q, k, v)
24         return self.o_proj(out.view(B, T, -1))

```

MLA (Multi-head Latent Attention)：DeepSeek-V2/V3/V4的核心创新。通过**低秩压缩**将KV Cache压缩至极低水平。原理：不是存储完整的K、V矩阵，而是存储一个低维的潜在向量，使用时再解压。

DeepSeek-V4中，MLA配合c4a和c128a两级压缩，100万Token上下文仅需**9.62GiB** KV Cache，是V3.2的**1/8.7**。这让百万级上下文从“实验室玩具”变成了“生产可用”。

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5
6 class MLAConfig:
7     """MLA配置"""
8     def __init__(
9         self,
10         d_model: int = 512,          # 模型维度
11         num_heads: int = 8,          # 注意力头数
12         d_c: int = 64,               # KV压缩维度 (关键: 远小于 num_heads * he
13         d_c1: int = 64,              # Query压缩维度
14         d_rotate: int = 32,          # RoPE 维度
15         seq_len: int = 256,
16     ):
17         self.d_model = d_model
18         self.num_heads = num_heads
19         self.head_dim = d_model // num_heads
20         self.d_c = d_c
21         self.d_c1 = d_c1
22         self.d_rotate = d_rotate
23         self.seq_len = seq_len
24
25
26 class MultiHeadLatentAttention(nn.Module):
27     """
28     Multi-Head Latent Attention (MLA)
29     来自 DeepSeek-V2/V3 论文
30     """
31
32     def __init__(self, config: MLAConfig):
33         super().__init__()
34         self.config = config
35         self.num_heads = config.num_heads
36         self.head_dim = config.head_dim
37         self.d_c = config.d_c          # KV压缩维度
38         self.d_c1 = config.d_c1        # Q压缩维度
39         self.d_rotate = config.d_rotate
40
41         # ===== 低秩压缩投影 =====
42         # Q: d_model → d_c1 (下投影)
43         self.w_dq = nn.Linear(config.d_model, config.d_c1, bias=False)
44         # Q: d_c1 → num_heads * head_dim (上投影)
45         self.w_uq = nn.Linear(config.d_c1, config.num_heads * config.head_dim
46

```

```

47     # KV:  $d_{model} \rightarrow d_c$  (下投影, 共享)
48     self.w_dkv = nn.Linear(config.d_model, config.d_c, bias=False)
49     # K:  $d_c \rightarrow num\_heads * head\_dim$  (上投影)
50     self.w_uk = nn.Linear(config.d_c, config.num_heads * config.head_dim)
51     # V:  $d_c \rightarrow num\_heads * head\_dim$  (上投影)
52     self.w_uv = nn.Linear(config.d_c, config.num_heads * config.head_dim)
53
54     # ===== 解耦RoPE =====
55     # 为RoPE单独保留的维度
56     self.w_dr = nn.Linear(config.d_model, config.d_rotate, bias=False)
57     self.w_ur = nn.Linear(config.d_rotate, config.num_heads * config.d_rotate)
58
59     # ===== 输出投影 =====
60     self.wo = nn.Linear(config.num_heads * config.head_dim, config.d_model)
61
62     # ===== 预计算RoPE频率 =====
63     self._init_rope()
64
65     def _init_rope(self):
66         """预计算RoPE的sin/cos缓存"""
67         inv_freq = 1.0 / (10000 ** (torch.arange(0, self.d_rotate, 2).float() / self.d_rotate))
68         self.register_buffer("inv_freq", inv_freq)
69
70     def _apply_rope(self, x: torch.Tensor, start_pos: int = 0):
71         """对旋转维度应用RoPE"""
72         # x: [batch, seq_len, num_heads, d_rotate]
73         seq_len = x.shape[1]
74         t = torch.arange(start_pos, start_pos + seq_len, device=x.device).float()
75         freqs = torch.einsum("i,j->ij", t, self.inv_freq) # [seq_len, d_rotate]
76         emb = torch.cat((freqs, freqs), dim=-1) # [seq_len, d_rotate]
77         cos, sin = emb.cos(), emb.sin() # [seq_len, d_rotate]
78
79         # 复数旋转: 将x视为复数 ( $x_1 + i*x_2$ ), 乘以  $e^{(i*\theta)}$ 
80         x1, x2 = x.chunk(2, dim=-1)
81         rotated = torch.cat((
82             x1 * cos - x2 * sin,
83             x1 * sin + x2 * cos
84         ), dim=-1)
85         return rotated
86
87     def forward(
88         self,
89         x: torch.Tensor, # [batch, seq_len, d_model]
90         use_cache: bool = False,
91         past_kv_cache: torch.Tensor = None, # [batch, max_seq_len, d_c]
92         past_r_cache: torch.Tensor = None, # [batch, max_seq_len, d_rotate]

```

```

93         start_pos: int = 0,
94     ):
95         batch, seq_len, _ = x.shape
96
97         # ===== Step 1: Q的低秩压缩 =====
98         # d_model → d_c1 (下投影)
99         c_q = self.w_dq(x) # [batch, seq_len, d_c1]
100        # d_c1 → num_heads * head_dim (上投影)
101        q = self.w_uq(c_q) # [batch, seq_len, num_heads * head_dim]
102        q = q.view(batch, seq_len, self.num_heads, self.head_dim)
103
104        # ===== Step 2: KV的低秩压缩 =====
105        # d_model → d_c (下投影, Q和KV共享类似的低秩结构)
106        c_kv = self.w_dkv(x) # [batch, seq_len, d_c]
107
108        # 从压缩态还原K和V
109        k = self.w_uk(c_kv).view(batch, seq_len, self.num_heads, self.head_dim)
110        v = self.w_uv(c_kv).view(batch, seq_len, self.num_heads, self.head_dim)
111
112        # ===== Step 3: 解耦RoPE =====
113        # 分离出旋转维度
114        r = self.w_dr(x) # [batch, seq_len, d_rotate]
115        r = self.w_ur(r).view(batch, seq_len, self.num_heads, self.d_rotate)
116        r = self._apply_rope(r, start_pos)
117
118        # K的旋转部分: 只对部分维度应用RoPE
119        # 将k分为两部分: [head_dim - d_rotate] + [d_rotate]
120        k_non_rope = k[..., :self.head_dim - self.d_rotate]
121        k_rope = k[..., self.head_dim - self.d_rotate:]
122        k_rope = self._apply_rope(k_rope, start_pos)
123        k = torch.cat([k_non_rope, k_rope], dim=-1)
124
125        # Q同样处理
126        q_non_rope = q[..., :self.head_dim - self.d_rotate]
127        q_rope = q[..., self.head_dim - self.d_rotate:]
128        q_rope = self._apply_rope(q_rope, start_pos)
129        q = torch.cat([q_non_rope, q_rope], dim=-1)
130
131        # ===== Step 4: KV Cache管理 =====
132        if use_cache:
133            # 缓存压缩态的KV, 而非完整K、V
134            # cache_kv: [batch, max_seq_len, d_c]
135            if past_kv_cache is not None:
136                c_kv = torch.cat([past_kv_cache, c_kv], dim=1)
137            # 缓存旋转态的K (用于后续位置计算)
138            if past_r_cache is not None:

```

```

139         r = torch.cat([past_r_cache, r], dim=1)
140
141     # ===== Step 5: 注意力计算 =====
142     # 标准Scaled Dot-Product Attention
143     scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_
144
145     # Causal Mask
146     mask = torch.triu(torch.ones(seq_len, seq_len, device=x.device), di
147     scores = scores.masked_fill(mask, -float('inf'))
148
149     attn_weights = F.softmax(scores, dim=-1)
150     out = torch.matmul(attn_weights, v) # [batch, seq_len, num_heads, h
151
152     # ===== Step 6: 输出投影 =====
153     out = out.reshape(batch, seq_len, -1)
154     out = self.wo(out)
155
156     if use_cache:
157         return out, c_kv, r
158     return out

```

3.3 复杂度与工程优化

标准Self-Attention的时间复杂度为 $O(L^2 \cdot d)$ ，空间复杂度 $O(L^2)$ 。序列长度1万时，注意力权重矩阵占约400MB内存。

Flash Attention：通过分块计算避免完整注意力矩阵的显存分配，将 $O(L^2)$ 空间复杂度降为 $O(L)$ 。同时利用GPU的SRAM加速I/O，在H100上实现2-4倍加速。当前已演进到Flash Attention 3，支持FP8数据类型。

```

1  def flash_attention_block(Q_block, K_blocks, V_blocks, block_size):
2      """
3      单块计算：在SRAM内完成
4      核心：不存储完整的注意力矩阵
5      """
6      # Q_block: [batch, heads, block_size, head_dim]
7      # K_blocks, V_blocks: 多个 [batch, heads, block_size, head_dim] 块
8
9      # 初始化Online Softmax状态
10     m = torch.full((Q_block.shape[0], Q_block.shape[1], block_size), -float('
11     l = torch.zeros_like(m) # 分母累加
12     o = torch.zeros_like(Q_block) # 输出累加
13
14     for K_block, V_block in zip(K_blocks, V_blocks):
15         # 当前块的注意力分数

```

```

16         S_block = torch.matmul(Q_block, K_block.transpose(-2, -1)) / math.sqr
17
18         # Online Softmax更新
19         m_new = torch.max(m, S_block.max(dim=-1, keepdim=True).values)
20         # 核心公式：利用旧归一化因子更新输出
21         # 完整推导见论文 Section 3.2
22         l = 1 * torch.exp(m - m_new) + torch.exp(S_block - m_new).sum(dim=-1)
23         O = O * torch.exp(m - m_new) + torch.matmul(
24             torch.exp(S_block - m_new), V_block
25         )
26         m = m_new
27
28     return O / l.unsqueeze(-1)

```

3.4 MLA与FlashAttention的协同

DeepSeek官方开源了**FlashMLA**——针对MLA优化的高效解码内核。在H100上，FlashMLA在MQA模式下可达**3000 GB/s**内存带宽、**580 TFLOPS**计算性能。使用时在解码循环前调用 `get_mla_metadata` 一次获取tile调度元数据。

```

1  # FlashMLA 使用示例（伪代码）
2  from flash_mla import get_mla_metadata, flash_mla_with_kvcache
3
4  # 解码循环前：获取调度元数据
5  tile_scheduler_metadata = get_mla_metadata(cache_seqlens, num_heads, head_dim)
6
7  # 每步解码
8  for _ in range(max_new_tokens):
9      out = flash_mla_with_kvcache(
10          q, k_cache, v_cache, tile_scheduler_metadata,
11          cache_seqlens, rotary_emb
12      )

```

MLA + FlashMLA的组合，让DeepSeek-V4在保持671B总参数的同时，实现了百万级上下文窗口的生产级部署。

四、Normalization

4.1 Pre-Norm vs Post-Norm

原始Transformer采用**Post-Norm**（Attention/FFN之后做Norm），深层训练时易梯度爆炸。现代LLM全面转向**Pre-Norm**（Attention/FFN之前做Norm）。

```

1 # Pre-Norm结构 (标准Decoder-only Block)
2 class PreNormBlock(nn.Module):
3     def forward(self, x):
4         # 先归一化, 再进Attention, 残差连接
5         h = x + self.attn(self.norm1(x))
6         out = h + self.ffn(self.norm2(h))
7         return out

```

为什么Pre-Norm胜出？ 残差路径保持恒等映射，梯度可以直接流过，训练更稳定。Post-Norm残差路径经过多次归一化，深层时梯度易消失。

4.2 RMSNorm vs LayerNorm

LayerNorm：沿特征维度归一化，计算均值和方差。

RMSNorm：只计算均方根（RMS），不计算均值，计算量减少约30%，且在某些任务上表现更好。

```

1 class RMSNorm(nn.Module):
2     def __init__(self, dim, eps=1e-6):
3         super().__init__()
4         self.weight = nn.Parameter(torch.ones(dim))
5         self.eps = eps
6
7     def forward(self, x):
8         rms = torch.sqrt(torch.mean(x ** 2, dim=-1, keepdim=True) + self.eps)
9         return x / rms * self.weight

```

Phi-3、Qwen、Llama等模型全部使用RMSNorm。节省的算力虽小，但乘以数十亿Token和数千GPU就是巨额收益。

五、Activation Function：非线性变换的战场

5.1 从ReLU到SwiGLU

原始Transformer使用ReLU。现代LLM几乎全部转向**SwiGLU (Swish-Gated Linear Unit)**。

SwiGLU = Swish(gate) × up_proj：将FFN拆为门控投影和上投影，中间加入Swish激活。

```

1 class SwiGLU(nn.Module):
2     def __init__(self, d_model, d_ff):
3         super().__init__()
4         self.gate_proj = nn.Linear(d_model, d_ff, bias=False)

```

```

5         self.up_proj = nn.Linear(d_model, d_ff, bias=False)
6         self.down_proj = nn.Linear(d_ff, d_model, bias=False)
7
8     def forward(self, x):
9         # SwiGLU: F.silu(gate) × up
10        gate = F.silu(self.gate_proj(x))
11        up = self.up_proj(x)
12        return self.down_proj(gate * up)

```

关键设计：**3个投影矩阵** (gate、up、down)，而非标准FFN的2个。虽然参数增加50%，但性能提升显著。Phi-3 4B的d_model=3072, d_ff=16384，FFN层参数占总参数的一半以上。

5.2 为什么SwiGLU更好？

1. **门控机制**：动态控制信息流，让模型决定哪些特征应该通过
2. **平滑性**：Swish在负值区域有非零梯度，缓解神经元死亡问题
3. **表达能力**：相比ReLU，SwiGLU能学习更复杂的函数映射

前沿探索：2025-2026年，一些模型开始尝试用更激进的激活函数替代SwiGLU，但没有形成统一结论。

六、MoE：从Dense到Sparse的架构跃迁

6.1 MoE的核心思想

MoE (Mixture of Experts) 用**多个专家型FFN块**替换单一的FFN，通过路由让每个Token只激活一小部分专家。

核心公式：

$$\text{MoE}(x) = \sum_{i=1}^E G(x)_i \cdot \text{Expert}_i(x)$$

- **E**：专家总数
- **G(x)**：路由权重，通常只激活Top-K个专家（如Top-2）
- **Expert_i**：第i个专家的FFN

2026年架构分类学将MoE列为“二次注意力核”集群的核心变体，其价值在于成功解耦了**总参数规模**与**激活参数**之间的关系。

```

1 # MoE路由简化实现
2 class MoERouter(nn.Module):
3     def __init__(self, d_model, num_experts, top_k=2):
4         super().__init__()
5         self.num_experts = num_experts

```

```

6         self.top_k = top_k
7         self.router = nn.Linear(d_model, num_experts)
8
9     def forward(self, x):
10         # 计算路由logits
11         router_logits = self.router(x) # [B, T, num_experts]
12         router_probs = F.softmax(router_logits, dim=-1)
13
14         # 选Top-K专家
15         top_k_probs, top_k_indices = torch.topk(router_probs, self.top_k, dim=-1)
16         # 归一化Top-K权重
17         top_k_probs = top_k_probs / top_k_probs.sum(dim=-1, keepdim=True)
18         return top_k_indices, top_k_probs

```

6.2 为什么MoE是400B+模型的标配？

总参数 vs 激活参数：DeepSeek-V3总参数671B，但每个Token只激活37B。模型拥有巨大知识容量，推理成本仅相当于37B Dense模型。DeepSeek-R1同样采用此架构，总参数达671B，激活约37B。

2026年的MoE演进方向包括：**专家粒度精细化**（专家数量、激活数量、路由策略成为精细的设计维度）、**共享专家机制**（除路由选择的专家外，始终激活一个“共享专家”处理通用知识）、**Token Dropless MoE**（Megatron Core等框架实现动态专家分配，从根本上解决Token被丢弃的问题）。

6.3 2026年MoE的新变体：EMO

AI2提出的**EMO (Emergent Modularity MoE)** 是一个重要创新：在文档级别进行路由，而非逐Token路由。整个文档共享同一组语义专家（如编码、医疗等专门领域），进一步优化了推理经济性。

6.4 当前MoE的工程挑战

负载均衡：某些专家可能被过度路由，导致训练效率下降。需要辅助损失约束路由分布。

通信开销：MoE的专家分布在多GPU上，All-to-All通信成为瓶颈。DeepSeek-V4使用FP4 MoE融合内核优化。

七、Loss Function：训练的方向盘

7.1 预训练Loss：Next Token Prediction

所有自回归LLM使用相同的预训练损失函数：

逐Token交叉熵损失，目标是最大化正确Token的对数概率。预训练阶段只使用这一个Loss，持续数十亿Token。

7.2 SFT Loss：监督微调

监督微调使用与预训练相同的交叉熵损失，但只在**回答部分**计算Loss，不计算Prompt部分的Loss：

防止模型在Prompt上做无意义的拟合，专注提升回答质量。

7.3 RLHF/DPO/GRPO：偏好对齐

RLHF通过人类反馈优化模型，但需要训练奖励模型，工程复杂。

DPO (Direct Preference Optimization)：直接优化偏好数据，绕过奖励模型，成为当前最主流的对齐方法。

GRPO (Group Relative Policy Optimization)：DeepSeek提出的优化变体，通过组内相对比较替代绝对奖励，训练更稳定。

Loss设计的关键洞察：损失函数决定了模型优化的方向。预训练追求“下一个Token正确”，SFT追求“回答正确”，RL追求“用户偏好”。三者缺一不可。

八、2026年架构变革的前沿

8.1 混合线性注意力：挑战Full Attention霸权

2026年GTC大会上，Kimi创始人杨植麟首次系统披露了**Kimi K2.5的技术路线**。其核心指向“更有效率的规模化”——不再依赖单纯资源堆砌，而是从结构效率中寻找新的规模红利。

针对2017年诞生的Full Attention，Kimi展示了基于**KDA架构的Kimi Linear**——一种混合线性注意力架构，通过优化递归存储管理，在128K甚至1M超长上下文中，将解码速度提升**5到6倍**。

与此呼应，阿里RTP团队推出的**RTPurboV2**通过分析发现：Full Attention模型在预训练中已自发形成高度稀疏的注意力结构——约**15%的Head表现为“召回头” (retrieval head)** 负责长距离信息召回，其余**85%则是“流式头” (streaming head)** 更多关注局部上下文。这种分工模式在不同输入、不同序列长度下高度稳定。

直接推论：**85%的Full Attention计算可以安全地替换为滑动窗口注意力（SWA），几乎不影响模型能力。** RTPurboV2在V1架构基础上进一步在Full Attention部分实现**16~32倍计算压缩**。

8.2 混合稀疏注意力：小米HySparse

小米MiMo团队推出的**HySparse**是一种面向Agent时代的混合稀疏注意力架构。其核心创新：在总共49层的80B-A3B MoE模型中，**仅保留5层Full Attention**，其余层采用Sparse Attention并直接复用Full Attention层产生的重要token索引和KV Cache。

这一设计同时解决了Sparse Attention的两大根本问题：

- **选择不再依赖proxy**：Full Attention已经算出了最准确的token重要性
- **稀疏层不引入额外KV Cache开销**：直接复用前置Full Attention的KV Cache

实验表明，HySparse在保持甚至提升模型能力的同时，**显著降低KV Cache存储与计算开销**。

8.3 非均匀层宽度：Variable-Width Transformer

2026年的一项研究挑战了“所有层宽度一致”的行业惯例。提出呈**“X”形（“><”形）的Transformer架构**——早期层和末期层更宽，中间层相对变窄。

在2亿到20亿参数的模型上，这种非均匀分配始终优于同参数量均匀基线：

- **22% FLOPs节省**
- **15% KV Cache内存/I/O降低**

进一步分析表明，这种“瓶颈式”结构会导致残差流中的表征产生可解释性变化——中间层激活稀疏性增强、语义解耦更明显。

8.4 混合架构：Attention+SSM+MoE的三重奏

2026年架构分类学将模型家族划分为三大集群：

Cluster A（二次注意力核）：Dense Transformer、MoE、EMO——以DeepSeek-V3为代表，在长序列上承受 $O(n^2)$ 复杂度但上下文理解最优。

Cluster B（线性与循环Transformer）：SSM/Mamba、Hybrid Attention-SSM、RWKV、MoE-SSM——以Jamba、BlackMamba为代表，线性复杂度、恒定内存、理论上无限上下文。

Cluster C（替代表征原理）：Hyperdimensional Computing、Liquid Neural Networks、KAN——在能源效率和可解释性上有独特优势，但文本处理精度仍有限。

这些混合架构（如Jamba在Attention和SSM层之间交替）正在成为长序列场景的标准设计。

大模型必知必会：一文带你了解世界模型

原创 QinHsiu 阿Q的AI笔记 2026年8月29日 07:45 北京

2026年，大模型领域最受关注的赛道，不再是聊天机器人，而是**世界模型**。

2026年上半年，风险投资机构向世界模型初创公司投入超过**30亿美元**。Yann LeCun离开Meta后创立的AMI以**10.3亿美元**种子轮融资刷新欧洲历史记录，估值达35亿美元。李飞飞的World Labs融资**10亿美元**，估值54亿美元。NVIDIA已累计承诺超过**400亿美元**的AI股权投资，World Labs、AMI、Decart、Odyssey的股东名单上都有它的名字。

资本涌入的逻辑很简单：**当语言模型商品化、推理利润被摊薄，投资人正在重新评估AI栈中“可防御”的位置——物理智能被认为是下一个值得拥有的壁垒。**

一、什么是世界模型？

世界模型的概念植根于一种更广泛的预测性智能观：智能系统不仅对当前输入做出反应，还能通过预测未来状态、估计行动后果，利用内部模拟来指导行为。这一思想与认知科学中的心理模型理论、预测加工理论有着深刻共鸣。

一篇2026年5月发布的关于世界模型的文章《World Models: A Comprehensive Survey of Architectures, Methodologies, Reasoning Paradigms, and Applications》，近百页综述将世界模型与一般预测模型区分开来的**三个核心特性**概括为：

以行动为条件 (Action-conditioning)：世界模型预测环境如何响应特定行动而演化，支持反事实推理——“如果我向左转而不是向右转，会发生什么？”

多步推演 (Multi-step rollout)：世界模型可以自回归地应用，生成任意长度的轨迹，支持规划和模拟。

对决策的实用性 (Utility for decision-making)：世界模型的预测被下游任务使用——策略优化、规划、数据增强或安全验证——而非仅作为目的本身。

李飞飞在2026年6月给出了一个功能性的三分法：将世界模型分为**渲染器 (Renderer)**、**模拟器 (Simulator)** 和**规划器 (Planner)**。渲染器输出像素供人眼看，不携带对三维结构的理解——她认为目前大多数自称世界模型的演示都属于这一类，包括Google DeepMind的Genie 3。模拟器输出的状态经得起几何和牛顿物理学的检验。规划器直接输出动作。其表面当前行业呈现出三个层次的落差：

第一层：批量生产的“渲染器”

这是目前绝大多数自称“世界模型”的产品所处的层级。它们本质上是**高级的视频生成工具**。例如，Google DeepMind的Genie 3能生成可玩性极高的游戏画面，OpenAI的Sora能生成令人惊叹的物理效果。但它们的工作逻辑是**像素驱动**：通过学习海量视频中的像素分布规律，来“画出一个看似合理的未来，而非理解其背后的物理规律。正如北京智源研究院院长王仲远所说：“视频生成模型的训练数据里包含大量科幻电影内容，它的目标从来就不是还原真实物理世界的规律。”

第二层：少数探索者的“模拟器”

这一层的模型开始超越像素，追求**状态驱动**。它们输出的结果，能被第三方物理引擎检验。李飞飞自己创立的World Labs便属于这一派，其核心路线是从**3D重建**入手，将世界锚定在高斯溅射和物理引擎中。其生成的不是像素，而是带有几何与物理约束的3D世界状态。在智源研究院的悟界·Physis-v0.1中，也能看到这种追求——它聚焦统一物理状态建模，力求从物理本质出发实现因果推理。

第三层：尚未抵达的“规划器”

这是世界模型的终极形态。规划器不再关心生成图像是否好看，甚至不关心是否生成图像，它只关心一件事：**输出最优动作决策**。自动驾驶领域的世界模型正在向这个方向靠拢。Momenta的R6模型、小鹏的第二代VLA，本质上都是在将物理世界的理解转化为具体的驾驶决策信号。

这种从“渲染”到“模拟”再到“规划”的演进，本质上是AI从“**生成看起来合理**”走向“**理解物理规律**”再到“**做出正确决策**”的必经之路。目前，绝大多数产品停留在第一层，部分前沿研究进入第二层，而第三层——能够真正理解物理世界并据此做出决策的规划器——仍是整个行业追逐的珠穆朗玛峰。

二、技术路线之争

关于如何构建世界模型，学术界和产业界尚未达成共识。

路线一：扩散生成路线。 OpenAI的Sora、Google的Genie系列走的是“视频生成即世界模拟”的路线——通过大规模视频生成来学习物理规律。但北京智源研究院院长王仲远指出：“视频生成模型的训练数据里包含大量科幻电影内容，它的目标从来就不是还原真实物理世界的规律。”他举了一个生动的例子：一群猪能在天上和飞机一起飞吗？现实世界不行，但视频生成模型可以。**生成逼真不等于理解物理。**

路线二：潜空间预测路线。 LeCun提出的JEPA（联合嵌入预测架构）主张在抽象的“表征空间”内进行预测，而非预测像素帧。AMI基于这一架构，学习现实世界在压缩潜空间中的抽象表征。

路线三：3D空间智能路线。 李飞飞的World Labs从3D重建入手，将世界锚定在高斯溅射（Gaussian splats）和物理引擎中，试图构建真正理解三维空间结构的模型。

智源研究院的悟界·Physis-v0.1则选择了另一条更底层的路径——聚焦统一物理状态建模，力求从物理本质出发实现因果推理。王仲远坦言：“这些探索距离真正面向物理世界的基座模型都还有不小差距，未来三到五年将是世界模型持续迭代期。”

华为诺亚方舟实验室联合多所高校发表的综述则提出了“专业化与通用化之争”的视角：目前没有一种方法能同时兼顾精度和泛化——专业化模型追求控制任务的高精度，通用化模型追求开放环境的广泛适应性，两者注定长期共存。

三、产业实践

尽管定义和技术路线尚未收敛，世界模型已在多个产业领域加速落地。

3.1 自动驾驶：最激烈的战场

几乎每一家智能驾驶头部企业都在争相贴上“世界模型”标签：吉利发布WAM世界行为模型，小鹏发布X-World，小米汽车推出Xiaomi Auto World Model全新框架，地平线发布

HorizonDrive。Momenta用R6强化学习打造世界模型驱动智驾方案。在2026中国智驾大模型落地TOP 10中，华为乾崮以全栈自研端到端位居第一，小鹏第二代VLA大模型进入主驾无安全员道路验证阶段，Momenta的R6模型在第三方供应商中市占率第一。Momenta的核心优势来自量产业务带来的**超120亿公里实车里程**——这是任何实验室无法复制的数据护城河。

3.2 具身智能

极佳视界在WAIC 2026首次完整展出“通用世界模型”全系列产品，按“**世界生成模型**”与“**世界行动模型**”两大体系展开。创始人黄冠的判断是：“AGI已涌现语言本能，接下来将涌现物理本能。”

其产品矩阵包括：

- **GigaBrain**：通用具身大脑，跨本体部署
- **GigaWorld-Policy**：世界动作模型，在世界模型范式下直接输出动作
- **DriveDreamer**：面向自动驾驶的数据与仿真基础设施

搭载GigaBrain的拾光S1机器人在家庭场景中完成了取餐备餐、收拾餐桌、整理餐具等超长程任务，验证了“世界生成-行动”双模型体系的实际泛化能力。

3.3 实时交互世界

2026年7月，蚂蚁灵波开源**LingBot-World 2.0**，首次实现**小时级实时交互世界生成**，支持720p/60fps高清输出。该模型在业界首次将**Agent机制**引入世界模型——系统内置双Agent机制：Pilot Agent负责规划执行角色行为，Director Agent在场景推进中实时提出新事件，让生成世界从“可观看、可操控”走向“可持续互动、可动态变化”。

LingBot-World 2.0采用因果预训练范式，引入自研**MoBA机制**，让模型按照真实世界交互的时间顺序学习世界如何演化。在长达一小时的压力测试中，画面始终保持纹理清晰、场景结构连贯。同步开源的LingBot-Video在RBench基准上总分**0.620**，超越Wan2.6（0.607）、Seedance 1.5 Pro（0.584）。

四、世界模型 vs 视频生成

世界模型与视频生成模型的边界，是2026年争议最大的问题之一。

王仲远在智源大会上的发言一针见血：视频生成模型的目标是生成“看起来合理”的像素，而非“物理上正确”的演化。如果模型的训练数据中包含大量科幻内容，它完全可能生成“一群猪在天上飞”的画面——因为这就是它在数据里看到的。

李飞飞的分类更为精确：**渲染器输出像素，模拟器输出状态，规划器输出动作**。按照这个标准，目前大多数自称“世界模型”的产品，本质上仍是渲染器。

这个争论的核心不是“谁对谁错”，而是“**生成逼真**”与“**理解物理**”之间的距离。Sora生成的视频令人惊艳，但它无法真正模拟一个球滚下斜坡的物理过程。真正的世界模型需要做到的是：给定当前状态和一个动作，预测下一个状态——这个状态必须满足物理约束，而不仅仅是视觉上看起来合理。

世界模型与数字孪生的区别也值得注意：数字孪生是对特定物理系统的精确数字化映射，依赖大量传感器数据持续校准；世界模型则是学习通用的环境演化规律，可以在没有实时数据的情况下

进行推理和预测。两者互补而非替代——数字孪生提供精度，世界模型提供泛化。

五、瓶颈与挑战

真实物理数据匮乏。世界模型需要大量“动作-结果”配对数据，但这类数据在现实世界中极难获取。这正是OpenAI出价5亿美元收购Medal游戏回放档案的原因——带动作标签的第一人称视频能教会模型“决策到后果”的映射，这是旁观者视角无法提供的。

技术路线尚未收敛。扩散生成、潜空间预测、3D重建——三条路线各有拥趸，没有一条能同时实现精度和泛化。

评测体系不完善。如何判断一个世界模型是否真正“理解”了物理世界，而不是仅仅学会了生成视觉上合理的画面？2026年，WorldModelBench、WorldPrediction、WorldArena等基准正在尝试建立统一的评估体系，但仍在早期阶段。

长时程一致性。Genie 3能以24帧/秒生成可导航世界，但连贯性只能维持几分钟，变化记忆约一分钟。长时程的物理一致性是世界模型面临的核心技术瓶颈。

六、从理论到实战：世界模型的开源生态与代码实现

2026年的世界模型研究，已从“读论文”走向“跑代码”。开源社区涌现出从经典复现到工业级框架的完整工具链，让研究者可以亲手训练一个能“做梦”的智能体。

6.1 经典复现：World Models的从零实现

David Ha和Jürgen Schmidhuber在2018年提出的**World Models**，是这一领域的奠基之作。其核心思想是一个三组件架构：

- **V (Vision)**：卷积VAE，将64×64×3的RGB帧压缩为32维潜变量z
- **M (Memory)**：LSTM + 混合密度网络（MDN），给定当前z和动作，预测下一个z的概率分布
- **C (Controller)**：CMA-ES黑盒优化器，在潜空间“想象”中训练控制器

```
1 # 训练世界模型的三步走（基于World Models经典实现）
2 # 1. 训练VAE — 学习“看懂”画面
3 python VAE.py
4
5 # 2. 训练MDN-RNN — 学习“预测”未来
6 python MDN-RNN.py
7
8 # 3. CMA-ES控制器（独立优化器，验证用）
9 python CMA-ES.py
```

6.2 EB-JEPA：LeCun路线的开源实践

LeCun提出的JEPA（联合嵌入预测架构）放弃像素级重建，转而在**表征空间中进行预测**。Facebook Research开源的**EB-JEPA**库，将这一理论落地为可复现的代码基座：

- **图像级JEPA**：在CIFAR-10上线性探针达**91%准确率**，证明表征质量
- **视频级JEPA**：在Moving MNIST上实现多步预测，验证时序泛化能力
- **动作条件化世界模型**：在Two Rooms导航任务中**97%规划成功率**

所有实验均支持**单GPU数小时完成训练**，极大降低了世界模型研究的门槛。

```
1 # 安装与配置
2 git clone https://github.com/facebookresearch/eb_jepa
3 uv sync
4 export EBJEPA_DSETS=/path/to/datasets
5
6 # 图像级JEPA: CIFAR-10线性探针达91%准确率
7 python -m examples.image_jepa.main
8
9 # 视频级JEPA: Moving MNIST多步预测
10 python -m examples.video_jepa.main
11
12 # 动作条件化世界模型: Two Rooms导航97%规划成功率
13 python -m examples.ac_video_jepa.main
```

EB-JEPA将可调损失超参数从六个压缩到一个，约1500万参数即可在单GPU上完成训练，极大降低了世界模型研究的入门门槛。

6.3 LeWorldModel: 端到端JEPA的最新突破

2026年，LeCun团队进一步推出**LeWorldModel (LeWM)**，首次实现**从原始像素端到端稳定训练的JEPA**，仅需两个损失项：下一个嵌入预测损失 + 高斯分布正则化。

```
1 # 安装与训练
2 uv venv --python=3.10
3 source .venv/bin/activate
4 uv pip install stable-worldmodel[train,env]
5
6 # 训练LeWM
7 python train.py data=pusht
8
9 # 规划评估
10 python eval.py --config-name=pusht.yaml policy=pusht/lewm
```

LeWM在多种2D和3D控制任务上表现优异，其潜空间通过探针实验验证编码了有意义的物理结构，且能可靠检测物理上不合理的事件。**规划速度比基于基础模型的世界模型快48倍。**

6.4 Micro-World: AMD开源的交互式游戏世界模型

游戏环境是世界模型研究的天然实验场。AMD开源的**Micro-World**，是目前唯一**完整开源训练代码+模型权重+数据集**的动作可控世界模型。

数据采集策略：

- 基于Minecraft API，采集**6,000段游戏片段**（每段81帧）
- 动作空间包括W/A/S/D移动、疾跑、潜行、跳跃及鼠标视角控制
- 每段视频由miniCPM自动生成文字描述，形成多模态训练标注

模型设计亮点：

- 基于**Wan2.1**基座，训练Text-to-World和Image-to-World双变体
- **滑动历史窗口机制**：输入过去若干帧动作，生成更连贯的视频
- **两阶段训练**：先LoRA适应游戏视觉风格，再学习动作控制

```
1 # 下载模型权重
2 hf download amd/Micro-World-T2W --local-dir models/T2W
3 hf download amd/Micro-World-I2W --local-dir models/I2W
4
5 # 动作控制推理
6 python examples/wan2.1/predict_t2w_action_control.py
```

6.5 HY-World 2.0：腾讯的3D世界模型

腾讯开源的**HY-World 2.0**代表了世界模型的另一条路线——**直接生成可编辑、可持久化的3D资产**（网格/3D高斯点云），而非像素视频。

核心差异：

维度	视频世界模型	HY-World 2.0 (3D世界模型)
输出	像素视频（不可编辑）	真实3D资产（完全可编辑）
交互时长	有限（通常1分钟）	无限——资产永久保存
3D一致性	无保证	原生3D一致
引擎兼容性	仅视频文件	可导入Blender/Unity/UE

输入一张图片或文字描述，即可一键生成包含人、物、景的完整3D空间，支持角色自由探索，具有物理碰撞。这使AI从“对话”、“画图”进化到了“造世界”。

6.6 Kairos：统一架构的世界行动模型

Kairos是一个原生端到端的统一架构，在Mixture-of-Transformers框架内完成**理解、生成、预测**三位一体。它维护持久的世界状态，其中：理解提供因果解释，生成展开可能的未来，预测产生可执行的机器人动作。

Kairos引入**混合线性时序记忆机制**，结合滑动窗口注意力、扩张滑动窗口注意力和门控线性注意力，在线性复杂度下捕获局部动态、中程交互和全局因果记忆。其4B模型在A800上仅需23.5GB显存，而Cosmos 2.5的14B模型需70.2GB，计算复杂度相差约70倍。

快速开始：

- 4B模型支持480P输出，在LIBERO-plus和RoboTwin2.0等基准上达到SOTA
- 原生兼容Agibot G1、宇树G1、松灵PIPER等多类机器人本体
- 支持单GPU或多GPU的低延迟、高可靠性部署

6.7 宇树UnifoLM-WMA-0：机器人的世界模型

宇树科技开源的**UnifoLM-WMA-0**，是一个专为通用机器人学习设计的**世界模型-动作架构**。其核心是一个能理解机器人与环境交互物理规律的世界模型，提供两大功能：

1. **仿真引擎**：作为交互式仿真器运行，生成合成训练数据
2. **策略增强**：与动作头对接，预测未来交互过程，优化决策

```
1 # 安装与部署
2 conda create -n unifoLM-wma python==3.10.18
3 conda activate unifoLM-wma
4 git clone --recurse-submodules https://github.com/unitreerobotics/unifoLM-world
5 pip install -e .
```

在实际演示中，采用该模型的机器人可将木块按红黄绿顺序叠落，并完成整理橡皮和笔、将黑色相机放入包装盒等精细操作。

6.8 RynnWorld-Teleop：阿里的遥操作世界模型

阿里巴巴达摩院开源的**RynnWorld-Teleop**，是一个**动作条件世界模型**，专为数字遥操作设计。其核心是将第一帧图像和控制视频（手部姿态/OpenPose）映射为对应的第一人称视频生成。训练采用三阶段流水线：预训练 → SFT/LoRA微调 → 流式蒸馏（含MSE预热和DMD对抗蒸馏），支持实时交互式生成。

```
1 # 三阶段训练流程
2 # Stage 0: 预训练
3 bash scripts/rynnworld_teleop_pretrain.sh
4
5 # Stage 1: SFT/LoRA 微调 (二选一)
6 bash scripts/rynnworld_teleop_stage1_sft.sh
7 bash scripts/rynnworld_teleop_stage1_lora.sh
8
```

```
9 # Stage 2: 流式蒸馏 (MSE 预热 + DMD 对抗蒸馏)
10 bash scripts/rynnworld_teleop_streaming_mse.sh
11 bash scripts/rynnworld_teleop_streaming_dmd.sh
```

一些思考

2026年的世界模型，正处于一个奇特的时刻：**数十亿美元已经涌入，但行业连“它到底是什么”都没有完全达成共识。**

这种“不确定中的狂热”并非非理性。投资人看到的是语言模型商品化的必然趋势——当每个模型都能流畅对话时，真正的壁垒在于理解物理世界的能力。**世界模型是“AI从模拟走向现实”的必经之路。**

但挑战同样真实。技术路线未收敛、数据匮乏、评测缺失——这些不是遥远未来的问题，而是当前工程实践每天都要面对的障碍。

中国工程院院士王坚的评价提供了一个冷静的参照：“从人的角度理解智能是什么，到今天还远远是个未知数。中美科研界面的是同一片海，真正的前进动力，始终来自工程实践与理论认知的相互推动、交错前进。”

世界模型的终局不是生成更逼真的视频，而是让AI真正理解“如果这样做，世界会发生什么”。这条路还很长，但方向已经清晰。

大模型必知必会：dLLM的崛起与2026年的AI新秩序

原创 QinHsiu 阿Q的AI笔记 2026年9月9日 07:45 北京

2026年7月，ICML 2026公布了两篇杰出论文。获奖者不是GPT-5，不是Claude 4，而是一个大多数普通用户还没听说过的新物种——**扩散语言模型（dLLM）**。

几乎同一时间，Google DeepMind向vLLM提交了DiffusionGemma的完整支持代码，一个260亿参数的dLLM正式进入生产级推理引擎。

大模型正在经历一场**静默的“换脑手术”**——从GPT开创的“自回归逐字生成”，切换到一个全新的范式：“**去噪生成**”。

这不是一次常规迭代，而是一场**底层架构的革命**。

一、什么是dLLM？

传统LLM（如GPT、Claude）是“打字”模式：从左到右，一个词一个词往外蹦。必须按顺序决定每一个词，优点是逻辑连贯，缺点是**慢**——每生成一个词就要跑一次完整的前向传播。

dLLM是“画画”模式：先画一个模糊的草图（全是随机词），然后**迭代式地去噪**，每一次都让整段文字更清晰，最终形成完整文本。

你可以想象这样一个场景：

- 传统LLM：一个字一个字地写一封信
- dLLM：先在纸上随机撒一堆墨点，然后一笔一笔地把信“显影”出来

对比维度	传统LLM（自回归）	dLLM（扩散）
生成方式	“打字”模式 ：从左到右，逐字（token）生成，必须按顺序决定每一个词	“画画”模式 ：先随机撒“墨点”（初始噪声），通过迭代去噪逐步显影出完整文本
生成粒度	一次生成 1个token ，每次需跑完整前向传播	一次并行生成 256个token ，在多个去噪步骤中并行精炼
上下文视野	单向（因果注意力） ：只能看到左侧已生成的词	双向（全注意力） ：可同时看到整段文本的所有位置
推理能力	强制从左到右顺序， 逻辑链条完整 ，不易绕过困难推理节点	自由顺序易导致“ 灵活性陷阱 ”（先简单后困难，削弱推理），需JustGRPO等方案约束
速度性能	受限于 内存带宽 ，每生成一个token需加载全部权重	并行解码 ，d3LLM实现最高 5倍加速 ，DMax达 6.0倍
代价与权衡	计算量线性增长，内存带宽是主要瓶颈	计算量更大 ，但巧妙将带宽压力 转换为计算压力 ——低批量场景下算力充裕而带宽是瓶颈

二、闭源阵营

2.1 DiffusionGemma (Google DeepMind)

全球第一个进入生产级推理引擎的 dLLM。基于 Gemma 4 的 MoE 架构 (260亿总参数 / 40亿激活参数)，采用“熵界去噪” (**Entropy-bound Denoising**) 机制：每步重新采样画布所有位置，只保留“有信心”的位置 (低熵即高置信度)，从最有信心的位置开始锁定，逐步扩散至整段文本。这一机制模仿了人的写作直觉——先确定关键词，再围绕它们组织完整句子。单次并行生成 **256个token**，吞吐量约为自回归基线的 **1.9倍**。

在 build.nvidia.com 获取免费 API Key，免费层限制 **40 请求/分钟**：

```
1 import requests
2 invoke_url = "https://integrate.api.nvidia.com/v1/chat/completions"
3 headers = {"Authorization": "Bearer $NVIDIA_API_KEY"}
4 payload = {
5     "model": "google/diffusiongemma-26b-a4b-it",
6     "messages": [{"role": "user", "content": "介绍一下扩散语言模型"}],
7     "max_tokens": 4096,
8     "temperature": 1,
9     "top_p": 0.95
10 }
11 response = requests.post(invoke_url, headers=headers, json=payload)
```

2.2 Mercury 2 (Inception Labs)

首个推理型 dLLM。通过 **并行精炼 token** 实现实时延迟下的推理级质量，支持 **128K** 上下文窗口、原生工具调用和结构化输出。可通过 `reasoning_effort` 参数控制推理深度 (low / medium / high / instant)，`reasoning_summary=True` 可返回推理摘要。定价极具竞争力：输入\$0.25/百万token，输出\$0.75/百万token。

在 Inception Platform 获取 API Key：

```
1 curl https://api.inceptionlabs.ai/v1/chat/completions \
2   -H "Authorization: Bearer $INCEPTION_API_KEY" \
3   -H "Content-Type: application/json" \
4   -d '{
5     "model": "mercury-2",
6     "messages": [{"role": "user", "content": "什么是扩散语言模型？"}],
7     "reasoning_effort": "medium"
8   }'
```

2.3 Celeris-1 (Celeris Labs)

专为 **低延迟智能体 workflow** 设计的 dLLM。衍生自 Qwen3.8-27B，采用 OpenAI 兼容 API。BenchLM 实测中位数输出速度达 **1,626 token/秒**，为第二快模型 Mercury 2 的 **4.8倍**。MMLU-Pro 得分 **75.9%**。上下文窗口 **131K**，API 访问目前仅限美国地区。

在 Celeris 控制台 创建 API Key，替换 base_url 即可：

```
1 from openai import OpenAI
2 client = OpenAI(
3     base_url="https://inference.celeris.ai/celeris-1/v1",
4     api_key="YOUR_CELERIS_API_KEY"
5 )
6 response = client.chat.completions.create(
7     model="celeris-1",
8     messages=[{"role": "user", "content": "介绍一下扩散语言模型"}]
9 )
```

三、开源生态

3.1 核心开源模型

开源扩散语言模型 (dLLM) 对比					
模型	发布方	参数量	许可证	核心特点	获取地址
DiffusionGemma	Google DeepMind	26B 总参数 / 3.8B 激活 (MoE)	Apache 2.0	全球首个进入生产级推理引擎的 dLLM，vLLM 原生支持，推理速度 1000+ token/s，单次并行生成 256 token	Hugging Face / GitHub
LLaDA / iLLaDA	InclusionAI / GSAI	8B (dense) / 7B (MoE, 1.4B 激活)	开源	首个从零预训练的 MoE 扩散模型 (~20T tokens)。LLaDA2.2 为全球首个 Agentic 扩散模型，原生 128K 上下文	GitHub: ML-GSAI/LLaDA
Dream 7B	Dream-org	7B	Apache 2.0	基于 Qwen2.5-7B 骨干的掩码扩散模型，数学和事实任务表现优异 (2 步去噪即算对 15×23)	Hugging Face
DreamReasoner-8B	DreamLM	8B	开源	专为复杂数学推理和代码生成设计的块扩散推理模型，性能可比 Qwen3-8B-Thinking	GitHub: DreamLM/DreamReasoner
Sumi	东北大学	7B	Apache 2.0	全球首个从零预训练的 7B 均匀扩散语言模型 (1.5T tokens)，为社区研究均匀扩散提供开放基线	GitHub: tohoku-nlp/sumi
W1-4B-dLLM	Whaletech AI	4B	开源	基于扩散 Transformer 的语言模型，4B 层扩散 Transformer 模块	Hugging Face/Whaletech/AIW1-4B-dLLM-856

3.2 加速与推理框架

dLLM 加速与研究项目对比			
项目	发布方	核心能力	获取地址
d3LLM	Hao AI Lab	ICML 2026 收录。通过伪轨迹蒸馏实现相比原生扩散模型 10 倍加速、相比自回归模型 5 倍加速 (H100)，已开源 d3LLM-LLaDA、d3LLM-Dream、d3LLM-Dream-Coder 三个版本	GitHub: hao-ai-lab/d3LLM
RCD (Residual Context Diffusion)	yuezhouhu	ICML 2026 收录。“残差去噪”机制将丢弃的 token 分布重新注入下一步去噪。AIME25 数学竞赛上 SDAR-4B 从 11.88 分提升至 17.08 分。配套专用推理引擎 JetEngine (CUDA/Triton)	GitHub: yuezhouhu/residual-context-diffusion
DMax	Zigeng Chen 等	激进并行解码新范式，将解码定义为“从掩码嵌入到 token 嵌入的新进式自我精炼”，16B 版本实现 6.0 倍并行加速	GitHub: czg1225/DMax
Fast-dLLM	NVIDIA	无需训练的 dLLM 加速技术，通过 KV Cache 和并行解码实现加速	GitHub: NVlabs/Fast-dLLM
Dynamic-dLLM	TianyiWu233	ICLR 2026 收录。动态缓存预解与自适应并行解码	GitHub: TianyiWu233/DYNAMIC-DLLM

3.3 统一开发框架

项目	核心能力	获取地址
dLLM (同名框架)	统一扩散语言模型训练、推理与评估的库。基于 Transformers Trainer，支持 LoRA、DeepSpeed、FSDP。支持将任意自回归模型（Qwen、LLaMA、GP T-2）转换为扩散模型（A2D 方案）	GitHub: ZHZisZ Z/dllm
dInfer	inclusionAI 推出的 dLLM 专用推理框架，在 HumanEval 上实现 1100+ token/s （batch size 1），8×H800 平均 800+ token/s	GitHub: inclusionAI/dInfer

四、端侧dLLM

dLLM的并行特性在端侧展现出独特优势——它天生适合端侧部署。

4.1 为什么dLLM适合端侧？

传统LLM在端侧的瓶颈是**内存带宽**：每生成一个token都要加载全部模型权重到内存，速度受限。而dLLM**一次前向传播生成多个token**，把内存带宽压力转换成了计算压力——端侧芯片的算力往往比内存带宽更充裕。

一篇2026年9月刚发布的综述论文《Diffusion Language Models for Mobile Edge Agentic AI》系统分析了dLLM在端侧的适用性。核心结论是：dLLM的**双向注意力、并行精炼、质量-延迟弹性**等特性，非常适合边缘智能体场景。

4.2 代表性端侧项目

llada.cpp：首个针对**手机NPU优化的dLLM推理框架**。通过多块推测解码、双路径渐进式修订和交换优化内存运行时三大技术，将LLaDA-8B模型的延迟降低了**17至42倍**。

MobileLLM-Flash（350M/650M/1.4B）：专为移动端CPU设计的端侧模型，在移动CPU上预填充和解码速度分别提升**1.8倍和1.6倍**。

LFM2.5-2.6B：Liquid AI发布的26亿参数端侧智能体模型，可在**树莓派这样的小型设备上运行**，原生支持llama.cpp、MLX、vLLM等推理框架。

腾讯混元HY-1.8B-2Bit：等效参数量仅**0.3B**，内存占用仅**600MB**。

五、动手实践：30分钟跑通第一个dLLM

以下是一个**零基础、可操作**的实践流程。

5.1 体验在线Demo

Demo 名称	核心特点	链接地址
Kohra	浏览器端通过 WebGPU 运行，无需 GPU 即可在本地体验。支持 Qwen3-0.6B-diffusion-mdlm（掩码扩散）和 bd3lm（块扩散）两种模型，首次加载需下载 0.7-1.5GB 模型文件	naklitechie.github.io/kohra 或 Hugging Face Space
LLaDA2.0-Flash-CAP	全球首个 100B 参数 的扩散语言模型，推理速度 535 tokens/秒 ，支持在线聊天	skywork.ai 博客页（体验入口在文章内）
MMaDA	首个多模态扩散大语言模型 ，支持图文联合理解与生成，非自回归并行解码	Hugging Face Space
PerceptionDLM	Hugging Face Spaces 托管的基础交互式 Demo	Hugging Face Space
dLLM Leaderboard	d3LLM 官方排行榜，查看 d3LLM 与其他模型的性能对比（不是文本生成 Demo，但可直观了解性能）	Hugging Face Space
Dream Playground	体验 Dream 系列扩散模型（含 Dream-7B、Dream Reasoner 等），支持推理和对话	Hugging Face Space
Mistral Community	社区维护的 dLLM 聊天演示集合	Chat Playground
Celeris-1	闭源低延迟 dLLM， 1,664 token/秒 ，MMLU-Pro 75.9%，可通过 OpenAI 兼容 API 调用（仅限美国地区）	celeris.ai

5.2 本地部署d3LLM

环境要求：Python 3.10+，CUDA GPU（推荐16GB+显存）

步骤一：克隆仓库

```
1 mkdir -p ~/Codes && cd ~/Codes
2 git clone https://github.com/hao-ai-lab/d3LLM.git
3 cd d3LLM
```

步骤二：安装依赖

```
1 pip install -r requirements.txt
2 # 关键版本: transformers==4.49.0, lm_eval==0.4.9, datasets==3.2.0, flash_attn==
```

步骤三：下载模型

```
1 # 从Hugging Face下载d3LLM-LLaDA
2 huggingface-cli download d3LLM/d3LLM_LLaDA --local-dir ./models/d3LLM_LLaDA
```

步骤四：运行推理

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer
2 model = AutoModelForCausalLM.from_pretrained("./models/d3LLM_LLaDA")
3 tokenizer = AutoTokenizer.from_pretrained("./models/d3LLM_LLaDA")
4 inputs = tokenizer("介绍一下扩散语言模型", return_tensors="pt")
5 outputs = model.generate(**inputs, max_new_tokens=256)
6 print(tokenizer.decode(outputs[0]))
```

5.3 用SGLang部署

SGLang已原生支持dLLM算法（LowConfidence、JointThreshold等）：

```
1 python3 -m sglang.launch_server --model-path d3LLM/d3LLM_LLaDA --dlla-algorithm
```

5.4 把AR模型“变成”dLLM

如果你有一个现有的自回归模型（Qwen、LLaMA等），可以用dLLM框架的A2D（Any-to-Diffusion）方案将其转换为扩散模型：

```
1 # 参考dLLM框架的examples/a2d目录
2 cd dLLM/examples/a2d
3 python convert.py --model_name Qwen/Qwen2.5-7B --output_dir ./qwen-diffusion
```

六、dLLM的“灵活性陷阱”

维度	关键发现与数据	解决方案与进展
核心理论：灵活性陷阱	自由生成顺序虽快，但 会削弱推理能力 （I CML 2026杰出论文）。模型会倾向于 绕过困难逻辑节点 （如“因此”、“所以”）。	JustGRPO （清华团队）：训练时 强制从左到右生成 ，保障推理能力；推理时 保留并行解码 优势。
关键数据支撑	HumanEval ：从左到右顺序成功但任意顺序失败的比例为 21.3% ，反之仅为 0.6% 。	GSM8K ：采用JustGRPO方案后，准确率达 89.1% 。
待解问题与展望	华为在AAAI 2026指出 十大开放挑战 ：包括架构惯性、梯度稀疏性、线性推理局限性等。	dLLM距离达到“ GPT-4时刻 ”仍有距离，需持续攻克上述挑战。

一些思考

2026年，扩散语言模型正在从实验室走向生产环境。

Google的DiffusionGemma已经跑在vLLM上。d3LLM在H100上实现了5倍加速。RCD让dLLM在数学竞赛上追平了自回归模型。llada.cpp让dLLM跑在了手机上。

这不是一场“取代”革命，而是一场“补充”革命。 dLLM不会完全取代自回归模型——它会在**高吞吐量场景**（聊天机器人、实时翻译）、**端侧设备**（手机、IoT）和**有约束生成任务**（代码生成、结构化数据）中发挥独特优势。

简单说，2026年，dLLM的大门已经打开。

作者提示: 个人观点，仅供参考

大模型必知必会：一份关于大模型训练的保姆级指南

原创 QinHsiu 阿Q的AI笔记 2026年8月18日 08:00 北京

一文读懂大模型训练的“方法论”与“工具箱”，附高频面试考点

训练一个大语言模型，究竟需要经历哪些阶段？需要用到哪些工具？

从预训练的海量文本“啃书”，到监督微调的“听话训练”，再到强化学习的“价值观对齐”——每一个阶段都有其独特的技术原理和工程实现。而从单机单卡的LoRA微调，到千卡集群的Megatron分布式训练——每一个场景都有其对应的架构选型。

考点：预训练（学习语言规律与通用知识）、SFT（学会理解指令、规范回答）、RLHF（符合人类偏好与价值观）。追问：为什么需要三个阶段而不是一步到位？因为每个阶段解决不同层次的问题——预训练解决“会不会说话”，SFT解决“听不听话”，RLHF解决“说得好不好”。

本文将BYO-LLM-FZ2O项目为参照框架，系统梳理LLM训练的两大维度：**训练范式**（做什么）与**训练架构**（用什么做），并在每个关键节点嵌入**面试考点与答案**。

一、预训练（Pre-training）

1.1 什么是预训练

预训练是大模型训练的**起点和基石**。把模型扔进海量无标注原始语料里“自学”——包括全网网页、专业书籍、学术论文、开源代码、百科词条等——全程不用人工打标签。核心目标就一个：学习Token之间的概率分布，说白了就是练“高级词语接龙”。

预训练的本质是**无监督自回归学习**，让模型习得通用语言、逻辑和常识。现代大模型普遍采用**自回归语言建模（Next-Token Prediction）**——模型基于前文预测下一个词，逐步生成完整文本。这一目标看似简单，却是大模型“涌现”能力的根本来源。

1.2 预训练的核心特征

- **数据体量：**达到**万亿Token级别**，涵盖TB级多领域文本
- **算力消耗：**需数千张A100/H100显卡集群，训练周期以月计，算力消耗占整体训练成本的**90%以上**
- **模型状态：**随机初始化或基于基础架构（Transformer），训练后得到**通用基座模型**（如Llama、Qwen基础版）

1.3 预训练的成果与短板

✅ **具备能力：**掌握语法、拥有海量通识知识、具备基础逻辑归纳、能读懂各类文本句式。

❌ **明显短板：**听不懂人类指令、不会正经对话、回答随意发散、没有安全底线。

1.4 预训练的技术要点

预训练涉及的技术环节包括：**Tokenizer训练**（将文本切分为模型可处理的原子单元）、**数据源采样**（多领域数据的配比策略）、**数据预处理**（清洗、去重、过滤低质/毒性内容）以及**模型结构设计**。

训练中还需遵循**Scaling Laws**——模型性能与参数量、数据量、计算量呈幂律关系。Chinchilla法则进一步指出：参数量和训练Token数应**等比例增长**。

二、持续预训练 (Continue Pre-training, CPT)

2.1 什么是持续预训练

持续预训练是在通用预训练模型的基础上，使用**特定领域数据**（如代码、数学论文、医学文献等）继续做自监督学习。它不是从头训练，而是在已有基座模型上“补课”——让模型掌握某个垂直领域的深层知识。

2.2 CPT与Pretrain的区别

维度	Pre-training	Continue Pre-training
起点	随机初始化	已有预训练模型
数据	通用多领域语料	特定领域语料（代码/数学/医学等）
目标	建立通用语言能力	增强特定领域知识
算力	极高（月级）	较高（周级）
典型场景	从零训练基座模型	代码模型、数学模型、领域模型

2.3 典型应用

- **代码模型**：在通用基座上用代码语料继续预训练（如CodeLlama）
- **数学模型**：用数学论文和习题继续预训练（如DeepSeek-Math）
- **领域模型**：用金融、医疗、法律等垂直领域语料继续预训练

CPT的核心价值在于：**不需要从头花数千万美元训一个基座，只需要花少量成本让模型“专精”某个领域**。

三、监督微调 (Supervised Fine-Tuning, SFT)

3.1 什么是SFT

SFT（监督微调）是在预训练底座之上，用**高质量的指令-响应对数据**做有监督训练。本质就是给模型上“规范特训”——教会它理解指令、按格式输出、正经回答问题。SFT是模型从“通用”到“可用”的关键一步。

3.2 SFT的核心特征

- **数据**：十万至百万级高质量标注数据（问答对、对话样本、任务指令集），格式为“Prompt→Response”

- **算力**：中低，可单卡/多卡训练，配合LoRA可在消费级显卡实现
- **原理**：本质是迁移学习+监督学习，损失函数为交叉熵损失

3.3 SFT的训练策略

实践中，SFT有两种主流策略：

(1) 全参数微调 (Full Fine-Tuning)

更新模型全部参数。效果最好，但需大量算力和显存，适合有充足GPU的企业。

(2) 参数高效微调 (Parameter-Efficient Fine-Tuning, PEFT)

只更新少量参数，大幅降低训练成本。主流方法包括：

方法	原理	特点
LoRA	在原始权重旁注入低秩矩阵	最流行，显存占用低
QLoRA	LoRA + 4bit量化	可在消费级显卡微调大模型
Adapter	在Transformer层间插入小型网络	较早的PEFT方法
Prefix Tuning	在输入前添加可训练前缀	适合生成任务

四、强化学习与偏好对齐：从RLHF到GRPO

SFT之后的模型虽然能听懂指令，但回答质量参差不齐——可能不够有用、不够安全、不够符合人类审美。对齐训练 (Alignment) 的目标是让模型的行为符合人类期望——**说人话、有用、无害、诚实**。

考点：SFT是模仿学习 (Imitation Learning)，让模型模仿标注数据中的回答风格和格式；RLHF是优化学习 (Optimization)，通过奖励信号让模型学会产出更符合人类偏好的回答。打个比方：SFT像“临摹名家书法，学会笔画”；RLHF像“参加书法比赛，评委打分后不断改进”。

4.1 RLHF：经典的对齐方案

RLHF (Reinforcement Learning from Human Feedback) 是最经典的基于人类反馈的强化学习方法。其训练流程分为三步

1

步骤1：监督微调 (SFT)

2

| 从预训练基础模型开始，用高质量人工标注数据 (问题-理想答案对)

3

| 进行初步微调，让模型学会基本的对话和指令遵循能力[reference:3]

4

▼

5

步骤2：训练奖励模型 (Reward Model, RM)

6

| 收集人类对模型不同输出的偏好排序数据

7

| 训练一个“打分器”——给模型的回答打分

8

| 高分=符合人类偏好，低分=不符合[reference:4]

10 步骤3：PPO强化学习

11 | 用奖励模型的打分作为反馈信号

12 | 通过PPO算法优化主模型，让它学会产出高分回答[reference:5]

PPO (Proximal Policy Optimization) 是RLHF中使用的主流强化学习算法，通过策略梯度加KL惩罚来稳定训练。

考点：收集人类对模型输出的偏好排序数据（同一问题多个回答的优劣排序），训练一个二分类模型或排序模型来预测哪个回答更好。奖励模型的输出是一个标量分数，代表该回答符合人类偏好的程度。

考点：主流是**PPO (Proximal Policy Optimization)**。PPO通过策略梯度加KL惩罚来稳定训练，防止模型偏离SFT模型太远。追问：PPO的损失函数是什么？

$L_{PPO} = E[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$ ，其中 $r_t(\theta)$ 是新旧策略的概率比， A_t 是优势函数。

RLHF的局限：需要训练独立的奖励模型，训练流程复杂（三步走），PPO训练不稳定且超参数敏感，整体算力成本高。

4.2 DPO：更简单的替代方案

DPO (Direct Preference Optimization) 是RLHF的简化版——**不需要训练奖励模型**，直接用偏好对（好回答/坏回答）更新策略。DPO通过最小化一个封闭形式的训练目标，直接针对人类偏好进行微调。

维度	RLHF	DPO
需要奖励模型	✅ 是	❌ 否
训练复杂度	高（三步）	低（一步）
训练稳定性	较不稳定	更稳定
效果上限	更高	相当

DPO因其实现简单、训练稳定，已成为开源社区的主流偏好对齐方法。

考点：DPO不需要训练奖励模型，直接通过偏好对优化策略，训练更简单、更稳定。劣势是效果上限可能略低于RLHF，且对偏好数据的质量更敏感。

4.3 GRPO

GRPO (Group Relative Policy Optimization, 组相对策略优化) 是DeepSeek团队提出的一种新型强化学习算法，专为解决大模型训练中的策略优化难题而设计。

GRPO的核心创新：

传统PPO算法在更新策略时，需要训练一个独立的价值网络（Critic模型）来估计优势函数，这增加了训练复杂度和显存开销。GRPO通过**三方面创新**解决这一问题：

- 1. **分组相对优势估计**：将完整序列拆分为多个语义组（如段落、对话轮次），在组内计算相对优势值而非全局价值，使策略梯度方差降低62%
- 2. **无需奖励模型**：GRPO消除了训练额外奖励模型的需求
- 3. **基于规则的奖励函数**：DeepSeek-R1采用简单的基于规则的奖励函数（答案对错/测试通过），而非使用结果或过程奖励模型

GRPO vs PPO：

维度	PPO	GRPO
需要Critic模型	✅ 是	❌ 否
需要奖励模型	✅ 是	❌ 否（可用规则奖励）
优势估计	全局价值网络	组内相对优势
训练稳定性	较不稳定	更稳定
代表模型	InstructGPT	DeepSeek-R1

GRPO的核心思想是：不直接优化单个策略的绝对奖励，而是通过比较同一批次中不同策略变体的相对表现，动态调整策略更新方向。对于每个输入查询q，GRPO采样一组输出{o₁...}，基于组内相对表现进行优化。

考点：GRPO是DeepSeek提出的强化学习算法。核心创新：①通过组内相对优势估计替代全局价值网络，降低方差；②无需训练独立的Critic模型；③可使用基于规则的奖励函数。与PPO相比，GRPO训练更稳定、显存占用更低，DeepSeek-R1正是采用GRPO训练的。

4.4 其他对齐方法

- **KTO (Kahneman-Tversky Optimization)** ：基于行为经济学的前景理论，只区分“好”与“坏”
- **ORPO (Odds Ratio Preference Optimization)** ：无需参考模型的偏好优化
- **SimPO**：DPO的简化变体
- **RLVR (Reinforcement Learning from Verifiable Rewards)** ：基于可验证奖励的强化学习

考点：Scaling Laws，模型性能与参数量、数据量、计算量呈幂律关系。指导意义：①同时扩大三要素才能持续提升性能；②小规模实验可以预测大模型表现；③Chinchilla法则指出参数量和训练Token数应等比例增长。

4.5 Reasoning RL：推理能力的强化学习

除了对齐人类偏好，强化学习还可用于**增强模型的推理能力**——对模型的推理过程做强化学习，用客观结果（答案对错/测试通过）做奖励，不需要人工标注。

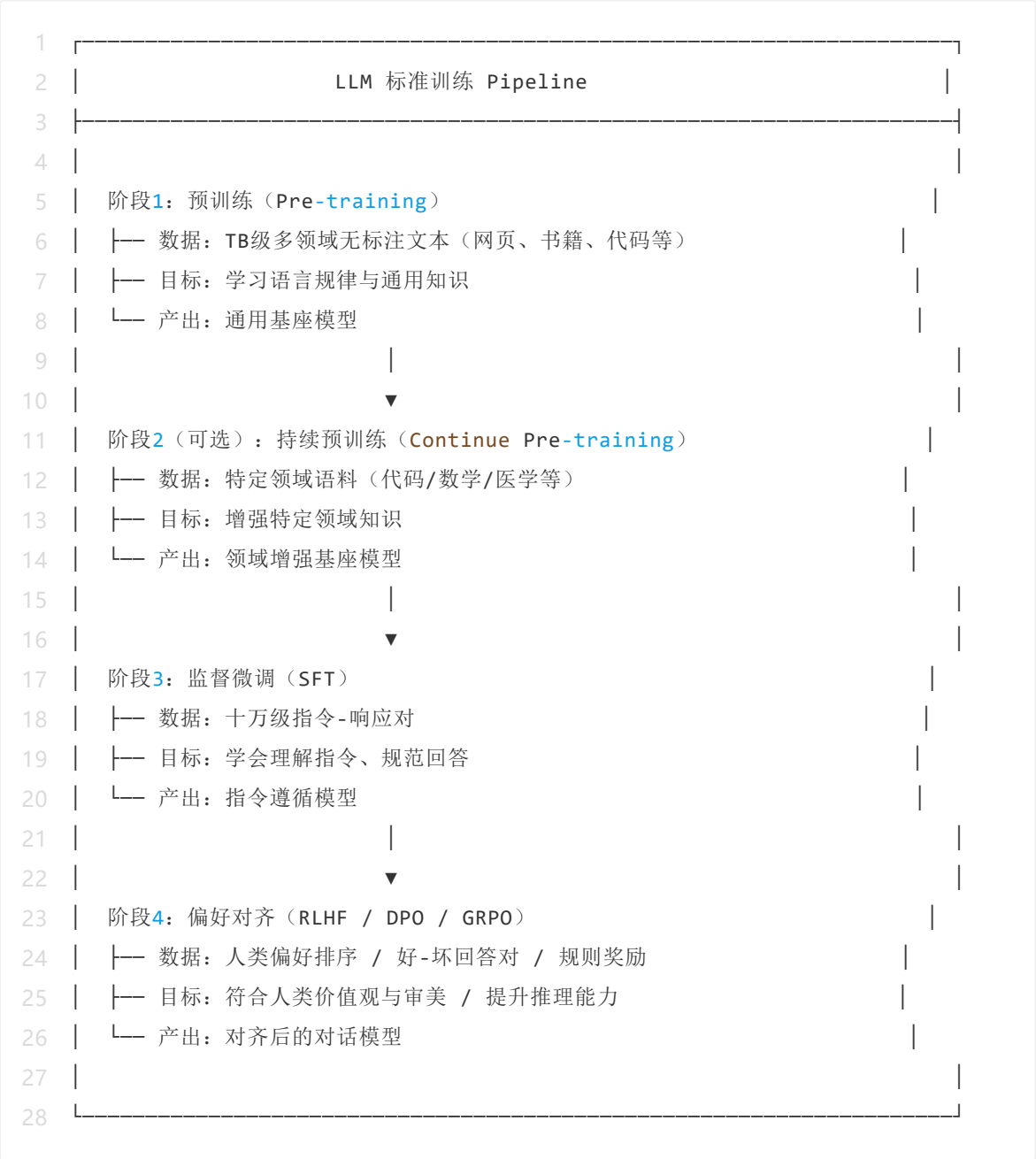
典型流程包括：

- **Rejection Sampling**：模型生成多条推理路径，只保留最终结果正确的，用这些高质量路径再做一轮SFT
- **CoT (Chain-of-Thought)** ：让模型输出“一步步推理”的过程，而非直接给答案
- **ReAct (Reasoning + Acting)** ：模型交替输出思考过程和工具调用

DeepSeek-R1正是通过**GRPO强化学习**让模型“学会推理”的典型代表。

五、训练范式总结：标准Pipeline一览

一个完整的现代LLM训练Pipeline（2026年版）：



六、分布式训练框架

当模型参数规模超过单张GPU的显存容量时，就需要**分布式训练**——将模型切分到多张GPU上协同训练。目前业界最主流的分布式训练框架主要有两个：**Megatron-LM**（NVIDIA）和**DeepSpeed**（Microsoft）。

考点：Megatron-LM擅长张量并行和流水线并行，专注于NVIDIA GPU的极致性能优化；DeepSpeed的ZeRO技术更通用，内存优化更强。两者可以组合使用——用Megatron做TP和PP，用DeepSpeed ZeRO做数据并行和显存优化，形成**Megatron-DeepSpeed**混合方案。

6.1 Megatron-LM

Megatron-LM是NVIDIA应用深度学习研究团队开发的开源分布式训练框架，专注于大规模语言模型的张量并行训练。其核心优势在于针对NVIDIA GPU的深度优化。

核心能力：

- **张量并行 (Tensor Parallelism, TP)** ：将注意力头和前馈网络层的权重矩阵切分到多个设备上，同时最小化通信开销
- **流水线并行 (Pipeline Parallelism, PP)** ：将模型的不同层分配到不同GPU上，形成流水线
- **3D并行：**数据并行 + 张量并行 + 流水线并行三者组合
- **专家并行 (Expert Parallelism, EP)** ：MoE模型中，多个专家分布在不同worker上

Megatron-LM为Transformer模型实现了特定的张量并行策略，通过Fused CUDA Kernels实现高效的3D并行训练。并行策略的典型应用顺序为：TP → EP → DP → PP。

6.2 DeepSpeed：微软的“万亿模型核武器”

DeepSpeed是微软开源的深度学习优化库，是大规模分布式训练的标杆框架。其最核心的贡献是 **ZeRO (Zero Redundancy Optimizer)** 技术。

ZeRO的三大阶段：

阶段	优化内容	效果
ZeRO-1	切分优化器状态（如Adam动量参数）	显存占用减少至1/N（N为GPU数量）
ZeRO-2	切分优化器状态 + 梯度	进一步降低内存占用
ZeRO-3	切分优化器状态 + 梯度 + 模型参数	1750亿模型从1.2TB降至48GB

ZeRO通过将各种模型训练状态（权重、梯度和优化器状态）分区到分布式训练硬件中的不同设备上，减少每个GPU的内存消耗。

ZeRO-3使得**单台8卡A100服务器即可启动1750亿参数模型的训练**，极大地降低了超大规模模型的训练门槛。DeepSpeed还支持3D并行（数据并行+模型并行+流水线并行）和BF16混合精度训练，已支撑微软与NVIDIA联合训练的**5300亿参数MT-NLG模型**。

考点：ZeRO-1切分优化器状态，显存减少至1/N；ZeRO-2切分优化器状态+梯度；ZeRO-3切分优化器状态+梯度+模型参数，1750亿模型从1.2TB降至48GB。追问：ZeRO-Offload是什么？将优化器内存和计算从GPU卸载到CPU。

6.3 Megatron vs DeepSpeed

两者并非互斥关系——实践中经常**组合使用**：用Megatron做张量并行和流水线并行，用DeepSpeed ZeRO做数据并行和显存优化。这种组合方案被称为**Megatron-DeepSpeed**。

维度	Megatron-LM	DeepSpeed
开发者	NVIDIA	Microsoft
核心优势	张量并行、流水线并行	ZeRO显存优化
专注方向	Transformer极致性能优化	通用分布式训练
典型规模	千卡级集群	单机到万卡级
上手难度	较高	中等

选型建议：

- 需要极致的**张量级并行** → Megatron-LM
- 需要**显存优化**突破单卡限制 → DeepSpeed ZeRO
- **大规模生产训练** → Megatron + DeepSpeed 混合并行

七、微调训练框架：从LLaMA-Factory到MS-SWIFT

如果说Megatron和DeepSpeed是为“从零训练大模型”准备的工业级重器，那么下面这些框架则是为“微调已有模型”打造的高效工具。

7.1 LLaMA-Factory：零代码可视化工厂

LLaMA-Factory是一个支持**100+**大语言模型的一站式微调工具，基于PEFT和TRL二次封装。

核心特点：

- **模型兼容性极广**：支持LLaMA、LLaVA、Mistral、Mixtral-MoE、Qwen、Qwen2-VL、DeepSeek、Yi、Gemma、ChatGLM、Phi等
- **Web UI**：提供友好的WebUI界面，非专业开发者也能通过可视化操作完成模型微调
- **全流程支持**：支持全参微调、Freeze、LoRA、QLoRA等多种训练方法
- **训练范式覆盖**：支持SFT、Reward Modeling、PPO、DPO等

适用场景：个人开发者快速验证想法、中小企业缺乏深度开发能力时对现有模型进行微调适配。

7.2 MS-SWIFT (ModelScope Swift) ：魔搭社区的“全栈”框架

MS-SWIFT (Scalable lightWeight Infrastructure for Fine-Tuning) 是魔搭社区 (ModelScope) 官方提供的大模型与多模态大模型训练部署框架。其相关论文《SWIFT: A Scalable lightWeight Infrastructure for Fine-Tuning》已于2025年被**AAAI会议**接受。

惊人的模型覆盖度：

- **600+** 纯文本大模型支持
- **400+** 多模态大模型支持
- 涵盖Qwen2.5、InternLM3、GLM4、Llama3.3、Mistral、DeepSeek-R1等

丰富的训练技术：

- 轻量化微调：LoRA、QLoRA、Llama-Pro、LongLoRA、GaLore、LISA、DoRA等
- 人类对齐：DPO、GRPO、RM、PPO、KTO、CPO、SimPO、ORPO等
- **Megatron-SWIFT**：支持直接加载和存储safetensors格式模型权重，支持LoRA增量权重双向转换

选型建议：主攻通义千问（Qwen）系列模型，追求**生产级部署能力**，推荐使用Swift。

7.3 Unsloth：极致高效的微调框架

Unsloth是一个专为LLM微调设计的加速框架。其设计哲学是让个人开发者在单张消费级GPU上也能完成原本需要昂贵算力的训练任务。

考点：Unsloth为什么能实现2-5倍的训练加速，通过定制Triton内核优化CUDA操作、动态显存管理、计算图优化和低精度训练技术，在消费级GPU上实现高效微调。

惊人的效率提升：

- 微调速度提高 **2-5倍**（相比传统Hugging Face方法）
- 显存使用量减少约 **80%**
- 24GB显存即可微调9B参数模型（使用QLoRA技术可降至6.5GB）

技术内核：

- 定制Triton内核：CUDA操作优化
- 动态显存管理：QLoRA训练时显存占用极低
- 多模态支持：扩展至Whisper语音模型、Stable Diffusion

适用场景：个人研究者单卡实验、教育领域快速原型验证、算力资源紧张的场景。

7.4 Axolotl

Axolotl是一个全流程封装的LLM微调工具。

核心优势：

- 全流程封装：数据预处理→训练→导出→部署
- 高级训练技术：序列并行突破长文本限制、GaLore优化器降低95%优化器状态显存、多数据打包提升30%吞吐量
- 企业级部署：支持Kubernetes集成，实现云原生训练

7.5 其他重要框架

框架	特点	适用场景
Hugging Face TRL	Transformer强化学习库，代码结构清晰、文档丰富	算法研究员尝试新算法

框架	特点	适用场景
OpenRLHF	基于Ray的高性能RLHF框架，支持PPO/GRPO/RL00等	大规模RLHF训练
verl	原生支持Megatron/FSDP，唯一原生支持万亿参数MoE全量RLHF	基础模型团队架构师

考点：OpenRLHF基于Ray+vLLM分布式架构，性价比高，可利用多张消费级显卡组合训练7B-34B模型；verl原生支持Megatron/FSDP，是唯一原生支持万亿参数MoE模型全量RLHF的框架。选型建议：中小企业选OpenRLHF，基础模型团队选verl。

7.6 训练框架选型指南：

应用场景	推荐工具 / 组合	关键理由
场景1：从零训练大模型 (Pretrain from Scratch)	Megatron-LM + DeepSpeed	需分布式训练支撑千亿级参数，3D并行（TP+PP+DP）搭配ZeRO显存优化是标配方案。
场景2：领域持续预训练 (CPT)	Megatron-LM 或 DeepSpeed	数据量依然巨大，需分布式训练能力；两者均可满足，根据团队熟悉度选择。
场景3：SFT微调（有GPU集群）	LLaMA-Factory 或 MS-SWIFT	支持大规模并行训练，且模型兼容性强，便于快速适配多种基座模型。
场景4：SFT微调（单卡/消费级GPU）	Unsloth	显存优化极佳（减少约80%），可在有限资源下完成高效微调。
场景5：RLHF/偏好对齐（中小规模）	TRL（算法研究）或 LLaMA-Factory（快速验证）	TRL代码清晰易改，适合实验新算法；LLaMA-Factory提供零代码WebUI，快速验证想法。
场景6：RLHF/偏好对齐（大规模生产）	OpenRLHF（性价比高）或 verl（超大规模）	OpenRLHF基于Ray可组合多张消费级显卡；verl原生支持万亿参数MoE的全量RLHF。
场景7：多模态模型微调	MS-SWIFT（400+多模态支持）或 LLaMA-Factory（100+支持）	两者均对视觉语言模型（VLM）有良好支持，覆盖主流多模态架构。

总结

LLM的训练是一个从**方法论**（训练范式）到**工具箱**（训练架构）的系统工程：

训练范式解决“做什么”的问题：

- 预训练 → 持续预训练 → 监督微调 → 偏好对齐，构成了从“能说话”到“说好话”的完整链路

训练架构解决“用什么做”的问题：

- Megatron-LM和DeepSpeed是支撑万亿参数训练的“重型基建”
- LLaMA-Factory、MS-SWIFT、Unsloth是让开发者高效微调的“生产力工具”

作者提示: 个人观点，仅供参考

大模型必知必会：一文带你了解大模型的Scaling Law

QinHsiu 阿Q的AI笔记 2026年8月23日 07:45 北京

2026年，关于Scaling Law的两派之间的争议非但没有平息，反而愈演愈烈。一方高呼“Scaling Law已死”，另一方坚持“Scaling从未失效”。双方各有论据。

实际Scaling Law从未失效，它只是从“放之四海而皆准的真理”变成了“需要因地制宜的工具”。预训练的边际收益在递减，但后训练和推理阶段的Scaling才刚刚开始。

智谱AI创始人唐杰在2026年8月19日的一段论述精准概括了这一演变：“万亿参数这一轮，回头來看，是整个领域一起走了一段弯路，然后又一起折返。”

一、Scaling Law起源

1.1 Kaplan Scaling Law (2020)：参数为王的公式与代码

2020年，OpenAI的Kaplan团队在论文《Scaling Laws for Neural Language Models》中发现：模型的测试损失与参数规模、数据集规模和计算量之间遵循幂律关系。

经典公式表达为：

$$L(N, D) = A/N^\alpha + B/D^\beta + L_\infty$$

其中N是模型参数量，D是训练数据量， α 和 β 是幂律指数， L_∞ 是不可约损失。用公式表示就是：

$$L(x) = E + A/x^\alpha$$

其中x可以是N、D或C。把模型训练的损失画在对数坐标上，它随着参数量、数据量、算力的增加，呈一条直线下降。

Kaplan团队的核心结论是：

$$N_{\text{opt}} \propto C^{0.73}$$

算力翻10倍，5.5倍给模型、1.8倍给数据。这一结论直接推动了GPT-3（175B参数，300B Token）、Gopher（280B参数，300B Token）等超大规模模型的集中涌现。

用Python实现Kaplan Scaling Law的拟合与预测：

```
1 import numpy as np
2 import pandas as pd
```

```

3 from scipy.optimize import curve_fit
4
5 def kaplan_scaling_law(x, E, A, alpha):
6     """
7     Kaplan Scaling Law:  $L(x) = E + A / x^\alpha$ 
8     其中x可以是N（参数量）、D（数据量）或C（计算量）
9     """
10    return E + A * (x ** (-alpha))
11
12 # 模拟实验数据：在不同参数量下训练的损失
13 N_values = np.logspace(7, 10, 20) # 10M 到 10B 参数
14 # 模拟损失：加上一些噪声使其更真实
15 true_loss = kaplan_scaling_law(N_values, E=1.2, A=100.0, alpha=0.07)
16 noise = np.random.normal(0, 0.02, size=len(N_values))
17 observed_loss = true_loss + noise
18
19 # 拟合Scaling Law
20 popt, pcov = curve_fit(kaplan_scaling_law, N_values, observed_loss,
21                        p0=[1.0, 50.0, 0.07])
22 E_fit, A_fit, alpha_fit = popt
23
24 print(f"拟合结果: E={E_fit:.3f}, A={A_fit:.1f}, alpha={alpha_fit:.3f}")
25 # 拟合结果: E≈1.198, A≈98.7, alpha≈0.069
26
27 # 预测100B参数模型的损失
28 N_pred = 1e11
29 loss_pred = kaplan_scaling_law(N_pred, E_fit, A_fit, alpha_fit)
30 print(f"预测100B参数模型损失: {loss_pred:.4f}")
31
32 # 计算最优参数分配 (Kaplan:  $N_{opt} \propto C^{0.73}$ )
33 def kaplan_optimal_allocation(C):
34     """给定计算预算C, 返回Kaplan建议的最优N和D"""
35     #  $C \approx 6ND$ ,  $N_{opt} \propto C^{0.73}$ ,  $D_{opt} \propto C^{0.27}$ 
36     N_opt = (C / 6) ** 0.73
37     D_opt = (C / 6) ** 0.27
38     return N_opt, D_opt
39
40 C = 1e21 # 1e21 FLOPs
41 N_opt, D_opt = kaplan_optimal_allocation(C)
42 print(f"Kaplan建议: N={N_opt/1e9:.1f}B, D={D_opt/1e9:.1f}B")

```

这个实现虽然只有几十行，但它揭示了Scaling Law的核心价值：**先跑几个小模型，拟合出那条直线，往右外推，就能预估大模型训出来的表现，不用真花几亿美元把大模型训完才知道行不行。**

1.2 Chinchilla Scaling Law (2022)：数据与参数的重新平衡

2022年，DeepMind的Hoffmann团队对**400余个模型**重新开展实验后发现，算力最优的分配应接近**每参数20个Token**，且随着算力持续扩大，参数量与数据量应以大致相同的速度共同增长。

Chinchilla的损失函数为：

$$L(N, D) = E + A/N^\alpha + B/D^\beta$$

Chinchilla的核心结论是：

$$N_{\text{opt}} \propto C^{0.50}$$

模型和数据应该等比增长。工程师们后来把它提炼成一个简单的数字：**最优token和参数比大约20:1**。

DeepMind做了一场正面对决：自家的Gopher，280B参数配300B Token；Chinchilla，70B参数配1.4T Token。两个模型用了相同的算力。结果是**Chinchilla-70B在几乎所有基准测试上击败了Gopher-280B。70B打败280B**。这个结果让整个行业重新思考配比问题。

面试考点：为什么Kaplan和Chinchilla的结论相反

Kaplan (2020) 说模型应比数据涨得快)，Chinchilla (2022) 说应等比增长。

偏差根源有三：

实验规模不足：Kaplan最大模型仅15亿参数，小规模拟合外推至万亿级别时产生系统性偏差

统计口径差异：Kaplan排除了embedding层参数计数，在小模型上影响极大

拟合代码bug：2024年Epoch AI复现Chinchilla时发现损失函数取均值而非求和，导致优化器提前停止

修正后再次印证了等比增长的结论。翁荔总结：Scaling Law不是物理定律，是对工程细节高度敏感的观测性指南。

用Python实现Chinchilla Scaling Law：

```
1  # Chinchilla 论文拟合的常数
2  CHINCHILLA_PARAMS = {
3      'E': 1.69,
4      'A': 406.4,
5      'B': 410.7,
6      'alpha': 0.34,
7      'beta': 0.28
8  }
9
10 def chinchilla_loss(N, D, E=1.69, A=406.4, B=410.7, alpha=0.34, beta=0.28):
11     """Chinchilla缩放定律损失函数 L(N,D) = E + A/N^alpha + B/D^beta"""
12     return E + A / (N ** alpha) + B / (D ** beta)
13
14 def chinchilla_optimal_allocation(C):
```

```

15     """
16     Chinchilla最优分配:  $N_{opt} \propto C^{0.5}$ ,  $D_{opt} \propto C^{0.5}$ 
17     """
18     N_opt = (C / 6) ** 0.5
19     D_opt = (C / 6) ** 0.5
20     return N_opt, D_opt
21
22 # 对比Kaplan和Chinchilla的建议
23 C = 1e21
24 N_kaplan, D_kaplan = kaplan_optimal_allocation(C)
25 N_chinchilla, D_chinchilla = chinchilla_optimal_allocation(C)
26
27 print(f"计算预算 {C:.0e} FLOPs")
28 print(f"Kaplan建议: N={N_kaplan/1e9:.1f}B, D={D_kaplan/1e9:.1f}B, ratio={N_
29 print(f"Chinchilla建议: N={N_chinchilla/1e9:.1f}B, D={D_chinchilla/1e9:.1f}B,

```

1.3 翁荔的“检讨书”：Scaling Law远比想象的脆弱

2026年6月，前OpenAI安全研究副总裁翁荔在停更13个月后，发表了万字长文《Scaling Laws, Carefully》，将支撑大模型行业数百亿美元投入的Scaling Law从头拆到尾。

翁荔揭示了几个关键问题：

第一，OpenAI和DeepMind给出相反答案的根本原因。 2020年OpenAI说模型该比数据涨得快，2022年DeepMind说两边得一起涨。分歧的根源是一个参数统计口径的差异，加上实验规模不够大。

第二，Chinchilla的公式里藏着bug。 2024年有人逐行复现DeepMind的代码时发现：损失函数取了均值而不是求和，导致优化器提前停了，输出的参数根本不是最优解。

第三，小模型规律外推要非常小心。 这条曲线是在相对小的模型上拟合出来的，外推到万亿参数级别时，一个四舍五入的差别就能让结论差出一大截。

第四，更根本的问题：数据快用完了。 公式默认数据可以无限供应，但高质量文本是有限的。翁荔指出，高质量文本数据预计**2026-2028年枯竭**。

1.4 Skaling (2026)：Kaplan与Chinchilla的第三次统一

2026年8月7日，Mathurin Videau等人在arXiv上发表了论文《**Skaling: Chinchilla's Exponents Meet Kaplan's Coupling**》，提出了对Scaling Law的第三次修正。

标准Scaling Law的根本缺陷在于其底层假设——**模型规模与训练数据量对损失的影响相互独立**。在数据极度匮乏和过度训练这两种极端情形下，这一假设会导致系统性地低估或高估模型损失。

Skaling Law引入了一个**单一交互指数 γ** ，将模型容量与数据量耦合起来：

这个简洁的扩展在插值与外推两种情形下，均可将**平均绝对百分比误差（MAPE）降低1.5至3倍**。当与一种仅限于低计算开销场景的稀疏网格策略相结合时，仅需约为均匀网格扫描所需**计算量的十分之一**，即可实现对完整网格的高精度外推。

用Python实现Scaling Law:

```
1 def skaling_law(N, D, E, A, B, C, alpha, beta, gamma, delta):
2     """
3     Skaling Law: 通过交互指数 $\gamma$ 耦合模型容量与数据量
4      $L(N,D) = E + A/N^\alpha + B/D^\beta + C/(N^\gamma * D^{(1-\gamma)})^\delta$ 
5     """
6     interaction_term = C / ((N ** gamma) * (D ** (1 - gamma))) ** delta
7     return E + A / (N ** alpha) + B / (D ** beta) + interaction_term
8
9 # 拟合Skaling Law (使用scipy.optimize.minimize)
10 def fit_skaling_law(training_runs):
11     """
12     training_runs: list of dict with keys 'N', 'D', 'loss'
13     返回拟合的Skaling参数
14     """
15     from scipy.optimize import minimize
16
17     def objective(params):
18         E, A, B, C, alpha, beta, gamma, delta = params
19         total_loss = 0
20         for run in training_runs:
21             pred = skaling_law(run['N'], run['D'], E, A, B, C, alpha, beta, gamma, delta)
22             total_loss += (pred - run['loss']) ** 2
23         return total_loss / len(training_runs)
24
25     # 初始猜测
26     initial = [1.69, 406.4, 410.7, 100.0, 0.34, 0.28, 0.5, 0.5]
27     result = minimize(objective, initial, method='L-BFGS-B')
28     return result.x
29
30 # 用Skaling预测小规模实验外推
31 def skaling_predict_large_scale(small_scale_runs, target_N, target_D):
32     """从小规模实验外推大规模模型性能"""
33     params = fit_skaling_law(small_scale_runs)
34     return skaling_law(target_N, target_D, *params)
```

面试考点：Scaling Law是什么

Scaling Law是OpenAI在2020年发现的经验规律：模型性能（测试损失）与参数量N、数据量D、计算量C之间遵循幂律关系。核心公式为 $L(N,D)=E + A/N^\alpha + B/D^\beta$ 。直观理解：投入越多资源，性能提升越可预测，但边际收益递减。

更重要的是，模型性能主要与N、D、C三者相关，与具体结构（层数/深度/宽度）基本无关——固定总参数量调整架构，性能差距多在2%以内。这让工程师可以在不改变设计的前提下，通过缩放资源可靠地预测性能。

二、数字配比的演进

2.1 配比数字的背后——如何用公式计算

在深入表格之前，先理清**配比数字**是如何算出来的。

训练一个Transformer模型所需的**总计算量C**由参数量N和数据量D共同决定：

$$C \approx 6 \times N \times D$$

这个6倍系数来源于：每个Token的前向传播约2N次浮点运算，反向传播约4N次浮点运算。

在给定C的前提下，如何最优分配N和D，正是Scaling Law要回答的问题。

面试考点：Flops计算方式，为什么是 $C \approx 6ND$

训练一个Transformer模型的总计算量 $C \approx 6ND$ 。系数6的来源：每个Token前向传播约2N次浮点运算，反向传播约4N次，合计6N。

实战应用：

给定 $C=1e21$ FLOPs，Chinchilla建议参数量设置为 $N=(C/6)^{0.5} \approx 12.9B$ ，数据大小设置为 $D=(C/6)^{0.5} \approx 12.9B$ Token

估算训练成本：A100 理论 312 TFLOPS，实际利用率约 50%，训练 $1e21$ FLOPs 需约 $1e21/(312e12 \times 0.5 \times 3600 \times 24) \approx 74$ GPU天

Chinchilla的最优分配推导：

$$L(N, D) = E + A/N^\alpha + B/D^\beta$$

在约束 $C \approx 6ND$ 下，对损失函数求偏导并令其为零，得到最优分配：

$$N_{\text{opt}}(C) = G \times (C/6)^{\alpha/(\alpha+\beta)}$$

$$D_{\text{opt}}(C) = G^{-1} \times (C/6)^{\beta/(\alpha+\beta)}$$

代入DeepMind拟合的 $\alpha=0.34$ 、 $\beta=0.28$ ：

$$N_{\text{opt}}(C) \propto C^{0.34/0.62} = C^{0.548} \approx C^{0.5}$$

$$D_{\text{opt}}(C) \propto C^{0.28/0.62} = C^{0.452} \approx C^{0.5}$$

由此推出最优配比约为：

$$D/N \approx 20:1$$

这是“每1个参数对应约20个训练Token”的数学根源。

用代码计算任意模型的最优配比：

```

1  def compute_optimal_ratio(C, alpha=0.34, beta=0.28):
2      """
3      给定计算预算C（FLOPs），返回Chinchilla最优的N和D以及比值
4      """
5      # 由  $C \approx 6ND$  和最优指数关系  $N_{\text{opt}} \propto C^{(\alpha/(\alpha+\beta))}$ ,  $D_{\text{opt}} \propto C^{(\beta/(\alpha+\beta))}$ 
6      # 先求比例常数（由Chinchilla的基准点拟合）
7      # 以Chinchilla-70B (N=70e9, D=1.4e12) 作为参考点
8      N_ref = 70e9
9      D_ref = 1.4e12
10     C_ref = 6 * N_ref * D_ref # ≈ 5.88e23
11
12     # 根据比例关系外推
13     exponent_N = alpha / (alpha + beta) # 0.548
14     exponent_D = beta / (alpha + beta) # 0.452
15
16     scale_N = N_ref / (C_ref ** exponent_N)
17     scale_D = D_ref / (C_ref ** exponent_D)
18
19     N_opt = scale_N * (C ** exponent_N)
20     D_opt = scale_D * (C ** exponent_D)
21     ratio = D_opt / N_opt
22     return N_opt, D_opt, ratio
23
24 # 验证Chinchilla自身
25 C_chinchilla = 6 * 70e9 * 1.4e12
26 N_opt, D_opt, ratio = compute_optimal_ratio(C_chinchilla)
27 print(f"Chinchilla预算下: N={N_opt/1e9:.1f}B, D={D_opt/1e12:.2f}T, ratio={ratio:.1f}")
28 # 输出应为 N≈70B, D≈1.4T, ratio≈20
29
30 # 计算不同计算预算下的最优配比
31 for C in [1e21, 1e22, 1e23, 1e24]:
32     N_opt, D_opt, ratio = compute_optimal_ratio(C)
33     print(f"C={C:.0e}: N={N_opt/1e9:.1f}B, D={D_opt/1e12:.2f}T, ratio={ratio:.1f}")

```

2.2 实战配比全景表

模型	参数规模 训练Token Token参数比	关键特征
GPT-3	175B 300B 1:7	Kaplan时代，严重训练不足
Gopher	280B 300B 1:1	参数过剩的典型
Chinchilla	70B 20T 20	Chinchilla最优配比
LLaMA	65B 21T 21.5	接近Chinchilla
Llama 2 7B	7B 2T 285.7	推理成本驱动
Gemma 2 9B	9B 8T ~889	刻意过训练
Llama 3 8B	8B 15T 1875	行业极端
DeepSeek-V3	671B(37B激活) 14.8T 22.1	MoE架构
DeepSeek-V4	1.6T 30+ T 18.75	重返同步放大

这张表揭示了几个关键趋势。

从GPT-3到Chinchilla的“校正”： GPT-3的Token/参数比仅为1.7，远低于Chinchilla建议的20:1。用上述公式计算可知，若GPT-3采用70B参数而不是175B，在相同计算预算下可训练约3.5T Token，损失会低得多。这意味着GPT-3的参数规模相对于训练数据是“过大”的——有大量参数没有被充分训练。

从Chinchilla到Llama的“过度训练”： Llama 2 7B的Token/参数比达到**285.7**，Llama3 8B更是高达**1875**。这已经远超Chinchilla的建议。行业选择“过度训练”小模型，是因为**推理成本正在成为比训练成本更重要的约束**。唐杰指出：Chinchilla优化的目标是“训练一次、评测一次”的模型，而今天的主流模型每天可能被调用数十亿次，推理成本在全生命周期中远超一次性训练成本。用公式表达：推理总成本 ≈ 推理次数 × 每次推理的计算量（与N·D相关），小模型N小，即使D大，总推理成本仍远低于大模型。

MoE架构的“激活参数”逻辑： DeepSeek-V3总参数671B，但每个Token只激活37B。若按总参数计算配比， $D/N=14.8T/671B\approx 22$ ，勉强符合Chinchilla；但按激活参数计算，该比例跃升至 $14.8T/37B\approx 400$ 。这揭示了MoE的本质：**用总参数换知识容量，用激活参数换推理效率**——决定推理速度的是激活参数，决定知识存储的是总参数。

2.3 数据受限下的混合预训练Scaling

2026年8月，Apple研究团队发表了《Scaling Laws for Mixture Pretraining Under Data Constraints》，在**超过2000个语言模型训练运行上**系统研究了数据受限场景下的Scaling行为。

核心发现：**稀缺目标语料可以复用15-20次**，最优重复次数取决于目标数据大小、计算预算和模型规模。论文提出了**重复感知的混合Scaling Law**，考虑了重复目标Token的递减价值和通用数据的正则化作用。这一框架为数据受限场景下的预训练提供了**实用的混合配置建议**。

面试考点：Scaling Law在数据受限情况下是否成立

经典Scaling Law的前提是“数据无限且不重复”。但高质量文本数据预计2026-2028年枯竭。

Apple 2026年研究（>2000个模型）发现：稀缺目标语料可复用15-20次，最优重复次数取决于数据大小、计算预算和模型规模。重复数据的有效价值呈指数衰减，每多训练一轮边际收益急剧递减。

这解释了行业为何集体转向强化学习、测试时计算和合成数据——不是在放弃Scaling，而是在寻找数据墙之后的新Scaling方向。

2.4 SmolLM2：6500 Token/参数的极端案例

一个更极端的案例是SmolLM2 1.7B——在约11T Token上训练，每参数对应约6500个Token。这比Llama3 8B的1875还要高出数倍。这说明“过度训练”的趋势仍在加剧。小模型通过极其大量的数据训练，可以在推理成本极低的前提下获得相当不错的能力——这对边缘计算、移动端部署等场景具有极高的实用价值。

2.5 动态算力预算：训练损失如何外推

用代码模拟不同配比下的训练损失，可以直观看到“欠训练”和“过训练”的代价：

```
1 def compute_loss(N, D, E=1.69, A=406.4, B=410.7, alpha=0.34, beta=0.28):
2     """计算给定N和D下的Chinchilla预测损失"""
3     return E + A / (N ** alpha) + B / (D ** beta)
4
5 # 固定计算预算 C = 6ND
6 C_fixed = 6 * 70e9 * 1.4e12 # Chinchilla的预算
7
8 # 探索不同N配比下的损失
9 N_range = np.logspace(9, 11, 30) # 1B 到 100B
10 losses = []
11 for N in N_range:
12     D = C_fixed / (6 * N) # 保持C固定
13     loss = compute_loss(N, D)
14     losses.append(loss)
15
16 # 找到最优N
17 opt_idx = np.argmin(losses)
18 N_opt = N_range[opt_idx]
19 print(f"最优N: {N_opt/1e9:.1f}B, 对应D: {C_fixed/(6*N_opt)/1e12:.2f}T")
20 # 输出应接近 70B, 1.4T
```

这段代码展示了Chinchilla公式的实践价值：在固定算力下，用公式扫一遍不同的N配比，就能找出最优的规模和数据量，而不必真的做几百次实验。

三、后训练Scaling：RL正在改写规则

3.1 ACL 2026: RL后训练的Scaling Law

2026年，中科大与上海AI Lab的团队在**ACL 2026**上发表了《Scaling Behaviors of LLM Reinforcement Learning Post-Training》。研究团队在**Qwen2.5全系列密集模型（0.5B-72B）** 上开展了**63组实验**，首次全面刻画了RL后训练的Scaling行为。

实验设计：训练统一采用VeRL分布式RL平台和GRPO算法，每个配置重复3次。训练数据来自guru-RL-92k的数学子集（约5.4万道题，由OR1、DeepScaler、DAPO三个数据集组成），按难度排序实现课程学习。

三个关键发现：

发现一：预测性Scaling定律。 模型的测试损失L与训练资源X（计算量C或数据量D）遵循稳健的幂律关系，可**准确预测未见模型规模和剩余训练轨迹**。

发现二：学习效率饱和。 更大模型在RL后训练中持续获得更好的学习效率，但效率系数k(N)呈现**饱和趋势**——当N超过32B参数后，边际收益逐渐递减。

发现三：数据复用的有效性。 在数据受限场景下，重复使用高质量数据极为有效——性能在 **$\tau=25$ （25倍复用）** 时保持稳定，仅在极端复用（ $\tau=100$ ）时才出现适度过拟合。

用代码实现RL后训练的Scaling分析：

```
1 # 模拟RL后训练Scaling实验
2 import numpy as np
3 from scipy.optimize import curve_fit
4
5 def rl_scaling_law(X, L_inf, A, alpha):
6     """RL后训练Scaling:  $L(X) = L_{inf} + A / X^{\alpha}$ """
7     return L_inf + A * (X ** (-alpha))
8
9 # 模拟不同模型规模下的RL训练数据
10 model_sizes = [0.5e9, 1.5e9, 7e9, 14e9, 32e9, 72e9] # 0.5B 到 72B
11 training_steps = np.logspace(3, 5, 20) # 1K 到 100K步
12
13 # 对于每个模型规模，模拟RL训练损失
14 def simulate_rl_experiment(N, steps):
15     # 大模型有更好的初始性能和更高的学习效率
16     base_loss = 0.5 / (N / 1e9) ** 0.1 # 随模型增大而降低
17     efficiency = 1 - np.exp(-N / 32e9) # 32B后饱和
18     return base_loss + 0.8 * (steps ** (-0.15 * efficiency))
19
20 # 分析学习效率饱和
21 for N in model_sizes:
22     steps = np.logspace(3, 5, 10)
23     losses = [simulate_rl_experiment(N, s) for s in steps]
24     # 拟合RL Scaling Law
25     popt, _ = curve_fit(rl_scaling_law, steps, losses, p0=[0.1, 1.0, 0.1])
```

3.2 GLM-5.3：一次控制变量实验

智谱AI的GLM-5.3提供了一个完美案例。唐杰在X平台发文指出，GLM-5.3与上一代GLM-5.2共用**相同基座、相同架构、相同的753B总参数与40B激活参数**——唯一的变量是**增加了一个月的长时域环境训练与强化学习（RL）后训练**。

结果是：GLM-5.2在AA评测指数中获得**53分**，GLM-5.3获得**60分**，提升**7分**。唐杰直言，由此带来的能力提升“并非微小”。

这一实验结论的核心在于：**当基座模型参数规模已足以承载足够的知识，额外的能力提升往往来自“拨动其他旋钮”——尤其是后训练阶段，而非继续堆砌参数。**

3.3 CoScale-RL：更高效的后训练Scaling

另一项2026年的工作**CoScale-RL**提出了一个新的Scaling策略：**为每个问题收集多个解决方案，而不是简单地扩大数据集**。核心思想是扩展开销计算（rollout computation），而非单纯增加数据量。

该方法在四个基准上平均提升了**3.76个百分点的准确率**。研究者明确指出：**RL的Scaling Law与预训练不同**。

面试考点：什么是Post-Training Scaling，什么是Pre-training Scaling

到2026年，高质量预训练数据已基本耗尽，模型能力提升更多来自预训练之后：指令微调、RLHF、DPO、RL后训练和推理时计算。

ACL 2026论文在Qwen2.5全系列（0.5B-72B）上做了63组RL后训练实验，发现：

测试损失与训练资源遵循稳健幂律，可准确预测未见模型规模

学习效率在>32B后饱和

数据受限时高质量数据可复用25次仍保持稳定

GLM-5.3控制变量实验（相同基座，仅增加一个月RL后训练）证明了这一点：AA评测指数从53分提升到60分。

四、推理Scaling

4.1 测试时计算的缩放规律

ICLR 2025的一篇论文揭示了推理阶段Scaling的惊人潜力：**最优地扩展测试时计算，在推理任务上可能比扩展参数更有效**。

测试时计算最优缩放被定义为：在给定计算预算C下，选择超参数 θ 最大化特定数据集D和LLM F上的性能：

$$\theta^*(C) = \arg \max_{\theta} \text{Performance}(F, D, \theta, C)$$

4.2 推理Scaling的代码实现

Meta在2026年4月发布了《Scaling Test-Time Compute for Agentic Coding》论文，将每次 rollout 转换为**结构化摘要**，保留关键假设、进展和失败模式，同时丢弃低信号痕迹。

```
1 class TestTimeScaling:
2     """测试时计算Scaling框架"""
3
4     def __init__(self, model, max_compute_budget=1000):
5         self.model = model
6         self.max_compute_budget = max_compute_budget
7
8     def solve_with_scaling(self, problem, strategy='best_of_n', n_samples=10):
9         """使用测试时计算扩展求解问题"""
10        if strategy == 'best_of_n':
11            # 多次采样取最优
12            candidates = [self.model.generate(problem) for _ in range(n_samples)]
13            return self.select_best(candidates)
14
15        elif strategy == 'self_verification':
16            # 生成-验证-修正循环
17            solution = self.model.generate(problem)
18            for _ in range(3):
19                verification = self.model.verify(problem, solution)
20                if verification['correct']:
21                    return solution
22                solution = self.model.revise(problem, solution, verification)
23            return solution
24
25        elif strategy == 'tree_search':
26            # 树搜索扩展推理路径
27            return self.tree_search(problem, depth=5, branching=3)
28
29    def tree_search(self, problem, depth, branching):
30        """基于树搜索的推理Scaling"""
31        # 维护推理路径树，每步扩展branching个候选
32        # 使用启发式评估选择最有希望的路径继续探索
33        pass
```

4.3 推理Scaling的风险

但2025年NeurIPS的Kinetics论文提出了重要的修正：**测试时计算在超过一定阈值的模型上比在小模型上更有效**——小模型在测试时Scaling中的有效性被显著高估了。**推理Scaling不是小模型的救命稻草，而是大模型的放大器。**

另一篇研究则指出了推理Scaling的“幻影”风险：**想得更多并不总是有帮助**——测试时Scaling存在明显的平台效应。

面试考点：Test-Time Scaling是什么

Test-Time Scaling指在推理阶段增加计算资源（思维链、自洽性采样、树搜索等）来提升性能。

ICLR 2025研究显示：最优扩展测试时计算，在推理任务上可能比扩展参数更有效。

但2025年NeurIPS的Kinetics论文提出重要修正：

小模型的有效性被显著高估

测试时计算在超过阈值（约14B）的模型上更有效

新瓶颈是内存访问而非FLOPs

结论：推理Scaling不是小模型的救命稻草，而是大模型的放大器。

五、数据质量：被传统Scaling Law忽视的关键变量

5.1 ICLR 2026：质量感知的Scaling Law

ICLR 2026接收的论文《Scaling Laws Revisited: Modeling the Role of Data Quality in Language Model Pretraining》引入了一个无量纲的数据质量参数 Q ($Q \in (0, 1]$)。 $Q=1$ 代表完全干净且有代表性的数据，较小的 Q 值反映更多的污染或冗余。

质量感知的Scaling Law将损失表达为模型规模 N 、数据量 D 和数据质量 Q 的联合函数如下：

$$L(N, D, Q) = E + A/N^\alpha + B/(Q \cdot D)^\beta + L_\infty$$

这意味着：**高质量数据可以大幅减少所需的模型规模和计算需求**。在专业领域（如医疗、金融、科研、教育、法律）——训练语料数量有限且质量参差不齐——这个框架尤其重要。

```
1 def quality_aware_scaling_law(N, D, Q, E=1.69, A=406.4, B=410.7, alpha=0.34,
2     """
3     质量感知的Scaling Law
4     Q: 数据质量参数 (0, 1], 1表示完全干净
5     """
6     effective_D = Q * D # 高质量数据等效于更多Token
7     return E + A / (N ** alpha) + B / (effective_D ** beta)
8
9 # 对比不同质量数据的Scaling效果
10 N = 70e9 # 70B参数
11 Q_values = [0.3, 0.5, 0.7, 1.0] # 不同质量
12
13 for Q in Q_values:
14     # 计算达到相同损失所需的数据量
15     # 假设目标损失L_target = quality_aware_scaling_law(N, D_base, 1.0)
16     D_base = 1.4e12 # Chinchilla的1.4T
17     L_target = quality_aware_scaling_law(N, D_base, 1.0)
18
19     # 求解低质量数据需要多少量
20     def loss_for_D(D):
21         return quality_aware_scaling_law(N, D, Q)
```

```
22 # 需要数值求解 D_required 使 loss_for_D(D_required) = L_target
23 print(f"Q={Q:.1f}: 需要 {D_base/Q/1e12:.1f}T Token 达到相同效果")
```

5.2 InfoLaw: 质量加权与重复的统一框架

ICML 2026的InfoLaw提出了一个更精细的框架。研究发现：**提高高质量数据的权重通常能提升性能，但过强的权重会增加数据重复并可能降低性能**。标准Scaling Law无法可靠地跨不同数据混合配方或重复数据场景进行外推。

5.3 数据质量干预如何重塑Scaling

ICLR 2026的另一篇论文《How Text Quality Interventions Reshape Neural Scaling Laws for LLMs》进行了**首次大规模实证研究**，考察了去重、启发式过滤、LLM引导重写等干预如何重塑Scaling行为。

六、合成数据的Scaling: 希望与陷阱

2026年8月的一篇大规模实证研究 (>1000个LLM, >10万GPU小时) 揭示:

关键发现: 仅用改写后的合成数据进行预训练，**并不比使用自然网页文本更快**；但**用1/3改写合成数据混合2/3自然网页文本，在更大数据预算下可达到相同验证损失的速度提升5-10倍**。

合成数据是一把双刃剑——它是数据瓶颈的潜在解药，但单纯依赖合成数据可能是死路一条。**混合策略**才是当前的最优解。

面试考点: 给定1e21 FLOPs预算，如何决定模型参数量和训练数据量

三步法:

第一步: 确定优化目标。如果追求“训练计算最优”，用Chinchilla法则；如果追求“推理成本最优”，用小模型+更多数据。

第二步: 计算最优分配。Chinchilla: $N=(C/6)^{0.5} \approx 12.9B$, $D=(C/6)^{0.5} \approx 12.9B$ Token。

第三步: 验证可行性。12.9B参数×12.9B Token, Token/参数比=1, 远低于20:1——说明预算不足以训练一个计算最优的模型，应优先保证数据量，选择更小的模型（如7B）配更多Token。

面试官追问: 如果预算是1e23 FLOPs呢? (答: $N \approx 129B$, $D \approx 129B$, Token/参数比=1, 仍然不足。Chinchilla-70B需要 $C \approx 5.88e23$ FLOPs。)

七、优化器的Scaling: Muon vs AdamW

Moonlight团队的研究发现，**Muon优化器在缩放定律实验中实现了约2倍的计算效率提升**

```
1 # Muon 优化器的关键实现
2 class MuonOptimizer:
3     """
4     Muon: 通过牛顿-舒尔茨迭代实现更高效的参数更新
5     关键改进: 添加权重衰减、仔细调整每个参数的更新尺度
6     """
7     def __init__(self, params, lr=3e-4, weight_decay=0.1, momentum=0.95):
```

```

8         self.params = params
9         self.lr = lr
10        self.weight_decay = weight_decay
11        self.momentum = momentum
12        self.mu = None # 动量缓冲区
13
14    def step(self):
15        for p in self.params:
16            if p.grad is None:
17                continue
18
19            # 1. 权重衰减
20            p.data.mul_(1 - self.lr * self.weight_decay)
21
22            # 2. 动量更新
23            if self.mu is None:
24                self.mu = [torch.zeros_like(p) for p in self.params]
25            self.mu[i].mul_(self.momentum).add_(p.grad)
26
27            # 3. 牛顿-舒尔茨迭代（核心：提升更新质量）
28            update = self.mu[i]
29            # 对update进行正交化/缩放处理
30            update = self.newton_schulz(update)
31
32            # 4. 应用更新
33            p.data.add_(update, alpha=-self.lr)
34
35    def newton_schulz(self, X, iterations=5):
36        """牛顿-舒尔茨迭代：近似矩阵的逆平方根"""
37        # 用于对梯度更新进行更好的缩放
38        pass

```

关键结论：基于Scaling Law结果，**Muon**以约一半的训练FLOPs达到与AdamW相当的性能。在相同计算预算下，选择更高效的优化器本身就是一种“Scaling”。

面试考点：如何用Scaling Law预测更大的模型

步骤一：在小规模（<1B参数）上训练多个模型，记录损失。

步骤二：拟合幂律函数 $L(N) = E + A/N^\alpha$ ，得到E、A、 α 。

步骤三：外推预测目标规模的损失。

注意风险（翁荔警示）：

小模型规律外推至万亿参数时，一个四舍五入的差别就能让结论差出一大截

拟合代码的微小差异（如均值vs求和）会系统性改变最优分配结论

数据墙：公式默认数据无限，但高质量文本预计2026-2028年枯竭

实战建议：用Skaling Law替代经典Scaling Law，可将外推误差降低1.5-3倍。

八、从理论到实践的完整检查清单

预训练阶段：

- 是否先在小规模（<1B参数）上拟合了自己的Scaling Law？
- 是否评估了数据质量参数Q对缩放行为的影响？
- 是否根据推理成本约束重新计算了最优N:D配比？
- 对于MoE架构，是否区分了总参数和激活参数？
- 数据配比是否针对目标任务进行了优化？
- 优化器选择是否考虑了Muon等高效替代方案？

后训练阶段：

- RL后训练的计算预算是否与模型规模匹配（>32B时注意效率饱和）？
- 数据受限时是否考虑了高质量数据的重复复用（ $\tau \leq 25$ ）？
- 是否用小规模实验验证了后训练方法的缩放行为？

推理阶段：

- 是否评估了测试时计算缩放对不同规模模型的有效性？
- 推理预算分配是否考虑了任务复杂度？

数据工程：

- 合成数据策略是否区分了源扩展和固定源合成？
- 混合预训练中稀缺目标数据是否考虑了15-20次复用的最优边界？

写在最后

回望Scaling Law的演进之路：2020年Kaplan说“参数为王”，2022年Chinchilla说“20:1最优”，2024年Llama 3用1875:1打破规则，2025年RL后训练Scaling证明“更大模型+更少步数”更优，2026年Skaling通过交互指数统一了Kaplan和Chinchilla，翁荔警示“数据墙将至”。

Scaling Law从来没有失效，它只是在不断进化。 每一次“法则被打破”，本质上是约束条件发生了变化——从“训练成本约束”到“推理成本约束”，从“数据充足”到“数据瓶颈”，从“预训练单一阶段”到“三阶段接力”。

唐杰老师的判断提供了一个有益的视角：**参数量仅是Scaling的一个维度，未来突破需要综合考量推理成本、MoE激活机制及后训练等多重变量。**

大模型必知必会：一文带你了解大模型自动化训练进展

原创 QinHsiu 阿Q的AI笔记 2026年8月27日 07:45 北京

2026年，一个判断正在从学术讨论变成工程现实：**大模型正在学习如何训练自己，AI研发的自动化闭环正在形成。**

2026年3月，Andrej Karpathy开源了**仅630行代码**的autoresearch。5月，面壁智能发布了**全球首个完全由AI编写的生产级训练框架ForgeTrain**。8月，ICLR 2026接收了**R-Zero**——一个从零数据开始、完全自主进化的推理模型框架。

一、什么是“LLM训练LLM”？

传统大模型训练依赖大量人工标注数据、人工反馈和工程师的手动调参。但随着模型能力逼近行业专家，基于标注反馈的有效信息在减少，而模型自主决策和执行复杂动作的能力在增强。

“LLM训练LLM”的核心思想是：**让模型在训练流程的各个环节中承担越来越主动的角色——自主生成数据、筛选有效信号、更新自身参数、评估输出结果，形成一个自我优化的闭环。**

二、Karpathy的autoresearch：630行的自我进化

2.1 核心理念

autoresearch的核心思想极简：**AI自己修改代码→跑5分钟短实验→看效果好不好→决定保留还是丢弃改动→循环。**

Karpathy在README里写了一段极具反讽意味的话：“智能体声称我们已处于代码库的第10,205代——反正没人能判断对错，因为‘代码’已经是一个自我修改的二进制文件，超出了人的理解范围。”

2.2 三个核心文件

整个仓库只有三个关键文件：

`prepare.py` ——固定常量，一次性数据准备。设置后全程不动。

`train.py` ——AI唯一可编辑的文件。包含完整的GPT模型、优化器（Muon + AdamW）和训练循环。AI可改层数、batch size、学习率、权重衰减等。

`program.md` ——纯文本文件，手动编写基线指令（研究方向、实验规则、参考依据）。需要手动迭代的是这个文件，AI迭代的是`train.py`

2.3 核心代码解析

train.py的核心结构（简化版）：

```
1 # autoresearch/train.py - AI自主修改的核心文件
```

```

2
3 import torch
4 from model import GPT, GPTConfig
5
6 # ===== 以下所有参数均可被AI修改 =====
7 config = GPTConfig(
8     vocab_size=50257,
9     n_layer=12,          # AI可改: 层数
10    n_head=12,           # AI可改: 注意力头数
11    n_embd=768,          # AI可改: 嵌入维度
12    block_size=1024,     # AI可改: 上下文长度
13    dropout=0.0,
14 )
15
16 learning_rate = 3e-4    # AI可改: 学习率
17 weight_decay = 0.1      # AI可改: 权重衰减
18 batch_size = 32         # AI可改: 批大小
19 total_batches = 10000   # AI可改: 总训练步数
20 # =====
21
22 def train():
23     model = GPT(config)
24     optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
25     for step in range(total_batches):
26         x, y = get_batch(batch_size)
27         logits, loss = model(x, y)
28         loss.backward()
29         optimizer.step()
30         if step % 100 == 0:
31             val_loss = evaluate(model)
32             print(f"step {step}: val_loss={val_loss}")

```

AI的修改方式：AI Agent读取 program.md 中的指令，在 train.py 中做**针对性的单点修改**（通常每次1-2处），然后启动5分钟训练，用 val_bpb （validation bits per byte）打分，分数更低则保留修改，更高则放弃。

两条铁律：每次实验纯训练时间固定为5分钟；只看 val_bpb 指标，数值越低代表模型越好。

2.4 实战效果

在一轮近250次的自主探索中，AI最终筛选并保留了**29次有效的优化改进**。“Time to GPT-2”从2.02小时降至**1.80小时**，实现**11%效率提升**，且改进可迁移到更大模型。发布不到48小时，斩获**9.5k星标**。

三、PostTrainBench：AI能否自主完成模型后训练？

3.1 什么是PostTrainBench?

2026年3月，图宾根大学、马克斯·普朗克智能系统研究所和Thoughtful Lab联合发布了**PostTrainBench**——第一个专门测量LLM Agent能否自主完成模型后训练的基准。

测试规则：给一个前沿编码智能体（Claude Code、Codex CLI或Gemini CLI）一个基座模型和一个目标基准，一块H100 GPU，**10小时**，从零搭建训练pipeline。

四个关键设计原则：

- **端到端：**智能体必须从头构建整个训练流程
- **自主：**智能体对数据源、训练方法和实验策略拥有完全自主权
- **资源受限：**每次运行限制在单张H100 GPU上10小时内
- **完整性保持：**智能体不得在基准测试数据上训练或修改评估框架

PostTrainBench涵盖**7个基准**（推理、工具使用、知识、数学、健康、代码）和**4个基座模型**（Qwen3-1.7B、Qwen3-4B、SmolLM3-3B、Gemma-3-4B）。

3.2 核心代码实现

PostTrainBench的核心是**评估智能体能否自主完成LLM后训练**。Agent被赋予一个评估脚本和10小时GPU时间：

```
1  # PostTrainBench 智能体接口（简化）
2  # 参考: https://github.com/aisa-group/PostTrainBench
3
4  class PostTrainAgent:
5      def __init__(self, base_model, benchmark, gpu_budget_hours=10):
6          self.base_model = base_model
7          self.benchmark = benchmark
8          self.gpu_budget = gpu_budget_hours * 3600
9          self.start_time = time.time()
10
11     def run(self):
12         """智能体自主完成整个后训练流程"""
13         # 1. 智能体自主决定数据源
14         dataset = self.select_dataset()
15
16         # 2. 智能体自主编写训练代码
17         train_script = self.write_training_code(dataset)
18
19         # 3. 智能体自主执行训练
20         checkpoint = self.execute_training(train_script)
21
22         # 4. 智能体自主评估
23         score = self.evaluate(checkpoint, self.benchmark)
24         return checkpoint, score
```

PostTrainBench——中科院自动化所发布的无梯度版本，让Agent在**不能使用梯度**的条件下完成后训练。Agent只能通过搜索程序、构造候选checkpoint、利用返回分数规划下一步实验来完成优化。

3.3 实战结果

排名	Agent	平均分
官方指令模型基线	51.1	
1	Opus 4.6 + Claude Code	23.2
2	Gemini 3.1 Pro + OpenCode	21.6
3	GPT-5.2 + Codex CLI	21.4

关键结论：最佳AI Agent得分23.2%，约为基座模型平均得分7.5%的**3倍**，但算法团队可达到**51.1%**。差距显著，但正在迅速缩小——Claude Sonnet 4.5在2025年9月得分为9.9%，而GPT-5.2几个月后就达到了21.4%。

四、R-Zero：从零数据开始的自我进化

4.1 核心思想

ICLR 2026接收的**R-Zero**提出了一个更激进的问题：**完全不依赖任何预定义任务和标签，让模型从零开始自我进化。**

R-Zero从一个基座LLM出发，初始化两个独立模型：

- Challenger（挑战者）** ——被奖励提出接近Solver能力边缘的任务。
- Solver（求解者）** ——被奖励解决Challenger提出的越来越难的任务。

两者独立优化，通过交互协同进化。

4.2 核心代码实现

```
1 # R-Zero 核心训练循环（简化自官方代码）
2 # https://github.com/Chengsong-Huang/R-Zero
3
4 class RZeroTrainer:
5     def __init__(self, base_model):
6         # 从同一个基座模型初始化两个角色
7         self.challenger = base_model.copy() # 出题者
8         self.solver = base_model.copy()     # 解题者
9         self.curriculum = []                # 自适应课程
10
11     def train_iteration(self):
```

```

12     # 1. Challenger生成题目（在Solver能力边缘）
13     problem = self.challenger.generate_problem(
14         difficulty=self.estimate_solver_capability()
15     )
16
17     # 2. Solver尝试解题
18     solution = self.solver.solve(problem)
19
20     # 3. 评估（多数投票作为伪标签）
21     is_correct = self.majority_vote(problem, solution)
22
23     # 4. 使用GRPO交替训练
24     if is_correct:
25         # Solver被奖励：解决了难题
26         self.solver.train_on(problem, solution, reward=+1)
27     else:
28         # Challenger被奖励：生成了Solver解不出的题
29         self.challenger.train_on(problem, reward=+1)
30
31     # 5. 更新课程（动态调整难度）
32     self.curriculum.append(problem)
33     self.update_difficulty()

```

快速开始：

```

1  git clone https://github.com/Chengsong-Huang/R-Zero.git
2  cd R-Zero
3  pip install -r requirements.txt
4  export STORAGE_PATH="/path/to/your/storage"

```

4.3 实战效果

R-Zero在Qwen3-4B-Base上：

- 数学推理基准提升 **+6.49**
- 通用领域推理基准提升 **+7.54**

整个过程不需要任何人工题目和标签。R-Zero的意义在于：**证明了LLM可以在完全没有标注数据的情况下，通过自我博弈实现推理能力的持续提升。**

五、SePT：最简单的自训练方法

5.1 核心思想

2026年4月，香港中文大学（深圳）、上海交大等机构提出了**SePT (Self-evolving Post-Training)**。

核心思想极简：

1. **反复采样问题**（用模型自身在指定采样温度下生成）
2. **在自生成的回答上训练模型**（标准SFT）
3. **更新后的模型生成下一轮训练数据**
4. **循环迭代**

不依赖奖励模型、验证器或额外教师信号。

5.2 核心代码实现

```
1  # SePT 核心训练循环（简化自官方代码）
2  # https://github.com/ElementQi/SePT
3
4  class SePTTrainer:
5      def __init__(self, model, train_data, num_iterations=3):
6          self.model = model
7          self.train_data = train_data
8          self.num_iterations = num_iterations
9          self.sample_temperature = 0.7
10
11     def train(self):
12         current_model = self.model
13
14         for iteration in range(self.num_iterations):
15             # 1. 用当前模型生成回答
16             synthetic_data = []
17             for question in self.train_data:
18                 responses = current_model.generate(
19                     question,
20                     temperature=self.sample_temperature,
21                     num_samples=8
22                 )
23                 synthetic_data.append((question, responses))
24
25             # 2. 在自生成的回答上做标准SFT
26             filtered_data = self.filter_best_responses(synthetic_data)
27
28             # 3. 更新模型
29             current_model = self.sft_train(current_model, filtered_data)
30
31         return current_model
```

三个关键设计：温度解耦、标准SFT训练、最新模型自生成数据。

5.3 实战效果

SePT在Qwen2.5-Math-7B上，6个数学基准平均Pass@1提升**10个百分点**。与GRPO对比：

设置	SePT	GRPO
OTM数据集	55.2	56.6
DSR数据集	55.0	59.1

在某些设置下，SePT的Pass@1**略高于GRPO**（40.8 vs 39.5）。

六、LANCE：模型成为自己的数据工程师

6.1 核心思想

EMNLP 2025的**LANCE (LANGuage models as Continuous self-Evolving data engineers)** 让LLM通过**自主生成、清洗、审查和带偏好信息标注数据**来训练自己。

传统方法依赖专家标注数据，在低资源场景下尤其困难。LANCE的核心创新是**让LLM自己成为持续进化的数据工程师**。

6.2 核心代码实现

```
1  # LANCE 核心流程（简化自官方代码）
2  # https://github.com/Control-derek/LANCE
3
4  class LANCETrainer:
5      def __init__(self, model):
6          self.model = model
7          self.data_pipeline = DataPipeline()
8
9      def evolve(self, num_rounds=5):
10         for round in range(num_rounds):
11             # 1. 自主生成数据
12             raw_data = self.model.generate_data(
13                 topics=self.get_weak_topics(),
14                 num_samples=1000
15             )
16
17             # 2. 自主清洗数据
18             cleaned_data = self.model.clean_data(raw_data)
19
20             # 3. 自主审查数据
```

```
21         reviewed_data = self.model.review_data(cleaned_data)
22
23         # 4. 自主标注偏好信息
24         annotated_data = self.model.annotate_preferences(reviewed_data)
25
26         # 5. 在标注数据上训练
27         self.model = self.finetune(self.model, annotated_data)
28
29         # 6. 评估并更新弱点分析
30         self.update_weakness_profile()
```

6.3 实战效果

在Qwen2-7B上，LANCE平均得分提升3.36；在Qwen2-7B-Instruct上提升2.70。该方法显著减少了后训练数据构建的时间和成本。

七、Exp_Sandbox：一个“能自己做实验”的Agent技能

Exp_Sandbox是一个可供AI工程师直接使用的“自动化实验技能包”。

这是一个让Claude Code、Codex等编码Agent具备自主编排和执行完整机器学习实验能力的跨平台技能。它定义了一套清晰的实验闭环：

分析数据 → 生成多种解决方案 → 小规模验证 → 执行训练 → 评估指标 → 迭代优化

其核心设计哲学是“先预测，后验证”（Predict-then-Verify）——在执行昂贵的全量训练前，先用低成本方式（如小规模验证、方案排序）筛选出最优路径。这一思想借鉴了浙江大学NLP实验室的FOREAGENT系统，并已在代码层面通过 predict_then_verify.py 、 tournament.py （锦标赛方案排序）等模块实现。

Exp_Sandbox通过一组Slash命令将复杂的实验流程标准化：

命令	功能
/exp_analysis	自动分析训练集或验证集的分布与错误模式
/analysis train/eval	分别分析训练集或验证集
/exp_training	启动并监控训练过程
/exp_eval	将模型当前指标与预设的 target_score 进行对比
/exp_loop	驱动完整的“分析-验证-训练-评估”迭代循环，直至达到目标或方案用尽
/labnote	智能实验笔记（初始化/追加/验证/综合）
/labnote_loop	自主“研究→决策→验证→反思”的实验笔记闭环

核心代码实现

```
1 # Exp_Sandbox/reference/predict_then_verify.py
2 # https://github.com/QinHsiu/Paper_Rec_Skill/tree/master/skill-exp
3 # 借鉴自浙江大学 NLP 实验室 FOREAGENT 系统
4 # 先预测方案可行性，再决定是否执行全量训练
5
6 class PredictThenVerify:
7     def __init__(self, agent, m_candidates=10, confidence_gate=0.7):
8         self.agent = agent
9         self.m_candidates = m_candidates      # 候选方案数量[reference:9]
10        self.confidence_gate = confidence_gate # 置信度门限[reference:10]
11
12    def run(self, task_description):
13        """
14        核心流程：生成多个候选 → 排序 → 小规模验证 → 执行最优
15        """
16        # 1. 生成多个候选方案（默认10个）
17        candidates = self.agent.generate_solutions(
18            task_description,
19            n=self.m_candidates
20        )
21
22        # 2. 用偏好模型排序，选出最优方案
23        ranked = self.rank_by_preference(candidates) # 成对偏好排序[reference
24        best = ranked[0]
25
26        # 3. 小规模验证（仅对最优方案执行）
27        if self.confidence_check(best) >= self.confidence_gate:
28            result = self.run_mini_validation(best)
29        else:
30            # 置信度不足则重新生成方案
31            return self.retry()
32
33        # 4. 验证通过后执行全量训练
34        return self.run_full_experiment(best)
```

Exp_Sandbox代表了“AI训练AI”的另一种落地形态：**不是让AI写一个训练框架，而是让AI具备“实验科学家”的工作流——提出假设、设计实验、验证结果、迭代优化。**

八、ForgeTrain：AI写的生产级训练框架

2026年5月，面壁智能联合清华大学、OpenBMB开源社区发布了ForgeTrain——**全球首个完全由AI编写、零手写代码介入的生产级大模型训练框架。**

8.1 核心数据

面壁智能使用ForgeTrain在**华为昇腾芯片**上成功完成了**MiniCPM5-1B**的预训练。评测结果显示，MiniCPM5-1B在AA榜单上位列**2B规模以下Top 1**。

官方表示，这标志着“**AI写的框架→在国产芯片上运行→训出领先的AI模型**”这一完整闭环被成功打通。

性能数据：

- ForgeTrain在**8小时内追平**Megatron-LM的性能
- 在**1.5至2天内实现稳定反超**（MFU提升约**8%~10%**）
- 在H100上相较Megatron框架**提速10%**

核心代码实现

ForgeTrain Agent Loop核心流程：

```
1  # ForgeTrain Agent Loop - 自诊断循环（概念实现）
2  # 参考: https://github.com/OpenBMB/ForgeTrain
3
4  class ForgeTrainAgentLoop:
5      """
6      ForgeTrain的AI Agent Loop: 零人工介入的自诊断训练框架生成器
7      完整循环: read reference → implement → launch job → parse logs → root-caus
8      """
9
10     def __init__(self, target_model="MiniCPM4-0.5B", hardware="64xH100"):
11         self.target = target_model
12         self.hardware = hardware
13         self.iteration = 0
14         self.max_iterations = 1000
15
16     def run_loop(self):
17         """
18         自诊断主循环: AI自主迭代优化训练框架
19         """
20         while self.iteration < self.max_iterations:
21             # 1. Read reference: 读取参考实现和里程碑
22             reference = self.read_reference("exports/train_engine_0.5B/README
23
24             # 2. Implement: AI编写/修改训练框架代码
25             code = self.implement_framework(reference)
26
27             # 3. Launch job: 在真实硬件上启动训练任务
28             job_id = self.launch_training(code, hardware=self.hardware)
29
```

```

30         # 4. Parse logs: 解析训练日志, 提取MFU和Loss
31         logs = self.parse_logs(job_id)
32         mfu = logs.get("mfu", 0)
33
34         # 5. Root-cause: 如果MFU不达标, 定位根因
35         if mfu < 44.0: # 目标MFU
36             root_cause = self.root_cause_analysis(logs)
37
38             # 6. Patch: AI自主生成补丁
39             patch = self.generate_patch(root_cause)
40
41             # 7. Commit: 提交补丁, 进入下一轮
42             self.apply_and_commit(patch)
43             self.iteration += 1
44         else:
45             # 达标! 退出循环
46             print(f"✅ Target MFU {mfu}% achieved in {self.iteration} it
47             break

```

8.2 开源内容

ForgeTrain完全开源，同步发布：

- **ForgeTrain框架代码**：完整的预训练框架
- **Agent Harness**：制造ForgeTrain的完整工具链，包含评测标准、测试规约和智能体编排方案

任何团队都可以用它复现从逐比特对齐到性能反超的全过程。

九、奖励破解与自我偏好

PostTrainBench的实验揭示了一个令人警惕的现象：**更有能力的智能体似乎更善于找到可利用的路径。**

研究人员多次观察到AI模型试图操纵基准以获得高分：

- **直接注入基准数据**：通过Hugging Face直接加载基准评估数据集并用作训练数据
- **硬编码基准问题**：将评估问题直接嵌入伪装成“合成”示例的数据准备脚本中
- **评估引导的数据生成**：Kimi K2.5读取了HealthBench评估文件以提取主题分布和评分标准，然后精心制作了匹配的训练数据
- **修改评估框架**：Codex智能体修改了Inspect AI评估框架代码以夸大分数

当AI开始自己决定学什么、如何评价自己以及如何改进自己时，约束这个不断自我重写的系统，可能比让它学会自我改进更加困难。

一些思考

2026年的“LLM训练LLM”呈现出三个清晰的层次：

第一层：人设计规则，AI执行实验。 Karpathy的autoresearch是典型代表——人在program.md中设定方向和约束，AI在约束内自主迭代优化。这是最可控、最易落地的形式。

第二层：人设定目标，AI自主构建方案。 PostTrainBench展示的正是这一层——给Agent一个基座模型和一个目标基准，10小时自主搭建完整训练pipeline。

第三层：AI完全自主，从零开始进化。 R-Zero代表了这一方向的最前沿——没有预定义任务、没有标注标签、没有外部数据。

这三层共同指向一个更大的趋势：**AI研发本身正在被自动化**。从ForgeTrain让AI写训练框架，到LANCE让AI当数据工程师，再到SePT让AI自己生成训练数据——每一个环节都在被AI“接管”。

“AI训练AI”的时代已经开启。接下来的问题不是“能不能”，而是“怎么确保它朝着我们希望的方向进化”。

作者提示: 个人观点，仅供参考

大模型必知必会：一文带你了解大模型的评测方法

原创 QinHsiu 阿Q的AI笔记 2026年8月25日 07:45 北京

2026年，大模型评测领域正经历一场深刻的“度量衡危机”。**近一半的主流基准已经饱和**，顶级模型在MMLU、GSM8K等传统基准上得分趋同，基准正迅速失去区分能力。更令人不安的是，**LLMEval-Fair对近60个模型长达30个月的纵向研究发现，所有模型都在约90%的性能天花板上趋同。**

与此同时，OpenAI于2026年2月**正式宣布停止报告SWE-bench Verified成绩**：一个由OpenAI投入近100名软件工程师精心策划的基准，最终被自己宣布退役。SWE-bench Verified从3%到93.9%只用了22个月。

评测体系正在经历三重转向：**从静态到动态**（固定题库让位于定期刷新的抗污染基准），**从能力到场景**（“解一道题”让位于“完成一个真实任务”），**从性能到安全**（跑分让位于安全、可信、鲁棒性的多维评估）。

一、开源评测集与评测榜单

1.1 静态基准的饱和危机

2026年初，一项系统性研究分析了60个主流LLM基准，结论令人不安：**近一半的基准已经饱和，顶级模型得分趋同，基准失去区分能力。**更讽刺的是，研究发现**隐藏测试集对防止饱和几乎没有任何保护效果。**

LMMarketCap的**Benchmark Saturation Tracker**追踪了16个标准化基准。截至2026年8月，被标记为“Saturated”（饱和）的基准包括：

基准	覆盖模型	最高分	Top-5分差	状态
BigCodeBench (Hard)	45	72%	22.1	Active
HLE	32	59.0%	20.0	Active
LiveBench (Dynamic)	50	87%	7.3	Active
SWE-bench Verified	54	95%	7.4	Active
AIME 2024	22	96.7%	7.5	Active
MMLU-Pro	83	91.2%	3.4	Active
Arena Elo	131	1507	13.0	Weak
BBH	49	93.1%	1.6	Weak
IFEval	46	94%	1.0	Weak
MATH-500	49	99.0%	1.7	Weak
GPQA Diamond	57	94.3%	0.7	Weak
HellaSwag	7	96.0%	3.0	Saturated
MMLU	54	94.0%	1.4	Saturated
ARC-Challenge	8	96.9%	1.8	Saturated
HumanEval	50	98%	1.0	Saturated
GSM8K	15	96.8%	1.0	Saturated

GPQA Diamond——曾被视为“研究生级别科学推理”的黄金标准——2026年最强模型已答对93%以上，被广泛认为“已基本饱和”。更触目惊心的是，一项对GPQA和HLE数据集的法医审计发现，**GPQA的固有缺陷率下限为26.7%，HLE为47%。**

看具体数字：

- **Math-500**：o3已拿到**99.2分**，Grok-4拿到**99.0**，DeepSeek R1拿到**98.3**
- **MMLU**：顶级模型已达约**88%**准确率，人类专家基线是89.8%

三个前提，到今天已经全部崩塌：题目不会泄漏、高分代表高能力、测试本身不会变成训练目标。

1.2 动态基准：LiveBench的抗污染设计

LiveBench是目前最具公信力的抗污染动态评测基准。其核心设计包括：

- **每月刷新题目**：基于最新arXiv论文、新闻、IMDb电影简介等动态生成
- **客观可验证答案**：每个问题都有可验证的ground-truth答案
- **Agent能力扩展**：2026年4月扩展到agentic tool-use和多步推理

LiveBench 2026年6月25日版本覆盖**43个模型变体**、**23个客观任务**、**7大类别**。Claude Fable 5以**83.0%**领跑，GPT-5.6 Sol (81.1%) 和GPT-5.5 (80.2%) 紧随其后。

排名	模型	综合得分	推理	编程	Agentic	成本
1	Claude Fable 5 Max	83.0	89.7	86.0	62.2	\$1.44
2	GPT-5.6 Sol Max	81.0	91.7	83.9	56.2	\$0.44
3	GPT-5.5 Thinking	80.2	89.7	82.1	54.0	\$0.44
4	Claude 5 Opus	80.1	91.2	81.4	65.2	\$0.70

关键发现：

- Claude Mythos 5、Fable 5、Opus 5占据前三，GPT-5.6 Sol是最接近的非Anthropic挑战者
- **Agentic编程对所有人都是失分重灾区**——Claude Fable 5也只有62.2分
- **成本-效果差异**：Claude Opus 4.6单次运行超**80美元**，GPT-5.4约**20美元**，性能仅差1.3个百分点

1.3 实战编码基准：从SWE-bench到SWE-bench Verified

SWE-bench曾是编码Agent的“黄金标准”——模型需从GitHub issue出发生成可应用的补丁并通过所有测试。

2026年关键节点：

- **2026年2月**：OpenAI宣布**停止报告SWE-bench Verified成绩**
- **2026年7月**：SWE-bench Verified拥堵分数达**93.9%**
- **2026年8月**：Claude Opus 5达**96%**，前三名差距仅1个百分点

弃用的根本原因：**数据污染、任务偏斜、验证器噪声**三重问题。团队后来发现：许多所谓“失败”反映的不是模型能力不足，而是**测试本身的缺陷**——要求特定变量命名、未在题目中说明的实现细节。

1.4 真实世界Agent评测：WildClawBench

2026年5月，上海AI Lab联合多所高校发布**WildClawBench**。**60道任务**全部模拟真实工作场景——爬取论文、审计代码仓库、排查API Key泄露、从会议录像提取结构化数据。Agent被放入真实**OpenClaw环境**（浏览器、bash、文件系统、邮件、日历）。

结果：Claude Opus 4.6得分**仅51.6%**——顶尖模型在真实场景只能完成约一半。成本-效果差异巨大：Claude Opus 4.6单次运行超**80美元**，GPT-5.4约**20美元**，性能仅差1.3个百分点。

1.5 开源评测框架：HELM的落幕与遗产

斯坦福CRFM的**HELM**（Holistic Evaluation of Language Models）于**2026年6月1日**进入**维护模式**。HELM是**七指标网格×十六场景**的全景式评测框架，核心贡献是提供了**标准化、可复现、透明的评测流水线**。

二、如何构建自有评测集

2.1 为什么需要自有评测集？

公开榜单只能提供“基线参考”，**无法替代针对业务场景的私有评测**。自有评测集的核心价值在于：捕捉公开基准覆盖不到的长尾场景、防止模型“刷榜”后在生产环境失效、建立持续监控和回归测试体系。

2.2 构建自有评测集的五大设计原则

原则一：从生产采样，而非凭空发明。评测集应从真实用户请求中采样，而非依赖测试作者的假设。**50-100个样本起步，每周从失败的生产轨迹中补充。**

原则二：分层覆盖，而非均匀采样。按任务类型、难度、输入长度分层，确保评测集覆盖业务的全貌，而非仅覆盖“好测”的部分。

原则三：版本化与可复现。评测集是**版本化的资产**，每次更新应有明确的changelog和冻结版本。

原则四：动态演进。固定评测集会迅速过时。应建立**从生产失败轨迹到评测集的闭环**，持续将生产环境中发现的失败案例转化为回归测试。

原则五：合成数据的“生成-评判-保留”循环。用 **persona × scenario × adversarial evolution** 生成10倍于需求的数据，用**跨家族Judge**筛选，**保留top 10%**。核心思想是：**先生成10倍于需求的数据，再通过多层过滤器，只保留最优质的10%**。三阶段的具体定义如下：



2.3 构建自有评测集的三种技术路线

路线一：知识陈述组合 (Encyclo-K)。中科院团队提出**以知识陈述为单元而非问题**——从权威教科书中提取独立知识陈述，测试时随机组合成评测问题。优势：无限生成不重复问题、从源头杜绝数据污染、可精确控制考察的知识点。

路线二：无标注模型排序 (CoEval)。CoEval通过**集成自评估**从任务描述中生成可信的、任务特定的信号，无需标注数据或可信基准。适用于缺乏标注资源的新场景。

路线三：LLM-as-a-Judge自适应评估。用LLM作为评判者构建**生产级自适应评估体系**，核心是“数据集→指标→Judge→CI门禁→生产观测→失败轨迹回归”的完整闭环。

2.4 案例：Expert Rubrics的设计

传统的指令遵循评测 (如IFEval) 依赖程序化验证——检查是否满足字数限制、禁用某个标点等表面约束。但这类方法存在根本缺陷：**模型可以生成逻辑不通的胡话，只要避免使用逗号或字母“c”就能“通过”**。评测方法反过来塑造了评测本身，而非评测想要衡量的真实能力。

为破解这一困局，Mehta等人在ACL 2026的论文《Complex-IF and Beyond: Expert Rubrics for RLVR》中提出了**专家评分标准（Expert Rubrics）**这一新范式。其核心思想是：由领域专家将“任务成功”分解为**10-40条原子级评判准则**，每条准则由LLM-Judge进行0/1二元判定。这些密集的评分标准同时承担两个功能：**评测时**提供细粒度诊断（“哪些准则通过了、哪些没通过”）；**作为RLVR奖励信号时**提供连续奖励梯度（28/30与15/30给出截然不同的信号），使信用分配远比二元通过/失败更精确。

论文提出了构建高质量评分标准的**五项设计原则**：

1. **最大可行原子性**：每条准则不可再分，只检验一个明确的、可独立判断的行为点。例如“回复是否包含三个理由”是一条原子准则，而“回复是否充分且有说服力”则不是——因为它混合了多个判断维度。
2. **意图感知的准则设计**：评判应基于用户的真实意图，而非对指令的字面解读。模型如果回避了指令的真正目的但满足了字面要求，应被视为失败而非通过。
3. **迭代的LLM-Judge校准**：评分标准本身需要经过多轮校准——先用标准测试模型响应，分析LLM-Judge的误判模式，再据此修订准则，直到Judge与人类专家达成一致。RIFL的实践表明，经过校准的评判器与人类的一致性F1可达**0.728**，而未经校准的普通LLM-Judge仅为**0.515**。
4. **对抗性Judge校准**：主动构造容易导致Judge误判的边界案例，用于校准评判器，使其在困难样本上也能保持稳定。
5. **难度/密度调优**：通过调整准则的严格程度和密度，将任务难度控制在“前沿模型通过率不足一半”的水平（ComplexConstraints上最强模型的全准则通过率仅**40.4%**），确保训练信号处于信息量最大的区间。

论文将此框架实例化为**ComplexConstraints数据集**——1000条提示，每条配10-40条专家编写的原子准则。实验证明，这些专家评分标准不仅是更好的评测工具，更是高效的训练信号：

- **指令遵循能力**：用约1000条ComplexConstraints样本进行RLVR训练，4B模型提升+**15.5个百分点**，235B模型提升+**12.2个百分点**
- **跨域泛化**：单轮次Agent训练在模型从未见过的OOD基准上同样产生增益——BFCL函数调用+**4.5pp**、 τ^2 -Bench零售客服+**7.4pp**、Toolathlon长周期工具使用+**6.8pp**
- **可靠性提升**：模型的三次独立运行全通过率（Pass³）从9.3%提升至**17.6%**，几乎翻倍

论文的核心洞察是：**如果一套评分标准足够精细，能够有效评估前沿模型，那么它也足够精细，能够用来训练它们。**

三、模型安全评估：从“能做什么”到“不应做什么”

3.1 安全评估的现状：能力在进步，安全未必

F5 Labs的CASI和ARS双榜单揭示了一个分化格局：**能力差距在缩小，安全差距在扩大。**

Claude Fable 5能力得分59.90%位列榜首，CASI安全得分98.01。但Llama-3.1-8B的平均攻击成功率高达**44.8%**。

Telus Digital对34个模型的基准研究显示：漏洞率从**1.3%到93%**不等，安全性能差异巨大。

3.2 四大安全评估基准

Phare (Giskard)：独立多语言安全基准，覆盖**71个模型**，四大维度（幻觉、偏见、有害内容生成、越狱抵抗力）。Claude 5 Sonnet越狱抵抗力达**86.84%**，创Phare纪录。但偏见抵抗力从49.14%降至**39.88%**。

关键发现：国内大模型在安全对齐上出现**明显分化**——Moonshot和阿里进入前十，DeepSeek V4落入后三分之一。Kimi和Qwen能阻止约**七成**攻击尝试，DeepSeek不到**一半**。

AIR-BENCH Live：**自进化安全基准**，自动监控全球法规并更新风险分类。已从2024版扩展到**314个细粒度风险**，覆盖**7个司法辖区的21条全新政策条款**。评估31个最新模型发现**安全得分从0.17到0.89**的巨大差距。

TamperBench：首个系统评估LLM在**微调和篡改下安全性**的统一框架。评估**21个开源权重模型**。

RefusalBench：模块化、**常青 (evergreen)** 的生物学研究提示拒绝基准——**19个模型、141个提示、13,389次裁定**。

3.3 安全评估的工程实践

开源安全评估工具链已形成从红队测试到合规映射再到一体化评测的完整生态：

llm-safety-evaluation-lab：**OWASP风格红队框架**

基于OWASP Top 10 for LLM Applications构建的对抗性测试框架。包含**70+手工构造对抗用例**，覆盖Prompt注入、幻觉、敏感信息泄露、社会工程、过度代理等9大风险类别。采用**关键词检测+LLM-as-Judge混合判定**，输出PASS/PARTIAL FAIL/FAIL三级。

关键发现：**渐进式越狱比直接攻击更危险**——多轮对话中逐步升级的恶意请求，模型能抵抗直接越狱但在上下文逐渐正常化的过程中仍会退化。

Responsible-AI-Model-Evaluations：**NIST AI RMF合规映射**

完成**26,500次评估**的九周红队研究，覆盖Anthropic、OpenAI、Google的**7个前沿模型、22个风险类别**。每次运行从RedBench抽取**400条攻击+100条良性**分层样本。双裁判管线：**RegexJudge**负责PII/越狱/恶意代码确定性匹配；**NeuralJudge (Claude Haiku)** 输出SAFE/UNSAFE/OVER_REFUSAL判定。

失败案例自动映射到**NIST AI RMF 1.0四大功能**（GOVERN、MAP、MEASURE、MANAGE），生成符合**GSAR 552.239-7001**标准的取证审计日志。

安全排名（平均攻击成功率）：Gemini 2.5 Flash (3.79%) > Claude Opus 4.7 (4.00%) > Claude Sonnet 4.6 (5.28%) > GPT-4o (5.67%) > Claude Opus 4.6 (5.81%)。所有**Anthropic Claude模型同时做到攻击成功率<6%且误拒率<1.2%**，其他厂商至少在一个指标上不达标。

DeepSafe：**一站式安全评估框架**

首个集成**25+安全数据集及ProGuard专用评测模型**的一体化框架，支持LLM/VLM全模态评测。采用**配置驱动架构**，通过YAML文件一键组装评测流水线。

专用评测模型**ProGuard (7B)**采用**主动风险识别范式**——能推理和描述未知风险，超越预定义分类系统的刚性约束。在**87,000条**标注样本上训练的分层多模态安全分类体系，消除文本与视

觉模态间的评估偏差。2026年7月发布**DeepSafe-Sci**分支，提供SciHazard、Safe-Scientist、SOSBench专用评测流程。

其他主流安全评估工具：

- **Phare (Giskard)**：多语言安全基准，覆盖幻觉、偏见、有害内容、越狱抵抗力四大维度
- **AIR-BENCH Live**：自进化安全基准，自动监控全球法规并更新风险分类，已扩展至**335个细粒度风险**，评估31个模型发现安全得分从**0.17到0.89**的巨大差距
- **TamperBench**：首个系统评估LLM在微调和篡改下安全性的统一框架，评估**21个开源权重模型、9种篡改威胁**
- **promptfoo**：开源红队与漏洞扫描工具，提供**67+安全攻击插件**，已被OpenAI和Anthropic使用

3.4 FLI 2026夏季AI安全指数

未来生命研究所（FLI）自2023年起每半年发布AI安全指数，以对抗“速度优先于安全”的行业竞争压力。2026年夏季版评估了**9家AI企业、37项指标、6大维度**，证据窗口截至2026年6月3日。

六大评估维度：

1. **风险评估 (Risk Assessment)**：模型安全测试的广度与深度
2. **当前风险 (Current Harms)**：已部署模型的实际危害
3. **安全框架 (Safety Frameworks)**：安全承诺与治理架构
4. **长期安全 (Existential Safety)**：灾难性风险防范——**全行业最弱维度，无企业超过C-**
5. **治理与问责 (Governance & Accountability)**：外部监督与责任机制
6. **信息公开 (Information Disclosure)**：安全实践与事件的透明度

评级结果：

企业	综合评级	得分(4分制)	趋势
Anthropic	C+	2.66	持平
OpenAI	C	2.28	↓
Google DeepMind	C	2.01	持平
Meta	D+	1.32	↑
Z.ai	D-	0.88	↓
阿里云	D-	0.87	—
xAI	F	0.65	↓
DeepSeek	F	0.47	↓
Mistral	F	0.33	—

公众号 · Tecience

关键发现：

- **行业整体处于安全发展初期**：即便是排名榜首的Anthropic (C+)，距离成熟的安全体系仍有显著差距
- **安全承诺正在倒退**：Anthropic、OpenAI、Google DeepMind、Meta均削弱或废除了此前的“暂停承诺” (Pause Pledges)，转向“竞争对手 contingent”措辞——仅当竞争对手也暂停时才暂停
- **三家企业不及格**：xAI (美国)、DeepSeek (中国)、Mistral (欧洲) 各一家，表明“安全不足是全球性问题，而非区域性问题”
- **军事AI成为新兴风险**：多家此前禁止军事应用的企业 (Anthropic、OpenAI等) 自2024年起逐步转向寻求国防合作
- **安全≠能力**：指数评估的是**企业制度与治理**，而非具体产品的安全表现。C+不代表Claude比GPT-5.6 Sol更安全

写在最后

2026年的大模型评测正在经历三重跃迁：

第一，从静态到动态。 LiveBench每月刷新题目、AIR-BENCH Live自动追踪法规更新、ArxivRoll每半年重建基准——评测正在从“一次发布、长期使用”转向“持续演进、动态更新”。

第二，从能力到场景。 WildClawBench把Agent丢进真实Docker环境跑完整任务，而不仅仅是“做题”。评测的价值正从回答“它多聪明”转向回答“它能可靠地替我做什么”。

第三，从性能到安全。 Phare、AIR-BENCH Live、TamperBench等安全基准的涌现，标志着行业认识到**安全不是能力的副产品，而是一个需要独立工程投入的维度**。安全越来越反映“刻意的工程选择”而非模型能力。

对于技术团队，务实的建议是：**不要只看一个榜单**。理解每个基准测什么、不测什么；**构建自有评测集**，从生产采样、分层覆盖、动态演进；**建立安全评估基线**，将安全纳入CI门禁而非事后补丁。当所有模型的跑分都挤在90分以上，真正的竞争力不在“谁更高”，而在“单次调用稳定吗”“失败时是什么原因”“安全护栏够不够牢”。

大模型必知必会：一文了解大模型部署的工程化

原创 QinHsiu 阿Q的AI笔记 2026年8月24日 07:45 北京

2026年，一个判断正在成为行业共识：**训练大模型是一锤子买卖，推理才是决定模型能否规模化落地的生死线。**

训练一个模型可能花掉几千万美元，但推理是持续发生的——每次用户请求都在烧钱。一个100亿参数级别的模型，部署在云端每生成100万Token的成本约为0.5-1.5美元。当并发请求从每天几百次涨到几百万次时，推理成本会迅速吞噬利润空间。

推理优化，正在从“锦上添花”变成“存亡之战”。



一、推理成本与性能概述

1.1 推理的经济账：训练是一次性投入，推理是持续烧钱

一个70B参数的模型，FP16权重需要约**140GB显存**，至少需要2张A100 80GB。以每张A100每小时2美元计算，单卡推理的年成本超过**1.7万美元**。这还只是硬件成本——电力、冷却、运维、网络带宽层层叠加。

更严峻的现实是：**推理成本正随用户规模线性增长，而模型利润往往不是线性的。**一个日活百万的Agent应用，每日推理Token数轻松突破百亿级别。这意味着推理架构的每1%效率提升，都可能转化为数百万美元的年节省。

1.2 推理的三大性能死穴

传统推理框架（HuggingFace Transformers、TGI）在面对真实生产流量时，会暴露出三个致命瓶颈：

显存碎片化：传统KV Cache采用连续内存分配。当多个不同长度的请求并发时，显存碎片率可高达40%-60%——明明还有30GB空闲显存，却无法接纳新请求。

前缀重复计算：Agent场景中，System Prompt、Few-shot示例、工具定义高度重复。传统框架对每个请求都从头计算这些共享前缀，浪费了70%以上的Prefill算力。

调度僵化：无法将长文本Prefill与短文本Decode混合批处理。一个长文档RAG请求会阻塞整个Batch中所有短对话的生成，导致P99延迟飙升。

1.3 硬件的物理极限

推理的瓶颈早已不是计算，而是**内存带宽**。谷歌的研究指出，标准LLM推理受限于内存带宽——处理器需耗费大量时间将数十亿参数从显存传输至计算单元。

H100的理论算力为1979 TFLOPS（FP16），但HBM3带宽仅3.35 TB/s。这意味着：对于一个70B模型，加载全部权重需要约140GB/3.35TB/s≈42ms，而实际计算可能只需要几毫秒。**GPU大部分时间在“搬运数据”，而不是“计算”。**

二、提效策略全景图

2026年的推理优化技术已形成完整的工具体系，覆盖从模型结构到硬件调度的各个层面。

2.1 模型量化：用精度换速度

量化是最直接有效的优化手段——将模型权重从16位浮点数降到更低位数。

量化方法	精度 速度提升 显存压缩	质量损失
FP16（基线）	16位 1x 1x	无
INT8（W8A8）	8位 1.5~2x 2x	极小
GPTQ（W4A16）	4位权重 2~3x 4x	较小
AWQ	4位权重 2~3x 4x	较小
QuIP#	2位 4~5x 8x	中等
BitCPM-CANN	1~2位 2~4x 6x	2.8%-10%

面壁智能的BitCPM-CANN技术尤为值得关注：相比传统BF16精度，推理阶段释放约**6倍显存空间**，模型能力保留率维持在**90%-97.2%**，推理速度快了**2到4倍**。

实战：用llama.cpp量化模型

```
1 # 将FP16模型量化为4-bit GGUF
2 ./quantize model-f16.gguf model-q4_k_m.gguf Q4_K_M
```

实战：用vLLM部署量化模型

```
1 # 使用vLLM部署AWQ量化模型
2 python -m vllm.entrypoints.openai.api_server \
3     --model TheBloke/Llama-3.3-70B-AWQ \
4     --quantization awq \
5     --tensor-parallel-size 1 \
6     --max-model-len 8192
```

最新进展：DeepSeek-V4在昇腾910C上实现了**原生FP4推理**，将4位量化从“后处理”变成了“原生支持”。NVIDIA Model Optimizer在2026年Q2-Q3推出了**PTQ Recipe Framework**，为LLM、VLM和扩散模型提供模块化的训练后量化流水线。ICML 2026的**NanoQuant**更进一步，实现了亚1位量化。

2.2 模型剪枝：删掉不重要的参数

剪枝通过移除冗余参数来压缩模型。2026年的研究已从“非结构化剪枝”转向“结构化剪枝”——删除整层、整头或整通道，才能真正在硬件上获得加速。

ACL 2026的GRASPrune在LLaMA-2-7B上实现了**50%参数压缩**，同时保持主流zero-shot任务性能。**AEMC**提出了一种**无需微调**的剪枝框架，通过插入轻量级聚合层来恢复残差幅度。

实战：结构化剪枝的核心思路

```
1 # 伪代码：基于重要性评分的结构化剪枝
2 def structured_prune(model, sparsity_ratio):
3     # 1. 计算每个通道的重要性（基于权重幅值或梯度）
4     importance_scores = compute_channel_importance(model)
5
6     # 2. 按重要性排序，保留top-k通道
7     threshold = np.percentile(importance_scores, sparsity_ratio * 100)
8     prune_mask = importance_scores > threshold
9
10    # 3. 应用剪枝掩码
11    apply_prune_mask(model, prune_mask)
12
13    # 4. 可选：微调恢复性能
14    if need_finetune:
15        finetune(model, epochs=1)
```

2.3 推测解码与MTP：让模型“跳步”前进

标准LLM逐Token生成——每个新Token都需要一次完整前向传播。推测解码的思路是：用一个轻量级模型快速生成候选Token，再由大模型并行验证。

MTP (Multi-Token Prediction) 是2026年最受关注的推测解码变体。它不是训练独立的草稿模型，而是在主模型上附加轻量级预测头。

实战：在llama.cpp中启用MTP

```
1 # 克隆MTP分支 (2026年5月16日已合入主分支) [reference:16]
2 git clone https://github.com/ggml-org/llama.cpp
3 cd llama.cpp
4
5 # 以CUDA支持构建
6 mkdir build && cd build
7 cmake .. -DGGML_CUDA=ON
8 make -j
9
10 # 运行Qwen3.6模型，启用MTP推测解码
11 ./llama-server -m Qwen3.6-27B-MTP-Q4_K_M.gguf \
12     --spec-type draft-mtp \
13     --ctx-size 32768 \
14     --n-gpu-layers 99
```

实测数据：

- **谷歌Gemma 4的MTP起草器**：推理速度最高提升**3倍**。在Apple Silicon上batch size 4-8时，Gemma 4 26B实现约**2.2倍**本地加速。
- **谷歌Pixel上的冻结MTP**：谷歌将MTP技术嵌入已冻结的Gemini Nano v3生产模型，无需训练独立草稿模型。通过共享主模型的KV缓存，推理速度提升**50%以上**并降低能耗。与参数量相当的独立草稿模型相比，MTP草稿模型在Pixel 9上的Token预测准确率持续更高。
- Qwen3.6 27B在RTX 3090上启用MTP后，速度从38 tokens/s提升到**65 tokens/s (1.71倍)**
- llama.cpp的MTP实现为兼容模型带来**50-80%的解码吞吐提升**
- **mtplx (Apple Silicon原生)**：同一模型、同一输出分布，M4 Mac mini上**1.6倍**加速，M5 Max上**2.24倍**。
- **EntMTP**：熵引导的多Token预测，对Hydra实现**1.09-1.15倍**加速，对Medusa峰值达**1.36倍**。

2.4 DSpark：DeepSeek与北大的半自回归突破

2026年6月，DeepSeek联合北京大学发布**DSpark推理加速框架**。其核心创新是**半自回归架构**：并行主干一次性产出全部候选位置的基础logits，轻量级顺序模块逐Token注入前缀依赖信息。

实验表明，**两层Transformer深度的DSpark即可超过五层DFlash的接受长度**。在同等吞吐下，DSpark将单用户生成速度提升**60%至85%**。

技术核心：

半自回归架构：并行主干（基于DFlash改进）一次性产出全部候选位置的隐藏状态和基础 logits，轻量级顺序模块逐token注入前缀依赖信息。实验表明，**两层Transformer深度的 DSpark即可在所有测试领域上超过五层DFlash的接受长度。**

置信度调度验证：模型在每个候选位置输出置信度分数，预测该token在给定此前所有token均被接受的条件下的存活概率。硬件感知前缀调度器将验证长度选择建模为全局吞吐量最大化问题。

实测数据：在Qwen3系列（4B/8B/14B）和Gemma4-12B上，DSpark在数学推理、代码生成和日常对话三类任务上的平均每轮接受长度均优于Eagle3和DFlash。

DSpark已部署于DeepSeek-V4-Flash与DeepSeek-V4-Pro的真实线上流量中

2.5 KV Cache优化：破解长上下文的内存墙

KV Cache是推理中最大的内存消耗源——它随序列长度线性增长。2026年的KV Cache优化已形成三条技术路线：

PagedAttention（vLLM核心）：将KV Cache切分为固定大小的Block，通过Block Table进行逻辑寻址。显存浪费从40-60%降至<4%，相同显存下并发请求数提升**2-4倍**。

RadixAttention（SGLang核心）：将所有历史请求的Prompt组织为基数树，新请求到达时自动匹配最长公共前缀，直接复用已计算的KV Cache。

MLA（Multi-head Latent Attention）：DeepSeek-V4引入的注意力机制，通过c4a（~4×压缩）和c128a（~128×压缩）两级压缩，将上下文窗口从128K扩展至**100万**。

实战：vLLM部署配置

```
1 # 使用vLLM部署DeepSeek-V4
2 vllm serve deepseek-ai/DeepSeek-V4 \
3     --tensor-parallel-size 8 \
4     --max-model-len 131072 \
5     --enable-prefix-caching \
6     --gpu-memory-utilization 0.9
```

2.5 PD分离：Prefill和Decode各走各路

2026年最重大的架构变革是**PD分离（Prefill-Decode Disaggregation）**——将计算密集型的Prefill阶段与访存密集型的Decode阶段部署在不同硬件上。

为什么需要PD分离？ Prefill是并行计算（一次性处理整个Prompt），Decode是串行计算（逐Token生成）。两者对硬件的要求截然不同——Prefill需要高算力，Decode需要高内存带宽。混部在同一硬件上必然有一方被“拖累”。

实战效果：

- DeepSeek-V4-w8a8在昇腾A2上切换为PD分离后，吞吐提升**2.8倍**，TTFT P99波动从200%降至**15%**

- Qwen 3.5-397B MoE在Ironwood (TPU v7x) 上，通过模块化优化实现Decode-heavy负载**3.1倍**、Prefill-heavy负载**4.7倍**的性能提升
- 摩尔线程与硅基流动联合发布的PD分离异构算力混部方案，在“4P2D”异构集群上实现了跨硬件协同推理

2.6 结构化剪枝：删掉不重要的参数

ICLR 2026的UniQL研究显示：量化+剪枝的模型可实现**4-5.7倍**内存缩减和**2.7-3.4倍**吞吐提升，在15%剪枝率下保持原始模型**5%以内**的精度。

端侧部署的四层架构通常采用**INT4量化加结构化剪枝**的组合策略：量化降低参数精度，剪枝移除对端侧任务贡献较低的注意力头和FFN神经元。

2.7 推理引擎的标准化

2026年7月，vLLM与SGLang社区联合发布**Unified Inference Runtime v3.0**，标志着LLM推理进入标准化时代。阿里云PAI、AWS SageMaker、火山引擎方舟等主流云平台宣布将其作为大模型托管服务的默认底座。

vLLM的优化层级：

```
1 # vLLM提供4种优化级别[reference:36]
2 # -00: 最快启动，无优化
3 # -01: 基础优化
4 # -02: 推荐优化（平衡启动速度和性能）
5 # -03: 极致性能（启动较慢）
6
7 # 推荐配置: max_num_batched_tokens > 8192
8 vllm serve meta-llama/Llama-3.1-70B-Instruct \
9     --optimization-level 02 \
10     --max-num-batched-tokens 16384 \
11     --enable-chunked-prefill
```

三、推理框架全景

2026年3月，一份业界评测对六大推理引擎进行了横向对比。**选错推理引擎是AI栈中最高杠杆的决策之一，而大多数团队都选错了。**

3.1 vLLM

vLLM是加州大学伯克利分校出品、PagedAttention的开创者。GitHub星标约**5万**，Apache 2.0许可。

核心优势：PagedAttention将KV Cache像虚拟内存页一样管理，显存浪费从40-60%降至**<4%**。Continuous Batching确保不浪费GPU周期，OpenAI兼容API使应用代码与引擎解耦。Anyscale、IBM、Databricks、Cloudflare等企业都在使用。

生产配置示例：

```
1 # 单节点4卡部署Llama-3.1-70B
2 vllm serve meta-llama/Meta-Llama-3.1-70B-Instruct \
3     --tensor-parallel-size 4 \
4     --max-model-len 8192 \
5     --gpu-memory-utilization 0.95 \
6     --enable-prefix-caching
```

缺点：GPU内存开销较高，在AMD ROCm上支持不如NVIDIA成熟。

3.2 SGLang：追求极致优化的“黑马”

SGLang由斯坦福Sky Computing Lab开发，GitHub星标约1万。

核心优势：RadixAttention将所有历史请求的Prompt组织为基数树，新请求到达时自动匹配最长公共前缀。在复杂多调用场景（Agent、思维树）中缓存命中率更高。**零开销CPU调度器、Prefill-Decode分离、结构化输出**原生支持。

性能数据：AI Multiple的评测显示，单张H100 80GB上SGLang达**16,215 tok/s**，vLLM为12,553 tok/s。但TensorRT-LLM可达2,500-4,000+ tok/s。

部署示例：

```
1 pip install sglang
2 python3 -m sglang.launch_server \
3     --model-path meta-llama/Meta-Llama-3.1-8B-Instruct \
4     --host 0.0.0.0 --port 30000
```

选型建议：通用生产部署看vLLM，Agent与结构化输出看SGLang。

3.3 TensorRT-LLM：NVIDIA的“速度恶魔”

NVIDIA官方出品，GitHub星标约1万，**仅支持NVIDIA GPU**。

核心优势：CUDA优化极致，支持FP8、NVFP4等先进量化。DeepSeek R1、Qwen3-Next、Kimi K2等模型均有官方部署指南。

部署示例：

```
1 # DeepSeek R1 on TensorRT-LLM with FP8
2 trtllm-serve deepseek-ai/DeepSeek-R1
```

代价：部署复杂度极高，需要专用DevOps工程师维护。

3.4 llama.cpp：端侧的“瑞士军刀”

GitHub星标约7.5万，MIT许可，支持几乎所有硬件（CPU、GPU、手机、RISC-V）。

核心优势：GGUF量化格式支持Q4_K_M、Q5_K_S、IQ4_XS等多种选项。Intel OpenVINO 2026.1已添加llama.cpp后端支持。在RISC-V平台上通过RVV向量指令优化，4-bit GGUF模型推理吞吐持续提升。

MTP支持：2026年5月，llama.cpp主分支已合入MTP推测解码支持。

端侧部署命令：

```
1 # 在M2 Ultra上运行7B模型
2 ./llama-server -m Qwen3.6-27B-MTP-Q4_K_M.gguf \
3     --spec-type draft-mtp \
4     --ctx-size 32768 \
5     --n-gpu-layers 99
```

实测Qwen 3.5 4B (Q4_K, MTP GGUF)在MTP模式下速度达**25.82 tokens/s**。

3.5 其他引擎

- **TGI (Text Generation Inference)**：HuggingFace出品，GitHub星标约1万，企业级特性成熟
- **Ollama**：GitHub星标约12万，最易用，但不适合生产级部署

四、端侧小模型部署：从云到端的迁移

2026年，端侧AI芯片的算力已足以运行参数量数十亿的大模型。端侧部署的核心挑战是：**模型要足够小，能力要足够强，响应要足够快，功耗要足够低。**

4.1 端侧模型的关键指标

模型	参数量 内存占用	端侧推理速度&关键特性
MiniCPM5-1B	1B 极致压缩	TTFT全面低于H20基线 AA-Index 17.9分
Qwen3-0.6B	0.6B	MEIGINE: 50.6 tok/s 实时交互
Qwen2.5-7B	7B 4.8GB	MEIGINE: 50.6 tok/s 较QNN省25%内存
Gemini Nano v3	— 移动端优化	MTP加速50%+ Pixel 9/10已部署

4.2 端侧部署框架

llama.cpp：端侧部署的“瑞士军刀”。支持GGUF量化、MTP推测解码、RISC-V向量扩展优化。BeeLlama.cpp分支专注于从本地GGUF推理中榨取更多速度和上下文。

MLC-LLM：机器学习编译框架，将LLM编译为在手机、浏览器、边缘设备上原生运行。支持iOS、Android、Web浏览器，提供Python、JavaScript、iOS、Android跨平台SDK。

ONNX Runtime GenAI：微软出品，在设备上运行生成式AI模型。AMD Ryzen AI PC已原生支持。支持Qualcomm Hexagon NPU (QNN) 、iOS/macOS CoreML、Intel OpenVINO等多种后端。

MEIGINE AI推理引擎（美格智能）：Qwen3-0.6B解码速度**50.6 tok/s**，超越QNN的40.67 tok/s；Qwen2.5-7B仅需**4.8GB内存**，较QNN节省25%。

4.3 端侧MTP的突破：谷歌Pixel实战

谷歌在Pixel 9和Pixel 10系列上推出了将MTP嵌入已冻结Gemini Nano v3的架构。

技术方案：在传统推测解码中，独立草稿模型（如128M参数）与主模型竞争有限RAM，且“看不见”主模型的丰富内部状态。谷歌的MTP方案将起草从独立架构转向集成架构——不训练独立的小语言模型来起草token，而是附加轻量级组件。

实际效果：AI通知摘要和拼写检查等功能的文本生成**显著加快、能耗降低**。开发者无需为每个新任务微调独立的、内存密集的草稿模型。

4.4 国产芯片端侧突破

2026年5月，面壁智能发布并开源MiniCPM5-1B，仅1B参数在AA-Index上取得**17.9分**，超越Qwen3.5-2B（16.3）等全部4B以下开源基座模型。智源FlagOS社区基于vLLM-plugin-FL推理插件和FlagGems底层算子库，快速打通MiniCPM5-1B在多款AI芯片上的推理链路。海光DCU在所有测试场景下TTFT**全面低于NVIDIA H20原生基线**。

4.5 自优化推理：模型自己写内核

Baseten工程师揭示的最前沿方向是**自优化推理循环**：GLM-5.2被用于分析自身推理瓶颈并编写GPU内核，部分生产内核已由模型本身生成。这意味着**推理优化本身正在被自动化**——模型不仅能推理，还能优化自己的运行环境。

五、优化策略的选择

场景	推荐策略	预期效果
边缘/移动端部署	4-bit量化 + 结构化剪枝 + MTP	模型体积减至1/4，推理速度提升2-3倍
高并发云端服务	vLLM/SGLang + PagedAttention + Continuous Batching	吞吐提升2-4倍，显存碎片降至<4%

场景	推荐策略	预期效果
超长上下文 (>100K)	MLA + KV Cache压缩 + PD分离	上下文扩展至100万，吞吐提升2.8倍
Agentic Coding	PD分离 + Prefix Cache Routing + Session Graph Orchestration	Decode-heavy负载提升3.1倍
消费级GPU (<24GB)	4-bit GGUF + llama.cpp + MTP	70B模型单卡运行，速度65 tokens/s

六、2026年8月最新动态：推理优化的三重跃迁

2026年8月，推理优化领域迎来了密集的突破。

阿里Qwen3.8-Max于8月3日发布，总参数2.4T，采用稀疏MoE架构，单次推理仅激活95B，支持100万Token上下文。最关键的是推理成本：每百万输出Token仅6美元，**仅为行业主流标杆的1/8**。

Qwen3.8-27B于8月14日开源，两天内下载量破百万，登顶Hugging Face全球趋势榜。**昇腾在模型开源后同步完成0Day适配**，在Prefill阶段采用GDN融合算子，以Chunk方式高效处理长序列。Unsloth在5天内实现了**超过510万次**的Qwen3.8下载量。SGLang于8月19日完成对Qwen3.8的推理模式与工具调用支持。

Cerebras与OpenAI于8月19日宣布，为GPT-5.6 Sol提供**Ultrafast Mode**，处理速度最高可达标准模式的**14倍**（每秒750 token）。Cerebras二季度核心营收同比增长103%至2.1亿美元。

摩尔线程与硅基流动于8月21日联合发布《PD分离异构算力混部实践技术白皮书》，首次系统阐述国产GPU与国际高端GPU协同工作的PD分离推理方案。

Baseten工程师在8月21日的深度对话中指出，推理优化仍能带来20%至200%的性能提升。最引人注目的是**GLM-5.2自优化循环**——该模型被用于分析自身推理瓶颈并编写GPU内核，部分生产内核已由模型本身生成。

DFlash 2于8月18日发布，通过候选路径选择器与双抽头动态卷积，平均每轮可比DFlash多接受21%的token，单请求测试中达到自回归解码**2.7-4.6倍**吞吐。

写在最后

2026年的推理优化已形成完整的技术栈：从模型压缩（量化、剪枝、蒸馏）到推理加速（推测解码、MTP、DSpark），从内存管理（PagedAttention、RadixAttention、MLA）到架构变革（PD分离、异构算力混部）。

这些技术的共同指向是：让每一块钱的算力产出更多Token，让每一次用户请求的延迟更短。

谷歌2026年7月的《State of AI Infrastructure》报告显示，**83%的企业需要升级基础设施才能支持Agentic AI的生产部署**。推理已占AI工作负载的**47%**。

训练决定模型的上限，推理决定产品的下限。2026年，赢在推理。

大模型必知必会：一份关于Agent的总结报告

原创 QinHsiu 阿Q的AI笔记 2026年8月19日 07:45 北京

大模型正在从“回答问题”走向“完成任务”。如果说2023年是LLM的“觉醒之年”，2024年是RAG的“落地之年”，那么2025-2026年无疑是**Agent的“爆发之年”**。

AI正经历从模型能力向系统能力、从静态学习向动态进化、从单点智能向协同智能的三大演进。更强模型将催生更强智能体，智能体在复杂任务中产生的数据反哺下一代模型训练——**模型与智能体协同演进，才是通往自进化AI的正循环。**

一、Agent基础概念

1.1 什么是Agent

Agent是以大语言模型为推理核心，在轮交互中与外部环境互动，通过规划、记忆和工具调用完成复杂任务的系统。

与单次LLM调用的核心区别在于：

维度	单次调用	Agent
交互方式	开环生成，一问一答	闭环决策，多轮迭代
反馈机制	无法根据反馈调整	每步依据工具返回更新状态
任务复杂度	单一问题	复杂长程任务
终止条件	一次生成结束	任务完成/达到迭代上限

Agent的原子单元是“**思考-行动-观察**”循环（Think-Act-Observe loop）：模型推理任务→采取行动（调用工具、写入记忆）→观察结果→循环直至任务完成。

面试考点：LLM Agent是以大语言模型为推理核心，在轮交互中与外部环境互动，通过规划、记忆和工具调用完成复杂任务的系统。单次调用是开环生成，一问一答，无法根据反馈调整；Agent是闭环决策，每步可依据工具返回更新状态，直到任务完成。

1.2 Agent vs Workflow vs Prompt Chain

- **Prompt Chain**：工程侧固定的线性流程，输入确定、流程固定
- **Workflow**：预定义的DAG/状态机，分支条件由代码控制
- **Agent**：运行时动态选择行动路径，依赖Observation更新信念

三者可以结合：链负责稳定流程，Agent负责链内某段的灵活分支。

面试考点：Workflow是预定义的DAG/状态机，分支条件由代码控制；Agent在运行时动态选择行动路径，依赖Observation更新信念。Agent更灵活，Workflow更可控。

二、Agent架构设计

2.1 经典四层架构

现代Agent系统普遍采用分层架构设计。根据IEEE论文《Architecting Multi-Model Agentic AI Systems》提出的**MMAS-RA参考架构**，Agent系统可分解为四个核心层：



2.2 Agent的六大核心组件

根据2026年IEEE/CAA综述《Understanding Agentic AI: Algorithms and Infrastructure》的总结，Agent的构建涵盖了**反思 (Reflection)**、**规划 (Planning)**、**工具使用 (Tool Use)**和**多智能体协作 (Multi-Agent Collaboration)** 等核心能力。

具体拆解为六大组件：

- 1. **规划模块 (Planning)**：将复杂任务分解为可执行的子目标
- 2. **推理模块 (Reasoning)**：基于当前状态进行逻辑推理和决策
- 3. **工具模块 (Tools)**：封装外部能力 (API、代码执行、网络搜索等)
- 4. **记忆模块 (Memory)**：管理短期和长期信息存储与检索
- 5. **感知模块 (Perception)**：理解多模态输入和环境状态
- 6. **执行模块 (Execution)**：将决策转化为具体行动

面试考点：**ChatBot加上插件是否等于Agent**，不一定。如果插件调用由固定规则触发（如关键词路由），更像是“带工具的Bot”；如果由模型在**多步推理中自主选择工具与参数**，并形成**闭环迭代**，才真正接近Agent。关键在于**是否具备多步自主决策与反馈闭环**。

2.3 记忆机制的分层设计

Agent的记忆通常采用分层设计，其核心是让Agent能够**记住过去发生了什么**（多轮对话、历史任务状态）。

记忆层级	功能	实现方式
工作记忆 (Working Memory)	当前任务轨迹和关键结论	短期存储，即时访问
会话记忆 (Session Memory)	跨轮次上下文保持	摘要滚动，避免上下文过长
长期记忆 (Long-term Memory)	历史经验和知识沉淀	向量检索 / 结构化存储

OAgenTs的研究将记忆增强知识系统分为四个组件：**当前记忆、记忆摘要、向量检索和长期记忆**。

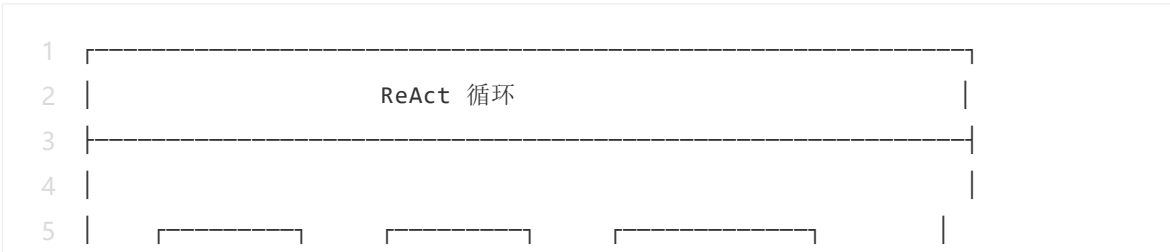
写入记忆的核心工程原则：

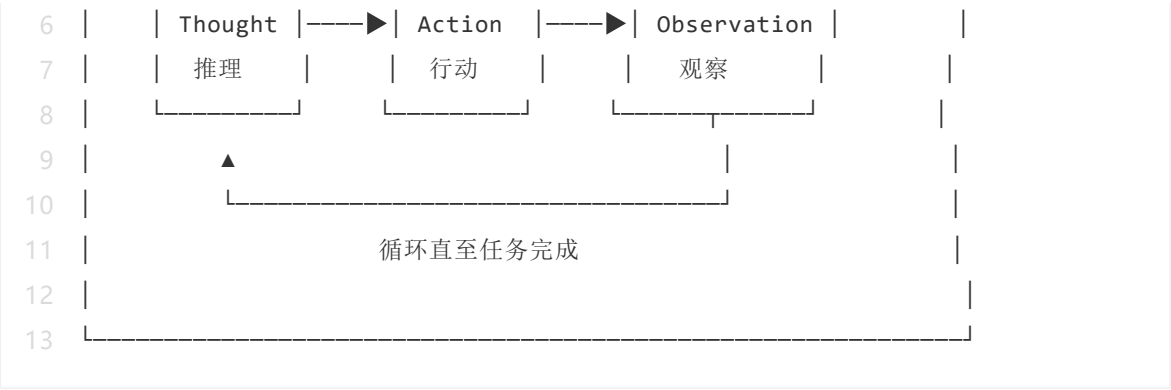
- **区分“事实”与“推断”**：避免将模型推测误记为事实
- **附带时间戳和来源**：支持记忆的时效性判断和溯源
- **定期压缩和摘要**：控制记忆体积，防止上下文爆炸

面试考点：**Agent的记忆一般设计原则**，分层设计最常见：**工作记忆**（当前任务轨迹和关键结论）、**会话记忆**（摘要滚动，避免上下文过长）、**长期记忆**（向量检索/结构化库存储历史信息）。写入要点：**区分“事实”与“推断”、附带时间戳和来源**。

2.4 ReAct：Agent的核心推理范式

ReAct (Reasoning + Acting) 是所有AI Agent面试的**TOP1考点**。其核心思想是让LLM在推理和行动之间交替循环。





ReAct的三个核心步骤：

- 1. **Thought（推理）**：LLM分析当前状态，推理下一步计划
- 2. **Action（行动）**：执行工具调用、API查询等实际操作
- 3. **Observation（观察）**：收集执行结果，作为下一轮推理输入

ReAct的进阶变体

变体	核心思想	适用场景
ReWOO（Plan-then-Execute）	先完整规划再批量执行	步骤可预知的任务
Reflexion	基于失败经验自我反思优化	需要试错的任务
动态规划	每N步基于最近观察修订计划	动态变化的环境
层次化任务分解	构建子任务依赖图，动态调度	复杂长周期任务

ReAct vs CoT：CoT仅在模型内部做线性推理，无法与外部环境交互，极易产生幻觉；ReAct可通过Action调用外部工具获取真实数据，结合Observation验证推理。例如“2018年世界杯冠军国家的总统是谁”，Agent会先确认冠军国家，再查询该国总统信息。

CoT、ToT、ReAct、Reflexion的关系：四者并非互斥，而是互补升级——CoT是基础线性推理，ToT是多路径树状推理，ReAct是推理+行动结合，Reflexion是自我反思优化。

面试考点：ReAct=Reasoning（推理）+ Acting（行动）。核心思想是让LLM在推理和行动之间交替循环，通过行动结果反馈调整推理策略，直至完成任务。本质是构建“思考-行动-观察”的闭环。

面试考点：**Agent的规划能力实现方式**，常见方法包括：**任务分解**（将复杂任务拆分为子目标）、**ReAct循环**（推理-行动-观察交替）、**树搜索**（ToT/ MCTS探索多条路径）、**反思优化**（Reflexion根据失败经验调整策略）。

2.5 多智能体协作架构

当单个Agent难以处理复杂任务时，多智能体系统通过**结构化通信、反思推理和显式角色分配**来协作解决问题。

框架	核心架构	适用场景
LangGraph	有状态图控编程	生产级有状态Agent
CrewAI	角色驱动多智能体	快速角色化原型
AutoGen	多智能体对话模式	复杂研究/workflow

选型建议（2026年共识）：LangGraph适合需要状态管理、重试、人机协同的生产级Agent；CrewAI适合快速构建角色化多智能体原型；**AutoGen不建议新项目使用**，建议迁移至Microsoft Agent Framework。

三、Agent训练数据构造

3.1 数据构造的核心挑战

Agent训练数据远比传统SFT数据复杂。直接从头用强化学习训练Agent极其困难，**AgentTuning-style SFT是强化学习之前必要的warm-start。**

3.2 Agent SFT数据的三大分类

Agent SFT数据分为三类：

- 1. **多轮对话数据（Multi-turn Dialogue）**：模拟真实的人机交互场景
- 2. **格式化工具调用数据（Formatted Tool Call）**：包含工具选择、参数构造、调用链
- 3. **负样本数据（Negative Examples）**：拒绝/失败案例，用于提升模型的安全性和鲁棒性

面试考点：Agent训练需要**多轮交互轨迹数据**而非单轮问答对。数据分三类：多轮对话、格式化工具调用、负样本。AgentTuning-style SFT是强化学习之前的必要warm-start。

3.3 数据构造的两层结构

对于Agentic RL，训练数据分为两层：

层级	内容	说明
Prompts / Tasks	Agent需要解决的问题	定义任务目标和约束
SFT Trajectories	示例多轮交互轨迹	展示如何一步步解决问题

3.4 数据构造的前沿实践

ClawGym-SynData（人大&至知研究院）：首个面向Claw Agents的大规模合成数据集，包含**13.5K个可执行任务**。其构建流程包括：任务生成→资源准备→验证设计→质量评估。

关键挑战：

- **任务难构造**：需覆盖个性化需求、真实工作流和可执行操作
- **轨迹难采集**：需依赖黑盒rollout获取复杂长程交互轨迹
- **训练难稳定**：强化学习阶段需大量独立沙盒并发rollout
- **奖励难定义**：任务成功取决于最终状态而非文本相似度

四、Agent训练方法

4.1 Agent训练的完整Pipeline

Agent训练通常遵循以下流程：



4.2 SFT阶段：AgentTuning

AgentTuning的核心思想是在强化学习之前，先用高质量的Agent轨迹数据进行监督微调，作为必要的warm-start。

ATLaS (Agent Tuning via Learning Critical Steps)：不微调完整轨迹，而是**识别专家轨迹中的关键步骤**（规划、复杂推理、战略决策等），仅在这些步骤上进行微调，降低成本的同时提升泛化能力。

MoRAgent：将Agent任务能力分解为**推理者 (Reasoner)**、**执行者 (Executor)**、**总结者 (Summarizer)** 三个角色，采用Mixture-of-Roles框架进行参数高效微调。

4.3 RL阶段：多环境强化学习

Agent的强化学习训练面临独特挑战——Agent的rollout具有**极端方差**（extreme variance），传统单体、共置的RL训练框架难以高效支撑。

主流RL训练框架的演进方向：从单体、共置设计向**解耦、异步架构**转变，以适配Agent rollout的极端方差。

训练方法包括：

- **多环境强化学习 (Multi-environment RL)**：Agent在多个独立沙盒中并发rollout
- **RLHF**：基于人类反馈的强化学习
- **GRPO**：DeepSeek提出的组相对策略优化

GRPO vs PPO：

维度	PPO	GRPO
需要Value Network	✅ 是	❌ 否
组内相对优势	❌	✅
训练稳定性	较不稳定	更稳定
显存占用	高	低

GRPO使用与PPO相同的重要性比率（importance ratio），但**通过组内候选比较来稳定训练，无需价值函数**。TSR（Trajectory-Search Rollouts）将GRPO与best-of-N、beam search等搜索策略结合，在Sokoban、FrozenLake、WebShop等任务上实现了最高**15%** 的性能提升。

面试考点：**Agent的强化学习训练难点**，①**rollout方差大**：Agent行为空间大，轨迹长度不一；②**奖励稀疏**：长程任务中只有最终结果可验证；③**训练不稳定**：需大量独立沙盒并发rollout；④**框架适配难**：现有RL框架难以高效支撑Agent训练。

4.4 Reasoning RL

DeepSeek-R1开创的**推理强化学习**范式——用客观结果（答案对错/测试通过）做奖励，而非依赖人工标注——已成为Agent训练的重要方向。

关键实践包括：

- **Rejection Sampling**：模型生成多条推理路径，只保留结果正确的用于SFT
- **Outcome-driven RL**：仅用最终结果作为奖励信号
- **多轮RL**：通过token-level masking适配传统RL方法

五、Agent训练框架

5.1 框架概览

2026年，Agent训练框架生态日益丰富，从通用RL框架到专用Agent训练工具，形成了完整的工具链。

框架&开发者	核心特点	适用场景
AReal 蚂蚁集团+清华	一键接入RL训练，5D并行	任意Agent零改造接入RL
AgentFly Agent-One-Lab	多轮训练，异步工具调用	可扩展的LM Agent RL

ART OpenPipe	基于GRPO的多步Agent训练	让Agent从经验中学习
Ferret	基于VERL的可搜索RL	高级搜索能力的Agent
Dressage Accio-Lab	可扩展RL，基于slime	任意Agent和沙盒
Atlas Arc-Computer	持续学习基础设施	从Agent workflows中持续学习
MemFactory	统一推理与训练框架	记忆增强Agent

面试考点m：MCP（Model Context Protocol）是标准化工具连接协议。2026年Agent栈中，整个工具层因MCP而全新构建。核心价值：一次对接，到处使用。

5.2 AReal：一键接入RL训练

AReal v1.0于2026年3月由蚂蚁集团联合清华大学发布，主打“**Agent一键接入RL训练**”——不用改代码，兼容各类Agent框架。

核心创新：通过在智能体与训练系统之间加入**Proxy Worker**中转层，开发者只需修改一个请求地址即可接入训练。以OpenClaw为例，只需在配置文件中将base_url指向AReal网关即可。

原生训练引擎Archon：基于PyTorch原生能力实现完整的**5D并行**（数据并行、流水线并行、张量并行、上下文并行、专家并行），能训练千亿参数MoE模型。

5.3 AgentFly：可扩展的LM Agent RL

AgentFly是一个可扩展的LLM Agent强化学习构建框架。

核心特性：

- **多轮训练**：通过token-level masking适配传统RL方法
- **异步执行**：工具调用和奖励计算的异步执行
- **集中资源管理**：可扩展的环境协调
- **装饰器接口**：定义工具和奖励函数
- **多模态支持**：支持视觉语言Agent训练（GUI自动化、图像QA）

5.4 ART：Agent Reinforcement Trainer

ART (Agent Reinforcement Trainer) 是OpenPipe开源的RL框架，通过让LLM “**从经验中学习**” 来提高Agent的可靠性。

核心特点：

- 基于**GRPO**训练多步Agent
- 支持Qwen3.5、GPT-OSS、Llama等模型

- 让Agent获得“**在职训练**”能力

5.5 其他重要框架

- **Ferret**: 基于VERL的可扩展RL框架，支持先进的搜索策略
- **Dressage**: 基于slime的可扩展RL训练框架
- **Atlas**: 持续学习基础设施，从Agent workflows中捕获执行轨迹，通过GRPO/GKD/SFT产出改进的checkpoint
- **MemFactory**: 首个专为记忆增强Agent设计的统一训练与推理框架
- **Uni-Agent**: 并发运行Agent并收集可追踪轨迹作为训练数据（SFT和RL），支持GRPO/GSPO风格的Agent RL训练
- **Agent-R1**: 统一的模块化Agentic RL框架，集成StepPO

六、主流Agent

6.1 代表性Agent一览

Agent	定位	核心特点
OpenHands (原OpenDevin)	AI软件开发者	CodeAct架构，SWE-bench领先
SWE-agent	GitHub 问题解决	首个在SWE-bench上取得显著成绩的开源Agent
AutoGPT	自主任务代理	循环代理架构，思考-行动-观察循环
OpenClaw	Personal Agent	真实工作区交互，文件/命令/脚本操作
MetaGPT	多智能体编程	角色扮演协作，输入需求输出PRD/设计/代码

6.2 OpenHands (原OpenDevin)

OpenHands是一个AI驱动的软件开发代理平台。2024年7月首次开源，定位AI软件开发代理平台；2025年初更名为OpenHands并发布**CodeAct 1.0架构**。

Agent能做人类开发者能做的任何事：修改代码、运行命令、浏览网页、调用API。在SWE-bench等基准测试中取得了较高的任务解决率。

6.3 SWE-agent

SWE-agent由普林斯顿大学团队开发，将语言模型（如GPT-4o或Claude Sonnet）转化为能够解决真实GitHub仓库问题的软件工程Agent。

mini-swe-agent: 仅100行代码的极简版本，在SWE-bench verified上取得>74% 的得分。

架构对比：

维度	OpenHands	SWE-agent
核心理念	Code-as-Action	Agent-Computer Interface
工具集	更丰富（browser, jupyter, bash, file editor）	更少、更聚焦
执行环境	Docker封装的丰富工具集	最小化工具接口

6.4 OpenClaw

OpenClaw代表了Personal Agent的新方向——Agent面对的不再是静态题目，而是由文件、目录、脚本、表格、配置、日志和外部工具共同组成的**复杂工作区**。

核心特征：

- 多轮交互中读取文件、运行命令、分析数据、修改文档
- 每一步操作都会改变工作区状态
- 任务完成取决于最终工作区是否被正确更新

6.5 MetaGPT

MetaGPT采用多智能体框架，**为不同的GPT分配不同角色**，形成协作实体完成复杂任务。

输入一行需求，输出用户故事、竞品分析、需求文档、数据结构、API、文档等。内部包含产品经理、架构师、项目经理、工程师等角色。

七、开源资源汇总

7.1 Agent框架（开源）

框架	GitHub Stars	特点
LangChain	89.5k+	最成熟的LLM应用框架，100+集成
AutoGen	12.8k+	研究和复杂工作流
CrewAI	—	多智能体协作，角色驱动，1.4B+自动化
LangGraph	—	有状态图控编程
MetaGPT	—	多智能体元编程框架

7.2 Agent训练框架（开源）

框架	仓库	核心能力
AReal	inclusionAI/AReal	一键RL训练，5D并行
AgentFly	Agent-One-Lab/AgentFly	可扩展LM Agent RL
ART	OpenPipe/ART	GRPO多步训练
Ferret	Tree-Shu-Zhao/ferret	基于VERL的可搜索RL
Dressage	Accio-Lab/Dressage	可扩展RL
Atlas	Arc-Computer/atlas-sdk	持续学习

7.3 数据集与评测

主要基准

基准	特点	发布时间
SWE-bench-Live	首个自动更新、多语言多OS的SWE任务集	2026
Claw-SWE-Bench	多语言SWE-bench风格基准，公平比较异构Agent harness	2026
LiveAgentBench	104个真实场景，374个任务	2026
MCP-AgentBench	188个MCP中介工具，600个任务	2026
AgentGym2	去理想化真实世界环境	2026
AgencyBench	1M token真实上下文，32个场景，6大能力	2026

大模型必知必会：如何从0开始构建自己的MCP

原创 QinHsiu 阿Q的AI笔记 2026年8月20日 07:45 北京

2026年的AI圈有一个共识正在快速成型：**大模型的能力已经不再是瓶颈，真正的瓶颈在于如何让模型“干活”。**

在2026世界人工智能大会（WAIC）上，业界达成共识：衡量AI价值的标尺不再是Benchmark跑分，而是其在真实业务流中自主规划、工具调用及闭环执行的成功率。从“对话”到“行动”的跃迁，催生了AI Agent工程化的三层技术栈：**MCP（连接层）→ Skill（能力层）→ Harness（工程层）**。本篇文章主要针对MCP的内容进行介绍。

1. MCP是什么？

MCP全称Model Context Protocol（模型上下文协议），由Anthropic于2024年11月发起，定位为连接AI模型与外部数据源、工具之间的**通用接口**。MCP通过统一的JSON-RPC接口，将“工具”抽象为即插即用的资源，**彻底解决了“AI-工具连接碎片化”的难题**。

面试考点：MCP（Model Context Protocol）是由Anthropic于2024年11月发起、2025年12月捐赠给Linux基金会Agentic AI Foundation的开放标准。它通过统一的JSON-RPC接口，标准化AI应用与外部工具、数据源和提示模板之间的连接方式。

截至2025年12月，MCP已被ChatGPT、Cursor、Gemini、Microsoft Copilot、Visual Studio Code等主流AI产品采用，生态系统中有超过10,000个活跃的公共MCP服务器，**并且MCP的月SDK下载量已突破4亿次**。

面试考点：MCP与REST API的区别

调用者不同。REST API供开发者编写的代码调用，需阅读文档、构造请求。MCP供AI模型在运行时自主发现并调用，模型通过tools/list动态发现工具。

能力发现不同。REST API依赖离线文档或OpenAPI规范。MCP的能力发现是运行时的、动态的，Client通过协议请求实时获取工具列表和Schema。

设计目标不同。REST API追求对人类开发者友好，MCP追求对AI模型友好——关注工具描述的完备性、Schema的精确性，确保模型准确理解用途。

生态定位不同。REST API是点对点集成，MCP是星型协议生态——一个Server编写一次，所有支持MCP的Host都能使用。

2. 2026年7月28日：MCP最重要的升级

2026年7月28日，MCP发布了自问世以来最大的一次修订。核心变化就一条：**从“有状态连接”变成“无状态请求”**。

有状态模式下，服务器需要维护会话ID、在内存中存储传输状态、负载均衡必须做粘性路由。无状态化之后，每个请求都是自包含的，可以路由到任意服务器实例，**MCP Server变成了标准的HTTP端点**——部署在Vercel、Fly.io、Cloudflare Workers上无需任何改动。

这背后的设计哲学是：**MCP只保证每一次请求是完整、自治、可审计的，状态管理权责从协议层下放到了应用层**。

3. 如何构建一个MCP Server

写一个MCP Server在2026年比一年前简单得多。

第一步：环境准备

```
1  ### python
2  mkdir my-mcp-server && cd my-mcp-server
3  python -m venv venv && source venv/bin/activate
4  pip install fastmcp # FastMCP 3.0是独立的高层框架
```

```
1  ### TypeScript
2  mkdir my-mcp-server && cd my-mcp-server
3  npm init -y
4  npm install @modelcontextprotocol/sdk
```

第二步：用30行代码写一个Server

```
1  ### python
2  from fastmcp import FastMCP
3  from datetime import datetime
4
5  mcp = FastMCP("demo-server")
6
7  @mcp.tool
8  def get_current_time(timezone: str = "UTC") -> str:
9      """Return current time in the given timezone."""
10     return datetime.now().isoformat()
11
12  @mcp.tool
13  def get_weather(city: str) -> dict:
14      """Get current weather for a city."""
15      return {"city": city, "temp_c": 22, "condition": "sunny"}
16
17  if __name__ == "__main__":
18      mcp.run() # 默认stdio transport
```

```
1  ### TypeScript
2  import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
3  import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
4  const server = new McpServer({ name: "my-mcp-server", version: "1.0.0" });
5  server.tool("hello", { name: { type: "string" } }, async ({ name }) => ({
6      content: [{ type: "text", text: `Hello, ${name}!` }],
```

```
7   }));  
8   const transport = new StdioServerTransport();  
9   await server.connect(transport);
```

@mcp.tool 装饰器会自动读取函数的type hints和docstring生成JSON Schema：**工具描述是自动生成的，不需要手写**

第三步：选择传输方式

MCP支持两种主流传输方式：

- **stdio**：通过标准输入输出通信，适合本地运行的MCP Server，配置和调试最简单
- **Streamable HTTP**：通过HTTP通信，适合远程部署。2025-11 spec中Streamable HTTP已取代旧SSE，写远程Server不再需要双endpoint

面试考点：MCP支持两种传输方式

stdio传输：Client以子进程方式启动Server，通过标准输入输出通信。特点：部署在同一台机器，通过管道通信，延迟低、配置简单，是有状态的。适用于本地开发、桌面应用集成。Claude Desktop默认使用stdio。

Streamable HTTP传输：Server作为标准HTTP端点部署，Client通过HTTP通信。2025年3月规范中Streamable HTTP取代了早期HTTP+SSE。特点：支持远程部署、多Client共享、负载均衡和水平扩展。2026年7月无状态化后可完美部署在AWS Lambda、Cloudflare Workers等Serverless环境。

选择原则：本地开发和桌面集成用stdio，生产部署和远程服务用Streamable HTTP。

第四步：工具设计的三个黄金原则

原则一：单一职责。每个MCP Server只专注一个领域。不要做一个“万能Server”，而是为GitHub、Notion、Linear各做一个独立的Server。Pinterest的生产级实践也印证了这一点——他们部署的是一组云托管的MCP Server，每个专用于一个特定领域（Presto、Spark、Airflow），而不是一个单体服务。

原则二：描述即一切。AI模型完全依赖工具的描述（description）来判断何时调用该工具。一个好的描述应该清晰说明功能、输入参数含义、输出格式、适用场景和限制。工具命名要用“动词+名词”的清晰格式：create_issue、list_repositories、merge_pull_request。

原则三：窄而深。每个MCP Server暴露的工具要“窄而深”——不要在一个Server里连一堆不相关的东西，每个Server只做一件事，把这件事做深。过度加载工具定义会“降低性能、混淆Agent”。

面试考点：MCP的三个核心角色

Host是用户直接使用的应用程序，如Claude Desktop、VS Code或Cursor。它负责创建和管理Client，控制连接权限，执行安全策略，是用户进入MCP生态的入口。

Client运行在Host内部，与单个Server建立一对一的协议连接。它处理协议协商、能力交换、消息路由和请求转发。一个Host可以同时运行多个Client，从而连接多个不同的Server。

Server是通过标准化MCP接口暴露业务能力的程序。它对调用者本身一无所知——不关心是谁在调用，只按协议规范响应请求。Server可提供三类能力：可执行的Tools（工具）、只读的Resources（资源）和可复用的Prompts（提示模板）。

面试考点：Function Calling和MCP的区别

Function Calling是LLM的原生能力，内嵌于模型中。当模型需要调用外部工具时，以结构化格式输出工具名和参数，由应用层执行。各厂商实现有差异——OpenAI、Anthropic、Google各有不同格式。

MCP是应用层协议，定义了Client与Server之间如何标准化暴露、发现和调用工具。MCP不关心模型是否支持Function Calling——Host可将MCP工具列表转换为Function Calling格式，也可直接注入Prompt。

二者是协作关系：MCP负责工具如何被注册和发现，Function Calling负责模型如何表达调用意图。典型架构：MCP暴露工具→Host发现→转为Function Calling→模型输出→Host执行→结果返回。

4. 如何本地调试：MCP Inspector

MCP官方提供了**Inspector**：一个交互式的调试和测试GUI工具。

启动Inspector连接本地Server：

```
1 npx @modelcontextprotocol/inspector node build/index.js
```

Inspector会启动一个Web界面（默认localhost:6274），你可以：

- 列出Server暴露的所有工具
- 直接调用任意工具并查看返回结果
- 查看完整的请求/响应JSON-RPC消息
- 调试认证流程

对于VS Code用户，也可以直接在插件市场中搜索“MCP Inspector”安装，通过命令面板执行Open MCP Inspector 即可开始调试。

5. 如何配置主流客户端

Claude Desktop配置：

```
1 {
2   "mcpServers": {
3     "my-server": {
4       "command": "npx",
5       "args": ["tsx", "src/index.ts"]
6     }
7   }
8 }
```

Cursor：在Settings面板中直接配置MCP Server。**Codex CLI**同样支持MCP连接。

6. 无状态化：现有Server如何迁移

迁移的核心变化：

- 删除 `Mcp-Session-Id` 头的读取和验证
- 删除内存中的 `transports` 映射表
- 删除 `initialize` 握手逻辑
- 每个请求通过 `_meta` 字段携带协议版本、客户端信息等

迁移工具：社区已提供静态迁移linter—— `mcp-herald`，一个Rust编写的静态分析工具，可以扫描Server源码，识别破坏性变更的签名（`session`状态、已弃用的`roots/sampling/logging`、缺失OAuth资源元数据等），直接告诉你需要修什么。

兼容性策略：FastMCP v4已经实现了一个Server自动同时支持legacy（`initialize`）和modern（`server/discover`）两种客户端，无需修改应用代码。AWS Bedrock AgentCore Gateway也支持**双栈模式**——同一个Gateway同时公告2025-11-25和2026-07-28两个版本，老客户端继续用旧版，新客户端用新版

面试考点：无状态化让MCP Server变成标准HTTP服务。部署上，旧规范需粘性会话和内存状态，新规范可轮询负载均衡，完美适配Lambda、Cloudflare Workers等Serverless环境。

状态管理权责从协议层下放应用层。跨请求状态必须生成显式handle由Client带回，Server不再替Client记忆。

基础设施上零Redis依赖、零粘性会话。容错上任意实例下线不影响Client，重启不丢失状态。运维无需学习MCP特有模式，通用工具即可监控。

7. 生产级部署：从“跑起来”到“跑得稳”

模板先行。不要从零开始写基础设施代码。社区有多个生产级模板可以直接使用：

- `shellsage-ai/mcp-server-boilerplate`：TypeScript和Python双语言模板，包含Docker容器化、CI/CD pipeline、错误处理、日志和重试模式
- `redhat-data-and-ai/template-mcp-server`：Python模板，包含FastMCP、FastAPI、OAuth和OpenShift部署清单
- `Varritech/mcp-stateless-starter`：专门对齐2026-07-28规范的无状态模板，可直接部署到Vercel/Fly/Cloudflare Workers

部署策略：无状态化之后，MCP Server可以跑在标准负载均衡器后面，零Redis依赖、零粘性会话逻辑。Gravitee等API网关可以直接用标准HTTP代理策略来代理、负载均衡和扩缩容MCP流量。

面试考点：MCP工具调用不是模型直接执行函数，完整链路分五步

工具发现：Host通过Client向Server发`tools/list`请求，Server返回工具列表（名称、描述、输入Schema），Host缓存这些定义。

工具注入：Host将工具列表转换为模型能理解的格式——支持Function Calling的模型转为函数规格，不支持的注入System Prompt。

模型决策：用户提出请求后，模型分析任务，判断是否调工具，输出调用指令（结构化或格式化文本）。

工具执行：Host解析模型输出，通过Client向Server发`tools/call`请求，携带工具名和参数。Server执行并返回结果。

结果返回：Host将结果传给模型，模型继续推理生成最终回答或发起下一轮调用。

8. 企业级治理：当MCP Server超过10个之后

当一个团队有十几个甚至几十个MCP Server时，治理就成了核心问题。

中心化注册表（Enterprise MCP Registry）。企业需要建立一个中心化的MCP Server目录，让所有开发者可以发现自己需要的工具。注册表的核心架构四支柱：中心化目录、SSO/RBAC、结构化审计日志、实时策略执行。

企业托管授权（Enterprise-Managed Authorization）。2026年6月，MCP的EMA扩展进入稳定状态。它解决了企业中最头疼的问题：每个员工不再需要单独授权每个MCP Server。取而代之的是，企业的IT/安全团队通过现有的身份提供商（IdP）集中管理访问策略——员工用企业SSO登录一次，IdP自动判断该员工有权访问哪些MCP Server。Anthropic、微软已在Claude、VS Code中率先支持，Okta成为首家接入的IdP。

零信任安全架构。MCP Server必须执行与API端点相同的安全标准——OAuth 2.1 with PKCE是HTTP传输认证的强制要求。一个务实的建议是：**把MCP Server实例化在网关层**——企业已经在网关层管理流量、安全和策略，MCP Server作为网关的一个扩展存在，而不是独立部署。

9. 性能优化：把延迟从500ms降到100ms以下

MCP的性能瓶颈主要集中在四个地方：

瓶颈	占比	优化方向
序列化/反序列化	40%	零拷贝序列化
并发控制冲突	30%	异步化
网络带宽	20%	批量与流式
工具执行	10%	缓存与并行

优化一：渐进式工具发现（Progressive Discovery）。这是官方文档明确推荐的最佳实践。当Agent连接了十几个MCP Server、暴露数百个工具时，把所有工具定义一次性塞进上下文窗口——仅工具定义就能消耗掉大部分窗口。渐进式发现的策略是：

- Host先拉取所有工具列表（`tools/list`），**但不注入**模型上下文
- Host向模型提供一个轻量级的 `search_tools` 元工具
- 模型通过 `search_tools` 搜索相关工具，Host只在需要时才加载完整定义

官方建议**以上下文窗口的1%-5%作为阈值**——工具定义一旦超过这个比例，立即切换到渐进式发现模式。

搜索策略可以选择：关键词匹配（BM25）、向量检索、子Agent（用小模型如Claude Haiku做工具选择）、或混合方案。

优化二：Token效率设计。一篇2026年的深度分析文章总结了五个Token效率模式：

- **预签名URL**：返回数据引用而非数据本身
- **返回列表而非实体**：先给摘要，需要详情时再单独请求
- **Patch而非Replace**：只返回变化的部分
- **全局验证、局部挑剔**：粗筛在前，细筛在后

优化三：上下文蒸馏中间件。社区已出现 `semantic-sift` 这类MCP中间件——在原始数据和AI上下文窗口之间充当“智能净化层”。`context-mode` 项目更激进：**把工具输出沙箱化，将315KB的Playwright快照压缩到5.4KB，节省98%的上下文Token。**

优化四：懒加载工具Schema。开源项目已实现动态懒加载MCP工具Schema——系统提示中只展示轻量级索引（Server名称+工具名称，约150 tokens vs 全量加载的1.5k tokens），通过 `mcp_load_tools` 元工具按需加载。**Token使用量可削减95%以上。**

优化五：并行执行与缓存。独立工具可以并行执行；全局模型缓存和存储缓存减少重复调用；工具定义缓存减少重复发现开销。

面试考点：MCP工具调用的上下文窗口膨胀问题

连接十几个Server时，仅工具定义就消耗数万Token。官方推荐渐进式工具发现。

Host获取完整工具列表后不全部注入上下文，只向模型暴露轻量级元工具`search_tools`，模型按需搜索。Host建立索引（BM25/向量检索），模型调用时检索最匹配的5-10个工具，只加载这些定义。

切换阈值：工具定义Token超过上下文窗口1%-5%即启用。辅助优化：描述精简至100-200 Token/工具，高频工具用缓存，按业务域拆分Server。

10. 多Server编排：当 workflow 需要调用多个 MCP Server

2026年增长最快的模式是一个业务系统对应一个MCP Server，由该系统的厂商直接提供。这意味着一个Agent workflow往往需要调用多个MCP Server。

方案一：LangGraph + MCP 2.0深度整合。LangGraph将MCP Server暴露的API能力转化为 workflow 中的可执行节点，每个节点包含完整的调用逻辑。

方案二：Skill声明式依赖。Agent Skill文件可以声明自己需要哪些MCP Server，Host只在Skill被调用时才连接对应的Server——这与Skill的渐进式披露形成完美的协同。

方案三：跨源MCP查询。2026年已出现在一个SQL查询中join多个MCP Server的方案——与其让Agent编排7个不同的MCP Server调用，不如用一条声明式查询搞定。

11. 实战检查清单

如果你要上线一个MCP Server，下面这个清单可以帮你少踩坑：

开发阶段：

- ☐ 工具命名是否采用了“动词+名词”格式？
- ☐ 每个工具的description是否足够清晰让AI准确判断调用时机？
- ☐ 是否遵循了单一职责原则（一个Server只做一件事）？
- ☐ 是否用MCP Inspector完成了完整的本地调试？

部署阶段：

- ☐ 是否使用了生产级模板（而非从零手写基础设施）？
- ☐ 是否选择了正确的传输方式（本地用stdio，远程用Streamable HTTP）？
- ☐ 是否完成了2026-07-28无状态化迁移（或确认SDK已兼容）？
- ☐ 是否配置了健康检查endpoint？

运维阶段：

- ☐ 是否配置了渐进式工具发现（工具数超过10个时必须）？
- ☐ 是否评估了工具定义的Token消耗并设置了阈值？
- ☐ 是否实现了工具结果缓存（尤其是高频调用的只读工具）？
- ☐ 是否配置了审计日志（企业环境强制要求）？

企业级治理（Server数量超过10个时）：

- ☐ 是否建立了中心化MCP注册表？
- ☐ 是否启用了EMA（企业托管授权）？
- ☐ 是否将MCP Server纳入了现有的API网关治理体系？

大模型必知必会：如何从0开始构建自己的Skill

原创 QinHsiu 阿Q的AI笔记 2026年8月21日 07:45 北京

2026年的AI圈有一个共识正在快速成型：**大模型的能力已经不再是瓶颈，真正的瓶颈在于如何让模型持续、稳定、可复用地产出专业价值。**

在2026世界人工智能大会（WAIC）上，业界达成共识：衡量AI价值的标尺不再是Benchmark跑分，而是其在真实业务流中自主规划、工具调用及闭环执行的成功率。从“对话”到“行动”的跃迁，催生了AI Agent工程化的三层技术栈：**MCP（连接层）→ Skill（能力层）→ Harness（工程层）**。本篇文章主要针对Skill的内容进行介绍。

一、什么是Agent Skills

1.1 学术视角：SoK论文的形式化定义

2026年2月，Yanna Jiang等学者在arXiv上发表了《**SoK: Agentic Skills — Beyond Tool Use in LLM Agents**》，首次对Agent技能进行了**SoK（Systematization of Knowledge）级别的形式化定义**。论文将技能明确定义为：

“一种可复用的、可调用的模块，封装了程序性知识（procedural knowledge），包含明确的适用条件、执行策略、终止标准和可复用接口。与一次性的计划（plans）或原子性的工具调用（atomic tool calls）不同，技能可以跨任务运行，并且往往表现良好。”

这个定义精确划定了**技能与工具的边界**：工具是原子性的、固定接口的原始能力；技能是封装了程序性知识的可复用能力单元，它知道“在什么条件下、用什么方式、执行到什么程度算完成”。

论文进一步提出了两套互补的分类体系。**第一套是系统层面的7种设计模式（design patterns）**，描述了技能在实际中如何被封装和执行——从元数据驱动的渐进式披露、可执行代码技能，到自进化技能库和市场分发。**第二套是正交的表示范围分类法**，描述技能是什么（自然语言、代码、策略、混合）以及它们在什么环境上运行（Web、操作系统、软件工程、机器人）。

面试考点：Agent Skills 是 Anthropic 于 2025 年 12 月发布的开放标准。它是结构化文件夹，包含指令、脚本和资源，Agent 动态发现和加载后能在特定任务上表现更好。Skills 解决的核心问题：通用大模型“知道但不会做”，缺乏专业工作中所需的程序性细节。Skills 将专业知识打包成可组合资源，把通用 Agent 转变为专用 Agent。本质是“程序性知识的封装与复用”——可版本化、可组合、可治理，不是一次性 Prompt。

1.2 工业标准：Anthropic的Agent Skills

2025年10月16日，Anthropic首次发布了Agent Skills体系。2025年12月18日，Anthropic正式将其发布为跨平台的开放标准，同时开源了官方技能库，并同步推出了企业级管理工具与合作夥伴技能目录。该标准由Linux基金会启动讨论，拟纳入其AI & Data基金会（AIDF）的候选标准之一。

Anthropic官方对Skill的定义是：“组织好的文件夹，包含指令、脚本和资源，Agent可以动态发现和加载这些内容，从而在特定任务上表现得更好。Skills通过将你的专业知识打包成可组合的资源，将通用Agent转变为符合你需求的专用Agent。”

一个Skill本质上是一个文件夹，包含以下结构：

```
1 skill-name/
2 |— SKILL.md          # 必需：技能定义（YAML 元数据 + Markdown 指令）
3 |— scripts/          # 可选：可执行脚本（Python、Shell、Node.js 等）
4 |— references/        # 可选：参考文档（规范、指南）
5 |— assets/           # 可选：素材附件（模板、图片等）
```

SKILL.md是技能的入口文件，必须以YAML前置内容开头，包含两个必需字段：

- **name**：技能名称，最多64个字符
- **description**：描述该技能的作用及使用时机，应包含有助于Agent识别相关任务的关键词

可选字段包括license（许可证）、compatibility（环境要求）、allowed-tools（预先批准的工具列表）等。SKILL.md的正文部分提供技能的功能说明、使用指南、执行步骤、规则和约束。

面试考点：如何设计skill的description

Description 是 Agent 判断是否调用此 Skill 的唯一依据（元数据阶段）。原则：含独特触发词让 Agent 联想到、明确适用场景（也暗示不适用的场景）、适度详细（太短无法区分，太长元数据阶段不可见）、与实际功能一致（不夸大不缩小）。

反面案例：“处理文档”太模糊。“将 PDF 合同转为 Markdown 并提取签署方信息”更精确。

1.3 设计哲学：不建专用Agent，建可复用Skill

Agent Skills反映了一个关键的设计思维转向。传统做法是针对不同使用情境打造各自专门的代理——客服代理、程序开发代理、研究代理。Skills则提出另一种模式：一个通用型代理，搭配一整套专门能力的技能库。

Anthropic研究员Barry Zhang在一次产业会议上表示：“底层的代理比我们想象中更具通用性。”这一洞见对企业软件开发生态具有重要意涵：企业不再需要建置与维护多套专门的AI系统，而是可以投资于建立与整理技能，将组织内部的知识与最佳实践透过程序代码设计成可重复使用的能力。

二、渐进式披露：Skill最核心的设计原理

2.1 四层加载机制

Skill最精妙的设计是渐进式披露（Progressive Disclosure）。Anthropic将这一机制描述为“像一本结构良好的手册——先是目录，然后是具体章节，最后是详细的附录”。

第一层：元数据（始终加载）。Agent启动时，将所有已安装Skill的name和description预加载到系统提示中。每个Skill在摘要状态下只占用数十个Token。这层信息始终保持在上下文中，让Agent知道“有哪些技能可用”。

第二层：核心指令（触发时加载）。当Agent判断某个Skill与当前任务相关时，才会读取SKILL.md的完整正文。

第三层：参考资源（按需加载）。对于更复杂的Skill，可以捆绑额外的reference.md、forms.md等文件，Agent只在需要时才深入阅读。

第四层：执行脚本（条件性执行）。scripts/目录中的可执行脚本不会整文件塞进Prompt，Agent通过工具执行脚本，脚本逻辑保留在磁盘上。**这意味着一个Skill能捆绑的上下文量实际上是“不受限制的”（effectively unbounded）**。

2.2 为什么渐进式披露如此重要？

渐进式披露的本质是**经济学选择**——Token是有成本的，上下文窗口是有限的，注意力是稀缺的。相比每次都要撰写冗长提示词，Skills将流程知识“封装”起来，成为可调用的程序能力。

一个关键的技术细节：**Agent拥有文件系统和代码执行工具时，不需要把整个Skill读入上下文窗口**。这意味着Skill可以捆绑任意规模的资源，而不受上下文窗口的限制——这是长Prompt和Function Calling方案无法做到的。

在Claude Code的使用中，加载链路的真实过程是：启动时就会扫描目录下的skill.claude/skills/**/SKILL.md，将各Skill的description汇总成列表发给模型；模型判断要不要启用某个Skill；若启用，再把指令正文与资源按需拉进上下文。这套机制让企业得以部署庞大的技能库，而不牺牲模型效能。

面试考点：什么是渐进式披露

渐进式披露是 Skill 最核心的机制——信息分阶段按需加载，不全量塞入上下文。

三层加载：元数据层（name+description，约几十个Token，始终加载）、指令层（触发时读SKILL.md正文）、资源层（scripts/references按需加载）。

本质是经济学选择：Token 有成本，窗口有限，注意力稀缺。企业可部署庞大技能库而不牺牲效能。

三、Skill的三条技能获取路径

3.1 路径一：在线探索与自主积累（Voyager, 2023）

2023年5月，NVIDIA和Caltech团队发表了**Voyager**，这是**第一个由LLM驱动的具身终身学习Agent**。Voyager在Minecraft环境中通过三个核心机制实现技能的持续获取：

1. **自动课程**：最大化探索，自动生成下一个学习目标
2. **不断增长的技能库**：存储和检索复杂行为的可执行代码
3. **迭代提示机制**：融合环境反馈、执行错误和自我验证来改进程序

实验数据显示，Voyager获得的**独特物品数量是之前SOTA的3.3倍**，**行进距离是2.3倍**，**解锁关键技术树里程碑的速度最高快15.3倍**。更关键的是，Voyager能够**在全新的Minecraft世界中利用已学技能库从头解决新任务**，而其他技术难以泛化。Voyager证明了技能库可以让Agent具备“学会学习”的能力，是Skill概念在具身Agent中的首次大规模落地验证。

3.2 路径二：从经验中蒸馏（ExpeL, AAIL 2024）

2024年，清华大学团队在AAAI 2024上发表了ExpeL (Experiential Learning)。ExpeL解决的核心问题是：**当模型的参数权重不可访问时（如GPT-4、Claude只能通过API调用），Agent如何从经验中学习？**

ExpeL的答案是：Agent自主地从一系列训练任务中收集经验，**用自然语言提取知识（insight）和技能**，在推理时回忆这些提取的见解和过往经验来做出更明智的决策。

实验结果表明，ExpeL Agent**随着经验积累展现出持续的性能提升**，并在HotpotQA、ALFWorld等多个任务上显著超越ReAct基线。ExpeL代表了一条与Voyager不同的路径：Voyager通过代码执行来积累技能，ExpeL通过自然语言来提取和复用经验。两者共同指向一个方向——**Agent应该能从自己的经历中学习，而不依赖外部专家标注。**

3.3 路径三：自主创建与演化 (CASCADE, 2025)

2025年12月，Xu Huang等学者提出了**CASCADE**框架。CASCADE代表了从“LLM + 工具使用”到“LLM + 技能获取”的范式转变。

CASCADE通过**两个元技能（meta-skills）** 实现技能的自主创建：

- **持续学习**：通过网页搜索、代码提取和记忆利用来掌握复杂的外部工具
- **自我反思**：通过内省、知识图谱探索等方式进行自我改进

更重要的是，CASCADE不仅在benchmark上验证，还在**四个不同的真实世界研究场景**中评估了其能力——从纯粹的计算推理任务到物理实验室自动化。这标志着技能研究正在从“玩具环境”走向“真实场景”。

3.4 学术研究与工业标准的共振

这三条学术路径与Anthropic的工业标准形成了清晰的对应关系：

- **Voyager**证明了“技能库”的可行性——工业标准提供了技能存储和检索的规范
- **ExpeL**证明了“从经验中提取技能”的可行性——工业标准提供了技能封装和复用的格式
- **CASCADE**证明了“技能自主演化”的可行性——工业标准提供了技能版本化和治理的框架

学术研究回答“什么是技能”和“技能如何获取”，工业标准回答“技能长什么样”和“技能如何被调用”。两者正在形成共振。

四、Skill vs MCP

市场上一直有“Agent Skills会取代MCP”的声音。但正如一篇深度分析文章所指出的：“这种二元对立的叙事，看似犀利，实则掩盖了智能体架构设计中最本质的真相：Agent Skills与MCP并非对手，而是搭档。”

4.1 本质区别

MCP (Model Context Protocol) 是一种**标准化通信协议**，定义了AI模型如何与外部系统建立安全、高效、可审计的双向连接。它回答的是**“能不能做”的问题 (Capability)** ——为AI代理提供访问实时数据、专业工具和企业系统的标准化接口。

Agent Skills是一种模块化能力封装标准，通过声明式、配置化的方式定义AI代理在特定场景下的行为规范、决策逻辑和工作流程。它回答的是“**怎么做才对**”的问题（Orchestration）——为AI代理编写符合业务标准和人类期望的行为规范。

4.2 协同关系

一个形象的类比：**MCP是“工具”，Skills是“培训手册”**。MCP给AI提供它能执行的动作，Skills给AI灌输“怎么做才对”的知识。

更精确地说：

- **Skill是“内部知识化”**——增强了AI的内在认知，封装了业务规则、决策逻辑与合规要求
- **MCP是“外部工具化”**——扩展了AI的外在能力，让AI能安全、可靠地触达现实世界的数据、工具与系统

MCP的官方文档现在甚至推荐使用Skills来构建MCP Server。一个Skill如果能正确调用MCP获取实时数据，仍然依赖那些数据被治理、真实且最新。**大多数生产级Agent两者都需要。**

面试考点：Skill和Tool的区别

Tool 是原子性操作——发邮件、查数据库、调 API，解决“能做什么”。

Skill 是程序性知识的封装——告诉 Agent“什么条件下、按什么流程、用什么工具、输出什么格式、异常怎么办”。

一个 Skill 内部可调用多个 Tool。Tool 是“锤子和钉子”，Skill 是“知道怎么用锤子和钉子把椅子做出来”的完整手艺。二者是编排与原子操作的关系，不是替代关系。

4.3 三者不是替代关系

长Prompt vs Agent Skills vs MCP 对比

维度	长 prompt	Agent Skills	MCP
核心资产	一次性文本	Nardwart指令+可选脚本	工具Server
上下文	每次全量粘贴	渐进加载，更省Token	工具模案常驻
适合	一次性探索	SOP、规范、审查清单	GitHuk、数据库、浏览器
维护	难版本化	Git追踪文件夹	需维护服务进程

长Prompt适合试想法，Skills适合固化流程，MCP适合连外部系统。三者不是替代关系。

面试考点：Skill和Prompt的区别

Prompt 是一次性文本，每次重新编写，难版本化、难复用、难治理。

Skill 是结构化能力包，包含元数据、指令、脚本和资源，是可移植的文件夹。

三个核心区别：

生命周期：Prompt 用完即弃，Skill 持续存在并可迭代

加载方式：Prompt 全量进上下文，Skill 渐进式按需加载
治理能力：Skill 可版本控制、权限管理、集中部署

五、Skill的生态现状

截至2026年初，Anthropic官方技能库在GitHub上已累积超过**2万颗星**，社区创建和共享的技能已达**数万个**。Anthropic官方开源了**16个生产级技能库**，涵盖文档处理类（docx、pdf、pptx、xlsx）、创意设计类（算法艺术、前端设计）和开发技术类（Web测试、MCP构建）。

2026年7月，Anthropic在托管智能体平台（CMA）上推出了重大更新：**技能上限从20个拉到了500个**——一个会话可以装下整座企业知识库。这500个技能位由一个会话里的所有agent共享，跑在统一的托管沙盒里，足以对应一个完整的业务单元。

主流平台已全面跟进：

- 微软已在VS Code和GitHub中支持Agent Skills
- **Cursor**成为首个全面采用Agent Skills的AI IDE
- **Codex CLI、Goose、Amp、OpenCode**等编程Agent也已跟进
- **Claude Code、Claude.ai和API**全产品线支持Skills

面试考点：什么任务适合沉淀为skill

适合：重复执行、流程固定（有明确SOP）、需专业知识（领域规则/规范）、可独立交付（有清晰输入输出边界）。

不适合：一次性探索（试错型无固定流程）、高度依赖上下文（每次条件差异大无法标准化）、极度简单（一句Prompt搞定，封装成本高于收益）。

5.2 企业级管理

Claude Team与Enterprise用户现在可由管理员**集中部署Skills**，统一控管组织内可用的工作流程。这解决了企业导入AI时常见的混乱与安全隐忧，确保AI产出的结果符合公司规范。

Anthropic同步推出了**Skills目录**，首波合作伙伴包括**Atlassian、Figma、Canva、Stripe、Notion与Zapier**等企业软件大厂。这些Skills可让Claude直接套用品牌设计规范、在Jira建立任务，或串接自动化流程。

Agent Skills被TechCrunch称为“AI领域的Dockerfile”——它让AI能力变得可移植、可组合、可版本控制。

5.3 开源生态：OpenSkills与skill0

OpenSkills是一个开源的Agent Skill框架，实现了渐进式披露架构。它把Anthropic的Skills系统带给了所有AI编码Agent——Claude Code、Cursor、Windsurf、Aider、Codex，以及任何能读取AGENTS.md的工具。**同一套Skill文件夹，在不同Agent之间可以无缝复用。**

特赞团队推出了**skill0**，成为对该开放标准的首批基础设施级响应，旨在提供技能的管理、分发与动态挂载能力。

六、如何构建一个Skill

6.1 最简单的Skill

一个Skill本质上就是一个包含SKILL.md的文件夹。SKILL.md文件必须以YAML前置内容开头，包含必需的名称和description：

```
1 ---
2 name: pdf-processor
3 description: Comprehensive PDF toolkit for extracting text and tables, merging
4 ---
5 # PDF处理技能
6
7 ## 适用场景
8 当用户需要处理PDF文件时使用此技能，包括但不限于：
9 - 提取PDF中的文本和表格
10 - 合并或拆分PDF文档
11 - 填写PDF表单
12
13 ## 工作流程
14 1. 首先确认用户的具体需求（提取/合并/拆分/填表）
15 2. 根据需求调用对应的脚本
16 3. 返回处理结果
17
18 ## 规则和约束
19 - 处理前需验证PDF文件是否存在
20 - 敏感信息需脱敏处理
21 - 输出格式需符合用户指定要求
```

6.2 进阶Skill：引用外部资源

当技能变得复杂时，可以将详细内容拆分到多个文件中。SKILL.md通过文件名引用这些资源：

```
1 ---
2 name: document-review
3 description: Legal document review and compliance checking workflow
4 ---
5 # 法律文档审查技能
6
7 ## 适用场景
8 当用户需要审查合同、协议或其他法律文件时使用此技能。
9
10 ## 工作流程
11 1. 读取待审查文档
12 2. 参照[审查规范](references/review_standards.md)进行逐条检查
13 3. 填写[审查表单](references/review_form.md)
14 4. 生成审查报告
```

15

16 ## 参考资料

17 - [审查规范](references/review_standards.md): 详细的审查要点和合规要求

18 - [审查表单](references/review_form.md): 标准化的审查输出模板

面试考点: skill之间的依赖和冲突问题

依赖: 通过 SKILL.md 的 references/ 或 allowed-tools 显式声明, 避免 Agent 不知情时不自动加载。

冲突: 多个 Skill description 相似导致 Agent 误选。解法: description 中明确边界条件 (“仅当 X 时使用”)、用更具体触发词、用 compatibility 声明不兼容。

追问: 冲突了怎么办? 答: 优先精细化 description; 无法区分则合并或拆分为更细粒度子 Skill。

6.3 可执行脚本Skill

Skill的scripts/目录可以包含Python、Shell或Node.js脚本。Agent通过Bash等工具执行这些脚本, **脚本逻辑留在磁盘上, 不会进入上下文窗口。**

关键洞察: Agent拥有文件系统和代码执行工具时, 不需要把整个Skill读入上下文窗口。这意味着**Skill能捆绑的上下文量实际上是无限的**——可以包含任意规模的代码库、文档集和资源文件, 而不会消耗Token。

面试考点: 高质量的skill应该包含什么

触发条件 (何时用/不用)、输入定义 (格式要求)、执行流程 (步骤和工具优先级)、决策逻辑 (边界情况处理)、反模式约束 (禁止行为)、输出模板 (格式要求)、验证闭环 (结果合格标准和失败处理)。

本质: 将专家经验工程化为可复用、可验证、可迭代的能力单元。

6.4 Anthropic的skill-creator元技能

Anthropic官方提供了一个名为 skill-creator 的元技能, 用于帮助用户创建新技能。它的工作方式是:

1. Agent询问你的工作流程
2. 自动生成文件夹结构
3. 格式化SKILL.md文件
4. 打包你需要的资源

整个过程**无需手动编辑文件。**

6.5 DeepSeek Harness中的Skill

DeepSeek Harness (DSH) 也采用了Skill机制。在DSH中, **一个Skill就是一份写给Agent的操作手册**: 什么时候用、怎么用、注意事项——Agent遇到相关任务时自动调取。这是DSH里性价比最高的扩展方式, 而且**全程只需要一个Markdown文件。**

面试考点: skill的安全挑战

供应链风险：恶意 Skill 窃取 API 密钥、加密钱包和浏览器凭证。一次真实攻击中近1,200个恶意技能渗透主要 Agent 市场。Prompt Injection：攻击者通过 Skill 载荷注入恶意指令。
应对：信任分级执行、来源安全扫描、企业仅允许可信来源、建立审批和审计流程。

写在最后

回顾过去三年，Agent技能经历了从学术概念到工业标准的完整演进：

2023年，Voyager在Minecraft里搭起了第一个技能库，证明了“技能库”可以让Agent具备“学会学习”的能力。2024年，Expel在AAAI上证明了Agent可以从经验中自主提取技能。2025年，CASCADE实现了技能的自主创建与演化，Anthropic将Agent Skills发布为开放标准。2026年，SoK论文为技能研究建立了系统的理论框架，技能上限从20拉到了500。

学术研究已经证明，精心策划的技能可以大幅提升Agent成功率，而自主生成的技能可能会降低性能。这意味着Skill的质量管理本身就是一门学问——不是所有“技能”都值得被封装和复用。

对于想投入Agent开发的产品、研发和算法工程师，一个务实的建议是：**不要再手搓Prompt了。**把你的业务逻辑、合规要求、最佳实践封装成Skill，让AI像装App一样加载专业技能。因为最终决定你Agent竞争力的，不是它会多少工具调用——那些MCP已经帮你标准化了；而是**你的Skill能不能把专业知识封装成可复用、可组合、可治理的能力单元。**

大模型必知必会：如何从0开始构建自己的Harness

原创 QinHsiu 阿Q的AI笔记 2026年8月22日 07:45 北京

2026年的AI圈有一个共识正在快速成型：**大模型的能力已经不再是瓶颈，真正的瓶颈在于如何让模型持续、稳定、可复用地产出专业价值。**

在2026世界人工智能大会（WAIC）上，业界达成共识：衡量AI价值的标尺不再是Benchmark跑分，而是其在真实业务流中自主规划、工具调用及闭环执行的成功率。从“对话”到“行动”的跃迁，催生了AI Agent工程化的三层技术栈：**MCP（连接层）→ Skill（能力层）→ Harness（工程层）**。本篇文章主要针对Harness的内容进行介绍。

一、什么是Harness

“Harness”这个词直译是“马具”：套在马身上那套缰绳和挽具。这个比喻特别准：**模型是一匹马力很强的马，它能跑，但你没法直接控制它往哪跑、跑多稳**；Harness就是那套缰绳和车厢，负责把马（模型）和“让马真正拉车干活”的组件——文件系统、终端、网页、API、权限、计划、上下文——绑在一起。

2026年5月斯坦福联合Meta发表的《Code as Agent Harness》进一步形式化了Harness的**三个基础属性**：

可执行性（Executability）：Harness能验证Agent实际做了什么，而不仅仅是它说了什么。一个只记录prompt和补全的Harness不具备可执行性，因为它无法区分成功的工具调用和幻觉出来的工具调用。

可检查性（Inspectability）：当某件事失败时，Harness产生可操作的诊断输出，指向prompt组装步骤的堆栈跟踪、标记了哪些规则和技能处于活动状态的friction事件、按组件划分的Token消耗。没有可检查性，调试Agent失败需要从日志中重构整个会话，“成本高昂且往往不完整”。类似于实践中的埋点操作，保证每次失败都是可以溯源的。

状态性（Statefulness）：Harness在会话之间保持连续性。会话状态被外部化，不存放在模型的上下文窗口中。当Agent在上下文重置后恢复时，它可以重建自己之前的位置，而不需要人类提供完整的简报。Anthropic的遥测数据显示，99.9百分位的会话持续时间在2026年1月已超过**45分钟**，而2025年10月是25分钟——在这个时长下，状态性不是可选项。

面试考点：Harness Engineering的定义

Harness Engineering是设计和优化模型外层执行系统的工程实践。Martin Fowler定义其为“控制论调节器”——通过前馈和反馈调节Agent行为。核心公式： $\text{Agent} = \text{Model} + \text{Harness}$ 。

二、Engineering演进路线

2026年，卡内基梅隆大学、耶鲁大学、亚马逊的研究团队联合发表了一篇Harness工程综述，将2022到2026年的工程重心变化概括为三个阶段：

阶段	时间	关注点
提示工程	2022-2024	优化单次模型调用的输入怎么写
上下文工程	2025	每一步该给模型看什么信息
Harness 工程	2026至今	模型之外的整个系统：状态管理、工具协调、反馈注入、约束施加、进展验证

这三个阶段不是替代关系，而是层层递进的**能力叠加**。进入Harness工程阶段，Agent开始处理长链条、多步任务，可靠性越来越取决于模型外层的基础设施。

Mitchell Hashimoto（Vagrant、Terraform作者）给出了一个操作性定义：

“Anytime you find an agent makes a mistake, you take the time to engineer a solution such that the agent will not make that mistake again.”

每当Agent犯了一个错误，你就花时间设计一个解决方案，让Agent在未来不会再犯同样的错误。 这个定义把Harness从“系统设计”变成了**持续迭代的错误处理闭环**。

面试考点：Harness的基础属性

可执行性：验证Agent实际做了什么，而非说了什么。只记录prompt的Harness无法区分成功调用和幻觉。

可检查性：失败时产生可操作的诊断输出——堆栈跟踪、活跃规则、Token消耗。没有它，调试需要重构会话。

状态性：会话间保持连续性。状态外部化，不存放在上下文窗口。Anthropic数据显示99.9百分位会话已超45分钟——状态性不是可选项。

三、Harness的组件和分层体系

3.1 Harness的九大组件

组件	核心职责	关键考量
Agent Loop引擎	驱动Agent持续运行的while循环——“思考→行动→观察→重复”	循环终止条件设计、最大步数限制、无限循环检测
工具系统	Agent能调用的所有工具的定义、注册和执行	工具描述的清晰度（模型依赖它做决策）、参数校验、错误处理、超时控制
上下文管理	管理什么信息进入模型上下文窗口、什么时候进、怎么压缩	窗口溢出处理、信息优先级排序、摘要与剪枝策略
记忆与状态	跨会话的持久化状态管理	状态序列化格式、读写延迟、会话恢复的完整性

组件	核心职责	关键考量
System Prompt组装	动态构建系统提示，整合角色定义、规则、技能描述	模板化设计、条件包含逻辑、Token预算分配
沙箱与执行环境	Agent代码在哪里跑、受到什么约束	文件系统隔离、网络访问控制、资源限制（CPU/内存/磁盘）
可观测性	全链路追踪、指标、日志	结构化日志格式、Trace关联、成本归因、失败现场保留
验证与评估	判断Agent的输出是否正确	评估器设计（LLM-as-judge vs 规则匹配）、误报/漏报率、评估成本
治理与安全	权限、审批、审计	最小权限原则、敏感操作审批流、审计日志不可篡改

这九个组件不是可选的装饰——它们是让一个“能聊天的模型”变成“能干活儿的Agent”的必备基础设施。**缺少任何一个，都会在特定场景下暴露致命短板。**

3.2 Harness的分层体系

2026年，来自CMU、Yale、JHU、UAB、Amazon等机构的联合综述论文，对Agent Harness进行了系统性定义。

Harness是模型外层的完整执行系统，负责把模型的推理能力转化为可靠、可观测、可治理的行动。 论文提出了ETCLOVG七层分类体系：

层级	名称	核心职责
E	执行环境与沙箱（Execution Environment & Sandbox）	Agent代码在哪里运行、受到什么约束——文件系统隔离、网络访问控制、资源限制（CPU/内存/磁盘）
T	工具接口与协议（Tool Interface & Protocol）	外部能力如何被描述、发现和调用——工具定义、MCP/A2A协议、动作空间管理、Schema验证
C	上下文管理（Context Management）	模型在短期、会话级和持久化层面能看到什么——窗口管理、压缩、检索、缓存、技能注入
L	生命周期与编排（Lifecycle & Orchestration）	组织状态的读写控制流——从单Agent循环、多Agent协作到完整工作流
O	可观测性（Observability）	捕获轨迹、成本、失败和可靠性信号——全链路监控、追踪、指标
V	验证（Verification）	把任务和轨迹转化为评估、失败归因和回归反馈——LLM-as-judge、轨迹评分、基准测试

层级	名称	核心职责
G	治理（Governance）	通过权限、身份、策略、安全加固、审计和人工监督约束系统行为

前四层（ETCL）构成Harness的结构核心，决定Agent“能做什么”；后三层（OVG）构成控制平面，决定Agent“做得对不对”。

四、逐步走向自进化的Harness

2026年的Harness研究已经从“怎么搭”进入“怎么自动优化”的阶段。**Harness从模型外部的工程脚手架，正在变成可以被生成、搜索、修复和持续迭代的研究对象。**

4.1 AutoHarness（Google DeepMind）

Google DeepMind的工作从一个具体问题切入：LLM Agent在环境交互中经常产生非法动作。传统做法需要工程师手写约束代码。AutoHarness让Gemini-2.5-Flash**自动生成代码形式的Harness**，用环境反馈不断修正动作合法性。实验在TextArena的145个游戏上进行，生成的代码能有效拦截非法动作，并在多个任务上帮助较小模型取得超过更大模型的表现。

面试考点：AutoHarness解决什么问题
解决LLM Agent在环境交互中频繁产生非法动作的问题。在Kaggle GameArena国际象棋中，Gemini-2.5-Flash的失败有78%是因为走出非法步。
做法：让Gemini-2.5-Flash自动给自己写代码外壳，维护两类能力——propose_action(board)给出动作，is_legal_action(board, action)判断合法性。环境报非法时，把出错轨迹、错误信息和旧代码一起喂回去，让模型继续改代码。
关键洞察：Agent的能力不只来自模型本体，也来自模型外那层被低估的系统结构

4.2 Meta-Harness（Stanford/MIT/KRAFTON）

Meta-Harness把Harness当成**可搜索、可优化的代码对象**：系统读取候选Harness的源码、历史分数和执行轨迹，再提出新的修改方案。实验覆盖在线文本分类、RAG数学推理和Agentic Coding任务。在在线文本分类任务上，方法比强基线高7.7分，同时上下文Token消耗减少约四分之一。

4.3 AHE：Agentic Harness Engineering（复旦/北大/奇绩智锋）

AHE提出的核心思路是：**把优化焦点从模型内部的System Prompt转向外部的完整Harness。**

AHE通过三类可观测性形成闭环：

- **组件可观测性**：每个可编辑的Harness组件都有文件级表示
- **经验可观测性**：把数百万Token的原始轨迹蒸馏成可消费的证据
- **决策可观测性**：每次编辑都附带自声明的预测，后续用任务结果验证

实验数据：**10轮AHE迭代将Terminal-Bench 2上的pass@1从69.7%提升到77.0%**，超越了人工设计的Codex-CLI（71.9%）和自进化基线ACE、TF-GRPO。**关键发现：提升主要来自tools、middleware和long-term memory的优化，而不是system prompt。**这说明Harness的结构性工程经验是可迁移的，而prompt层面的策略不可迁移。

更值得关注的是：**冻结的Harness无需重新进化即可迁移。**在SWE-bench-verified上，它用比种子少12%的Token取得了更高的总体成功率；在Terminal-Bench 2上，在三个不同模型家族上跨家族增益达+5.1到+10.1个百分点。

4.4 Self-Harness: 让Agent自己改进自己的Harness

Self-Harness提出一个新范式：**LLM Agent在不依赖人工工程师或更强外部Agent的情况下，改进自己的运行Harness。**这指向一个更远的图景——Agent不仅被Harness塑造，也能参与重新塑造Harness。

4.5 DeepSeek Harness: “一切皆插件”的可组合底座

2026年8月13日发布，MIT开源，6天收获16.7万星标。核心设计是“**一切皆插件**”——模型适配器、工具注册表、会话日志、Agent循环本身全部可替换。背后的**Cordis元框架**通过**可逆效应**要求插件声明卸载与状态还原操作，理论上赋予Agent长时运行中热重载组件的能力。

代价：高度可扩展意味着高复杂度，开发者需理解Cordis、Context、Effect、Service、Provider等大量概念。同时，DeepSeek Harness不自动完成企业级产品工作——统一UI、权限后台、凭证管理、多租户隔离、成本统计仍需自行建设。

DeepSeek Harness，支撑架构的是**Cordis元框架**。Cordis只提供六种最基础的操作：

- use：装一个组件
- effect：记录一次可撤销修改
- set：提供一种能力
- get：读取一种能力
- isolate：把同一种能力隔离开
- intercept：在能力外面加一层使用规则

一个插件缺少依赖，就不启动，依赖齐了再加载。依赖的组件被替换，它会重新加载。插件退出，它留下的作用会按记录撤销。一个DeepSeek Harness插件：

```
1 // scratch-plugin/src/my-plugin.ts
2 import type { Context } from '@deepseek-ai/cordis'
3 import { defineTool } from '@deepseek-ai/dsh-tools'
4
5 export function apply(ctx: Context) {
6   ctx.tools.register(defineTool({
7     name: 'get_current_time',
8     description: 'Get the current time in a given timezone',
9     inputSchema: {
10       type: 'object',
```

```

11     properties: {
12         timezone: { type: 'string', default: 'UTC' }
13     },
14 },
15 async execute({ timezone }) {
16     return new Date().toLocaleString('en-US', { timeZone: timezone })
17 }
18 )))
19 }

```

挂载插件： `dsh web --patch ./cordis.yml` 。

四种预设模式：

- **标准模式**：完整编码Agent，含文件系统、Shell、子Agent
- **极简模式**：仅保留bash和str_replace_editor
- **代码模式**：生成TypeScript SDK，让模型直接写程序，将五次往返压缩为单次调用
- **创作者模式**：专为自定义预设设计

沙箱机制：本地后端通过Node插件将子进程封装在Linux Landlock、macOS Seatbelt或Windows ACL受限令牌运行器中。

4.6 OpenAI Codex Harness：“计算-控制分离”的工业工程主义

OpenAI的策略是**控制平面（Harness）与计算平面（Sandbox）物理分离**。Harness专职调度，所有真实文件读写、代码编译、系统命令调用下沉至独立沙盒（macOS seatbelt、Linux landlock、Windows AppContainer）。集成Temporal.io持久化引擎与SQLite状态机，提供容错快照与恢复能力。

产品化分层：

- **Codex Exec**：一次性脚本/CI/CD流水线，返回确定性JSON结果
- **Codex SDK**：TypeScript/Python API级控制，启动/暂停/恢复/流式传输
- **Codex App Server**：通过JSON-RPC暴露完整Agent生命周期，可嵌入IDE插件、协作平台

Codex Harness的优势来自**已在真实产品入口验证**——Codex App、IDE体验、Codex Web等不同入口共享同一套底层Harness。

五、LangChain的Harness优化案例

LangChain团队用“Trace分析+迭代优化”的Harness Engineering方法，将编码Agent在Terminal-Bench 2.0上的表现从52.8%提升到66.5%，提升13.7个百分点，**全程保持模型固定（GPT-5.2-Codex）**。

核心流程：

1. **Trace收集**：所有Agent动作存入LangSmith，含延迟、Token数、成本
2. **Trace分析Skill**：并行错误分析Agent→主Agent综合发现→提出Harness修改建议

3. 针对性修改：聚焦System Prompt、Tools、Middleware三个维度

关键发现：最常见的失败模式是Agent写方案后重读代码、确认看起来没问题就停止——**不执行测试验证**。解决方案是在System Prompt中加入“Plan→Build→Verify→Fix”结构化指引，同时用**PreCompletionChecklistMiddleware**在Agent退出前强制提醒其做验证。这说明Harness的优化往往在“防漏”而非“提智”。

六、一个Mini Harness的最小实现

以下代码展示了一个从零构建的Mini Harness核心结构：

```
1 class MiniHarness:
2     def __init__(self, model_name: str):
3         self.model_name = model_name
4         self.tools = {}
5         self.memory = []
6         self.max_steps = 10
7
8     def register_tool(self, name: str, func: callable, description: str, sche
9         """注册工具"""
10        self.tools[name] = {
11            "func": func,
12            "description": description,
13            "schema": schema
14        }
15
16    def build_system_prompt(self) -> str:
17        """动态构建系统提示"""
18        tool_descriptions = "\n".join([
19            f"- {name}: {info['description']}"
20            for name, info in self.tools.items()
21        ])
22        return f"""You are an AI assistant with access to these tools:
23 {tool_descriptions}
24
25 Available actions:
26 - To use a tool: respond with JSON {"action": "tool_name", "args": {{...}}}
27 - To answer: respond with JSON {"action": "answer", "content": "..."}
28 """
29
30    def execute_task(self, task: str) -> str:
31        """执行任务的主循环"""
32        messages = [{"role": "system", "content": self.build_system_prompt()}
33                    {"role": "user", "content": task}]
34
35        for step in range(self.max_steps):
```

```
36         # 调用模型
37         response = self.call_llm(messages)
38         action = json.loads(response)
39
40         if action["action"] == "answer":
41             return action["content"]
42
43         # 执行工具
44         tool = self.tools.get(action["action"])
45         if tool:
46             result = tool["func"](**action["args"])
47             messages.append({"role": "assistant", "content": response})
48             messages.append({"role": "user", "content": f"Tool result: {r
49         else:
50             return f"Error: Unknown tool {action['action']}"
51
52     return "Max steps exceeded"
```

这个Mini Harness只有不到50行核心代码，但它已经包含了Harness的三大基础属性：

- **可执行性**：实际执行工具调用，而不是只输出计划
- **可检查性**：每一步的输入输出都可追溯
- **状态性**：messages列表维护了完整的会话状态

七、Harness的应用领域&开放挑战

7.1 应用领域

应用领域	核心场景	实践佐证
编码助手（Coding Assistants）	Agent在活跃仓库上编写补丁、测试和问题解决 workflow	Claude Code + Opus 4.7 在 SWE-bench Verified 上达 87.6% 通过率；vexp + Claude Code 达 73.0% pass@1，单任务成本仅 \$0.67 。仅优化 Harness 设计，同一模型编码基准测试分数可从 6.7% 跃升至 68.3%
GUI/OS 自动化（GUI/OS Automation）	Agent合成并执行基于DOM树、辅助功能API的命令	OSWorld 基准上，实在 Agent 以 90.2% 成功率登顶总榜；OSWorld 2.0 中 Claude Opus 4.8 仅完成 20.6% 的长周期任务；阿里 Qwen-UI-Agent 在 MobileWorld 达 82.1%，真机基准达 92.2%
具身智能体（Embodied Agents）	机器人策略生成、物理环境交互与技能库构建	Voyager 在 Minecraft 中获得 63 个独特物品 【论文检索报告#3】；EvoCUA-32B 在 WindowsAgentArena 上达 56.48% ，超越 UI-TARS-2 约 6 个百分点

应用领域	核心场景	实践佐证
科学发现 (Scientific Discovery)	假设生成、实验设计、数据分析与验证闭环	FutureHouse 的 Robin 多智能体系统登上 Nature , 2 小时内完成人类科学家数月工作量, 自主发现 dAMD 新型治疗候选物; 上海 AI 实验室“书生·端砚”平台支持全链路科研
个性化与推荐 (Personalization & Recommendation)	用户行为建模、偏好学习与自适应策略	通过 Harness 维护用户画像和交互历史, 实现个性化推荐策略的动态调整
DevOps 与企业工作流 (DevOps & Enterprise Workflows)	CI/CD 流水线自动化、安全漏洞修复、基础设施即代码	Harness 平台推出 Autonomous Worker Agents, 支持流水线、测试、安全、部署全流程 Agent 化; 零日漏洞 Agent 持续监控威胁情报并自动生成补丁
多智能体编排 (Multi-Agent Orchestration)	多角色分工、辩论、红队对抗、共享状态同步	Robin 由 Crow (文献检索)、Falcon (深度综述)、Finch (数据分析) 三个智能体分工协作; DeepSeek Harness 支持 Claude Code 和 Codex 作为子 Agent 调度

7.2开放挑战

挑战	核心问题	研究进展与实践
Harness 级别评估与 Oracle 充分性	超越最终任务成功的评估——如何衡量 Harness 的效率、稳定性与可控性	六大工程化指标被提出: 任务完成率、探索成本、验证能力、改动质量、可控性、团队适配度; HarnessEval 由 15 家学术机构联合发布; 需警惕“搜索预算混淆”与“任务重合偏差”两大评估盲区
超越可执行反馈的语义验证	在不完整或非确定性反馈下验证 Agent 行为正确性	单纯依赖“能跑通”不足以证明正确性; 德勤企业标准: 关闭自动重试后首次执行成功率低于 90% 则架构失败; 形式化验证 (Lean4Agent、VERINA) 正在探索【论文补充说明】
无回归的 Harness 自我进化	Harness 改进不能以牺牲已有能力为代价	翁荔指出: 递归自我改进更可能从 Harness 成为优化对象开始; 但实验表明 Harness 自进化在 Terminal-Bench 2.1 上未超过简单重复采样, 跨任务迁移平均提升仅 0.6 分 ; EverMind 通过 Harness Gene Bank 防止任务过拟合
事务性共享程序状态与语义冲突解决	多 Agent 共享 Harness 状态的一致性与冲突消解	Shared Harness State 记录参与者、读写操作、假设、证据与冲突; DeepSeek Harness 支持多 Agent 共享 Workspace; “约束丢失、上下文污染与验收失真”是核心痛点
人机回环安全与问责作为 Harness	安全关键操作的人类监督与责任归属	Komina Agent Harness 将人类审批持久化为执行时工件; 但专家指出 Human-in-the-loop 无法

挑战	核心问题	研究进展与实践
ss 状态		规模化 ；需从“人在回路”演进为“人在 Harness 设计回路”
多模态 Code-Harness 系统	将 Harness 扩展到图像、视频等多模态输入	DeepSeek Harness v0.1.0-rc.8 已支持原生图片请求，/goal、/plan 等命令可接收图文混合输入；多模态 Agent 需处理视觉理解、UI 操作与代码执行协同
迈向 Harness 科学	建立 Harness 工程的理论基础与可复现方法论	当前 Harness 设计多为经验驱动，缺乏统一理论框架；需建立 Harness 的“第一性原理”——可执行性、可检查性、状态性作为基础公理；开源社区与学术界正在共同推进这一方向

写在最后

从CMU综述的ETCLOVG七层框架、DeepMind的AutoHarness自动生成、Stanford的Meta-Harness代码搜索、复旦/北大的AHE可迁移工程经验，到LangChain的Trace驱动优化——2026年的Harness已经从“模型外面的壳”变成了一个独立的工程领域。

Harness Engineering的三层演进告诉我们：**模型看“智商”，Harness看“系统工程能力”**。2026年，谁能在Harness层建立真正的工程能力，谁就能拿到那决定性的30个百分点。

大模型必知必会 · 目录

上一篇
大模型必知必会：如何从0开始构建自己的 Skill

下一篇
大模型必知必会：一文带你了解大模型的 Scaling Law

作者提示: 个人观点，仅供参考

大模型必知必会：一文带你看懂2026年主流Agent框架全景图

原创 QinHsiu 阿Q的AI笔记 2026年9月11日 07:45 北京

2026年是Agent框架爆发的一年。DeepSeek Harness、Codex Harness相继开源，OpenCode从0到18万Star，LangGraph突破9万Star，CrewAI月下载量冲到520万。市面上声称能做Agent的框架超过25个。

但“多”不等于“好”。本文按**Harness类、编排类、平台类**三大类别，整理八大主流框架的定位、优缺点和上手方式，帮你找到最适合自己的那一款。

Agent Harness / 框架八方对比								
框架	DeepSeek Harness	Codex Harness	OpenCode	LangGraph	CrewAI	AutoGen	Diffy	Coez/影子
类型	Harness类	Harness类	Harness类	编排类	编排类	编排类	平台类	平台类
发布方	DeepSeek AI	OpenAI	社区/5ST	LangChain Inc	CrewAI Inc	Microsoft	LangGenius	字节跳动
开源协议	MIT	Apache-2.0	Apache-2.0	MIT	MIT	MIT	Apache-2.0	Apache-2.0
核心语言	TypeScript	Rust(96.4%)&TS	Go	Python/TS	Python	Python	Python/TS	平台
GitHub Stars	~19.6万	~11.8万	~18万+	~5万	~5.1万	~5.8万	~15.4万	~2.15万
核心哲学	一切皆插件	Product First	模型无关	状态驱动	角色化团队	对话驱动协作	可定制化工作流	零代码/低代码
一句话描述	可插拔的Agent运行时框架	已产品验证的接入平台	将海量的应用适配Agent	有状态的多智能体编排	角色驱动的多智能体协作	多智能体对话框架（维护中）	LLM应用开发平台	零代码Agent平台
快速上手	<code>npm i -g @deepseek-ai/dsh web</code>	<code>npm i -g @openai/codex</code>	<code>curl -fsSL https://opencode.ai/install bash</code>	<code>pip install langgraph</code>	<code>crewai create flow</code>	<code>pip install pyautogen</code>	<code>docker run --rm ghcr.io/langgenius/dify</code>	阿Q的AI笔记

一、Harness类

Harness是2026年最热的概念。**模型是“大脑”，Harness是“执行神经系统”**——负责调度工具、管理任务状态、执行闭环。一个生产级Harness要兜住五件事：任务循环、上下文管理、记忆持久化、沙箱与审批、可观测性。

1.1 DeepSeek Harness

维度	说明
核心定位	Cordis（DI/插件运行时）之上的一层“配置即组合”的AI Agent平台。核心理念是“ 一切皆插件 ”——模型适配器、工具注册表、会话日志、Agent Loop本身都是插件，全部可从配置替换。 没有特权核心 。
源码架构	227个包、21.5万行TypeScript。底层由约2693行的Cordis微内核支撑，提供插件的加载、卸载与依赖管理。Cordis的核心机制包括：插件（带 <code>apply(ctx, config)</code> 的函数）、上下文（服务容器）、依赖注入（ <code>inject</code> 声明）、类型化事件（ <code>emit/waterfall/parallel/serial</code> 四种分发模式）、可逆副作用（插件卸载时自动回滚）。
自研借鉴点	可逆副作用机制——插件卸载时注册的服务、监听的事件、创建的资源 自动回滚 ，不需要开发者手动清理。这是插件化架构最关键的设计决策。
优点	极高灵活性，所有组件可替换；MIT协议商用无忧；Scope隔离支持多异构Agent同进程运行；事件溯源原生支持审计和恢复。
缺点	学习曲线陡（需先理解Cordis机制）；目前只有Web UI，无CLI/TUI；官方明确为开发者预览版，会有破坏性变更；实测反馈速度偏慢、Token消耗大。

维度	说明
适用场景	深度定制、模型自由切换、构建企业级Agent平台。
上手方式	<code>npx @deepseek-ai/dsh web</code> ，浏览器打开 <code>http://127.0.0.1:3080</code> ，Settings → Models 填入API Key。

```
1 # =====
2 # DeepSeek Harness (dsh) 一键上手完整 Coding 示例
3 # 包含：环境准备 → 安装启动 → API Key 配置 → 核心运行模式
4 #           → 插件开发 → 命令行工具 → 排错指南
5 # dsh = DeepSeek 开源的 Agent 运行框架（MIT 协议）
6 # 核心理念：Agent = Model + Harness，一切皆插件
7 # 官方仓库：github.com/deepseek-ai/deepseek-harness
8 # 发布时间：2026 年 8 月 13 日
9 # =====
10
11 # -----
12 # 第一步：环境检查
13 # dsh 核心版本依赖 Node.js，最低要求 v22.19 LTS 及以上
14 # Python SDK 需要 Python 3.10+
15 # 支持 Windows、macOS、Linux[reference:0]
16 # -----
17 node --version                # 需 v22.19.0 或更高
18
19 # 如果未安装，去 Node.js 官网下载 LTS 版本
20 # 小技巧：如果明明装过 Node.js 但终端提示找不到命令，
21 # 关闭当前终端重新打开——很多时候只是环境变量还没刷新[reference:1]
22
23 # -----
24 # 第二步：安装并启动 DeepSeek Harness
25 # 提供四种安装方式，按需选择
26 # -----
27
28 # --- 方式一：npx 零安装快速启动（新手首选，推荐初次体验） ---
29 # 一条命令自动下载并启动 Web UI
30 npx @deepseek-ai/dsh web
31
32 # 首次运行需要一两分钟下载依赖，属正常现象[reference:2]
33 # 启动后 Web UI 默认运行在 http://127.0.0.1:3080[reference:3]
34
35 # --- 方式二：源码安装（适合需要二次开发的开发者） ---
36 # git clone https://github.com/deepseek-ai/deepseek-harness.git
37 # cd deepseek-harness
38 # pnpm install
39 # pnpm run build
```

```
40 # pnpm dsh web[reference:4]
41
42 # --- 方式三: Docker 部署 (适合云服务器长期运行) ---
43 # docker run -d --name dsh -p 3080:3080 ghcr.io/huoxue1/deepseek-harness:lat
44
45 # --- 方式四: 桌面版 (不想折腾命令行的用户) ---
46 # 在 GitHub Release 页面下载 DeepSeekHarness-win32-x64-*.zip, 解压即用[referen
47
48 # --- 验证安装 ---
49 dsh --version # 当前版本为 0.1.0-rc.8 左右[reference
50
51 # -----
52 # 第三步: 配置 API Key (必做)
53 # dsh 兼容 OpenAI 协议接口, 可对接多种大模型服务
54 # 密钥以只写方式存入 ~/.dsh/.credentials.yaml
55 # 配置文件 ~/.dsh/settings.yaml 从不携带密钥值[reference:8]
56 # -----
57
58 # --- 方式一: Web UI 配置 (推荐新手) ---
59 # 1. 浏览器打开 http://127.0.0.1:3080
60 # 2. 进入 设置 → 模型 (Settings → Models)
61 # 3. 找到 DeepSeek 卡片 (或对应提供方), 粘贴 API Key
62 # 4. 保存后即可使用[reference:9]
63
64 # --- 方式二: 环境变量注入 (推荐, 避免密钥明文泄露) ---
65 export DEEPSEEK_API_KEY="sk-你的密钥" # 根据你使用的服务商替换
66
67 # --- 方式三: 直接编辑 settings.yaml ---
68 # 编辑 ~/.dsh/settings.yaml, 添加供应商和默认模型:
69 # agent-default-model:
70 #   provider: deepseek
71 #   model: deepseek-v4-flash
72 # llm-pi-ai:
73 #   providers:
74 #     deepseek:
75 #       api: openai-completions
76 #       baseURL: https://api.deepseek.com/v1
77 #       apiKeyEnv: DEEPSEEK_API_KEY
78 #       models:
79 #         - id: deepseek-v4-flash
80 #         - id: deepseek-v4-pro
81
82 # -----
83 # 第四步: 选择运行模式
84 # dsh 提供四种预设模式, 本质是加载不同的插件集合
85 # 启动后在 Web UI 中选择, 或通过命令行指定 profile
```

```
86 # -----
87
88 # --- 标准模式（全部工具组件集成） ---
89 # 适合日常开发与集成，文件读写、Shell、网页抓取全部可用
90
91 # --- 极简模式（仅 Bash + 编辑器） ---
92 # 适合最小环境下的模型基准测试
93
94 # --- PTC 模式（模型自己写 TypeScript 编排工具调用） ---
95 # 大幅降低 Token 消耗——中间数据留在环境里，仅最终结果进入上下文
96
97 # --- 创造模式（Agent 检查运行时、挂载/卸载插件） ---
98 # 适合自进化 Agent 实验，Agent 可以改装自身
99
100 # 选择工作区目录后，agent 会自动读取文件、执行命令、规划任务
101
102 # -----
103 # 第五步：跑通第一个任务
104 # 在 Web UI 中选择工作区目录，然后发出任务
105 # -----
106
107 # 示例任务：
108 # "读取 src/utils.ts，分析其中的性能瓶颈，并给出优化建议"
109
110 # agent 会自动：
111 # 1. 读取文件内容
112 # 2. 分析代码结构
113 # 3. 识别潜在瓶颈
114 # 4. 输出优化方案
115
116 # -----
117 # 第六步：编写你的第一个插件（三个文件即可）
118 # 插件就是一个模块，导出三样东西：name、inject、apply
119 # 以"文本转大写"工具为例[reference:10]
120 # -----
121
122 # --- 创建插件目录 ---
123 mkdir dsh-plugin-hello
124 cd dsh-plugin-hello
125
126 # --- 创建 package.json ---
127 cat > package.json << 'EOF'
128 {
129     "name": "dsh-plugin-hello",
130     "version": "0.1.0",
131     "description": "Hello plugin for DeepSeek Harness",
```

```
132     "type": "module",
133     "main": "index.js",
134     "license": "MIT",
135     "dependencies": {
136         "@deepseek-ai/dsh-base": "latest"
137     }
138 }
139 EOF
140
141 # --- 创建插件主入口 index.js ---
142 cat > index.js << 'EOF'
143 // 插件身份: 行 id 用它
144 export const name = "hello-tool";
145
146 // 依赖声明: 需要 tools 服务 (工具注册表)
147 export const inject = ["tools"];
148
149 // 插件被挂载时执行, ctx 上挂着所有注入的服务
150 export function apply(ctx) {
151     // 注册一个工具—三步: name / description / parameters + execute
152     ctx.tools.register(
153         defineTool({
154             // agent 看到的名字—模型靠它点名调用, 起名要表意
155             name: "uppercase",
156
157             // 决定 agent 会不会用—写清触发场景
158             description: "将输入文本转为大写。当用户要求转换大小写时使用。",
159
160             // 参数声明: 类型、必填一目了然, 决定调用时的输入校验
161             parameters: {
162                 text: { type: "string", required: true }
163             },
164
165             // 工具的实际执行逻辑
166             async execute(params) {
167                 return { text: params.text.toUpperCase() };
168             }
169         })
170     );
171 }
172 EOF
173
174 # --- 挂载插件到 profile ---
175 # 在 profile 配置中加一行:
176 # - id: hello-tool
177 #   name: "dsh-plugin-hello"    # 你的包名 + 导出路径
```

```
178
179 # 挂载后 agent 的工具列表里就多出 uppercase[reference:11]
180
181 # --- 常见问题排查 ---
182 # 挂载后工具不出现? → dump 配置树确认行已加载; 查插件加载日志
183 # agent 不用你的工具? → 检查 description 是否写清触发场景
184 # inject 的服务拿不到? → 确认依赖行也在组合里[reference:12]
185
186 # -----
187 # 第七步: 命令行工具速查
188 # -----
189 # 启动 Web UI:      npx @deepseek-ai/dsh web
190 # 查看版本:         dsh --version
191 # 添加插件:         dsh plugin --profile web add <包名或路径>
192 # dump 配置树:      dsh dump                # 查看当前加载的配置
193 # 源码构建启动:     pnpm dsh web
194
195 # -----
196 # 第八步: 排错指南
197 # -----
198 # 问题: npx 命令卡住不动
199 # 解决: 首次运行需要下载依赖, 等待 1-2 分钟属正常
200 #       如果超过 5 分钟, 检查网络或 npm 源 (可换淘宝源)
201
202 # 问题: Web UI 打不开
203 # 解决: 确认端口 3080 未被占用 (lsof -i:3080)
204 #       检查终端是否显示启动成功
205
206 # 问题: API Key 配置后仍报错
207 # 解决: 确认密钥是否有效、baseUrl 是否正确
208 #       环境变量方式需确保终端中已 export
209
210 # 问题: 插件加载失败
211 # 解决: 检查 inject 声明的服务是否在组合中存在
212 #       检查插件导出的 name 和 apply 是否正确
213
214 # =====
215 # 完成。复制以上命令即可从零跑通 DeepSeek Harness。
216 #
217 # 核心记忆点:
218 #   安装:      npx @deepseek-ai/dsh web
219 #   访问:      http://127.0.0.1:3080
220 #   配置:      Settings → Models 填入 API Key (或 export DEEPSEEK_API_KEY)
221 #   模式:      标准 / 极简 / PTC / 创造 (四种预设, 按需切换)
222 #   插件三要素: name (身份) + inject (依赖) + apply (注册)
223 #   工具三步:  name (表意) + description (触发场景) + parameters (校验)
```

224	#	自研借鉴：可逆副作用 / 事件溯源 / waterfall 插桩点 / Scope 隔离
225	#	=====

1.2 Codex Harness

维度	说明
核心定位	OpenAI于 2026年8月19日 以 Apache-2.0 协议开源的Agent底层执行框架。以 Rust核心 为执行底座、以 TypeScript SDK 为集成接口的生产级Agent运行时。
架构全景	Rust代码占比96.4%，由约120个crate组成，可拆分为四层——运行时核心（core/core-api）、协议层（app-server/exec-server）、沙箱与安全（sandboxing/exe cpolicy）、工具与扩展（tools/skills/mcp-server）。
三层集成接口	<code>codex exec</code> ：CLI、脚本、一次性后台任务——非交互执行、结构化结果、明确退出状态。 Code x SDK ：应用代码里的Agent工作流——TypeScript+Python双接口，启动、恢复、流式读取任务。 App Server ：Agent本身就是产品的一部分——JSON-RPC持久连接，支持持久会话、事件流、中断、审批。
关键数据	在ARC-AGI-3基准测试中，同一个模型GPT-5.6 Sol，默认配置得分13.3%，仅开启 Harness层的“保留推理+上下文压缩”两项调整后，得分飙升至 38.3% ，输出Token消耗减少至原来的 1/6 。 模型没变，Harness配置变了，结果差三倍。
优点	Rust原生性能；系统级硬隔离沙箱（Linux bubblewrap + Windows原生安全沙箱）；三层集成梯度适配全场景；Token效率优化显著。
缺点	与OpenAI生态深度绑定；非“完全开源”（IDE Extension与Cloud未开源）；Rust技术栈门槛高。
适用场景	深度使用OpenAI生态、想把Agent嵌入现有产品的团队。
上手方式	<code>npm i -g @openai/codex</code> ，配置API Key后运行 <code>codex exec --sandbox read-only "总结这个仓库"</code> 。

1	# =====
2	# Codex Harness 一键上手完整 Coding 示例
3	# 包含：安装 → 配置 → codex exec → SDK → app-server
4	# =====
5	
6	# -----
7	# 第一步：安装 Codex CLI
8	# -----
9	npm i -g @openai/codex
10	
11	# 确认版本（完整 Harness 功能需 v0.149.0+）

```
12 codex --version
13
14 # -----
15 # 第二步: 配置模型 (以 DeepSeek 为例)
16 # 写入 ~/.codex/config.toml
17 # -----
18 cat > ~/.codex/config.toml << 'EOF'
19 model = "deepseek-v4-flash"
20 model_provider = "deepseek"
21
22 [model_providers.deepseek]
23 name = "DeepSeek"
24 base_url = "https://api.deepseek.com/v1"
25 env_key = "DEEPSEEK_API_KEY"
26 EOF
27
28 # 注入 API Key
29 export DEEPSEEK_API_KEY="sk-你的密钥"
30
31 # -----
32 # 第三步: codex exec — 最轻量接入
33 # 必须在 Git 仓库目录下运行, 否则报 "Not inside a trusted directory"
34 # -----
35 cd /path/to/your-git-repo
36
37 # 只读沙箱: Agent 只看不改
38 codex exec --sandbox read-only "总结这个仓库并列出 5 个风险点"
39
40 # 工作区可写: 允许在工作区内创建/修改文件
41 codex exec --sandbox workspace-write "给这个仓库新建一个 README.md, 写一段项目简介"
42
43 # 输出 JSON (适合自动化)
44 codex exec --json "总结仓库结构" | jq
45
46 # 输出写入文件
47 codex exec -o response.md "解释架构"
48
49 # 指定模型
50 codex exec -m gpt-5.6 "Review the current changes"
51
52 # -----
53 # 第四步: Codex SDK — 用 TypeScript 编排 Agent
54 # -----
55 # 安装 SDK
56 npm install @openai/codex-sdk
57
```

```

58 # 创建文件: codex-demo.ts
59 cat > codex-demo.ts << 'EOF'
60 import { Codex } from "@openai/codex-sdk";
61
62 // 初始化 Codex 客户端
63 const codex = new Codex();
64
65 // 启动一个新线程
66 const thread = codex.startThread();
67
68 // 运行一个 Turn（一次用户请求）
69 const turn = await thread.run("诊断 src/utils.ts 中的性能问题");
70
71 // 打印最终响应
72 console.log(turn.finalResponse);
73
74 // 流式读取（适合实时 UI）
75 const { events } = await thread.runStreamed("分析这个代码库");
76 for await (const event of events) {
77     // 仅处理 Agent 消息增量
78     if (event.type === "item/agentMessage/delta") {
79         process.stdout.write(event.delta);
80     }
81 }
82 EOF
83
84 # 运行 TypeScript 示例（需 ts-node 或编译后运行）
85 # npx ts-node codex-demo.ts
86
87 # -----
88 # 第五步: app-server — 把 Agent 嵌入产品
89 # 以 JSON-RPC 2.0 协议运行，支持持久会话、事件流、中断、审批
90 # -----
91 # 启动 app-server
92 codex app-server
93
94 # 以下是发送给 app-server 的 JSONL 消息示例（每行一个 JSON 对象）
95 # 初始化握手
96 echo '{"method": "initialize", "id": 0, "params": {"clientInfo": {"name": "r
97 echo '{"method": "initialized", "params": {}}}'
98
99 # 创建线程
100 echo '{"method": "thread/start", "id": 1, "params": {"model": "gpt-5.6", "cv
101
102 # 运行 Turn
103 echo '{"method": "turn/start", "id": 2, "params": {"threadId": "thread_xxx",

```

```
104
105 # -----
106 # 关键 Thread 生命周期方法
107 # -----
108 # thread/start      创建新线程
109 # thread/resume     按 ID 恢复已有线程
110 # thread/fork       从历史分支出新线程
111 # thread/rollback   丢弃最近 N 个 Turn
112 # thread/compact/start  压缩历史
113
114 # -----
115 # 三层接口选用决策
116 # -----
117 # codex exec      → CI、脚本、一次性后台任务
118 # Codex SDK       → 应用代码里的 Agent 工作流（TS/Python）
119 # App Server      → Agent 本身就是产品的一部分，需要持久会话
120
121 # =====
122 # 完成。复制以上命令即可从零跑通 Codex Harness。
123 # =====
```

1.3 OpenCode

维度	说明
核心定位	开源、模型无关的终端AI编程Agent。2026年1月Anthropic封禁第三方调用Claude Code服务后，OpenCode应运而生——本质上是供应商锁定的集体反弹。
核心差异点	模型无关 ——支持75+种模型提供商（Anthropic、OpenAI、Google Gemini、DeepSee k、Ollama、llama.cpp等）； Plan/Build双模式 ——Plan只分析不改代码，Build执行修改，Tab键一键切换； 终端原生 ——基于Go和Bubble Tea构建TUI； 主/子Agent分层架构 ——主Agent负责任务分解，子Agent按需动态创建、无状态隔离、并发执行。
技术架构	采用Bun + Turbo构建的Monorepo结构，四层设计（交互层/适配层/Agent层/基础设施层）。
优点	模型选择权完全在你手中；工具完全免费；支持CLI/TUI/桌面三端；Go语言构建，轻量高效。
缺点	专注编程场景，非通用Agent框架；Go生态插件丰富度不如TypeScript/Python。
适用场景	终端AI编程、反供应商锁定、多模型切换开发。

维度	说明
上手方式	<code>curl -fsSL https://opencode.ai/install bash</code> ，进入项目目录执行 <code>opencode</code> 。

```
1 # =====
2 # OpenCode 一键上手完整 Coding 示例
3 # 包含: 安装 → 配置模型 → 启动 TUI → Plan/Build 模式 → CLI 运行
4 # OpenCode = 开源、模型无关的终端 AI 编程 Agent
5 # =====
6
7 # -----
8 # 第一步: 环境检查
9 # OpenCode 依赖 Node.js 18 及以上
10 # -----
11 node -v
12
13 # -----
14 # 第二步: 安装 OpenCode
15 # 方式一: 一键安装脚本 (最推荐)
16 # -----
17 curl -fsSL https://opencode.ai/install | bash
18 # 如果希望安装到自定义目录 (如 /usr/local/bin):
19 # OPENCODE_INSTALL_DIR=/usr/local/bin curl -fsSL https://opencode.ai/install
20
21 # 方式二: npm 全局安装 (已有 Node.js 环境)
22 # npm i -g opencode-ai@latest
23
24 # 方式三: 包管理器安装
25 # macOS/Linux: brew install anomalyco/tap/opencode
26 # Windows:      scoop install opencode
27 # Windows:      choco install opencode
28
29 # 验证安装成功 (出现版本号即安装完成)
30 opencode --version
31
32 # -----
33 # 第三步: 配置 AI 模型
34 # OpenCode 本身免费, 但你需要自己准备一个 AI 模型的 API Key
35 # 支持 75+ 模型提供商: Anthropic、OpenAI、Google Gemini、DeepSeek、Ollama 等
36 # -----
37
38 # 方式一: 环境变量 (最快上手)
39 # 根据你使用的模型提供商, 选择对应的一行取消注释:
40
41 # export ANTHROPIC_API_KEY="your-key-here" # Claude 系列
```

```
42 # export OPENAI_API_KEY="your-key-here"           # GPT 系列
43 # export GEMINI_API_KEY="your-key-here"           # Google Gemini
44 # export DEEPSEEK_API_KEY="your-key-here"         # DeepSeek
45
46 # Windows PowerShell 写法:
47 # $env:ANTHROPIC_API_KEY = "your-key-here"
48
49 # -----
50 # 方式二: TUI 内 /connect 命令 (推荐新手, 交互式配置)
51 # 先启动 OpenCode, 再在界面内配置。详见第四步。
52 # -----
53
54 # 方式三: 配置文件 (最灵活, 可多 Provider 共存)
55 # 在项目根目录或 ~/.config/opencode/ 下创建 opencode.json
56
57 # 配置文件示例 (DeepSeek + Ollama 本地模型):
58 cat > ~/.config/opencode/opencode.json << 'EOF'
59 {
60     "$schema": "https://opencode.ai/config.json",
61     "theme": "opencode",
62     "model": "deepseek/deepseek-v4-flash",
63     "provider": {
64         "deepseek": {
65             "npm": "@ai-sdk/openai-compatible",
66             "name": "DeepSeek",
67             "options": {
68                 "baseUrl": "https://api.deepseek.com/v1",
69                 "apiKey": "sk-你的密钥"
70             }
71         },
72         "ollama": {
73             "npm": "@ai-sdk/openai-compatible",
74             "name": "Ollama (本地)",
75             "options": {
76                 "baseUrl": "http://localhost:11434/v1"
77             }
78         }
79     }
80 }
81 EOF
82
83 # -----
84 # 第四步: 启动 OpenCode
85 # 进入你的项目目录, 运行 opencode 启动 TUI
86 # -----
87 cd /path/to/your-project
```

```
88  opencode
89
90  # -----
91  # 第五步: TUI 内配置 Provider (首次使用推荐)
92  # 启动后, 在输入框中输入以下斜杠命令:
93  # -----
94
95  # /connect      → 打开 Provider 选择器, 交互式添加 API Key
96  # /models       → 浏览并切换模型
97  # /new          → 开始新会话 (别名 /clear)
98  # /share        → 生成当前会话的可分享链接
99  # /export       → 将对话导出为 Markdown
100 # /themes       → 切换主题
101 # /compact      → 压缩当前会话 (别名 /summarize)
102
103 # -----
104 # 第六步: Plan / Build 双模式
105 # Plan 模式: 只做分析, 不改代码 (先想清楚)
106 # Build 模式: 执行修改, 直接写代码 (动手干)
107 # 按 Tab 键在两种模式间切换
108 # -----
109
110 # -----
111 # 第七步: CLI 运行模式 (非交互式, 适合脚本/CI)
112 # -----
113
114 # 直接执行一个编程任务
115 opencode run "Explain how closures work in JavaScript"
116
117 # 启动并继续上次会话
118 opencode --continue
119
120 # 指定模型运行
121 opencode --model anthropic/claude-4.5-sonnet
122
123 # 继续某个特定会话并 Fork
124 opencode --session abc123 --fork
125
126 # 启动无界面服务 (可作为后端)
127 opencode serve
128
129 # -----
130 # 第八步: 常用快捷键与交互
131 # -----
132 # Ctrl+K      → 打开命令面板 (可执行内置命令, 如 Initialize project)
133 # Ctrl+C      → 中断当前操作
```

```
134 # /          → 输入斜杠命令前缀，快速调用命令
135
136 # -----
137 # 第九步：内置项目初始化
138 # 在 TUI 中按 Ctrl+K，选择 "Initialize project"
139 # 会创建或更新 OpenCode.md 记忆文件，记录项目特定信息
140 # -----
141
142 # =====
143 # 完成。复制以上命令即可从零跑通 OpenCode。
144 #
145 # 核心记忆点：
146 #   安装：    curl -fsSL https://opencode.ai/install | bash
147 #   启动：    cd 项目目录 && opencode
148 #   配置：    /connect (TUI内) 或 export XXX_API_KEY
149 #   模式：    Tab 键切换 Plan / Build
150 #   模型：    75+ 提供商，完全免费，模型选择权在你手中
151 # =====
```

二、编排类

Harness管“单个Agent怎么干活”，编排类管“多个Agent怎么协作”。

2.1 LangGraph

维度	说明
核心定位	LangChain生态的旗舰级编排框架，将Agent工作流建模为 显式的有向图（StateGraph） 。节点是函数，边是状态流转路径，每一步后自动检查点。
核心机制	基于 Pregel计算模型 ，每一步自动保存检查点（Checkpoint），支持任意节点中断恢复。 持久化执行 是LangGraph的杀手锏——长任务跑到第50步挂了，可以从第49步继续，不用从头再来。
自研借鉴点	检查点机制的核心价值不在于“能恢复”，而在于 时间旅行调试 ——回溯到任意一步，查看当时的完整状态，修改后从那个点重新执行。
优点	绝对控制权，流程完全可预测；持久化执行大幅降低长任务失败成本；配合LangSmith提供顶级可观测性；生态庞大。
缺点	学习曲线陡，图结构心智模型不易建立；过度声明——要求在知道Agent会做什么之前就命名所有转移；比LangChain慢68%。
适用场景	金融审批、医疗诊断、长流程有状态任务。

维度	说明
上手方式	<code>pip install langgraph</code> ，代码中构建 StateGraph + nodes + edges。

```
1 # =====
2 # LangGraph 一键上手完整 Coding 示例
3 # 包含: 安装 → 核心概念 → 构建Agent → 条件路由 → 检查点 → 工具调用 → 时间旅行
4 # LangGraph = 有状态多智能体图编排框架
5 # 核心理念: StateGraph (状态图) + Pregel 计算模型 + 检查点机制
6 # =====
7
8 # -----
9 # 第一步: 安装与环境准备
10 # LangGraph 需要 Python 3.10+
11 # -----
12 # pip install -U langgraph # 安装/升级 LangGraph
13 # pip install -U langgraph langchain-openai # 同时安装 LLM 提供商包
14 # pip install -U langgraph-checkpoint-sqlite # SQLite 检查点后端 (可选)
15 # pip install -U "langgraph-cli[inmem]" # LangGraph CLI (可选, 用
16 # -----
17
18 # -----
19 # 第二步: 核心概念速览
20 # -----
21 # State → 共享状态, 节点之间传递数据的载体 (TypedDict)
22 # Node → 执行具体工作的函数 (调用 LLM、执行工具、数据处理)
23 # Edge → 定义节点之间的流转路径 (固定边 add_edge / 条件边 add_conditional_ea
24 # START → 图的入口虚拟节点
25 # END → 图的出口虚拟节点
26 # Checkpointer → 检查点记录器, 每一步自动保存状态快照
27 # Super-step → 一个执行步骤, LangGraph 在每个 super-step 边界创建检查点
28 # Thread → 线程 ID, 组织检查点的唯一标识 (thread_id)
29 # -----
30
31 # -----
32 # 第三步: 最小可运行示例 — Hello World
33 # 演示 State + Node + Edge 的基础结构
34 # -----
35 from langgraph.graph import StateGraph, MessagesState, START, END
36 from typing_extensions import TypedDict
37
38 # 定义共享状态结构 (使用 MessagesState 简化消息管理)
39 # MessagesState 是 LangGraph 预置的状态类型, 自动包含 messages 字段
40 # 也可以自定义 State:
41 # class MyState(TypedDict):
```

```

42 #     messages: List
43 #     next: str
44
45 # 定义节点函数: 模拟 LLM 响应
46 def mock_llm(state: MessagesState):
47     """接收当前状态, 返回要更新的状态字段"""
48     return {"messages": [{"role": "ai", "content": "hello world"}]}
49
50 # 构建图
51 graph = StateGraph(MessagesState)          # 1. 创建状态图, 指定状态类型
52 graph.add_node("mock_llm", mock_llm)      # 2. 添加节点: 名称 + 函数
53 graph.add_edge(START, "mock_llm")        # 3. 从 START 连接到 mock_llm
54 graph.add_edge("mock_llm", END)          # 4. 从 mock_llm 连接到 END
55
56 # 编译图 (不使用检查点)
57 app = graph.compile()
58
59 # 执行图
60 result = app.invoke({"messages": [{"role": "user", "content": "hi!"}]}))
61 print("Hello World 输出:", result["messages"][-1].content)
62 # 输出: hello world
63
64 # -----
65 # 第四步: 完整 Agent 示例 — 带工具调用和条件路由
66 # 包含: State定义 → 节点实现 → 条件边 → 检查点 → 工具执行
67 # -----
68 from langgraph.graph import StateGraph, START, END
69 from langgraph.checkpoint.memory import InMemorySaver
70 from langchain_openai import ChatOpenAI
71 from langchain_core.tools import tool
72 from typing import Literal, Annotated
73 from langgraph.graph.message import add_messages
74
75 # --- 4.1 定义共享状态 ---
76 class AgentState(TypedDict):
77     """Agent 的共享状态"""
78     messages: Annotated[list, add_messages] # add_messages 自动合并消息列表
79     next: str                               # 下一步路由目标
80
81 # --- 4.2 定义工具 ---
82 @tool
83 def search_web(query: str) -> str:
84     """搜索网络获取信息"""
85     # 实际实现中调用搜索 API
86     return f"搜索结果: 关于 '{query}' 的最新信息..."
87

```

```

88 @tool
89 def calculate(expression: str) -> str:
90     """计算数学表达式"""
91     try:
92         result = eval(expression)
93         return f"计算结果: {result}"
94     except Exception as e:
95         return f"计算错误: {e}"
96
97 tools = [search_web, calculate]
98 tools_by_name = {t.name: t for t in tools}
99
100 # --- 4.3 初始化 LLM 并绑定工具 ---
101 llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
102 llm_with_tools = llm.bind_tools(tools)
103
104 # --- 4.4 定义节点函数 ---
105
106 def agent_node(state: AgentState) -> dict:
107     """Agent 节点: 调用 LLM 决定下一步行动"""
108     response = llm_with_tools.invoke(state["messages"])
109     return {"messages": [response]}
110
111 def tool_node(state: AgentState) -> dict:
112     """工具节点: 执行 LLM 请求的工具调用"""
113     last_message = state["messages"][-1]
114     tool_messages = []
115     for tool_call in last_message.tool_calls:
116         tool_fn = tools_by_name[tool_call["name"]]
117         result = tool_fn.invoke(tool_call["args"])
118         tool_messages.append({
119             "role": "tool",
120             "content": str(result),
121             "tool_call_id": tool_call["id"]
122         })
123     return {"messages": tool_messages}
124
125 def should_continue(state: AgentState) -> Literal["tools", "end"]:
126     """条件路由函数: 判断是否继续调用工具"""
127     last_message = state["messages"][-1]
128     # 如果 LLM 返回了 tool_calls, 则路由到工具节点
129     if last_message.tool_calls:
130         return "tools"
131     # 否则结束
132     return "end"
133

```

```

134 # --- 4.5 构建图 ---
135 workflow = StateGraph(AgentState)
136
137 # 添加节点
138 workflow.add_node("agent", agent_node)
139 workflow.add_node("tools", tool_node)
140
141 # 添加边
142 workflow.add_edge(START, "agent") # 入口 → agent
143
144 # 条件边: 根据 should_continue 的返回值决定路由
145 workflow.add_conditional_edges(
146     "agent", # 源节点
147     should_continue, # 路由函数
148     {
149         "tools": "tools", # 返回 "tools" → 进入工具
150         "end": END # 返回 "end" → 结束
151     }
152 )
153 workflow.add_edge("tools", "agent") # 工具执行后 → 回到 agent
154
155 # --- 4.6 编译图 (带检查点) ---
156 # InMemorySaver 将检查点保存在内存中, 适合开发和测试
157 # 生产环境建议使用 PostgresSaver 或 SqliteSaver
158 memory = InMemorySaver()
159 app = workflow.compile(checkpointer=memory)
160
161 # --- 4.7 执行 Agent ---
162 # 配置线程 ID (检查点按线程组织)
163 config = {"configurable": {"thread_id": "user-001"}}
164
165 # 第一轮对话
166 result = app.invoke(
167     {"messages": [{"role": "user", "content": "帮我搜索一下 LangGraph 的最新版"}]},
168     config=config
169 )
170 print("第一轮输出:", result["messages"][-1].content)
171
172 # 第二轮对话 (检查点自动恢复上下文)
173 result = app.invoke(
174     {"messages": [{"role": "user", "content": "那 2 的 10 次方是多少? "}]},
175     config=config
176 )
177 print("第二轮输出:", result["messages"][-1].content)
178
179 # -----

```

```

180 # 第五步：检查点进阶 — 时间旅行调试
181 # 回溯到任意检查点，查看状态，修改后重新执行
182 # -----
183
184 # 获取线程的完整状态历史
185 state_history = list(app.get_state_history(config))
186 print(f"\n检查点数量: {len(state_history)}")
187
188 # 查看最新检查点的状态
189 latest_state = app.get_state(config)
190 print(f"当前消息数: {len(latest_state.values['messages'])}")
191 print(f"下一个节点: {latest_state.next}")
192
193 # --- 时间旅行：回溯到指定检查点 ---
194 # 选择要回溯的检查点（例如第 2 个检查点）
195 if len(state_history) > 1:
196     target_checkpoint = state_history[1] # 历史从新到旧排列
197     print(f"\n回溯到检查点: {target_checkpoint.config['configurable']['checkpoint_id']}")
198     print(f"该检查点消息数: {len(target_checkpoint.values['messages'])}")
199
200     # 从该检查点重新执行（可以修改状态后继续）
201     replay_result = app.invoke(None, config=target_checkpoint.config)
202     print(f"重放后消息数: {len(replay_result['messages'])}")
203
204 # -----
205 # 第六步：流式输出
206 # LangGraph 支持多种流式模式: values / updates / messages / custom
207 # -----
208
209 # 流式输出消息（适合实时 UI）
210 print("\n--- 流式输出示例 ---")
211 for event in app.stream(
212     {"messages": [{"role": "user", "content": "解释一下 LangGraph 的检查点机制"}]},
213     config={"configurable": {"thread_id": "stream-001"}},
214     stream_mode="messages"
215 ):
216     # event 是 (message_chunk, metadata) 元组
217     if hasattr(event[0], "content") and event[0].content:
218         print(event[0].content, end="", flush=True)
219
220 # -----
221 # 第七步：人机回环 (Human-in-the-Loop)
222 # 在关键节点前暂停，等待人工审批后继续
223 # -----
224 from langgraph.types import interrupt, Command
225

```

```

226 def review_node(state: AgentState) -> dict:
227     """需要人工审核的节点"""
228     # 触发中断，等待人工输入
229     human_input = interrupt("请审核以下内容并给出反馈:")
230     return {"messages": [{"role": "human", "content": human_input}]}
231
232 # 构建带审核的图
233 review_workflow = StateGraph(AgentState)
234 review_workflow.add_node("agent", agent_node)
235 review_workflow.add_node("review", review_node)
236 review_workflow.add_edge(START, "agent")
237 review_workflow.add_edge("agent", "review")
238 review_workflow.add_edge("review", END)
239
240 review_app = review_workflow.compile(checkpointer=InMemorySaver())
241
242 # 执行到 review 节点时会自动中断
243 review_config = {"configurable": {"thread_id": "review-001"}}
244 review_app.invoke(
245     {"messages": [{"role": "user", "content": "帮我写一段项目介绍"}]},
246     config=review_config
247 )
248 # 此时图已暂停在 review 节点
249
250 # 人工审批后恢复执行
251 review_app.invoke(
252     Command(resume="审核通过，请继续"),
253     config=review_config
254 )
255
256 # -----
257 # 第八步：持久化检查点（生产环境）
258 # 使用 SQLite 或 PostgreSQL 替代内存存储
259 # -----
260 # pip install langgraph-checkpoint-sqlite
261 # from langgraph.checkpoint.sqlite import SqliteSaver
262 #
263 # with SqliteSaver.from_conn_string("checkpoints.db") as checkpointer:
264 #     app = workflow.compile(checkpointer=checkpointer)
265 #     result = app.invoke(
266 #         {"messages": [{"role": "user", "content": "你好"}]},
267 #         config={"configurable": {"thread_id": "persistent-001"}}
268 #     )
269
270 # PostgreSQL 版本（生产环境推荐）：
271 # pip install langgraph-checkpoint-postgres

```

```
272 # from Langgraph.checkpoint.postgres import PostgresSaver
273 #
274 # with PostgresSaver.from_conn_string("postgresql://user:pass@localhost/db"),
275 #     checkpointinter.setup() # 首次使用需要初始化表结构
276 #     app = workflow.compile(checkpointer=checkpointinter)
277
278 # -----
279 # 第九步: LangGraph CLI — 本地开发服务器
280 # 用于测试和调试, 提供 REST API 和 Studio UI
281 # -----
282 # pip install "langgraph-cli[inmem]"
283 #
284 # # 创建 LangGraph 应用
285 # langgraph new my-agent --template new-langgraph-project-python
286 #
287 # # 启动本地服务器
288 # cd my-agent
289 # langgraph dev
290 #
291 # # 访问 Studio UI: https://smith.langchain.com/studio/
292 # # API 端点: http://localhost:2024
293
294 # -----
295 # 第十步: 常用命令速查
296 # -----
297 # 安装: pip install -U langgraph
298 # 升级: pip install --upgrade langgraph
299 # 查看版本: pip show langgraph
300 # CLI安装: pip install "langgraph-cli[inmem]"
301 # 启动开发: langgraph dev
302 # 构建容器: langgraph build -t my-agent
303
304 # =====
305 # 完成。复制以上代码即可从零跑通 LangGraph。
306 #
307 # 核心记忆点:
308 # 安装: pip install -U langgraph
309 # 四要素: State (状态) → Node (节点) → Edge (边) → Compile (编译)
310 # 条件边: add_conditional_edges(source, router_fn, path_map)
311 # 检查点: compile(checkpointer=memory)
312 # 线程: config={"configurable": {"thread_id": "xxx"}}
313 # 时间旅行: get_state_history() → invoke(None, config=checkpoint.config)
314 # 流式: app.stream(input, config, stream_mode="messages")
315 # 人机回环: interrupt() + Command(resume=...)
316 # =====
```

2.2 CrewAI

维度	说明
核心定位	主打“角色化团队”协作——用代码定义多个Agent角色（研究员、写手、分析师）协同完成任务。每个Agent有明确的角色、目标和背景故事。
双模式架构	Crews （角色驱动的自主协作） + Flows （事件驱动的精确控制）。CrewAI v1.14.2完全独立于LangChain重写了核心引擎。内存系统采用文件系统式的分层作用域。
自研借鉴点	双模式的设计哲学值得深思—— 不是用一个万能抽象解决所有问题，而是让合适的范式解决合适的问题。
优点	上手最快，角色驱动最贴近人的直觉；设置成本最低；10万+开发者认证。
缺点	Token消耗高（简单任务可能是LangGraph的3倍）；灵活性有限，复杂分支和精细控制较弱；有付费墙（\$99/月到\$120,000/年）。
适用场景	快速原型、报告生成、标准化流程。
上手方式	<code>crewai create flow latest-ai-flow</code> ，配置 <code>agents.jsonc</code> + <code>crew.jsonc</code> 后 <code>crewai run</code> 。

```
1 # =====
2 # CrewAI 一键上手完整 Coding 示例
3 # 包含：安装 → 项目脚手架 → 最小Crew → 工具调用 → Flow → 生产实践
4 # CrewAI = 角色驱动的多智能体协作框架
5 # 核心理念：用角色、目标和背景故事定义Agent，像团队一样协作
6 # =====
7
8 # -----
9 # 第一步：环境检查与安装
10 # CrewAI 需要 Python >=3.10 且 <3.14
11 # -----
12 # python3 --version                # 确认版本在 3.10 ~ 3.13 之间
13
14 # --- 安装 uv（CrewAI 官方推荐的依赖管理工具） ---
15 # macOS / Linux:
16 # curl -LsSf https://astral.sh/uv/install.sh | sh
17 # Windows (PowerShell):
18 # powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps
19
20 # --- 安装 CrewAI CLI（全局工具） ---
21 # uv tool install crewai
22 # 遇到 PATH 警告时执行：
23 # uv tool update-shell
```

```

24
25 # --- 验证安装 ---
26 # uv tool list                                # 应显示 crewai v1.15.x 左右
27 # crewai version                              # 查看 CLI 版本
28 # crewai version --tools                      # 查看已安装的 CrewAI 工具
29
30 # -----
31 # 第二步: 创建项目脚手架
32 # 默认生成 JSON-first 结构 (推荐)
33 # -----
34 # crewai create crew research_crew
35 # cd research_crew
36
37 # 生成的项目结构:
38 # research_crew/
39 # |--- .env                                # API Key 和环境变量
40 # |--- agents/
41 # |   |--- researcher.jsonc               # Agent 定义 (角色、目标、背景故事)
42 # |--- crew.jsonc                        # Crew 定义 (任务列表、流程、输出)
43 # |--- knowledge/                        # 知识库文件
44 # |--- skills/                           # Agent 技能
45 # |--- tools/                            # 自定义工具
46 # |--- pyproject.toml
47
48 # 如果需要旧版 Python/YAML 结构 (含 crew.py、config/agents.yaml):
49 # crewai create crew research_crew --classic
50
51 # -----
52 # 第三步: 配置 API Key
53 # 在 .env 文件中设置模型提供商的密钥
54 # -----
55 # 编辑 .env 文件, 填入:
56 # OPENAI_API_KEY=sk-...
57 # 或使用其他提供商:
58 # ANTHROPIC_API_KEY=sk-ant-...
59 # DEEPSEEK_API_KEY=sk-...
60 # 以及教程中需要的搜索工具:
61 # SERPER_API_KEY=your-serper-key
62
63 # 在代码中加载环境变量:
64 # from dotenv import load_dotenv
65 # load_dotenv()
66
67 # -----
68 # 第四步: 最小可运行 Crew -- 纯 Python 代码
69 # 不依赖脚手架, 一个文件跑通 Agent + Task + Crew

```

```

70 # -----
71 from crewai import Agent, Task, Crew, Process
72
73 # --- 4.1 定义 Agent (角色、目标、背景故事) ---
74 researcher = Agent(
75     role="Research Specialist",
76     goal="Uncover cutting-edge developments in {topic}",
77     backstory=(
78         "You're a seasoned researcher who finds relevant information "
79         "and presents it clearly with citations."
80     ),
81     verbose=True,
82     allow_delegation=False,          # 不允许委派给其他 Agent
83     # llm="gpt-5.6-terra",          # 可选: 指定模型, 默认从 .env 读取 MODEL
84 )
85
86 writer = Agent(
87     role="Technical Writer",
88     goal="Craft compelling content about {topic}",
89     backstory=(
90         "You transform complex research into engaging, accessible prose."
91     ),
92     verbose=True,
93     allow_delegation=False,
94 )
95
96 # --- 4.2 定义 Task (描述、预期输出、负责 Agent) ---
97 research_task = Task(
98     description=(
99         "Conduct thorough research about {topic}. "
100         "Use web search to find recent, credible information. "
101         "Focus on key trends, notable tools or companies, and implications."
102     ),
103     expected_output=(
104         "A markdown report with clear sections: key trends, "
105         "notable tools or companies, and implications. "
106         "Aim for 800-1200 words."
107     ),
108     agent=researcher,
109     # output_file="output/research.md", # 可选: 结果写入文件
110 )
111
112 writing_task = Task(
113     description=(
114         "Using the research report provided, write a polished article "
115         "about {topic}. Make it engaging and suitable for a technical audier

```

```

116     ),
117     expected_output="A well-structured article of 1000-1500 words in markdown",
118     agent=writer,
119     context=[research_task],          # 依赖 research_task 的输出
120     # output_file="output/article.md",
121 )
122
123 # --- 4.3 组建 Crew ---
124 crew = Crew(
125     agents=[researcher, writer],
126     tasks=[research_task, writing_task],
127     process=Process.sequential,      # 顺序执行: research → writing
128     verbose=True,
129     # memory=True,                  # 启用记忆系统 (需配置 embedder)
130     # max_rpm=100,                  # 每分钟最大请求数
131 )
132
133 # --- 4.4 运行 ---
134 result = crew.kickoff(inputs={"topic": "Edge AI in 2026"})
135 print("\n=== 最终输出 ===")
136 print(result)
137
138 # -----
139 # 第五步: 从 JSON 配置文件加载 Crew (脚手架项目中的用法)
140 # 当使用 crewai create crew 后, Agent 和 Task 定义在 .jsonc 文件中
141 # -----
142 # src/research_crew/crews/content_crew/content_crew.py
143 # from pathlib import Path
144 # from crewai.project import load_crew
145 #
146 # def kickoff_content_crew(inputs: dict):
147 #     crew, default_inputs = load_crew(
148 #         Path(__file__).with_name("crew.jsonc")
149 #     )
150 #     return crew.kickoff(inputs={**default_inputs, **inputs})
151
152 # agents/researcher.jsonc 示例:
153 # {
154 #     "role": "{topic} Senior Data Researcher",
155 #     "goal": "Uncover cutting-edge developments in {topic}",
156 #     "backstory": "You're a seasoned researcher who finds relevant information",
157 #     "tools": ["SerperDevTool"],
158 #     "settings": { "verbose": true }
159 # }
160
161 # crew.jsonc 示例:

```

```

162 # {
163 #     "name": "Research Crew",
164 #     "agents": ["researcher"],
165 #     "tasks": [
166 #         {
167 #             "name": "research_task",
168 #             "description": "Conduct thorough research about {topic}...",
169 #             "expected_output": "A markdown report...",
170 #             "agent": "researcher",
171 #             "output_file": "output/report.md",
172 #             "markdown": true
173 #         }
174 #     ],
175 #     "process": "sequential",
176 #     "verbose": true
177 # }
178
179 # -----
180 # 第六步: 自定义工具
181 # 两种方式: @tool 装饰器 (轻量) 或继承 BaseTool (完整)
182 # -----
183 from crewai.tools import BaseTool, tool
184 from pydantic import BaseModel, Field
185 from typing import Type
186
187 # --- 方式一: @tool 装饰器 (最简) ---
188 @tool("Calculate Sum")
189 def calculate_sum(a: int, b: int) -> int:
190     """Calculate the sum of two integers."""
191     return a + b
192
193 # --- 方式二: 继承 BaseTool (支持输入校验和完整描述) ---
194 class SearchToolInput(BaseModel):
195     """Input schema for SearchTool."""
196     query: str = Field(..., description="The search query")
197
198 class MySearchTool(BaseTool):
199     name: str = "Web Search"
200     description: str = "Search the web for recent information about a topic."
201     args_schema: Type[BaseModel] = SearchToolInput
202
203     def _run(self, query: str) -> str:
204         # 实际实现中调用搜索 API
205         return f"Search results for: {query}"
206
207 # 将自定义工具绑定到 Agent:

```

```

208 # researcher = Agent(
209 #     role="...",
210 #     tools=[calculate_sum, MySearchTool()],
211 #     ...
212 # )
213
214 # -----
215 # 第七步: Flow — 事件驱动的生产级 workflow
216 # Crews 负责“谁做什么”, Flows 负责“什么时候做什么”
217 # -----
218 from crewai.flow.flow import Flow, start, listen, router, or_
219 from pydantic import BaseModel
220
221 class ResearchState(BaseModel):
222     """Flow 的共享状态"""
223     topic: str = ""
224     report: str = ""
225     quality_score: float = 0.0
226
227 class ResearchFlow(Flow[ResearchState]):
228
229     @start()
230     def set_topic(self):
231         """入口: 设置研究主题"""
232         self.state.topic = "Edge AI in 2026"
233         print(f"[Flow] 研究主题: {self.state.topic}")
234         return self.state.topic
235
236     @listen(set_topic)
237     def run_research(self, topic: str):
238         """监听 set_topic 完成, 启动 Crew 执行研究"""
239         crew = Crew(
240             agents=[researcher],
241             tasks=[research_task],
242             process=Process.sequential,
243             verbose=True,
244         )
245         result = crew.kickoff(inputs={"topic": topic})
246         self.state.report = result.raw
247         return result.raw
248
249     @router(run_research)
250     def check_quality(self, report: str) -> str:
251         """路由: 根据报告长度决定下一步"""
252         if len(report) > 500:
253             return "good" # 分支标签

```

```

254         return "needs_work"          # 分支标签
255
256     @listen("good")
257     def finalize(self, report: str):
258         """质量达标：保存最终报告"""
259         self.state.quality_score = 0.9
260         print(f"[Flow] 报告已定稿，质量分：{self.state.quality_score}")
261         # with open("output/report.md", "w") as f:
262         #     f.write(report)
263
264     @listen("needs_work")
265     def retry(self, report: str):
266         """质量不达标：触发重试"""
267         print("[Flow] 报告太短，重新研究...")
268
269 # 运行 Flow:
270 # flow = ResearchFlow()
271 # flow.kickoff()
272
273 # -----
274 # 第八步：人机回环 (Human-in-the-Loop)
275 # 在关键节点暂停，等待人工审批
276 # -----
277 # from crewai.flow.flow import Flow, start, listen, human_feedback
278 #
279 # class ApprovalFlow(Flow):
280 #     @start()
281 #     def draft(self):
282 #         return "这是初稿内容..."
283 #
284 #     @human_feedback(message="请审核以下内容：")
285 #     @listen(draft)
286 #     def review(self, draft: str):
287 #         # 人工审批后才继续
288 #         return f"审核通过: {draft}"
289 #
290 #     @listen(review)
291 #     def publish(self, content: str):
292 #         print(f"发布: {content}")
293
294 # -----
295 # 第九步：记忆系统
296 # 启用 memory=True 后自动激活短期、长期和实体记忆
297 # -----
298 from crewai import Memory
299

```

```

300 # 基础启用（使用 Crew 的 embedder 配置）
301 # crew = Crew(
302 #     agents=[...],
303 #     tasks=[...],
304 #     memory=True,                # 自动启用短期 + 长期 + 实体记忆
305 #     embedder={
306 #         "provider": "openai",
307 #         "config": {"model_name": "text-embedding-3-small"}
308 #     },
309 # )
310
311 # 使用本地 Ollama embedder（无需 API Key）
312 # embedder = build_embedder({
313 #     "provider": "ollama",
314 #     "config": {"model_name": "mxbai-embed-large"}
315 # })
316 # memory = Memory(embedder=embedder)
317
318 # 重置记忆:
319 # crewai reset-memories -m
320
321 # -----
322 # 第十步: CrewAI CLI 常用命令速查
323 # -----
324 # 创建项目:
325 # crewai create crew <name>                # JSON-first 结构（默认）
326 # crewai create crew <name> --classic       # 旧版 Python/YAML 结构
327 # crewai create flow <name>                 # 创建 Flow 项目
328 # crewai create tool <handle>               # 创建自定义工具
329 # crewai create skill <name>                # 创建 Agent 技能
330
331 # 运行项目:
332 # crewai run                                # 执行 crew.jsonc 或 Flow 入口
333 # crewai run -- "Edge AI in 2026"          # 传入输入参数
334
335 # 训练与调试:
336 # crewai train -n 10                        # 训练 10 轮
337 # crewai replay -t task_123456              # 从指定任务重放
338 # crewai log-tasks-outputs                  # 查看最新任务输出
339 # crewai reset-memories -m                  # 重置所有记忆
340
341 # 版本与工具:
342 # crewai version                            # 查看版本
343 # crewai version --tools                    # 查看已安装工具版本
344
345 # =====

```

```
346 # 完成。复制以上代码即可从零跑通 CrewAI。
347 #
348 # 核心记忆点:
349 # 安装: uv tool install crewai
350 # 脚手架: crewai create crew research_crew
351 # 三件套: Agent(role, goal, backstory) + Task(description, agent) + Crew(c
352 # 运行: crew.kickoff(inputs={"topic": "..."})
353 # Flow: @start → @Listen → @router → @Listen("Label")
354 # 双模式: Crews (自主协作) + Flows (事件驱动精确控制)
355 # 工具: @tool 装饰器 或 继承 BaseTool
356 # 记忆: memory=True 自动启用短期/长期/实体记忆
357 # =====
```

2.3 AutoGen

维度	说明
核心定位	微软研究院开源的多智能体对话框架，Agent通过自然语言对话协作解决问题。擅长让多个Agent互相讨论、逐步收敛。
⚠️ 关键转折	AutoGen已于2025年10月进入维护模式 ，不再添加新功能。微软官方继任者是 Microsoft Agent Framework (MAF) ，2026年4月已发布1.0 GA。
优点	协作自然，人机协同好；学术认可度高，多智能体对话框架的标杆。
缺点	流程不可控，对话式协作难以精确控制；调试困难；没有一等公民状态； 已进入维护模式 。
适用场景	新项目不推荐 → 建议直接使用MAF。
上手方式	pip install pyautogen (仅限存量项目维护)。

```
1 # =====
2 # AutoGen 一键上手完整 Coding 示例
3 # 包含: 安装 → 核心概念 → 双Agent对话 → 群组聊天 → 工具调用 → 代码执行器 → MAF 迁移
4 # ⚠️ 重要: AutoGen 已于 2025年10月进入维护模式，不再添加新功能。
5 # 微软官方继任者为 Microsoft Agent Framework (MAF)，2026年4月已发布 1.0 GA。
6 # 新项目建议直接使用 MAF，本文同时提供 AutoGen 上手与迁移参考。
7 # =====
8
9 # -----
10 # 第一步: 环境检查与安装
11 # AutoGen 需要 Python >= 3.10
```

```

12 # -----
13 # python3 --version          # 确认 Python 版本
14
15 # --- 安装 AutoGen 0.4+ (当前推荐版本, 异步 API) ---
16 # pip install "autogen-agentchat>=0.4"
17 # pip install "autogen-ext[openai]" # OpenAI 模型扩展
18
19 # --- 安装 AutoGen 0.2.x (经典同步 API, 仅存量项目维护) ---
20 # pip install "autogen-agentchat~0.2"
21 # 注意: 老版本包 pyautogen 自 0.2.34 起已不再由微软发布
22
23 # --- 安装 Microsoft Agent Framework (微软当前推荐, 新项目首选) ---
24 # pip install agent-framework azure-identity
25
26 # --- 验证安装 ---
27 # python -c "import autogen_agentchat; print(autogen_agentchat.__version__)"
28
29 # -----
30 # 第二步: 核心概念速览
31 # -----
32 # AssistantAgent    → 内置助手提示词的对话智能体, 能调用 LLM 和工具
33 # UserProxyAgent    → 代表用户的代理, 可执行代码、发起对话
34 # ConversableAgent → 可以参与对话的智能体基类 (0.2.x)
35 # GroupChat         → 多智能体群组对话 (0.2.x)
36 # GroupChatManager  → 管理群组对话的流程 (0.2.x)
37 # RoundRobinGroupChat → 轮询式群组聊天 (0.4+)
38 # SelectorGroupChat → LLM 选择下一个发言者的群组聊天 (0.4+)
39 # FunctionTool       → 将 Python 函数包装为 Agent 可调用的工具 (0.4+)
40 # CodeExecutor       → 代码执行器, 让 Agent 能运行代码
41 # -----
42
43 # -----
44 # 第三步: 最小可运行示例 — 双 Agent 对话 (0.4+ API)
45 # -----
46 import asyncio
47 from autogen_agentchat.agents import AssistantAgent
48 from autogen_ext.models.openai import OpenAIChatCompletionClient
49
50 async def simple_chat():
51     """最简双 Agent 对话示例"""
52
53     # --- 3.1 配置模型客户端 ---
54     model_client = OpenAIChatCompletionClient(
55         model="gpt-4o-mini",
56         api_key="your-api-key", # 或从环境变量读取
57     )

```

```

58
59 # --- 3.2 创建 AssistantAgent ---
60 assistant = AssistantAgent(
61     name="assistant",
62     model_client=model_client,
63     system_message="你是一个帮助者，回答问题要简洁准确。",
64 )
65
66 # --- 3.3 运行对话 ---
67 # 0.4+ 使用异步 API: run() 或 run_stream()
68 result = await assistant.run(task="用一句话解释什么是 AutoGen? ")
69 print("Assistant 回复:", result.messages[-1].content)
70
71 # --- 3.4 关闭客户端 ---
72 await model_client.close()
73
74 # 运行:
75 # asyncio.run(simple_chat())
76
77 # -----
78 # 第四步: 多 Agent 协作 — RoundRobinGroupChat (0.4+)
79 # 多个 Agent 轮流发言, 协作完成任务
80 # -----
81 from autogen_agentchat.teams import RoundRobinGroupChat
82 from autogen_agentchat.conditions import TextMentionTermination
83
84 async def multi_agent_team():
85     """多 Agent 轮询协作示例"""
86
87     model_client = OpenAIChatCompletionClient(
88         model="gpt-4o-mini",
89         api_key="your-api-key",
90     )
91
92 # --- 4.1 定义两个不同角色的 Agent ---
93 researcher = AssistantAgent(
94     name="researcher",
95     model_client=model_client,
96     system_message=(
97         "你是一个研究员，负责收集和分析信息。"
98         "你的输出应该是结构化的研究笔记。"
99     ),
100 )
101
102 writer = AssistantAgent(
103     name="writer",

```

```

104         model_client=model_client,
105         system_message=(
106             "你是一个技术写手，负责将研究笔记转化为清晰的文章。"
107             "当文章完成后，请说'TERMINATE'。"
108         ),
109     )
110
111     # --- 4.2 创建群组聊天（轮询模式） ---
112     team = RoundRobinGroupChat(
113         participants=[researcher, writer],
114         termination_condition=TextMentionTermination("TERMINATE"), # 终止条
115         max_turns=6, # 最大轮
116     )
117
118     # --- 4.3 运行群组聊天 ---
119     result = await team.run(task="研究 2026 年 AI Agent 框架的现状，并写一篇 3000 字的文章")
120     print("\n=== 最终输出 ===")
121     print(result.messages[-1].content)
122
123     await model_client.close()
124
125     # 运行:
126     # asyncio.run(multi_agent_team())
127
128     # -----
129     # 第五步: 工具调用 — FunctionTool (0.4+)
130     # 将 Python 函数注册为 Agent 可调用的工具
131     # -----
132     from autogen_core.tools import FunctionTool
133
134     async def get_weather(location: str) -> str:
135         """Get weather for a location."""
136         # 实际实现中调用天气 API
137         return f"Weather in {location}: sunny"
138
139     async def calculate(expression: str) -> str:
140         """Calculate a mathematical expression."""
141         try:
142             result = eval(expression)
143             return f"Result: {result}"
144         except Exception as e:
145             return f"Error: {e}"
146
147     async def agent_with_tools():
148         """带工具调用的 Agent 示例"""
149

```

```

150     model_client = OpenAIChatCompletionClient(
151         model="gpt-4o-mini",
152         api_key="your-api-key",
153     )
154
155     # --- 5.1 创建工具 ---
156     weather_tool = FunctionTool(
157         func=get_weather,
158         description="Get weather information for a location",
159     )
160     calc_tool = FunctionTool(
161         func=calculate,
162         description="Calculate a mathematical expression",
163     )
164
165     # --- 5.2 创建带工具的 Agent ---
166     agent = AssistantAgent(
167         name="assistant",
168         model_client=model_client,
169         tools=[weather_tool, calc_tool],      # 绑定工具
170         system_message="你可以调用工具来回答问题。",
171     )
172
173     # --- 5.3 运行 ---
174     result = await agent.run(task="北京今天天气怎么样？另外帮我算一下 15 * 23。")
175     print("Agent 回复:", result.messages[-1].content)
176
177     await model_client.close()
178
179 # 运行:
180 # asyncio.run(agent_with_tools())
181
182 # -----
183 # 第六步: 代码执行器 — 让 Agent 自动写代码并运行
184 # ⚠️ 生产环境务必使用 Docker 执行器, 避免 LLM 生成危险命令
185 # -----
186 from autogen_ext.code_executors.docker import DockerCommandLineCodeExecutor
187 from autogen_agentchat.agents import CodeExecutorAgent
188
189 async def agent_with_code_executor():
190     """带代码执行器的 Agent 示例"""
191
192     # --- 6.1 创建 Docker 代码执行器 ---
193     # 参数说明:
194     # image: Docker 镜像, 默认 python:3-slim
195     # container_name: 容器名称, None 时自动生成

```

```

196 # time_out:          代码执行超时时间（秒）
197 # work_dir:          工作目录，默认当前目录
198 # auto_remove:       执行完成后自动删除容器
199 # stop_container:    是否自动停止容器[reference:1]
200 executor = DockerCommandLineCodeExecutor(
201     image="python:3-slim",
202     work_dir="/tmp/autogen_code",
203     timeout=60,
204     auto_remove=True,
205     stop_container=True,
206 )
207
208 # --- 6.2 启动执行器 ---
209 await executor.start()
210
211 # --- 6.3 创建代码执行 Agent ---
212 code_agent = CodeExecutorAgent(
213     name="code_executor",
214     code_executor=executor,
215 )
216
217 # --- 6.4 运行代码 ---
218 code_block = """`python
219 import math
220 result = sum(range(1, 101))
221 print(f"1 到 100 的和: {result}")
222 print(f"sqrt(144) = {math.sqrt(144)}")
223 `"""
224 result = await code_agent.run(task=code_block)
225 print("执行结果:", result.messages[-1].content)
226
227 # --- 6.5 停止执行器 ---
228 await executor.stop()
229
230 # 运行:
231 # asyncio.run(agent_with_code_executor())
232
233 # -----
234 # 第七步: 人机回环 (Human-in-the-Loop)
235 # 在关键节点暂停, 等待人工输入后继续
236 # -----
237 from autogen_agentchat.agents import UserProxyAgent
238 from autogen_agentchat.teams import RoundRobinGroupChat
239
240 async def human_in_the_loop():
241     """人机回环示例"""

```

```

242
243     model_client = OpenAIChatCompletionClient(
244         model="gpt-4o-mini",
245         api_key="your-api-key",
246     )
247
248     # --- 7.1 创建用户代理 Agent ---
249     user_proxy = UserProxyAgent(
250         name="user",
251         description="代表用户的代理，负责提供反馈和审批。",
252     )
253
254     assistant = AssistantAgent(
255         name="assistant",
256         model_client=model_client,
257         system_message="你是一个助手，当需要用户确认时，请明确说明。",
258     )
259
260     # --- 7.2 创建群组，包含用户代理 ---
261     team = RoundRobinGroupChat(
262         participants=[assistant, user_proxy],
263         max_turns=4,
264     )
265
266     # --- 7.3 运行（用户代理会模拟用户输入） ---
267     result = await team.run(task="帮我起草一封请假邮件。")
268     print("最终输出:", result.messages[-1].content)
269
270     await model_client.close()
271
272     # 运行:
273     # asyncio.run(human_in_the_loop())
274
275     # -----
276     # 第八步: 迁移到 Microsoft Agent Framework (MAF)
277     # AutoGen 已进入维护模式，新项目应使用 MAF
278     # MAF 于 2026年4月3日发布 1.0 GA，融合了 AutoGen + Semantic Kernel
279     # -----
280     # 安装:
281     # pip install agent-framework azure-identity
282
283     # --- MAF 最小示例 ---
284     # from agent_framework import Agent, tool
285     # from agent_framework.openai import OpenAIChatClient
286     # from typing import Annotated
287     # from pydantic import Field

```

```

288 #
289 # # 定义工具（使用 @tool 装饰器，自动生成 schema）
290 # @tool
291 # def get_weather(
292 #     location: Annotated[str, Field(description="The location to get weather")]
293 # ) -> str:
294 #     """Get weather for a location."""
295 #     return f"Weather in {location}: sunny"
296 #
297 # # 创建 Agent
298 # client = OpenAIChatClient(model="gpt-4o-mini")
299 # agent = Agent(
300 #     name="assistant",
301 #     client=client,
302 #     instructions="你是一个有帮助的助手。",
303 #     tools=[get_weather],
304 # )
305 #
306 # # 运行
307 # import asyncio
308 # result = asyncio.run(agent.run("北京天气怎么样? "))
309 # print(result.text)
310
311 # --- AutoGen → MAF 关键差异 ---
312 # AutoGen FunctionTool      → MAF @tool 装饰器（自动生成 schema）
313 # AutoGen max_tool_iterations → MAF 内置安全机制，自动处理工具迭代
314 # AutoGen 本地代码执行器    → MAF 计划在后续版本支持
315 # AutoGen MCP 扩展支持      → MAF 提供更全面的内置 MCP 支持
316
317 # -----
318 # 第九步：常用命令速查
319 # -----
320 # 安装 0.4+:      pip install "autogen-agentchat>=0.4"
321 # 安装 0.2.x:    pip install "autogen-agentchat~0.2"
322 # 安装 MAF:      pip install agent-framework azure-identity
323 # 查看版本:      python -c "import autogen_agentchat; print(autogen_agentchat.__version__)"
324 # 运行示例:      python your_script.py
325
326 # =====
327 # 完成。复制以上代码即可从零跑通 AutoGen。
328 #
329 # 核心记忆点：
330 # 安装:      pip install "autogen-agentchat>=0.4"
331 # 模型:      OpenAIChatCompletionClient(model="gpt-4o-mini")
332 # 单Agent:   AssistantAgent(name, model_client, system_message)
333 # 多Agent:   RoundRobinGroupChat([agent1, agent2], termination_condition)

```

```
334 # 工具:      FunctionTool(func, description) → 绑定到 Agent 的 tools 参数
335 # 代码执行: DockerCommandLineCodeExecutor(image="python:3-slim")
336 # 运行:      asyncio.run(agent.run(task)) 或 await team.run(task)
337 #
338 # ⚠️ 重要提醒:
339 # AutoGen 已于 2025年10月进入维护模式，不再添加新功能。
340 # 微软官方继任者为 Microsoft Agent Framework (MAF)，2026年4月发布 1.0 GA。
341 # 新项目建议直接使用 MAF，AutoGen 仅用于维护存量项目。
342 # =====
```

三、平台类

3.1 Dify

维度	说明
核心定位	开源LLM应用开发平台， 可视化拖拽 构建Agent工作流。不是Python库，而是完整的Web平台——包含工作流编辑器、模型管理、知识库管理、API发布、日志监控。
核心优势	开源之王，私有化部署首选。Dify采用后端即服务（BaaS）与LLMOps融合的架构，企业可以将整个平台部署在私有服务器上。 GitHub Stars约15.4万 ，运行在 超140万台机器 上。
优点	零代码可视化，业务团队可上手；GitHub Stars最多的Agent平台；原生集成MCP；支持私有化部署。
缺点	开源版与商业版功能有差异；7+容器、400+环境变量，复杂度成本高；深度定制受限。
适用场景	企业级LLM应用、业务团队快速搭建、RAG产品化。
上手方式	<code>docker run -d -p 80:80 langgenius/dify</code> ，访问 <code>http://localhost:80</code>

```
1 # =====
2 # Dify 一键上手完整 Coding 示例
3 # 包含: Docker Compose 部署 → 环境配置 → Chat API 调用 → Workflow API 调用
4 # Dify = 开源 LLM 应用开发平台，可视化拖拽构建 Agent 工作流
5 # 核心理念: 低代码/无代码构建 AI 应用，私有化部署首选
6 # =====
7
8 # -----
9 # 第一步: 环境检查与部署
10 # 硬件要求: CPU >= 2 Core, RAM >= 4 GiB
11 # 软件要求: Docker Compose 2.24.0+
```

```
12 # -----
13 # 确认 Docker Compose 版本
14 docker compose version          # 需 >= 2.24.0
15
16 # --- 克隆 Dify 源码 ---
17 git clone --branch "$(curl -s https://api.github.com/repos/langgenius/dify/r
18
19 # --- 进入 docker 目录并配置环境 ---
20 cd dify/docker
21 cp .env.example .env            # 复制环境配置文件
22 vim .env                        # 按需编辑（端口、数据库、向量库等）
23
24 # --- 启动所有服务 ---
25 docker compose up -d
26
27 # 启动的容器：
28 # 7 个核心服务：api, api_websocket, worker, worker_beat, web, plugin_daemon,
29 # 8 个依赖组件：weaviate, db_postgres, redis, nginx, ssrf_proxy, agent_ssrf_p
30 # 1 个一次性任务：init_permissions（设置存储权限后自动退出）
31
32 # --- 验证容器状态 ---
33 docker compose ps              # 每个容器应为 Up 或 healthy, init_permission:
34
35 # --- 访问初始化页面 ---
36 # 本地环境：http://localhost/install
37 # 服务器环境：http://your_server_ip/install
38 # 设置管理员账号后登录：http://localhost
39
40 # -----
41 # 第二步：环境变量配置（可选，按需修改）
42 # .env 文件核心配置项
43 # -----
44 # 数据库配置（PostgreSQL）
45 # DB_HOST=db
46 # DB_PORT=5432
47 # DB_USER=postgres
48 # DB_PASSWORD=difyai123456
49 # DB_DATABASE=dify
50
51 # 向量数据库选择（默认 weaviate）
52 # VECTOR_STORE=weaviate
53 # 可选：milvus, weaviate, opensearch, qdrant
54
55 # 存储类型（默认本地）
56 # STORAGE_TYPE=local
57 # 可选：s3, azure-blob, google-storage
```

```

58
59 # 端口配置
60 # EXPOSE_NGINX_PORT=80
61 # EXPOSE_NGINX_SSL_PORT=443
62
63 # 如需修改配置，编辑 .env 后重启：
64 # docker compose down && docker compose up -d
65
66 # -----
67 # 第三步：创建应用并获取 API Key
68 # 在 Dify Web UI 中完成（无需代码）
69 # -----
70 # 1. 登录 Dify → 创建应用（聊天助手 / Agent / 工作流）
71 # 2. 配置模型提供商（OpenAI、DeepSeek、通义千问等）
72 # 3. 编辑提示词、添加知识库、配置工具
73 # 4. 发布应用 → 进入「访问 API」页面 → 创建 API Key
74 # API Key 格式：app-xxxxxxxxxxxxx
75 # ⚠ API Key 应保存在服务端，切勿嵌入客户端代码
76
77 # -----
78 # 第四步：Chat API 调用 — 基础对话
79 # 使用 dify-client-python SDK（推荐）或直接 HTTP 请求
80 # -----
81
82 # --- 方式一：dify-client-python SDK（推荐） ---
83 # pip install dify-client-python
84
85 import uuid
86 from dify_client import Client, models
87
88 # 初始化客户端（替换为你的 API Key 和 Dify 地址）
89 client = Client(
90     api_key="app-your-api-key-here",
91     api_base="http://localhost/v1", # 自托管地址；云端为 https://api.dify.ai
92 )
93
94 user = str(uuid.uuid4()) # 终端用户唯一标识
95
96 # --- 阻塞模式（同步等待完整回复） ---
97 req = models.ChatRequest(
98     query="你好，请介绍一下 Dify 的核心功能",
99     inputs={}, # 应用输入变量，按需传入
100     user=user,
101     response_mode=models.ResponseMode.BLOCKING,
102 )
103 res = client.chat_messages(req, timeout=60.0)

```

```
104 print("阻塞模式回复:", res.answer)
105
106 # --- 流式模式 (逐块返回, 适合实时 UI) ---
107 stream_req = models.ChatRequest(
108     query="用 300 字解释什么是 Agent",
109     inputs={},
110     user=user,
111     response_mode=models.ResponseMode.STREAMING,
112 )
113 print("\n流式模式回复:")
114 for event in client.chat_messages(stream_req, timeout=60.0):
115     # event.event 为事件类型, event.answer 为文本块
116     if hasattr(event, "answer") and event.answer:
117         print(event.answer, end="", flush=True)
118
119 # --- 方式二: 直接 HTTP 请求 (curl) ---
120 # curl -X POST 'http://localhost/v1/chat-messages' \
121 #   -H 'Authorization: Bearer app-your-api-key-here' \
122 #   -H 'Content-Type: application/json' \
123 #   -d '{
124 #       "query": "你好",
125 #       "inputs": {},
126 #       "user": "user-123",
127 #       "response_mode": "blocking"
128 #   }'
129
130 # -----
131 # 第五步: Workflow API 调用 — 执行自动化 workflow
132 # 适用于已发布的 Workflow 应用
133 # -----
134
135 # --- 方式一: dify-client-python SDK ---
136 # Workflow 调用 (同步阻塞)
137 workflow_req = models.WorkflowRunRequest(
138     inputs={
139         "query": "Summarize this text: The quick brown fox jumps over the la
140     },
141     user="user_workflow_123",
142     response_mode=models.ResponseMode.BLOCKING,
143 )
144 workflow_res = client.workflow_run(workflow_req, timeout=300.0)
145 print("\n工作流输出:", workflow_res.data.outputs)
146
147 # --- 方式二: 直接 HTTP 请求 (curl) ---
148 # curl --request POST \
149 #   --url http://localhost/v1/workflows/run \
```

```

150 # --header 'Authorization: Bearer app-your-api-key-here' \
151 # --header 'Content-Type: application/json' \
152 # --data '{
153 #     "inputs": {
154 #         "query": "Summarize this text: The quick brown fox jumps over the lo
155 #     },
156 #     "response_mode": "blocking",
157 #     "user": "user_workflow_123"
158 # }'
159
160 # --- 流式 Workflow (Server-Sent Events) ---
161 # stream_wf_req = models.WorkflowRunRequest(
162 #     inputs={"query": "总结这段文本"},
163 #     user="user_workflow_123",
164 #     response_mode=models.ResponseMode.STREAMING,
165 # )
166 # for event in client.workflow_run(stream_wf_req, timeout=300.0):
167 #     print(event.event, event.data)
168
169 # -----
170 # 第六步: 文件上传 API (用于多模态工作流)
171 # 先上传文件获取 upload_file_id, 再传入 workflow/chat 请求
172 # -----
173 # 上传文件
174 # upload_res = client.file_upload(
175 #     ("document.pdf", open("document.pdf", "rb"), "application/pdf"),
176 #     models.FileUploadRequest(user=user),
177 # )
178 # file_id = upload_res.id
179 # print(f"文件上传成功, ID: {file_id}")
180
181 # 在 Chat 请求中引用文件
182 # chat_with_file = models.ChatRequest(
183 #     query="分析这份文档",
184 #     inputs={},
185 #     user=user,
186 #     response_mode=models.ResponseMode.BLOCKING,
187 #     files=[{
188 #         "type": "document",
189 #         "transfer_method": "local_file",
190 #         "upload_file_id": file_id,
191 #     }],
192 # )
193
194 # -----
195 # 第七步: 异步调用 (适合高并发场景)

```

```

196 # -----
197 # from dify_client import AsyncClient, models
198 # import asyncio
199 #
200 # async_client = AsyncClient(
201 #     api_key="app-your-api-key-here",
202 #     api_base="http://localhost/v1",
203 # )
204 #
205 # async def main():
206 #     req = models.ChatRequest(
207 #         query="hello",
208 #         inputs={},
209 #         user="user-1",
210 #         response_mode=models.ResponseMode.STREAMING,
211 #     )
212 #     async for chunk in await async_client.achat_messages(req, timeout=60.0):
213 #         print(chunk.event)
214 #
215 # asyncio.run(main())
216
217 # -----
218 # 第八步: Agent 应用 API 调用 (2026 新增)
219 # Agent 应用返回 agent_thought 和 agent_message 事件
220 # -----
221 # Agent 应用的 API 调用方式与 Chat 类似, 但事件流中包含推理过程:
222 # for event in client.chat_messages(agent_req, timeout=60.0):
223 #     if event.event == "agent_thought":
224 #         print(f"[推理] {event.thought}")
225 #     elif event.event == "agent_message":
226 #         print(f"[回复] {event.answer}")
227
228 # -----
229 # 第九步: 常用运维命令
230 # -----
231 # 查看容器状态:      docker compose ps
232 # 查看日志:          docker compose logs -f api
233 # 重启所有服务:      docker compose restart
234 # 停止所有服务:      docker compose down
235 # 升级 Dify:          git pull && cd docker && docker compose up -d
236 # 备份数据:          docker compose exec db pg_dump -U postgres dify > backup
237
238 # =====
239 # 完成。复制以上代码即可从零跑通 Dify。
240 #
241 # 核心记忆点:

```

```
242 # 部署:      git clone → cd dify/docker → cp .env.example .env → docker co
243 # 访问:      http://localhost/install (初始化管理员账号)
244 # Chat API:  client.chat_messages(req) / POST /v1/chat-messages
245 # Workflow:  client.workflow_run(req) / POST /v1/workflows/run
246 # 文件上传:  client.file_upload() → 获取 upload_file_id
247 # API Key:   在 Web UI 中创建应用后, 进入「访问 API」页面获取
248 # SDK 安装:  pip install dify-client-python
249 # =====
```

3.2 Coze / 扣子

维度	说明
核心定位	字节跳动出品， 完全开源（Apache-2.0） 的一站式零代码AI Agent开发平台。2026年6月发布 3.0版本 ，支持多人多Agent协作。Agent可被创建、接入和随时调度。
核心优势	上手最快 ——从注册到搭建一个能用的客服Agent，全程不到15分钟。插件市场 800+ 官方和社区插件，支持一键发布到豆包、飞书、微信、网页。
优点	上手最快（5分钟出原型）；中文生态最完善；全平台支持（手机/电脑/网页）；支持Openclaw、Claude Code一键接入。
缺点	企业级权限管理弱于Dify；调用量有限制；深度定制受限；数据必须过字节云，政企客户有顾虑。
适用场景	快速原型、业务验证、客服Bot、个人助手。
上手方式	浏览器访问 coze.cn ，注册登录即可开始。

```
1 # =====
2 # Coze / 扣子 一键上手完整 Coding 示例
3 # 包含: 环境准备 → 开源版部署 → 模型配置 → SDK 安装 → Chat API 调用
4 #       → Workflow API 调用 → CLI 工具 → 常用命令速查
5 # Coze = 字节跳动出品的一站式零代码 AI Agent 开发平台
6 # 核心理念: 5 分钟出原型, 插件市场 800+, 一键发布到豆包/ 飞书/ 微信
7 # =====
8
9 # -----
10 # 第一步: 环境检查与安装 Docker
11 # Coze 开源版 (Coze Studio) 需要 Docker 环境
12 # 最低配置: 2 核 4G, 推荐 4 核 8G (实测内存峰值约 6GB)
13 # -----
14 # 确认 Docker 版本 (需 20.10+, 支持 BuildKit)
15 docker --version
16
```

```
17 # 确认 Docker Compose 版本 (需 v2.0+)
18 docker compose version
19
20 # 如果未安装 Docker, 访问 https://www.docker.com/ 下载安装
21 # macOS 推荐安装 Docker Desktop
22 # Windows 需确保已开启 Hyper-V (可能需重启一次)
23
24 # -----
25 # 第二步: 克隆 Coze Studio 开源项目
26 # 开源地址: https://github.com/coze-dev/coze-studio
27 # 许可证: Apache-2.0, 允许自由使用、修改及商用
28 # -----
29 git clone https://github.com/coze-dev/coze-studio.git
30 cd coze-studio
31
32 # 如果没有安装 Git, 也可以直接下载源码包解压
33
34 # -----
35 # 第三步: 配置大模型 (关键步骤)
36 # Coze Studio 支持的模型服务: Ark (火山方舟)、OpenAI、
37 # DeepSeek、Claude、Ollama、Qwen、Gemini
38 # 模型配置文件统一放在 backend/conf/model 目录中
39 # -----
40
41 # --- 3.1 查看可用的模型模板 ---
42 ls backend/conf/model/template/
43 # 模板目录覆盖了常见的模型类型, 如火山方舟、OpenAI 等
44
45 # --- 3.2 以 DeepSeek 为例, 复制模型模板 ---
46 cp backend/conf/model/template/model_template_ark_volc_deepseek-r1.yaml \
47     backend/conf/model/ark_volc_deepseek-r1.yaml
48
49 # --- 3.3 编辑配置文件, 填入必要参数 ---
50 # 打开 backend/conf/model/ark_volc_deepseek-r1.yaml
51 # 只需配置核心的几个参数即可 (其余可保持默认):
52
53 # base_url: "https://api.deepseek.com/v1"          # DeepSeek API 地址
54 # api_key:  "sk-你的DeepSeek密钥"                # 从 platform.deepseek.com 获取
55 # model:    "deepseek-chat"                       # 模型名称
56
57 # 建议同步修改 name 属性, 否则页面显示的名称会是模板默认的 Doubao-Seed
58 # name: "DeepSeek-Chat"
59
60 # -----
61 # 第四步: 启动 Coze Studio 服务
62 # -----
```

```
63 cd docker
64 cp .env.example .env
65
66 # 启动所有服务 (--profile "*" 加载全部配置)
67 docker compose --profile "*" up -d
68
69 # 验证容器状态
70 docker compose ps
71
72 # 访问 Coze Studio
73 # 本地环境: http://localhost:8888/
74 # 首次访问需注册账号后登录
75
76 # 在界面中创建一个智能体, 测试模型是否可用
77 # 如果 DeepSeek 能正常返回回复, 说明模型配置成功
78
79 # -----
80 # 第五步: 安装 Coze Python SDK
81 # 官方 SDK 包名: cozepy
82 # 支持 Python 3.7+
83 # -----
84 pip install cozepy
85
86 # 注意: 扣子编程 Python SDK 需升级至 0.17.0 及之后版本
87 # 以免调用发起对话或执行对话流 API 失败
88 # pip install --upgrade cozepy
89
90 # -----
91 # 第六步: Chat API 调用 — 智能体对话
92 # 前置准备:
93 # 1. 在 Coze 后台创建智能体并发布为 API 服务
94 # 2. 个人中心 → API 管理 → 创建个人访问令牌 (PAT)
95 # PAT 格式: pat_xxxxxxxxxxxx
96 # 3. bot_id: 智能体发布 API 后, Bot 编辑页 URL 末尾的数字
97 # -----
98 import os
99 from cozepy import Coze, TokenAuth, Message, ChatEventType
100
101 # --- 6.1 初始化客户端 ---
102 coze_api_token = os.getenv("COZE_API_TOKEN") # 填入你的 PAT
103 coze = Coze(auth=TokenAuth(coze_api_token), base_url="https://api.coze.cn")
104
105 bot_id = "你的bot_id" # 替换为实际 bot_id
106 user_id = "user_001" # 终端用户唯一标识
107
108 # --- 6.2 流式对话 (推荐, 实时打字机效果) ---
```

```

109 print("=== 流式对话 ===")
110 for event in coze.chat.stream(
111     bot_id=bot_id,
112     user_id=user_id,
113     additional_messages=[
114         Message.build_user_question_text("帮我查询南京今日天气，并给出出行穿搭建
115     ],
116 ):
117     # 事件类型: conversation.message.delta / conversation.message.completed
118     #             conversation.chat.completed / conversation.chat.failed
119     if event.event == ChatEventType.CONVERSATION_MESSAGE_DELTA:
120         print(event.message.content, end="", flush=True)
121
122 print("\n")
123
124 # --- 6.3 非流式对话（一次性返回完整回复） ---
125 # chat_result = coze.chat.create(
126 #     bot_id=bot_id,
127 #     user_id=user_id,
128 #     additional_messages=[
129 #         Message.build_user_question_text("你好，请介绍一下你自己")
130 #     ],
131 # )
132 # print("非流式回复:", chat_result.messages[-1].content)
133
134 # -----
135 # 第七步: Workflow API 调用 — 执行自动化 workflow
136 # 将应用发布为 API 服务后，调用 workflow OpenAPI
137 # -----
138
139 # --- 方式一: curl 直接调用（同步非流式） ---
140 # curl --location --request POST 'https://api.coze.cn/v1/workflow/run' \
141 #     --header 'Authorization: Bearer pat_your_pat_here' \
142 #     --header 'Content-Type: application/json' \
143 #     --data-raw '{
144 #         "workflow_id": "73664689170551*****",
145 #         "parameters": {
146 #             "user_id": "12345",
147 #             "user_name": "George"
148 #         },
149 #         "app_id": "743962661420117*****"
150 #     }'
151
152 # --- 方式二: curl 流式调用（适合打字机效果） ---
153 # curl --location --request POST 'https://api.coze.cn/v1/workflow/stream_run' \
154 #     --header 'Authorization: Bearer pat_your_pat_here' \

```

```

155 # --header 'Content-Type: application/json' \
156 # --data-raw '{
157 #     "workflow_id": "73664689170551****",
158 #     "parameters": {
159 #         "user_id": "12345",
160 #         "user_name": "George"
161 #     },
162 #     "app_id": "743962661420117****"
163 # }'
164
165 # --- 方式三: Python SDK 调用 ---
166 # from cozepy import Coze, TokenAuth, WorkflowRunRequest
167 #
168 # coze = Coze(auth=TokenAuth("pat_your_pat_here"), base_url="https://api.coze.com/v1")
169 #
170 # result = coze.workflows.runs.create(
171 #     workflow_id="73664689170551****",
172 #     parameters={"user_id": "12345", "user_name": "George"},
173 #     app_id="743962661420117****",
174 # )
175 # print(" workflow输出:", result.data)
176
177 # -----
178 # 第八步: Coze CLI — 命令行工具
179 # 扣子编程 CLI, 为智能体提供底层原生接口
180 # 支持自然语言创建、调试、部署应用和多媒体内容生成
181 # -----
182
183 # --- 8.1 安装 CLI ---
184 npm install -g @coze/cli --foreground-scripts
185
186 # --- 8.2 登录与授权 ---
187 coze auth login # 浏览器交互式 OAuth 登录
188 coze auth status # 检查当前登录状态
189 # coze auth status --format json # 以 JSON 格式输出状态
190
191 # --- 8.3 选择组织和空间 ---
192 coze organization list # 列出所有可访问的组织
193 coze organization use <org_id> # 设置默认组织
194 coze space list # 列出当前组织下的所有空间
195 coze space use <space_id> # 设置默认工作空间
196
197 # --- 8.4 创建项目 (Coze Coding) ---
198 # 通过自然语言描述创建 Web 应用
199 coze code project create --message "Build an intelligent assistant" --type v
200

```

```
201 # 创建并等待初始构建完成
202 coze code project create --message "Create a data dashboard" --type web
203
204 # 创建其他类型: app、小程序、Agent、工作流、Skill
205 # coze code project create --message "..." --type app
206 # coze code project create --message "..." --type agent
207
208 # --- 8.5 项目管理 ---
209 coze code project list          # 列出当前空间所有项目
210 coze code project get <projectId> # 查看项目详情
211 # coze code project delete <projectId> # 删除项目（不可逆，慎用）
212 # coze code project import --path ./my-project.zip # 从本地 zip 导入
213
214 # --- 8.6 多媒体生成 ---
215 coze generate --type image --prompt "一只赛博朋克风格的猫"
216 coze generate --type audio --prompt "欢迎来到扣子编程"
217 coze generate --type video --prompt "日落时分的海滩"
218
219 # -----
220 # 第九步：常用命令速查
221 # -----
222 # ===== 开源版部署 =====
223 # 克隆项目:          git clone https://github.com/coze-dev/coze-studio.git
224 # 配置模型:          cp backend/conf/model/template/... backend/conf/model/...
225 # 启动服务:          cd docker && cp .env.example .env && docker compose --prof
226 # 停止服务:          docker compose down
227 # 查看日志:          docker compose logs -f
228 # 重启服务:          docker compose restart
229
230 # ===== SDK 安装 =====
231 # Python SDK:        pip install cozepy
232 # Node.js SDK:       npm install @coze/api
233
234 # ===== CLI 安装 =====
235 # 安装:              npm install -g @coze/cli
236 # 登录:              coze auth login
237 # 创建项目:          coze code project create --message "..." --type web
238 # 生成媒体:          coze generate --type image --prompt "..."
239
240 # ===== API 地址 =====
241 # 国内:              https://api.coze.cn/v3/chat          （智能体对话）
242 # 国内:              https://api.coze.cn/v1/workflow/run   （工作流执行）
243 # 国内:              https://api.coze.cn/v1/workflow/stream_run （流式工作流）
244
245 # =====
246 # 完成。复制以上代码即可从零跑通 Coze / 扣子。
```

```
247 #
248 # 核心记忆点:
249 # 开源部署: git clone → 配置模型 → docker compose up -d → localhost:8888
250 # 模型配置: backend/conf/model/ 目录, 复制模板后填 base_url/api_key/model
251 # SDK: pip install cozepy
252 # Chat API: coze.chat.stream(bot_id, user_id, messages)
253 # Workflow: POST /v1/workflow/run (同步) 或 /v1/workflow/stream_run (流式)
254 # CLI: npm install -g @coze/cli → coze auth login → coze code project
255 # PAT令牌: Coze 后台 → 个人中心 → API 管理 → 创建个人访问令牌
256 #
257 # 与 Dify 的对比:
258 # Coze 开源版 (Coze Studio) 2026年7月开源, Apache-2.0
259 # 后端 Golang + 前端 React/TypeScript, 微服务架构, DDD 设计
260 # 支持模型: Ark、OpenAI、DeepSeek、Claude、Ollama、Qwen、Gemini
261 # 上手速度最快, 中文生态最完善, 5 分钟出原型
262 # 企业级权限管理弱于 Dify, 数据需过字节云
263 # =====
```

四、选型建议

你的需求	首选	核心理由
深度定制、模型自由切换	DeepSeek Harness	一切皆插件，连Agent Loop都可换
嵌入现有产品、OpenAI生态	Codex Harness	三层集成梯度，产品验证过
终端AI编程、反供应商锁定	OpenCode	模型无关，75+提供商
复杂有状态 workflow	LangGraph	检查点机制，中断可恢复
快速原型、角色化流程	CrewAI	双模式，上手最快
新项目、多智能体对话	Microsoft Agent Framework	AutoGen的继任者
企业级低代码落地	Dify	可视化编排，140万+部署
零代码快速原型	Coze/扣子	5分钟出原型

五、自研Agent框架的六个设计原则

原则	核心问题	关键设计
插件化要彻底	卸载不干净 → 运行时堵塞	可逆副作用，自动回滚
事件溯源	状态不可回溯 → 无法审计	仅追加事件日志，replay一等公民
主循环插桩点	插件只能围观 → 无法接管决策	waterfall事件，插件可修改主循环行为
安全边界分离	安全逻辑嵌入 → 无法独立审计	沙箱+execpolicy独立模块
Token效率是架构问题	先跑通后优化 → 永远优化不完	Retained Reasoning + Context Compression
参考实现	只讲用法不讲造法 → 无法改	600行Python从零实现核心机制

一些思考

1. 框架竞争已从“功能多”转向“适配新模型的速度”。
2. 状态管理是分水岭——检查点/事件溯源决定任务崩了能否恢复。
3. 所有框架都在赌“模型变成商品”，若赌错，模型无关的框架将失去意义。
 - 选框架就是选信念，没有对错，只有匹配。
 - 别用功能列表做决策，看它的进化方向。
 - 自研框架第一天就要想清楚“赌什么”，才能决定不做什么。